LINUX NETWORK TROUBLESHOOTING

OSI 7계층 장애 진단·대응 실습 명령어 총정리

상황별 장애 재현 · 진단 · 복구 명령어 모음 (L1~L7 + 시스템/컨테이너/클라우드/보안/자동화 + 부록 H 변수분리 대응)

사용 방법

1. 0장의 network namespace 랩에서 장애를 일으키고 진단·복구를 반복해 익힙니다.
2. 실제 서버에서는 **읽기 전용 점검 명령**부터 사용하고, 설정 변경·방화벽 규칙 삽입은 반드시 영향도를 확인한 뒤 실행합니다.
3. 항상 **아래(L1)에서 위(L7)로** 확인하고, 이상 지점의 계층에서 원인을 좁힙니다.
4. **전역 상수와 매개변수:** 조합론에서 계산한 값은 `K_` , 이름·IP·포트·경로처럼 고른 값은 `P_` (예: $P_NS_C, $P_C_IP, $P_PORT)로 0-1의 `lab.env` 에 분리되어 있습니다. 값을 바꿀 때는 lab.env 의 P_ 만 수정하세요. 어느 터미널에서든 문서의 명령을 실행하기 전에 그 터미널에서 `source ./lab.env` 를 먼저 실행하세요.
5. <꺾쇠> 부분은 환경에 맞게 바꿔 쓰세요. 일부 명령은 별도 패키지 설치가 필요합니다 (ethtool, tcpdump, mtr, iperf3, nmap, dnsutils 등).

주의 nmap·부하·DDoS 방어 실험·보안 점검 명령은 **본인이 관리하거나 허가받은 시스템에만** 사용하세요.

목차

0. 실습 랩 구성: 전역 상수와 매개변수 (network namespace)

1계층: 물리 계층 (Physical)

2계층: 데이터 링크 계층 (Data Link)

3계층: 네트워크 계층 (Network)

4계층: 전송 계층 (Transport)

5계층: 세션 계층 (Session)

6계층: 표현 계층 (Presentation)

7계층: 응용 계층 (Application)

부록 A. 시스템 자원 병목 (네트워크처럼 보이는 장애)

부록 B. 패킷 캡처 · 분석 종합

부록 C. 컨테이너 · 가상화 · 클라우드 환경

부록 D. 보안 관점 점검 (침입·공격 징후)

부록 E. 종합 진단 자동화 스크립트

부록 F. 증상 → 계층 빠른 판별표

부록 G. 계층별 핵심 명령어 한 장 요약

부록 H. 일방향 벡터·패턴·바이트 다리 (변수분리 대응 총정리)

부록 I. 개념별 시니어 학습 가이드

부록 J. 시니어 영역 (커널·TCP·추적·한도·LB·K8s·클라우드)

부록 K. 장애 사례 28건

부록 L. 학습 로드맵과 검증 코드

부록 M. 진단은 수식으로, 해결은 기존 방식으로 (호환성 가이드)

0. 실습 랩 구성: 전역 상수와 매개변수 (network namespace)

운영 서버가 아닌 격리된 가상 환경에서 장애를 일으키고 진단·복구합니다. 값은 두 종류로 나눕니다. 전역 상수 K_ 는 조합론의 덧셈 점화식(파스칼 삼각형)에서 계산으로 만든 값이고, 매개변수 P_ 는 이름·IP·포트·경로처럼 고른 값입니다. 환경이 달라지면 lab.env 의 P_ 값만 고치면 문서 전체 명령에 반영됩니다. 모든 명령은 root 권한 셸에서 실행하세요(sudo 는 변수를 넘기지 않으므로 sudo -i 로 root 셸을 연 뒤 사용).

0-1 전역 상수 K_ 와 매개변수 P_ 정의 (lab.env) 와 불러오기

판단 K_ 는 계산해서 읽기 전용으로 만들고(이미 있으면 건너뜀), P_ 는 값이 비어 있을 때만 기본값을 넣음. 문서의 모든 명령은 이 source 를 한 터미널에서 먼저 실행해야 동작

```
cat > lab.env <<'EOF'
# ===== 랩 설정: P_ 값만 고치면 아래 모든 명령에 반영 =====
# --- [전역 상수 K_] 파스칼 점화식 C(n,r)=C(n-1,r-1)+C(n-1,r) 으로 계산 (덧셈만 사용, 고른 값 아님) ---
_ROW() { local n=$1 i j; local -a a=(1) b
         for ((i=1;i<=n;i++)); do b=(1); for ((j=1;j<i;j++)); do b[j]=$((a[j-1]+a[j])); done; b[i]=1; a=
("${b[@]}"); done
         echo "${a[@]}"; }                                          # 파스칼 삼각형의 n행
_SUM() { local s=0 x; for x in $(_ROW $1); do s=$((s+x)); done; echo $s; }   # sum_r C(n,r)
_C()   { local -a a=($(_ROW $1)); echo ${a[$2]}; }                # C(n,r)
_K()   { [[ -v $1 ]] || declare -grx "$1=$2"; }                   # 읽기 전용 전역 상수 (두 번 source 해도 오류 없
음)
_K K_N_PORT   $(_SUM 16)     # 포트 개수 (16비트 필드)
_K K_N_IPV4   $(_SUM 32)     # IPv4 주소 개수
_K K_N_MAC    $(_SUM 48)     # MAC 주소 개수
_K K_N_TTL    $(_SUM 8)      # TTL 값 개수
_K K_LINKS_LAB $(_C 2 2)     # 호스트 2대의 전체 연결 수 C(2,2) = 1 → veth 1개
# --- [매개변수 P_] 고른 값: 호출 때 확정, 환경마다 다름 ---
# --- 네임스페이스(가상 컴퓨터) 이름 ---
: "${P_NS_C:=client}"          # 클라이언트 역할
: "${P_NS_S:=server}"          # 서버 역할
# --- 가상 NIC(veth) 이름 ---
: "${P_C_IF:=veth-c}"          # client 쪽 NIC
: "${P_S_IF:=veth-s}"          # server 쪽 NIC
# --- IP 주소 ---
: "${P_NET:=10.0.0.0/24}"      # 랩 네트워크 대역
: "${P_C_IP:=10.0.0.1}"        # client IP
: "${P_S_IP:=10.0.0.2}"        # server IP
: "${P_PREFIX:=24}"            # 서브넷 마스크 길이(/24)
# --- 포트 ---
: "${P_PORT:=8080}"            # 실습용 HTTP 포트
: "${P_TLS_PORT:=8443}"        # 실습용 TLS 포트
# --- 경로 / 이름 ---
: "${P_WEBROOT:=/tmp/www}"     # 웹 문서 폴더
: "${P_KEY:=/tmp/k.pem}"       # 테스트용 개인키
: "${P_CRT:=/tmp/c.pem}"       # 테스트용 인증서
: "${P_TLS_CN:=test.local}"    # 테스트 인증서의 CN(호스트명)
EOF
source ./lab.env
```

단어	뜻	단어	뜻
cat	파일 내용을 그대로 출력	s+x	합계 s에 x를 더함
>	출력을 파일로 보냄(덮어씀)	$s	셸 변수 s의 값
lab.env	랩 변수 정의 파일(전역 상수 K_와 매개변수 P_)	_C	lab.env 함수: 조합 C(n,r)을 파스칼 점화식으로 계산
<<	여기 문서(heredoc): 다음 줄들을 입력으로 사용	$(_ROW $1)	_ROW 함수를 $1로 호출한 결과
'EOF'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: EOF	${a[$2]}	배열 a의 $2번째 원소
_ROW	lab.env 함수: 파스칼 삼각형의 n번째 행을 점화식으로 만듦	_K	lab.env 함수: 이름과 값을 받아 읽기 전용 전역 상수로 선언
(	묶음 시작: 서브셸(별도 셸)에서 실행	[[	조건식 시작(bash 확장)
)	묶음 끝	-v	변수가 설정되어 있으면 참
{	명령 묶음 시작(현재 셸)	$1	셸 특수 변수: 함수·스크립트에 넘어온 인자/상태
local	함수 안에서만 유효한 지역 변수 선언	]]	조건식 끝

n=$1	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)	||	앞 명령이 실패하면 뒤 명령 실행
i	반복·인덱스 변수	declare	셸 변수 선언(속성 지정)
j	반복·인덱스 변수	-grx	전역(g)·읽기 전용(r)·내보내기(x) 속성으로 선언
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	"$1=$2"	조합 인자 → "$1=$2"
-a	배열로 선언	K_N_PORT	전역 상수: 포트 개수 65536 = Σ C(16,r)
a=	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)	$(_SUM 16)	명령 치환: 안쪽 명령을 실행한 출력이 이 자리에 들어감
1	숫자 1	16	숫자 값 16
b	파스칼 한 행을 담는 배열 b	K_N_IPV4	전역 상수: IPv4 주소 개수 2^32
for	목록의 각 값마다 반복 시작	$(_SUM 32)	명령 치환: 안쪽 명령을 실행한 출력이 이 자리에 들어감
i=1	변수 대입 이름=값	32	숫자 값 32
i	반복·인덱스 변수	K_N_MAC	전역 상수: MAC 주소 개수 2^48
<	파일을 입력으로 사용	$(_SUM 48)	명령 치환: 안쪽 명령을 실행한 출력이 이 자리에 들어감
=n	비교·대입에 쓰이는 값 n	48	숫자 값 48
i++	i를 1 증가	K_N_TTL	전역 상수: TTL 값 개수 256
do	반복문 본문 시작	$(_SUM 8)	명령 치환: 안쪽 명령을 실행한 출력이 이 자리에 들어감
b=	변수 대입 이름=값	8	숫자 값 8
j=1	변수 대입 이름=값	K_LINKS_LAB	전역 상수: 호스트 2대의 전체 연결 수 C(2,2)=1
j	반복·인덱스 변수	$(_C 2 2)	명령 치환: 안쪽 명령을 실행한 출력이 이 자리에 들어감
<	파일을 입력으로 사용	2	숫자 값 2
i	반복·인덱스 변수	:	아무 일도 안 하는 명령(뒤의 ${VAR:=값} 식만 평가하려고 사용)
j++	j를 1 증가	"${P_NS_C:=client}"	조합 인자 → "${P_NS_C · =client}"
b[j]=$	새 행 b의 j번째 값에 대입	"${P_NS_S:=server}"	조합 인자 → "${P_NS_S · =server}"
a[j-1]+a[j]	윗 행의 이웃한 두 값의 합(파스칼 점화식)	"${P_C_IF:=veth-c}"	조합 인자 → "${P_C_IF · =veth-c}"
done	반복문 끝	"${P_S_IF:=veth-s}"	조합 인자 → "${P_S_IF · =veth-s}"
b[i]=1	새 행 양 끝을 1로 둠	"${P_NET:=10.0.0.0/24}"	조합 인자 → "${P_NET · =10.0.0.0 · 24}"
a=	변수 대입 이름=값	"${P_C_IP:=10.0.0.1}"	조합 인자 → "${P_C_IP · =10.0.0.1}"
"${b[@]}"	따옴표로 묶은 명령 문자열: 다른 명령(예: C, S, bash -c)이 한 덩어리로 받아 실행	"${P_S_IP:=10.0.0.2}"	조합 인자 → "${P_S_IP · =10.0.0.2}"
echo	인자를 그대로 출력	"${P_PREFIX:=24}"	조합 인자 → "${P_PREFIX · =24}"
"${a[@]}"	조합 인자 → "${a[·]}"	"${P_PORT:=8080}"	조합 인자 → "${P_PORT · =8080}"
}	명령 묶음 끝	"${P_TLS_PORT:=8443}"	조합 인자 → "${P_TLS_PORT · =8443}"
_SUM	lab.env 함수: 비트 수 n을 받아 파스칼 삼각형 n행의 합 Σ C(n,r)=2^n 을 계산	"${P_WEBROOT:=/tmp/www}"	조합 인자 → "${P_WEBROOT · = · tmp · www}"
s=0	합계 s를 0으로 초기화	"${P_KEY:=/tmp/k.pem}"	조합 인자 → "${P_KEY · = · tmp · k.pem}"
x	임시 변수	"${P_CRT:=/tmp/c.pem}"	조합 인자 → "${P_CRT · = · tmp · c.pem}"
x	임시 변수	"${P_TLS_CN:=test.local}"	조합 인자 → "${P_TLS_CN · =test.local}"
in	for 반복에서 값 목록 앞에 붙는 말	EOF	여기 문서(heredoc)의 끝 표시
$(_ROW $1)	명령 치환: 안쪽 명령을 실행한 출력이 이 자리에 들어감	source	파일의 명령을 현재 셸에서 실행(변수가 현재 셸에 남음)
$1	셸 특수 변수: 함수·스크립트에 넘어온 인자/상태	./lab.env	파일·디렉터리 경로
s=$	변수 대입 이름=값		

0-2 랩 생성: client ↔ server 를 veth 케이블로 연결

판단 ping 2회 성공하면 정상. alias 는 source 이후, 다음 줄부터 사용 가능

```
ip netns add $P_NS_C; ip netns add $P_NS_S
ip link add $P_C_IF type veth peer name $P_S_IF
ip link set $P_C_IF netns $P_NS_C; ip link set $P_S_IF netns $P_NS_S
ip -n $P_NS_C addr add $P_C_IP/$P_PREFIX dev $P_C_IF
ip -n $P_NS_S addr add $P_S_IP/$P_PREFIX dev $P_S_IF
for n in $P_NS_C $P_NS_S; do ip -n $n link set lo up; done
ip -n $P_NS_C link set $P_C_IF up; ip -n $P_NS_S link set $P_S_IF up
alias C="ip netns exec $P_NS_C"     # 큰따옴표: 정의 시점에 변수가 값으로 치환됨
alias S="ip netns exec $P_NS_S"
C ping -c 2 $P_S_IP
```

단어	뜻	단어	뜻
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	in	for 반복에서 값 목록 앞에 붙는 말
netns	네트워크 네임스페이스 대상	$P_NS_C	client 네임스페이스 이름
add	추가	$P_NS_S	server 네임스페이스 이름
$P_NS_C	client 네임스페이스 이름	do	반복문 본문 시작
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	$n	셸 변수 n의 값
$P_NS_S	server 네임스페이스 이름	lo	루프백 인터페이스(자기 자신)
link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	up	인터페이스를 켬(또는 UP 상태만)
$P_C_IF	client 쪽 인터페이스 이름	done	반복문 끝
type	인터페이스 종류 지정	alias	긴 명령에 짧은 이름을 붙임
veth	가상 이더넷 쌍(양끝이 이어진 케이블)	C="ip	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)
peer	반대편 끝의 이름	netns	네트워크 네임스페이스 대상
name	이름	exec	네임스페이스·컨테이너 안에서 명령 실행
$P_S_IF	server 쪽 인터페이스 이름	$P_NS_C"	변수 P_NS_C 값(client 네임스페이스 이름)과 닫는 따옴표
set	속성 변경 (예: up/down, mtu)	S="ip	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)
-n	네임스페이스 이름 지정	$P_NS_S"	변수 P_NS_S 값(server 네임스페이스 이름)과 닫는 따옴표
addr	IP 주소 대상	C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)
$P_C_IP/$P_PREFIX	조합 인자 → $P_C_IP(client 쪽 IP) · $P_PREFIX(서브넷 prefix 길이)	ping	ICMP 에코 요청으로 도달성·지연 확인
dev	뒤에 오는 값이 장치(인터페이스) 이름임을 표시	-c	보낼 횟수
$P_S_IP/$P_PREFIX	조합 인자 → $P_S_IP(server 쪽 IP) · $P_PREFIX(서브넷 prefix 길이)	2	숫자 값 2
for	목록의 각 값마다 반복 시작	$P_S_IP	server 쪽 IP
n	행 번호 n (함수 인자)		

0-3 간이 웹서버 기동 (HTTP)

판단 client 에서 ok 가 출력되면 L1~L7 전 구간 정상

```
mkdir -p $P_WEBROOT && echo ok > $P_WEBROOT/index.html
S python3 -m http.server $P_PORT --bind $P_S_IP --directory $P_WEBROOT &
C curl -s http://$P_S_IP:$P_PORT/
```

단어	뜻	단어	뜻
mkdir	디렉터리 생성	http.server	파이썬 내장 간이 웹 서버 모듈
-p	중간 경로도 만들고, 이미 있어도 오류 없음	$P_PORT	실습 HTTP 포트(고른 값, 기본 8080)
$P_WEBROOT	웹 문서 디렉터리	--bind	바인드할 주소
&&	앞 명령이 성공하면 뒤 명령 실행	$P_S_IP	server 쪽 IP
echo	인자를 그대로 출력	--directory	서비스할 디렉터리
ok	성공 문구	$P_WEBROOT	웹 문서 디렉터리
>	출력을 파일로 보냄(덮어씀)	&	명령을 백그라운드로 실행
$P_WEBROOT/index.html	조합 인자 → $P_WEBROOT(웹 문서 디렉터리) · index.html	C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트

python3	Python 3 인터프리터	-s	진행 표시 숨김(silent)
-m	모듈을 프로그램으로 실행(예: http.server)	http://$P_S_IP:$P_PORT/	http 주소: 방식://호스트[:포트]/경로 → $P_S_IP(server 쪽 IP) · $P_PORT(실습 HTTP 포트(고른 값, 기본 8080))

0-4 랩 초기화 / 정리

판단 웹서버(프로세스)를 먼저 종료한 뒤 네임스페이스 삭제

```
pkill -f "http.server $P_PORT"; pkill -f s_server
ip netns del $P_NS_C; ip netns del $P_NS_S
rm -f $P_KEY $P_CRT
```

단어	뜻	단어	뜻
pkill	이름·조건으로 프로세스 종료	del	삭제
-f	전체 명령줄에서 일치	$P_NS_C	client 네임스페이스 이름
"http.server $P_PORT"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: http.server $P_PORT	$P_NS_S	server 네임스페이스 이름
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	rm	파일 삭제
s_server	openssl의 TLS 서버 모드	-f	없어도 오류 없이, 확인 없이 삭제
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	$P_KEY	개인 키 파일 경로
netns	네트워크 네임스페이스 대상	$P_CRT	인증서 파일 경로

1계층: 물리 계층 (Physical)

케이블, 전원, NIC, 스위치 포트, 광모듈(SFP), 링크 협상. 증상: 링크 다운, 속도·듀플렉스 불일치, CRC 에러 증가.

1-1 인터페이스 DOWN / 케이블 분리

판단 state DOWN, NO-CARRIER, Link detected: no → 케이블·포트·관리자 down

```
S ip link set $P_S_IF down            # 장애 발생
C ping -c 2 -W 1 $P_S_IP             # 실패
S ip -br link show $P_S_IF            # DOWN
ip -n $P_NS_C link show $P_C_IF        # NO-CARRIER
S ip link set $P_S_IF up              # 복구
```

단어	뜻	단어	뜻
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	-W	응답 대기 시간(초)
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	1	숫자 1
link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	$P_S_IP	server 쪽 IP
set	속성 변경 (예: up/down, mtu)	-br	한 줄 요약(brief)
$P_S_IF	server 쪽 인터페이스 이름	show	조회(표시)
down	인터페이스를 끔	-n	네임스페이스 이름 지정
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	$P_NS_C	client 네임스페이스 이름
ping	ICMP 에코 요청으로 도달성·지연 확인	$P_C_IF	client 쪽 인터페이스 이름
-c	보낼 횟수	up	인터페이스를 켬(또는 UP 상태만)
2	숫자 값 2		

1-2 링크 상태·속도·듀플렉스 점검

판단 Speed 10/100Mb 또는 Half 협상 = 케이블/포트/오토네고 문제

```
ip -br link
ethtool eth0 | egrep "Speed|Duplex|Auto-negotiation|Link detected"
ethtool -S eth0 | egrep -i "err|drop|crc|fifo|miss"
ethtool -i eth0                      # 드라이버/펌웨어 버전
ethtool -a eth0                      # 흐름제어(pause)
ethtool -k eth0 | egrep "tso|gso|gro|rx-checksum"    # 오프로드
cat /sys/class/net/eth0/{operstate,carrier,speed,duplex}
cat /sys/class/net/eth0/carrier_changes    # 링크 플래핑 횟수
```

단어	뜻	단어	뜻
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	-i	드라이버·펌웨어 정보
-br	한 줄 요약(brief)	-a	흐름 제어(pause) 설정
link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	-k	오프로드 기능 상태
ethtool	네트워크 카드(NIC) 속도·링크·통계 조회와 설정	"tso|gso|gro|rx-checksum"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: tso|gso|gro|rx-checksum
eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)	cat	파일 내용을 그대로 출력
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	/sys/class/net/eth0/	커널 장치 정보 가상 파일(/sys)
egrep	확장 정규식으로 줄 검색(grep -E 와 같음)	{	명령 묶음 시작(현재 셸)
"Speed|Duplex|Auto-negotiation|Link detected"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: Speed|Duplex|Auto-negotiation|Link detec	operstate,carrier,speed,duplex	읽을 sysfs 항목 이름들(링크 상태·캐리어·속도·duplex)
-S	NIC 통계 카운터	}	명령 묶음 끝
-i	대소문자 무시	/sys/class/net/eth0/carrier_changes	커널 장치 정보 가상 파일(/sys)
"err|drop|crc|fifo|miss"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: err|drop|crc|fifo|miss		

1-3 링크 플래핑 · NIC 하드웨어 오류 로그

판단 link is Down/Up 반복 = 케이블 불량, SFP 불량, 스위치 포트 오류

```
dmesg -T | egrep -i "eth0|link is (up|down)|nic|firmware|tx timeout|reset adapter"
journalctl -k --since "1 hour ago" | grep -i link
watch -n1 'cat /sys/class/net/eth0/carrier_changes'
ip monitor link                       # 링크 상태 변화 실시간 감시
```

단어	뜻	단어	뜻
dmesg	커널 메시지 버퍼 출력	grep	패턴에 맞는 줄만 출력
-T	사람이 읽는 시각으로 표시	-i	대소문자 무시
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성
egrep	확장 정규식으로 줄 검색(grep -E 와 같음)	watch	명령을 주기적으로 반복 실행해 화면 갱신
-i	대소문자 무시	-n1	갱신 간격(초) → 값 1
"eth0|link is (up|down)|nic|firmware|tx timeout|reset adapter"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: eth0|link is (up|down)|nic|firmware|tx t	'cat /sys/class/net/eth0/carrier_changes'	따옴표로 묶은 명령 문자열: 다른 명령(예: C, S, bash -c)이 한 덩어리로 받아 실행
journalctl	systemd 저널 로그 조회	ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구
-k	커널 메시지만	monitor	변화를 실시간으로 감시
--since	이 시각 이후	link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성
"1 hour ago"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 1 hour ago		

1-4 속도·듀플렉스 강제 설정 / 오토네고 복구

판단 양단 설정이 달라 Duplex mismatch 가 나면 late collision, CRC 증가

```
ethtool -s eth0 autoneg off speed 1000 duplex full   # 강제(양단 동일해야 함)
ethtool -s eth0 autoneg on                           # 복구
ethtool -r eth0                                      # 오토네고 재시작
```

단어	뜻	단어	뜻
ethtool	네트워크 카드(NIC) 속도·링크·통계 조회와 설정	1000	숫자 값 1000
-s	속도·duplex 등 설정 변경	duplex	통신 방식(full/half)
eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)	full	전이중
autoneg	속도·duplex 자동 협상	on	켬
off	끔	-r	오토네고시에이션 재시작
speed	링크 속도		

1-5 광모듈(SFP/SFP+) · 케이블 진단

판단 RX power 가 낮으면 광케이블/모듈 오염·불량

```
ethtool -m eth0                       # DOM: 온도, TX/RX power, 파장
ethtool --cable-test eth0             # 지원 NIC 의 TDR 케이블 테스트(드라이버 의존)
lspci | grep -i ethernet; lspci -vv -s <BDF> | grep -i -E "LnkSta|LnkCap"   # PCIe 링크 폭/속도
lshw -class network -short
```

단어	뜻	단어	뜻
ethtool	네트워크 카드(NIC) 속도·링크·통계 조회와 설정	-s	지정한 버스 위치의 장치만
-m	광모듈(SFP) 정보	<	파일을 입력으로 사용
eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)	BDF	PCI 장치 위치 자리표시자(버스:장치.기능)
--cable-test	케이블 진단	>	출력을 파일로 보냄(덮어씀)
lspci	PCI 장치 목록	-E	확장 정규식
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	"LnkSta|LnkCap"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: LnkSta|LnkCap
grep	패턴에 맞는 줄만 출력	lshw	하드웨어 목록
-i	대소문자 무시	-class	하드웨어 종류 지정
ethernet	이더넷 계층	network	네트워크 종류의 대상
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-short	요약 표시

-vv	아주 자세히

1-6 하드웨어 모니터링 (전원·온도·디스크)

판단 서버 자체 물리 장애 확인

```
ipmitool sdr; ipmitool sel list      # BMC 센서 / 이벤트 로그
sensors                              # lm-sensors 온도
smartctl -a /dev/sda                 # 디스크 SMART
dmidecode -t system                  # 장비 정보
```

단어	뜻	단어	뜻
ipmitool	서버 관리 보드(BMC) 조회	smartctl	디스크 S.M.A.R.T. 상태 조회
sdr	BMC 센서 데이터 기록	-a	디스크 건강 정보 전체
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	/dev/sda	장치 파일(/dev)
sel	BMC 시스템 이벤트 로그	dmidecode	BIOS/하드웨어 정보 표시
list	목록 조회	-t	표시할 정보 종류
sensors	온도·팬 센서 조회	system	시스템 전체

2계층: 데이터 링크 계층 (Data Link)

MAC, ARP, 스위치 학습, VLAN, 본딩, 브리지, STP. 같은 서브넷 내 통신 문제를 다룹니다.

2-1 잘못된 ARP 엔트리

판단 IP·라우팅 정상인데 통신 불가, MAC 이 실제와 다름

```
C ip neigh replace $P_S_IP lladdr 00:11:22:33:44:55 dev $P_C_IF nud permanent
C ping -c 2 -W 1 $P_S_IP            # 실패
C ip neigh show; S ip link show $P_S_IF
C tcpdump -i $P_C_IF -n arp -c 5
C ip neigh del $P_S_IP dev $P_C_IF   # 복구
C ip neigh flush all                 # 전체 캐시 삭제
```

단어	뜻	단어	뜻
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	$P_S_IP	server 쪽 IP
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	show	조회(표시)
neigh	이웃(ARP/NDP) 표 대상	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
replace	있으면 바꾸고 없으면 추가	S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)
$P_S_IP	server 쪽 IP	link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성
lladdr	링크 계층(MAC) 주소	$P_S_IF	server 쪽 인터페이스 이름
00:11:22:33:44:55	MAC 주소 예시	tcpdump	패킷 캡처·표시
dev	뒤에 오는 값이 장치(인터페이스) 이름임을 표시	-i	캡처할 인터페이스
$P_C_IF	client 쪽 인터페이스 이름	$P_C_IF	client 쪽 인터페이스 이름
nud	이웃 도달성 상태(Neighbor Unreachability Detection)	-n	이름 변환 안 함
permanent	영구 항목	arp	ARP 프로토콜(IP→MAC 해석)
ping	ICMP 에코 요청으로 도달성·지연 확인	-c	N개 잡고 종료
-c	보낼 횟수	5	숫자 값 5
2	숫자 값 2	del	삭제
-W	응답 대기 시간(초)	flush	항목 전체 비움
1	숫자 1	all	전체

2-2 ARP 상태 해석

판단 REACHABLE/STALE/DELAY/PROBE 정상, INCOMPLETE/FAILED = 응답 없음(L1·L2 문제)

```
ip neigh show
ip neigh show nud failed
ip neigh show nud incomplete
arp -n
arping -I eth0 -c 3 <대상IP>          # L2 도달성 (ICMP 와 무관)
cat /proc/net/arp
```

단어	뜻	단어	뜻
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	-I	사용할 인터페이스
neigh	이웃(ARP/NDP) 표 대상	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
show	조회(표시)	-c	보낼 요청 횟수
nud	이웃 도달성 상태(Neighbor Unreachability Detection)	3	숫자 값 3
failed	실패 상태	<	파일을 입력으로 사용
incomplete	해석 미완료 상태	대상IP	자리표시자: 확인할 대상 IP
arp	ARP 캐시(IP와 MAC 대응표) 조회	>	출력을 파일로 보냄(덮어씀)
-n	이름 변환 없이 숫자 주소로 표시	cat	파일 내용을 그대로 출력
arping	ARP 요청을 직접 보내 같은 링크의 호스트 응답을 확인	/proc/net/arp	커널이 보여 주는 가상 파일(/proc)

2-3 IP/MAC 중복 충돌

판단 arping -D 에서 응답이 오면 IP 중복

```
arping -D -I eth0 -c 3 <IP>          # DAD: 중복 IP 검사
arping -I eth0 -c 3 <IP>             # 응답한 MAC 목록 확인(2개 이상이면 충돌)
ip neigh show | sort -k5 | awk '{print $5}' | uniq -d    # 같은 MAC 이 여러 IP
# MAC 중복 실험
S ip link set $P_S_IF down
S ip link set $P_S_IF address $(C cat /sys/class/net/$P_C_IF/address)
S ip link set $P_S_IF up
```

단어	뜻	단어	뜻
arping	ARP 요청을 직접 보내 같은 링크의 호스트 응답을 확인	awk	열(필드) 단위로 줄을 가공하는 텍스트 처리기
-D	주소 중복 검사 모드(DAD)	'{print $5}'	awk 프로그램: 5번째 열 출력
-I	사용할 인터페이스	uniq	이웃한 같은 줄을 합침
eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)	-d	중복된 줄만
-c	보낼 요청 횟수	S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)
3	숫자 값 3	link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성
<	파일을 입력으로 사용	set	속성 변경 (예: up/down, mtu)
IP	자리표시자: 실제 IP 주소로 바꿔 넣음	$P_S_IF	server 쪽 인터페이스 이름
>	출력을 파일로 보냄(덮어씀)	down	인터페이스를 끔
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	address	addr 의 전체 이름
neigh	이웃(ARP/NDP) 표 대상	$(C cat /sys/class/net/$P_C_IF/address)	명령 치환: 안쪽 명령을 실행한 출력이 이 자리에 들어감
show	조회(표시)	C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	cat	파일 내용을 그대로 출력
sort	줄 정렬	/sys/class/net/$P_C_IF/address	조합 인자 → sys · class · net · $P_C_IF(client 쪽 인터페이스 이름) · address
-k5	5번째 열 기준	up	인터페이스를 켬(또는 UP 상태만)

2-4 패킷 손실 · 프레임 드롭 · 지연 주입

판단 tc netem 으로 장애를 재현하고 통계로 확인

```
S tc qdisc add dev $P_S_IF root netem loss 30%
S tc qdisc change dev $P_S_IF root netem delay 100ms 20ms loss 5% duplicate 1% corrupt 0.1% reorder 10%
C ping -c 20 $P_S_IP | tail -3
S ip -s -s link show $P_S_IF          # errors/dropped/overruns/carrier/collisions
S tc -s qdisc show dev $P_S_IF
S tc qdisc del dev $P_S_IF root       # 복구
nstat -az | egrep -i "InErrors|InDiscards|OutDiscards"
cat /proc/net/dev
```

단어	뜻	단어	뜻
S	랩의 server 네임스페이스에서 실행하는 별칭 (ip netns exec ns_s)	ping	ICMP 에코 요청으로 도달성·지연 확인
tc	트래픽 제어(지연·손실 모의 포함)	-c	보낼 횟수
qdisc	큐잉 규칙(트래픽 제어 단위)	20	숫자 값 20
add	추가	$P_S_IP	server 쪽 IP
dev	뒤에 오는 값이 장치(인터페이스) 이름임을 표시	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
$P_S_IF	server 쪽 인터페이스 이름	tail	뒤쪽 N줄 출력
root	최상위(루트) 위치	-3	마지막 3줄
netem	네트워크 에뮬레이터(지연·손실 모의)	ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구
loss	패킷 손실률 모의	-s	통계 카운터 포함
30%	값 30 (단위 %)	link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성
change	기존 규칙 값 변경	show	조회(표시)

delay	지연 모의	$P_S_IF	server 쪽 인터페이스 이름
100ms	값 100 (단위 ms)	-s	통계 포함
20ms	값 20 (단위 ms)	show	조회(표시)
5%	값 5 (단위 %)	del	삭제
duplicate	패킷 복제 모의	nstat	커널 네트워크 카운터 증가량 표시
1%	값 1 (단위 %)	-az	0인 카운터도 포함한 절대값(-a -z)
corrupt	패킷 손상 모의	egrep	확장 정규식으로 줄 검색(grep -E 와 같음)
0.1%	값 0.1 (단위 %)	-i	대소문자 무시
reorder	패킷 순서 뒤섞기 모의	"InErrors|InDiscards|OutDiscards"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: InErrors|InDiscards|OutDiscards
10%	값 10 (단위 %)	cat	파일 내용을 그대로 출력
C	랩의 client 네임스페이스에서 실행하는 별칭 (ip netns exec ns_c)	/proc/net/dev	커널이 보여 주는 가상 파일(/proc)

2-5 L2 트래픽 캡처 (ARP, 브로드캐스트, STP)

판단 ARP 폭주·루프 징후 = 브로드캐스트 스톰

```
tcpdump -i eth0 -n -e arp
tcpdump -i eth0 -n -e ether broadcast -c 100
tcpdump -i eth0 -n -e ether multicast
tcpdump -i eth0 -n -e -c 20 stp      # STP BPDU
tcpdump -i eth0 -n -e vlan           # 802.1Q 태그 확인
tcpdump -i eth0 -n -e ether host aa:bb:cc:dd:ee:ff
```

단어	뜻	단어	뜻
tcpdump	패킷 캡처·표시	-c	N개 잡고 종료
-i	캡처할 인터페이스	100	숫자 값 100
eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)	multicast	멀티캐스트
-n	이름 변환 안 함	20	숫자 값 20
-e	링크 계층(MAC) 헤더 표시	stp	스패닝 트리 프로토콜
arp	ARP 프로토콜(IP→MAC 해석)	vlan	VLAN(가상 LAN) 종류
ether	이더넷(MAC) 계층	host	호스트 조건 (tcpdump: 이 주소와 오가는 패킷)
broadcast	브로드캐스트 주소	aa:bb:cc:dd:ee:ff	MAC 주소 예시

2-6 VLAN 설정 · 점검

판단 VLAN ID 불일치 = 같은 스위치인데 통신 불가

```
ip link add link eth0 name eth0.100 type vlan id 100
ip addr add 192.168.100.10/24 dev eth0.100; ip link set eth0.100 up
ip -d link show eth0.100             # VLAN 상세
cat /proc/net/vlan/config
tcpdump -i eth0 -n -e vlan 100
ip link del eth0.100
```

단어	뜻	단어	뜻
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	set	속성 변경 (예: up/down, mtu)
link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	up	인터페이스를 켬(또는 UP 상태만)
add	추가	-d	상세(details)
eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)	show	조회(표시)
name	이름	cat	파일 내용을 그대로 출력
eth0.100	eth0 위의 VLAN 100 하위 인터페이스	/proc/net/vlan/config	커널이 보여 주는 가상 파일(/proc)
type	인터페이스 종류 지정	tcpdump	패킷 캡처·표시
vlan	VLAN(가상 LAN) 종류	-i	캡처할 인터페이스
id	VLAN/장치 식별 번호	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
100	숫자 값 100	-n	이름 변환 안 함
addr	IP 주소 대상	-e	링크 계층(MAC) 헤더 표시
192.168.100.10/24	IPv4 주소 (/뒤는 prefix 길이)	vlan	VLAN(가상 LAN) 종류
dev	뒤에 오는 값이 장치(인터페이스) 이름임을 표시	100	숫자 값 100
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	del	삭제

2-7 본딩(bonding)/팀 상태 점검

판단 slave 하나가 down 이면 이중화 깨짐, mode 불일치(LACP) 주의

```
cat /proc/net/bonding/bond0          # Mode, MII Status, Slave 상태, LACP Partner
ip -d link show bond0
nmcli con show; nmcli dev status
ip link set eth1 down; ip link set eth1 up      # slave failover 시험
dmesg | grep -i bond
```

단어	뜻	단어	뜻
cat	파일 내용을 그대로 출력	dev	뒤에 오는 값이 장치(인터페이스) 이름임을 표시
/proc/net/bonding/bond0	커널이 보여 주는 가상 파일(/proc)	status	서비스 상태 조회
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	set	속성 변경 (예: up/down, mtu)
-d	상세(details)	eth1	두 번째 이더넷 인터페이스
link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	down	인터페이스를 끔
show	조회(표시)	up	인터페이스를 켬(또는 UP 상태만)
bond0	본딩(NIC 묶음) 인터페이스	dmesg	커널 메시지 버퍼 출력
nmcli	NetworkManager 제어	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
con	NetworkManager 연결 프로파일	grep	패턴에 맞는 줄만 출력
show	조회(표시)	-i	대소문자 무시
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	bond	본딩 상태 파일

2-8 브리지 · 스위칭 점검 (가상환경/KVM/Docker)

판단 MAC 학습 테이블 / 포트 상태

```
bridge link show
bridge fdb show
bridge vlan show
brctl show; brctl showmacs br0
bridge -s link show
```

단어	뜻	단어	뜻
bridge	브리지·FDB(MAC 학습표) 조회	show	조회(표시)
link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
show	조회(표시)	showmacs	학습한 MAC 표시(brctl)
fdb	포워딩 DB(MAC 학습표)	br0	브리지 인터페이스
vlan	VLAN(가상 LAN) 종류	-s	통계 포함 표시
brctl	리눅스 브리지 관리(구형 도구)		

2-9 MTU / 점보프레임 불일치 (L2~L3 걸침)

판단 작은 패킷 OK, 큰 패킷 실패 = MTU 불일치

```
S ip link set $P_S_IF mtu 1000
C ping -c 2 -s 100 $P_S_IP
C ping -c 2 -s 1400 -M do $P_S_IP   # 실패(DF 설정)
C tracepath $P_S_IP                 # pmtu
ip link show | grep mtu
S ip link set $P_S_IF mtu 1500        # 복구
ping -M do -s 8972 <IP>              # 점보프레임(9000) 경로 검증
```

단어	뜻	단어	뜻
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	1400	숫자 값 1400
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	-M	MTU 탐색 방식(do이면 단편화 금지)
link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	do	반복문 본문 시작
set	속성 변경 (예: up/down, mtu)	tracepath	MTU까지 알려 주는 경로 추적
$P_S_IF	server 쪽 인터페이스 이름	$P_S_IP	server 쪽 IP
mtu	최대 전송 단위 지정	show	조회(표시)

`1000`	숫자 값 1000	`|`	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
`C`	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	`grep`	패턴에 맞는 줄만 출력
`ping`	ICMP 에코 요청으로 도달성·지연 확인	`mtu`	최대 전송 단위 지정
`-c`	보낼 횟수	`1500`	숫자 값 1500
`2`	숫자 값 2	`8972`	숫자 값 8972
`-s`	데이터 크기(바이트)	`<`	파일을 입력으로 사용
`100`	숫자 값 100	`IP`	자리표시자: 실제 IP 주소로 바꿔 넣음
`$P_S_IP`	server 쪽 IP	`>`	출력을 파일로 보냄(덮어씀)

3계층: 네트워크 계층 (Network)

IP 주소, 서브넷, 게이트웨이, 라우팅, ICMP, NAT, 방화벽(L3), IPv6.

3-1 IP/서브넷 설정 오류

판단 Network is unreachable → 경로 없음. 서브넷 마스크 확인

```
C ip addr flush dev $P_C_IF
C ip addr add 10.0.1.1/24 dev $P_C_IF
C ping -c 2 -W 1 $P_S_IP            # Network is unreachable
C ip addr; C ip route
C ip addr flush dev $P_C_IF; C ip addr add $P_C_IP/$P_PREFIX dev $P_C_IF   # 복구
```

단어	뜻	단어	뜻
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	-c	보낼 횟수
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	2	숫자 값 2
addr	IP 주소 대상	-W	응답 대기 시간(초)
flush	항목 전체 비움	1	숫자 1
dev	뒤에 오는 값이 장치(인터페이스) 이름임을 표시	$P_S_IP	server 쪽 IP
$P_C_IF	client 쪽 인터페이스 이름	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
add	추가	route	라우팅 표 대상
10.0.1.1/24	IPv4 주소 (/뒤는 prefix 길이)	$P_C_IP/$P_PREFIX	조합 인자 → $P_C_IP(client 쪽 IP) · $P_PREFIX(서브넷 prefix 길이)
ping	ICMP 에코 요청으로 도달성·지연 확인		

3-2 게이트웨이/라우팅 누락

판단 ip route get 으로 실제 선택 경로를 확인

```
S ip addr add 192.168.100.1/24 dev lo
C ping -c 2 -W 1 192.168.100.1       # unreachable
C ip route get 192.168.100.1
C ip route add 192.168.100.0/24 via $P_S_IP
C ip route add default via $P_S_IP  # 기본 게이트웨이
C ip route del 192.168.100.0/24
ip route show table all
ip rule show                         # 정책 라우팅
ip route show table main
```

단어	뜻	단어	뜻
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	route	라우팅 표 대상
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	get	하나를 조회·계산해서 보여 줌
addr	IP 주소 대상	192.168.100.1	IPv4 주소
add	추가	192.168.100.0/24	IPv4 주소 (/뒤는 prefix 길이)
192.168.100.1/24	IPv4 주소 (/뒤는 prefix 길이)	via	이 게이트웨이를 거쳐 감
dev	뒤에 오는 값이 장치(인터페이스) 이름임을 표시	$P_S_IP	server 쪽 IP
lo	루프백 인터페이스(자기 자신)	default	기본 경로(0.0.0.0/0)
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	del	삭제
ping	ICMP 에코 요청으로 도달성·지연 확인	show	조회(표시)
-c	보낼 횟수	table	라우팅 표 번호·이름 지정
2	숫자 값 2	all	전체
-W	응답 대기 시간(초)	rule	정책 라우팅 규칙
1	숫자 1	main	주 라우팅 표
192.168.100.1	IPv4 주소		

3-3 ICMP 만 차단 (ping 불가, 서비스 정상)

판단 ping 실패 ≠ 네트워크 장애

```
S iptables -A INPUT -p icmp -j DROP
C ping -c 2 -W 1 $P_S_IP
C curl -s -o /dev/null -w "%{http_code}\n" http://$P_S_IP:$P_PORT    # 200
S iptables -D INPUT -p icmp -j DROP
```

단어	뜻	단어	뜻
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	-W	응답 대기 시간(초)
iptables	netfilter 패킷 필터 규칙 조회·추가·삭제	1	숫자 1
-A	규칙을 체인 끝에 추가	$P_S_IP	server 쪽 IP
INPUT	이 호스트로 들어오는 패킷 체인	curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트
-p	프로토콜(tcp, udp, icmp)	-s	진행 표시 숨김(silent)
icmp	ICMP 프로토콜	-o	본문을 파일로 저장(/dev/null이면 버림)
-j	일치 시 동작(DROP, ACCEPT, REJECT, LOG)	/dev/null	출력을 버리는 특수 파일
DROP	응답 없이 조용히 버림	-w	끝나고 지정한 형식으로 측정값 출력(예: 단계별 시간)
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	"%{http_code}\n"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: %{http_code}\n
ping	ICMP 에코 요청으로 도달성·지연 확인	http://$P_S_IP:$P_PORT	http 주소: 방식://호스트[:포트]/경로 → $P_S_IP(server 쪽 IP) · $P_PORT(실습 HTTP 포트(고른 값, 기본 8080))
-c	보낼 횟수	-D	규칙 삭제
2	숫자 값 2		

3-4 경로 추적 · 병목 구간 찾기

판단 * * * 는 ICMP 차단일 수 있음. mtr 로 구간별 손실 확인

```
traceroute -n 8.8.8.8
traceroute -T -p 443 -n <IP>          # TCP 로 추적(ICMP 차단 우회)
traceroute -I -n <IP>                 # ICMP 모드
tracepath -n <IP>
mtr -rwzbc 100 8.8.8.8                # 리포트 모드, 손실·지연 통계
ping -c 100 -i 0.2 <IP> | tail -3
```

단어	뜻	단어	뜻
traceroute	패킷이 거치는 라우터 추적	mtr	traceroute와 ping을 합친 경로 품질 측정
-n	숫자 주소로 표시	-rwzbc	보고서 모드(r) + 전체 호스트명(w) + AS 번호(z) + 이름·IP 둘 다(b) + 횟수(c)
8.8.8.8	IPv4 주소	100	숫자 값 100
-T	TCP SYN 사용	8.8.8.8	IPv4 주소
-p	목적지 포트	ping	ICMP 에코 요청으로 도달성·지연 확인
443	숫자 값 443	-c	보낼 횟수
<	파일을 입력으로 사용	100	숫자 값 100
IP	자리표시자: 실제 IP 주소로 바꿔 넣음	-i	보내는 간격(초)
>	출력을 파일로 보냄(덮어씀)	0.2	숫자·단위 값 0.2
-I	ICMP 사용	<	파일을 입력으로 사용
tracepath	MTU까지 알려 주는 경로 추적	IP	자리표시자: 실제 IP 주소로 바꿔 넣음
-n	숫자 주소로 표시	>	출력을 파일로 보냄(덮어씀)
<	파일을 입력으로 사용	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
IP	자리표시자: 실제 IP 주소로 바꿔 넣음	tail	뒤쪽 N줄 출력
>	출력을 파일로 보냄(덮어씀)	-3	마지막 3줄

3-5 비대칭 라우팅 · rp_filter 로 인한 드롭

판단 여러 NIC 에서 응답이 다른 NIC 로 나가면 rp_filter 가 차단

```
sysctl net.ipv4.conf.all.rp_filter net.ipv4.conf.eth0.rp_filter
sysctl -w net.ipv4.conf.all.rp_filter=2     # loose 모드
nstat -az | grep -i IPReversePathFilter
sysctl -w net.ipv4.conf.all.log_martians=1; dmesg | grep -i martian
ip route get <외부IP> from <내IP>
```

단어	뜻	단어	뜻
sysctl	커널 파라미터 조회·설정	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
net.ipv4.conf.all.rp_filter	커널 파라미터 net.ipv4.conf.all.rp_filter: 역방향 경로 검사(0 끔, 1 엄격, 2 느슨)	dmesg	커널 메시지 버퍼 출력
net.ipv4.conf.eth0.rp_filter	커널 파라미터 net.ipv4.conf.eth0.rp_filter: 역방향 경로 검사(0 끔, 1 엄격, 2 느슨)	martian	불가능한 출발지 주소 패킷
-w	값을 지금 바꿈	ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구
net.ipv4.conf.all.rp_filter=2	커널 파라미터 net.ipv4.conf.all.rp_filter: 역방향 경로 검사(0 끔, 1 엄격, 2 느슨) → 값 2로 설정	route	라우팅 표 대상
nstat	커널 네트워크 카운터 증가량 표시	get	하나를 조회·계산해서 보여 줌
-az	0인 카운터도 포함한 절대값(-a -z)	<	파일을 입력으로 사용
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	외부IP	자리표시자: 외부 IP
grep	패턴에 맞는 줄만 출력	>	출력을 파일로 보냄(덮어씀)
-i	대소문자 무시	from	이 출발지 주소 기준
IPReversePathFilter	역방향 경로 검사 카운터 이름	내IP	자리표시자: 내 IP
net.ipv4.conf.all.log_martians=1	커널 파라미터 net.ipv4.conf.all.log_martians: 비정상 출발지 주소 패킷을 로그에 기록 → 값 1로 설정		

3-6 IP 포워딩 · NAT (라우터/게이트웨이 역할)

판단 포워딩 꺼짐 또는 MASQUERADE 누락

```
sysctl net.ipv4.ip_forward
sysctl -w net.ipv4.ip_forward=1
iptables -t nat -L -n -v
iptables -t nat -A POSTROUTING -s $P_NET -o eth0 -j MASQUERADE
iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT
conntrack -L | head; conntrack -S
```

단어	뜻	단어	뜻
sysctl	커널 파라미터 조회·설정	-o	나가는 인터페이스
net.ipv4.ip_forward	커널 파라미터 net.ipv4.ip_forward: 패킷 전달(라우터 동작) 허용	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
-w	값을 지금 바꿈	-j	일치 시 동작(DROP, ACCEPT, REJECT, LOG)
net.ipv4.ip_forward=1	커널 파라미터 net.ipv4.ip_forward: 패킷 전달(라우터 동작) 허용 → 값 1로 설정	MASQUERADE	나가는 인터페이스 주소로 출발지를 바꿈(NAT)
iptables	netfilter 패킷 필터 규칙 조회·추가·삭제	FORWARD	이 호스트를 통과해 전달되는 패킷 체인
-t	테이블 지정(filter, nat, mangle)	-i	들어오는 인터페이스
nat	NAT 테이블	eth1	두 번째 이더넷 인터페이스
-L	규칙 나열	ACCEPT	허용
-n	이름 변환 없이 숫자로 표시	conntrack	커널 연결 추적표(conntrack) 조회
-v	패킷·바이트 카운터 표시	-L	연결 추적 항목 목록
-A	규칙을 체인 끝에 추가	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
POSTROUTING	라우팅 결정 뒤(나가기 직전) 체인	head	앞쪽 N줄만 출력
-s	출발지 주소	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
$P_NET	랩 네트워크 대역	-S	CPU별 통계

3-7 IPv6 점검

판단 IPv6 우선 선택으로 지연이 생기는 경우도 많음

```
ip -6 addr; ip -6 route; ip -6 neigh
ping -6 -c 3 ::1
ping -6 -c 3 2001:4860:4860::8888
traceroute6 -n 2001:4860:4860::8888
curl -4 -sI https://example.com | head -1; curl -6 -sI https://example.com | head -1
sysctl net.ipv6.conf.all.disable_ipv6
rdisc6 eth0                          # Router Advertisement 확인
```

단어	뜻	단어	뜻
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	2001:4860:4860::8888	IPv6 주소
-6	IPv6 대상	curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트
addr	IP 주소 대상	-4	IPv4만 사용
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-sI	조용히 + 헤더만(-s -I)
route	라우팅 표 대상	https://example.com	https 주소: 방식://호스트[:포트]/경로 → example.com
neigh	이웃(ARP/NDP) 표 대상	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
ping	ICMP 에코 요청으로 도달성·지연 확인	head	앞쪽 N줄만 출력
-6	IPv6 사용	-1	숫자 값 -1
-c	보낼 횟수	-6	IPv6만 사용
3	숫자 값 3	sysctl	커널 파라미터 조회·설정
::1	IPv6 주소	net.ipv6.conf.all.disable_ipv6	커널 파라미터 net.ipv6.conf.all.disable_ipv6: IPv6 끔
2001:4860:4860::8888	IPv6 주소	rdisc6	IPv6 라우터 광고 요청
traceroute6	IPv6 경로 추적	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
-n	숫자 주소로 표시		

3-8 DHCP · 주소 할당 문제

판단 169.254.x.x 또는 주소 없음 = DHCP 응답 없음

```
dhclient -v eth0                     # 요청 과정 출력
dhclient -r eth0                     # 반납
tcpdump -i eth0 -n port 67 or port 68
journalctl -u NetworkManager | grep -i dhcp
nmcli device show eth0 | grep -i -E "ip4|gateway|dns"
cat /var/lib/dhcp/dhclient.leases
networkctl status eth0
```

단어	뜻	단어	뜻
dhclient	DHCP로 IP 주소를 받아오는 클라이언트	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
-v	상세 출력	grep	패턴에 맞는 줄만 출력
eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)	-i	대소문자 무시
-r	임대한 주소 반납(release)	dhcp	주소 자동 할당 프로토콜
tcpdump	패킷 캡처·표시	nmcli	NetworkManager 제어
-i	캡처할 인터페이스	device	장치
eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)	show	조회(표시)
-n	이름 변환 안 함	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
port	포트 번호 조건	-E	확장 정규식
67	숫자 값 67	"ip4|gateway|dns"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: ip4|gateway|dns
or	조건 중 하나	cat	파일 내용을 그대로 출력
68	숫자 값 68	/var/lib/dhcp/dhclient.leases	파일·디렉터리 경로
journalctl	systemd 저널 로그 조회	networkctl	systemd-networkd 링크 상태 조회
-u	특정 서비스(unit)만	status	서비스 상태 조회
NetworkManager	네트워크 관리 서비스	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)

3-9 ICMP 에러 메시지 해석

판단 원인 분류

```
# Destination Net Unreachable: 경로 없음 / Host Unreachable: ARP 실패
# Frag needed (type3 code4): MTU / TTL exceeded: 루프 또는 홉 초과
tcpdump -i eth0 -n icmp
tcpdump -i eth0 -n 'icmp[icmptype] != icmp-echo and icmp[icmptype] != icmp-echoreply'
ping -t 1 8.8.8.8                    # TTL=1 로 첫 홉 확인(ttl 지정)
```

단어	뜻	단어	뜻
tcpdump	패킷 캡처·표시	'icmp[icmptype] != icmp-echo and icmp[icmptype] != icmp-echoreply'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: icmp[icmptype] != icmp-echo and icmp[icm
-i	캡처할 인터페이스	ping	ICMP 에코 요청으로 도달성·지연 확인
eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)	-t	TTL 값 지정
-n	이름 변환 안 함	1	숫자 1
icmp	ICMP 프로토콜	8.8.8.8	IPv4 주소

4계층: 전송 계층 (Transport)

TCP/UDP 포트, 3-way handshake, 방화벽(L4), 소켓 상태, 포트/연결 고갈, 재전송.

4-1 포트 미리슨 (Connection refused)

판단 즉시 refused = 리스너 없음 또는 REJECT

```
C nc -zv -w 3 $P_S_IP 9999         # refused
S ss -tulnp | grep 9999
S python3 -m http.server 9999 --bind $P_S_IP &
C nc -zv -w 3 $P_S_IP 9999         # succeeded
ss -tlnp; ss -ulnp; ss -lntp sport = :80
lsof -i :80; fuser -v 80/tcp
```

단어	뜻	단어	뜻
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	--bind	바인드할 주소
nc	TCP/UDP 연결을 직접 열어 보는 도구(netcat)	$P_S_IP	server 쪽 IP
-zv	데이터 없이 포트만 확인(z) + 상세(v)	&	명령을 백그라운드로 실행
-w	연결 제한 시간(초)	-tlnp	TCP LISTEN + 숫자 + 프로세스
3	숫자 값 3	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
$P_S_IP	server 쪽 IP	-ulnp	UDP LISTEN + 숫자 + 프로세스
9999	숫자 값 9999	-lntp	LISTEN(l) + 숫자(n) + TCP(t) + 프로세스(p)
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	sport	출발지 포트 조건
ss	소켓 상태 조회(netstat의 후속)	=	같다(문자열 비교)
-tulnp	TCP·UDP LISTEN + 숫자 + 프로세스	:80	포트 80 (호스트 생략)
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	lsof	열린 파일·소켓 목록
grep	패턴에 맞는 줄만 출력	-i	네트워크 연결만
9999	숫자 값 9999	:80	포트 80 (호스트 생략)
python3	Python 3 인터프리터	fuser	파일·포트를 쓰는 프로세스 찾기
-m	모듈을 프로그램으로 실행(예: http.server)	-v	상세 표시
http.server	파이썬 내장 간이 웹 서버 모듈	80/tcp	80번 포트(tcp)
9999	숫자 값 9999		

4-2 방화벽 DROP vs REJECT

판단 타임아웃 = DROP/경로 문제, 즉시 거부 = REJECT/리스너 없음

```
S iptables -A INPUT -p tcp --dport $P_PORT -j DROP
time C nc -zv -w 5 $P_S_IP $P_PORT    # 5초 후 timed out
S iptables -D INPUT -p tcp --dport $P_PORT -j DROP
S iptables -A INPUT -p tcp --dport $P_PORT -j REJECT --reject-with tcp-reset
time C nc -zv -w 5 $P_S_IP $P_PORT    # 즉시 refused
S iptables -D INPUT -p tcp --dport $P_PORT -j REJECT --reject-with tcp-reset
```

단어	뜻	단어	뜻
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)
iptables	netfilter 패킷 필터 규칙 조회·추가·삭제	nc	TCP/UDP 연결을 직접 열어 보는 도구(netcat)
-A	규칙을 체인 끝에 추가	-zv	데이터 없이 포트만 확인(z) + 상세(v)
INPUT	이 호스트로 들어오는 패킷 체인	-w	연결 제한 시간(초)
-p	프로토콜(tcp, udp, icmp)	5	숫자 값 5
tcp	TCP 프로토콜	$P_S_IP	server 쪽 IP
--dport	목적지 포트	$P_PORT	실습 HTTP 포트(고른 값, 기본 8080)
$P_PORT	실습 HTTP 포트(고른 값, 기본 8080)	-D	규칙 삭제
-j	일치 시 동작(DROP, ACCEPT, REJECT, LOG)	REJECT	거부하고 거부 사실을 통지
DROP	응답 없이 조용히 버림	--reject-with	거부 시 돌려줄 ICMP 종류
time	명령 실행 시간 측정	tcp-reset	TCP RST로 거부

4-3 방화벽 정책 확인 (iptables / nftables / firewalld / ufw)

판단 규칙 순서·기본 정책(policy) 확인

```
iptables -L -v -n --line-numbers; iptables -S
iptables -t nat -L -n -v
nft list ruleset
firewall-cmd --state; firewall-cmd --list-all
firewall-cmd --permanent --add-port=$P_PORT/tcp && firewall-cmd --reload
ufw status verbose
iptables -I INPUT 1 -p tcp --dport $P_PORT -j ACCEPT   # 임시 허용
iptables -A INPUT -j LOG --log-prefix "DROP: "      # 드롭 로깅
journalctl -k | grep "DROP: "
```

단어	뜻	단어	뜻
iptables	netfilter 패킷 필터 규칙 조회·추가·삭제	status	서비스 상태 조회
-L	규칙 나열	verbose	자세히
-v	패킷·바이트 카운터 표시	-I	규칙을 체인 앞쪽에 삽입
-n	이름 변환 없이 숫자로 표시	INPUT	이 호스트로 들어오는 패킷 체인
--line-numbers	규칙 번호 표시	1	숫자 1
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-p	프로토콜(tcp, udp, icmp)
-S	규칙을 명령 형태로 출력	tcp	TCP 프로토콜
-t	테이블 지정(filter, nat, mangle)	--dport	목적지 포트
nat	NAT 테이블	$P_PORT	실습 HTTP 포트(고른 값, 기본 8080)
nft	nftables 규칙 조회·설정	-j	일치 시 동작(DROP, ACCEPT, REJECT, LOG)
list	목록 조회	ACCEPT	허용
ruleset	규칙 전체	-A	규칙을 체인 끝에 추가
firewall-cmd	firewalld 방화벽 설정·조회	LOG	커널 로그에 기록
--state	firewalld 실행 여부	--log-prefix	로그 앞에 붙일 문자열
--list-all	현재 zone 설정 전체 표시	"DROP: "	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: DROP:
--permanent	재부팅 후에도 유지되게 저장	journalctl	systemd 저널 로그 조회
--add-port=$P_PORT/tcp	포트 허용 추가 → 값 $P_PORT/tcp	-k	커널 메시지만
&&	앞 명령이 성공하면 뒤 명령 실행	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
--reload	저장된 설정 다시 적용	grep	패턴에 맞는 줄만 출력
ufw	Ubuntu 방화벽 도구	"DROP: "	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: DROP:

4-4 3-Way Handshake 관찰

판단 SYN 만 보이고 SYN-ACK 없으면 서버/방화벽 문제, SYN-ACK 는 있는데 ACK 없으면 반대 방향 문제

```
S tcpdump -i $P_S_IF -n "tcp port $P_PORT" -c 10
C curl -s -o /dev/null http://$P_S_IP:$P_PORT
tcpdump -i eth0 -nn 'tcp[tcpflags] & tcp-syn != 0'
tcpdump -i eth0 -nn 'tcp[tcpflags] & tcp-rst != 0'      # RST 추적
tcpdump -i eth0 -nn 'tcp[tcpflags] == tcp-syn'          # SYN 만
tcpdump -i eth0 -nn 'tcp[tcpflags] & (tcp-syn|tcp-ack) == (tcp-syn|tcp-ack)'  # SYN-ACK
```

단어	뜻	단어	뜻
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	-s	진행 표시 숨김(silent)
tcpdump	패킷 캡처·표시	-o	본문을 파일로 저장(/dev/null이면 버림)
-i	캡처할 인터페이스	/dev/null	출력을 버리는 특수 파일
$P_S_IF	server 쪽 인터페이스 이름	http://$P_S_IP:$P_PORT	http 주소: 방식://호스트[:포트]/경로 → $P_S_IP(server 쪽 IP) · $P_PORT(실습 HTTP 포트(고른 값, 기본 8080))
-n	이름 변환 안 함	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
"tcp port $P_PORT"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: tcp port $P_PORT	-nn	호스트·포트 이름 변환 안 함
-c	N개 잡고 종료	'tcp[tcpflags] & tcp-syn != 0'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: tcp[tcpflags] & tcp-syn != 0
10	숫자 값 10	'tcp[tcpflags] & tcp-rst != 0'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: tcp[tcpflags] & tcp-rst != 0
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	'tcp[tcpflags] == tcp-syn'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: tcp[tcpflags] == tcp-syn

curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트	'tcp[tcpflags] & (tcp-syn|tcp-ack) == (tcp-syn|tcp-ack)'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: tcp[tcpflags] & (tcp-syn|tcp-ack) == (tc

4-5 소켓 상태 분석

판단 TIME_WAIT 다수=정상 범위 점검, CLOSE_WAIT 다수=앱이 close 안 함, SYN_RECV 다수=SYN flood 의심

```
ss -s
ss -tan | awk 'NR>1{print $1}' | sort | uniq -c | sort -rn
ss -tan state time-wait | wc -l
ss -tan state close-wait | wc -l
ss -tan state syn-recv | wc -l
ss -tanp state close-wait             # 어느 프로세스인가
ss -tan | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn | head
ss -tnoi dst $P_S_IP                # rtt, cwnd, 재전송, 타이머
netstat -ant | awk '{print $6}' | sort | uniq -c
```

단어	뜻	단어	뜻
ss	소켓 상태 조회(netstat의 후속)	close-wait	CLOSE_WAIT 상태(상대가 닫았는데 내 쪽이 안 닫음)
-s	소켓 통계 요약	syn-recv	SYN을 받고 3-way 완료 대기 중
-tan	TCP 전체 상태 + 숫자	-tanp	TCP 전체 + 숫자 + 프로세스
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	'{print $5}'	awk 프로그램: 5번째 열 출력
awk	열(필드) 단위로 줄을 가공하는 텍스트 처리기	cut	줄에서 지정한 열만 잘라냄
'NR>1{print $1}'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: NR>1{print $1}	-d:	구분 문자 지정 → 값 :
sort	줄 정렬	-f1	뽑을 열 번호 → 값 1
uniq	이웃한 같은 줄을 합침	head	앞쪽 N줄만 출력
-c	개수 함께 표시	-tnoi	TCP + 숫자 + 타이머 + 내부 정보
-rn	숫자 내림차순	dst	목적지 주소 조건
state	상태로 필터	$P_S_IP	server 쪽 IP
time-wait	TIME_WAIT 상태(닫은 뒤 잠시 보관)	netstat	소켓·라우팅·통계 조회(구형 도구)
wc	줄·단어·바이트 수 세기	-ant	모든 TCP 소켓, 숫자로(-a -n -t)
-l	줄 수	'{print $6}'	awk 프로그램: 6번째 열 출력

4-6 재전송 · 지연 · 처리량

판단 retrans 증가 = 손실/혼잡. 대역폭과 지연을 직접 측정

```
nstat -az | egrep "TcpRetransSegs|TcpExtTCPLostRetransmit|TcpExtTCPTimeouts|TcpExtTCPSynRetrans"
netstat -s | egrep -i "retrans|timeout|failed|overflow|drop"
ss -ti | grep -E "retrans|rtt"
S iperf3 -s &
C iperf3 -c $P_S_IP -t 10          # TCP 처리량
C iperf3 -c $P_S_IP -u -b 100M     # UDP 손실/지터
C iperf3 -c $P_S_IP -P 4 -R        # 병렬·역방향
```

단어	뜻	단어	뜻
nstat	커널 네트워크 카운터 증가량 표시	iperf3	두 지점 사이 처리량 측정
-az	0인 카운터도 포함한 절대값(-a -z)	-s	서버 모드
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	&	명령을 백그라운드로 실행
egrep	확장 정규식으로 줄 검색(grep -E 와 같음)	C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)
"TcpRetransSegs|TcpExtTCPLostRetransmit|TcpExtTCPTimeouts|TcpExtTCPSynRetrans"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: TcpRetransSegs|TcpExtTCPLostRetransmit|T	-c	클라이언트로 서버에 접속
netstat	소켓·라우팅·통계 조회(구형 도구)	$P_S_IP	server 쪽 IP
-s	프로토콜별 통계	-t	측정 시간(초)
-i	대소문자 무시	10	숫자 값 10

"retrans|timeout|failed|overflow|drop"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: retrans|timeout|failed|overflow|drop	-u	UDP 사용
ss	소켓 상태 조회(netstat의 후속)	-b	목표 대역폭
-ti	TCP + 내부 정보(rtt, cwnd …)	100M	숫자·단위 값 100M
grep	패턴에 맞는 줄만 출력	-P	병렬 스트림 수
-E	확장 정규식	4	숫자 값 4
"retrans|rtt"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: retrans|rtt	-R	역방향(서버→클라이언트) 측정
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)		

4-7 포트·소켓 고갈 / conntrack 가득 참

판단 Cannot assign requested address, nf_conntrack: table full

```
sysctl net.ipv4.ip_local_port_range
ss -tan | wc -l
sysctl net.ipv4.tcp_tw_reuse net.ipv4.tcp_fin_timeout
cat /proc/sys/net/netfilter/nf_conntrack_{count,max}
dmesg | grep -i "nf_conntrack: table full"
sysctl -w net.netfilter.nf_conntrack_max=262144
ulimit -n; cat /proc/sys/fs/file-nr; cat /proc/sys/fs/file-max
ls /proc/<PID>/fd | wc -l            # 프로세스 FD 사용량
```

단어	뜻	단어	뜻
sysctl	커널 파라미터 조회·설정	-i	대소문자 무시
net.ipv4.ip_local_port_range	커널 파라미터 net.ipv4.ip_local_port_range: 임시(출발지) 포트 범위	"nf_conntrack: table full"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: nf_conntrack: table full
ss	소켓 상태 조회(netstat의 후속)	-w	값을 지금 바꿈
-tan	TCP 전체 상태 + 숫자	net.netfilter.nf_conntrack_max=262144	커널 파라미터 net.netfilter.nf_conntrack_max: 연결 추적 표 최대 항목 수 → 값 262144로 설정
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	ulimit	프로세스 자원 한도 조회·설정
wc	줄·단어·바이트 수 세기	-n	열 수 있는 파일 수
-l	줄 수	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
net.ipv4.tcp_tw_reuse	커널 파라미터 net.ipv4.tcp_tw_reuse: TIME_WAIT 소켓 재사용	/proc/sys/fs/file-nr	커널이 보여 주는 가상 파일(/proc)
net.ipv4.tcp_fin_timeout	커널 파라미터 net.ipv4.tcp_fin_timeout: FIN_WAIT 대기 시간(초)	/proc/sys/fs/file-max	커널이 보여 주는 가상 파일(/proc)
cat	파일 내용을 그대로 출력	ls	디렉터리 내용 나열
/proc/sys/net/netfilter/nf_conntrack_	커널이 보여 주는 가상 파일(/proc)	/proc/	커널이 보여 주는 가상 파일(/proc)
{	명령 묶음 시작(현재 셸)	<	파일을 입력으로 사용
count,max	conntrack 현재 개수와 최대값	PID	자리표시자: 프로세스 번호
}	명령 묶음 끝	>	출력을 파일로 보냄(덮어씀)
dmesg	커널 메시지 버퍼 출력	/fd	파일·디렉터리 경로
grep	패턴에 맞는 줄만 출력		

4-8 백로그 · SYN flood 점검

판단 리슨 큐 넘침 → 접속 지연/실패

```
ss -ltn                              # Recv-Q(현재 대기) / Send-Q(백로그 한도)
nstat -az | egrep "ListenOverflows|ListenDrops|TCPReqQFullDrop|TCPReqQFullDoCookies"
sysctl net.core.somaxconn net.ipv4.tcp_max_syn_backlog net.ipv4.tcp_syncookies
sysctl -w net.ipv4.tcp_syncookies=1
dmesg | grep -i "SYN flooding"
```

단어	뜻	단어	뜻
ss	소켓 상태 조회(netstat의 후속)	net.ipv4.tcp_max_syn_backlog	커널 파라미터 net.ipv4.tcp_max_syn_backlog: SYN 대기열 크기
-ltn	LISTEN + TCP + 숫자	net.ipv4.tcp_syncookies	커널 파라미터 net.ipv4.tcp_syncookies: SYN 폭주 시 쿠키로 방어

nstat	커널 네트워크 카운터 증가량 표시	-w	값을 지금 바꿈
-az	0인 카운터도 포함한 절대값(-a -z)	net.ipv4.tcp_syncookies=1	커널 파라미터 net.ipv4.tcp_syncookies: SYN 폭주 시 쿠키로 방어 → 값 1로 설정
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	dmesg	커널 메시지 버퍼 출력
egrep	확장 정규식으로 줄 검색(grep -E 와 같음)	grep	패턴에 맞는 줄만 출력
"ListenOverflows|ListenDrops|TCPReqQFullDrop|TCPReqQFullDoCookies"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: ListenOverflows|ListenDrops|TCPReqQFullD	-i	대소문자 무시
sysctl	커널 파라미터 조회·설정	"SYN flooding"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: SYN flooding
net.core.somaxconn	커널 파라미터 net.core.somaxconn: listen 대기열 상한		

4-9 UDP 포트 점검

판단 UDP 는 응답이 없어 판단이 어려움 → ICMP port unreachable 로 판단

```
nc -zvu <IP> 53
ss -ulnp
nmap -sU -p 53,123,161 <IP>
tcpdump -i eth0 -n udp port 53
S nc -ul 5000 &
echo hi | C nc -u -w1 $P_S_IP 5000
```

단어	뜻	단어	뜻
nc	TCP/UDP 연결을 직접 열어 보는 도구(netcat)	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
-zvu	UDP 포트 확인(-z -v -u)	-n	이름 변환 안 함
<	파일을 입력으로 사용	udp	UDP 프로토콜
IP	자리표시자: 실제 IP 주소로 바꿔 넣음	port	포트 번호 조건
>	출력을 파일로 보냄(덮어씀)	53	숫자 값 53
53	숫자 값 53	S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)
ss	소켓 상태 조회(netstat의 후속)	-ul	UDP로 수신 대기
-ulnp	UDP LISTEN + 숫자 + 프로세스	5000	숫자 값 5000
nmap	포트·서비스 스캐너	&	명령을 백그라운드로 실행
-sU	UDP 스캔	echo	인자를 그대로 출력
-p	검사할 포트	hi	자리표시자(예: 메시지 내용)
53,123,161	포트 목록·범위 53,123,161	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
<	파일을 입력으로 사용	C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)
IP	자리표시자: 실제 IP 주소로 바꿔 넣음	-u	UDP 사용
>	출력을 파일로 보냄(덮어씀)	-w1	연결 제한 시간 1초
tcpdump	패킷 캡처·표시	$P_S_IP	server 쪽 IP
-i	캡처할 인터페이스		

4-10 포트 스캔 · 서비스 노출 확인

판단 내 서버가 외부에 무엇을 열고 있는지 (허가된 대상에만 사용)

```
nmap -sT -p- <IP>                  # 전체 TCP
nmap -sV -p 22,80,443 <IP>         # 서비스/버전
nmap -Pn -p 443 <IP>               # ping 차단 환경
ss -tulnp                          # 로컬에서 직접 확인
nc -zv <IP> 20-25
```

단어	뜻	단어	뜻
nmap	포트·서비스 스캐너	443	숫자 값 443
-sT	TCP 연결 스캔	ss	소켓 상태 조회(netstat의 후속)
-p-	전체 포트(1–65535)	-tulnp	TCP·UDP LISTEN + 숫자 + 프로세스
<	파일을 입력으로 사용	nc	TCP/UDP 연결을 직접 열어 보는 도구(netcat)
IP	자리표시자: 실제 IP 주소로 바꿔 넣음	-zv	데이터 없이 포트만 확인(z) + 상세(v)
>	출력을 파일로 보냄(덮어씀)	<	파일을 입력으로 사용
-sV	서비스 버전 탐지	IP	자리표시자: 실제 IP 주소로 바꿔 넣음

`-p`	검사할 포트	`>`	출력을 파일로 보냄(덮어씀)
`22,80,443`	포트 목록·범위 22,80,443	`20-25`	포트 목록·범위 20-25
`-Pn`	호스트 생존 확인(ping) 생략		

5계층: 세션 계층 (Session)

세션 수립·유지·종료, 타임아웃, keepalive, NFS/RPC, 로그인 세션.

5-1 수립된 세션과 타이머

판단 timer:(keepalive,...) / (on,...) 로 재전송·유휴 확인

```
S python3 -m http.server $P_PORT --bind $P_S_IP &
C nc $P_S_IP $P_PORT &
S ss -o state established "( sport = :$P_PORT )"
S ss -tnoi state established
ss -tn state established | wc -l
```

단어	뜻	단어	뜻
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	$P_PORT	실습 HTTP 포트(고른 값, 기본 8080)
python3	Python 3 인터프리터	ss	소켓 상태 조회(netstat의 후속)
-m	모듈을 프로그램으로 실행(예: http.server)	-o	타이머 정보 표시
http.server	파이썬 내장 간이 웹 서버 모듈	state	상태로 필터
$P_PORT	실습 HTTP 포트(고른 값, 기본 8080)	established	연결이 맺어진 상태
--bind	바인드할 주소	"(sport = :$P_PORT)"	조합 인자 → "(sport = · $P_PORT)"
$P_S_IP	server 쪽 IP	-tnoi	TCP + 숫자 + 타이머 + 내부 정보
&	명령을 백그라운드로 실행	-tn	TCP + 숫자
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
nc	TCP/UDP 연결을 직접 열어 보는 도구(netcat)	wc	줄·단어·바이트 수 세기
$P_S_IP	server 쪽 IP	-l	줄 수

5-2 세션 중간 단절 · 재전송 관찰

판단 응답 경로 DROP 시 재전송 타이머가 증가

```
S iptables -A OUTPUT -p tcp --sport $P_PORT -j DROP
echo "GET / HTTP/1.0" | C nc -w 2 $P_S_IP $P_PORT
C ss -tno state established          # timer:(on,...,retrans) 확인
S iptables -D OUTPUT -p tcp --sport $P_PORT -j DROP
cat /proc/sys/net/ipv4/tcp_retries2
```

단어	뜻	단어	뜻
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)
iptables	netfilter 패킷 필터 규칙 조회·추가·삭제	nc	TCP/UDP 연결을 직접 열어 보는 도구(netcat)
-A	규칙을 체인 끝에 추가	-w	연결 제한 시간(초)
OUTPUT	이 호스트가 내보내는 패킷 체인	2	숫자 값 2
-p	프로토콜(tcp, udp, icmp)	$P_S_IP	server 쪽 IP
tcp	TCP 프로토콜	$P_PORT	실습 HTTP 포트(고른 값, 기본 8080)
--sport	출발지 포트	ss	소켓 상태 조회(netstat의 후속)
$P_PORT	실습 HTTP 포트(고른 값, 기본 8080)	-tno	TCP + 숫자 + 타이머
-j	일치 시 동작(DROP, ACCEPT, REJECT, LOG)	state	상태로 필터
DROP	응답 없이 조용히 버림	established	연결이 맺어진 상태
echo	인자를 그대로 출력	-D	규칙 삭제
"GET / HTTP/1.0"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: GET / HTTP/1.0	cat	파일 내용을 그대로 출력
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	/proc/sys/net/ipv4/tcp_retries2	커널이 보여 주는 가상 파일(/proc)

5-3 Keepalive · 유휴 세션 끊김 대응

판단 방화벽/NAT/LB 의 idle timeout 이 keepalive 주기보다 짧으면 세션 끊김

```
sysctl net.ipv4.tcp_keepalive_time net.ipv4.tcp_keepalive_intvl net.ipv4.tcp_keepalive_probes
sysctl -w net.ipv4.tcp_keepalive_time=300
ssh -o ServerAliveInterval=30 -o ServerAliveCountMax=3 user@host
grep -E "ClientAlive|TCPKeepAlive" /etc/ssh/sshd_config
ss -tnoe state established | grep keepalive
```

단어	뜻	단어	뜻
sysctl	커널 파라미터 조회·설정	grep	패턴에 맞는 줄만 출력
net.ipv4.tcp_keepalive_time	커널 파라미터 net.ipv4.tcp_keepalive_time: 연결이 한가한 뒤 keepalive 첫 확인까지 대기(초)	-E	확장 정규식
net.ipv4.tcp_keepalive_intvl	커널 파라미터 net.ipv4.tcp_keepalive_intvl: keepalive 재시도 간격(초)	"ClientAlive|TCPKeepAlive"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: ClientAlive|TCPKeepAlive
net.ipv4.tcp_keepalive_probes	커널 파라미터 net.ipv4.tcp_keepalive_probes: keepalive 실패 허용 횟수	/etc/ssh/sshd_config	설정 파일 경로(/etc)
-w	값을 지금 바꿈	ss	소켓 상태 조회(netstat의 후속)
net.ipv4.tcp_keepalive_time=300	커널 파라미터 net.ipv4.tcp_keepalive_time: 연결이 한가한 뒤 keepalive 첫 확인까지 대기(초) → 값 300로 설정	-tnoe	TCP + 숫자 + 타이머 + 확장 정보
ssh	원격 접속	state	상태로 필터
-o	설정 옵션을 한 줄로 지정	established	연결이 맺어진 상태
ServerAliveInterval=30	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
ServerAliveCountMax=3	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)	keepalive	연결 유지 확인
user@host	자리표시자: 사용자@호스트		

5-4 NFS / RPC 세션

판단 hard/soft 마운트, stale file handle, portmapper 문제

```
rpcinfo -p <NFS서버>
showmount -e <NFS서버>
mount | grep nfs; nfsstat -m; nfsstat -c
mount -t nfs -o vers=4,timeo=600,retrans=2 <서버>:/export /mnt
systemctl status nfs-server rpcbind
umount -f -l /mnt                    # 응답 없는 마운트 강제 해제
dmesg | grep -i -E "nfs|rpc"
exportfs -v
```

단어	뜻	단어	뜻
rpcinfo	RPC 서비스 목록 조회	vers=4,timeo=600,retrans=2	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)
-p	등록된 RPC 서비스 목록	<	파일을 입력으로 사용
<	파일을 입력으로 사용	서버	자리표시자: 서버 이름·주소
NFS서버	자리표시자: NFS 서버 이름	>	출력을 파일로 보냄(덮어씀)
>	출력을 파일로 보냄(덮어씀)	:/export	NFS 서버의 공유 디렉터리 경로 앞부분
showmount	NFS 서버가 내보낸 목록 조회	/mnt	파일·디렉터리 경로
-e	내보낸 공유 목록	systemctl	systemd 서비스 제어·상태 조회
<	파일을 입력으로 사용	status	서비스 상태 조회
NFS서버	자리표시자: NFS 서버 이름	nfs-server	NFS 서버 서비스
>	출력을 파일로 보냄(덮어씀)	rpcbind	RPC 포트 매핑 서비스
mount	파일시스템 마운트·조회	umount	마운트 해제
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	-f	강제 해제
grep	패턴에 맞는 줄만 출력	-l	바로 분리하고 나중에 정리(lazy)
nfs	NFS 파일 공유 프로토콜	/mnt	파일·디렉터리 경로
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	dmesg	커널 메시지 버퍼 출력
nfsstat	NFS 통계	-i	대소문자 무시
-m	마운트 옵션	-E	확장 정규식
-c	클라이언트 통계	"nfs|rpc"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: nfs|rpc
-t	파일시스템 종류	exportfs	NFS 서버의 공유 목록 조회·갱신

nfs	NFS 파일 공유 프로토콜	-v	상세 출력
-o	마운트 옵션		

5-5 로그인 세션 관리

판단 누가 접속 중인지, 세션 정리

```
who -a; w; last -n 20; lastb | head
loginctl list-sessions; loginctl show-session <ID>
loginctl terminate-session <ID>
pkill -KILL -t pts/2
ss -tnp state established '( sport = :22 )'
tmux ls; screen -ls                  # 끊김 대비 세션 유지 도구
```

단어	뜻	단어	뜻
who	로그인한 사용자 목록	>	출력을 파일로 보냄(덮어씀)
-a	전체 정보	terminate-session	세션 종료
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	pkill	이름·조건으로 프로세스 종료
w	로그인 사용자와 작업 표시	-KILL	강제 종료 신호(SIGKILL)
last	로그인 기록	-t	해당 터미널의 프로세스
-n	출력 줄 수	pts/2	가상 터미널 2번
20	숫자 값 20	ss	소켓 상태 조회(netstat의 후속)
lastb	로그인 실패 기록	-tnp	TCP + 숫자 + 프로세스
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	state	상태로 필터
head	앞쪽 N줄만 출력	established	연결이 맺어진 상태
loginctl	systemd 로그인 세션 조회	'(sport = :22)'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: (sport = :22)
list-sessions	로그인 세션 목록	tmux	터미널 멀티플렉서
show-session	세션 상세	ls	목록 나열 (s3 ls 등 하위 명령)
<	파일을 입력으로 사용	screen	터미널 세션 유지 도구
ID	자리표시자: 식별자	-ls	세션 목록

5-6 SMB / LDAP / DB 세션

판단 세션 기반 서비스의 연결 상태

```
smbstatus; smbclient -L //<서버> -N
ldapsearch -x -H ldap://<서버> -b "" -s base
mysql -e "SHOW PROCESSLIST; SHOW STATUS LIKE 'Threads_connected'"
psql -c "select state,count(*) from pg_stat_activity group by 1;"
ss -tn dport = :3306 | wc -l
```

단어	뜻	단어	뜻
smbstatus	Samba 접속 현황	""	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용:
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-s	검색 범위
smbclient	SMB/CIFS 공유 접속	base	기준 위치
-L	서버의 공유 목록	mysql	MySQL 클라이언트
//	파일·디렉터리 경로	-e	SQL을 바로 실행
<	파일을 입력으로 사용	"SHOW PROCESSLIST; SHOW STATUS LIKE 'Threads_connected'"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: SHOW PROCESSLIST; SHOW STATUS LIKE 'Thre
서버	자리표시자: 서버 이름·주소	psql	PostgreSQL 클라이언트
>	출력을 파일로 보냄(덮어씀)	-c	SQL을 바로 실행
-N	비밀번호 묻지 않음	"select state,count(*) from pg_stat_activity group by 1;"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: select state,count(*) from pg_stat_activ
ldapsearch	LDAP 디렉터리 검색	ss	소켓 상태 조회(netstat의 후속)
-x	단순 인증 사용	-tn	TCP + 숫자
-H	LDAP 서버 주소	dport	목적지 포트 조건

`ldap://`	ldap 주소: 방식://호스트[:포트]/경로 →	`=`	같다(문자열 비교)
`<`	파일을 입력으로 사용	`:3306`	포트 3306 (호스트 생략)
서버	자리표시자: 서버 이름·주소	`|`	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
`>`	출력을 파일로 보냄(덮어씀)	`wc`	줄·단어·바이트 수 세기
`-b`	검색 시작 위치(base DN)	`-l`	줄 수

6계층: 표현 계층 (Presentation)

TLS/SSL 인증서, 암호군, 프로토콜 버전, 문자 인코딩, 압축, 직렬화 형식.

6-1 TLS 서버 구성 및 핸드셰이크 확인

판단 인증서 체인·SAN·만료일 확인

```
openssl req -x509 -newkey rsa:2048 -nodes -keyout $P_KEY -out $P_CRT -days 30 -subj "/CN=$P_TLS_CN"
S openssl s_server -accept $P_TLS_PORT -cert $P_CRT -key $P_KEY -www &
C openssl s_client -connect $P_S_IP:$P_TLS_PORT -servername $P_TLS_CN </dev/null 2>&1 | head -30
C openssl s_client -connect $P_S_IP:$P_TLS_PORT </dev/null 2>/dev/null | openssl x509 -noout -subject -is
suer -dates -ext subjectAltName
C curl -vk https://$P_S_IP:$P_TLS_PORT/ 2>&1 | egrep "SSL|TLS|subject|expire|issuer"
```

단어	뜻	단어	뜻
openssl	암호·인증서·TLS 도구	-connect	접속할 host:port
req	인증서 요청 생성 모드	$P_S_IP:$P_TLS_PORT	조합 인자 → $P_S_IP(server 쪽 IP) · $P_TLS_PORT(실습 TLS 포트(고른 값))
-x509	X.509 인증서로 처리	-servername	SNI로 보낼 호스트 이름
-newkey	새 키 생성	$P_TLS_CN	인증서의 공통 이름(도메인)
rsa:2048	RSA 2048비트 키	<	파일을 입력으로 사용
-nodes	개인 키를 암호화하지 않음	/dev/null	출력을 버리는 특수 파일
-keyout	개인 키 저장 파일	2>&1	오류 출력을 표준 출력과 합침
$P_KEY	개인 키 파일 경로	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
-out	출력 파일	head	앞쪽 N줄만 출력
$P_CRT	인증서 파일 경로	-30	숫자 값 -30
-days	인증서 유효 일수	2>	오류 출력(2번)을 파일로 보냄
30	숫자 값 30	x509	X.509 인증서 처리 모드
-subj	인증서 주체(subject)를 명령줄로 지정	-noout	인코딩된 본문은 출력하지 않음
"/CN=$P_TLS_CN"	조합 인자 → " · CN=$P_TLS_CN"	-subject	주체 출력
S	랩의 server 네임스페이스에서 실행하는 별칭(ip netns exec ns_s)	-issuer	발급자 출력
s_server	openssl의 TLS 서버 모드	-dates	유효 시작·종료일 출력
-accept	수신 대기할 포트	-ext	확장 필드만 출력
$P_TLS_PORT	실습 TLS 포트(고른 값)	subjectAltName	인증서의 대체 이름 확장(SAN)
-cert	사용할 인증서	curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트
-key	사용할 개인 키	-vk	자세히 + 인증서 검증 생략(-v -k)
-www	접속한 클라이언트에 상태 페이지를 응답	https://$P_S_IP:$P_TLS_PORT/	https 주소: 방식://호스트[:포트]/경로 → $P_S_IP(server 쪽 IP) · $P_TLS_PORT(실습 TLS 포트(고른 값))
&	명령을 백그라운드로 실행	2>&1	오류 출력을 표준 출력과 합침
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	egrep	확장 정규식으로 줄 검색(grep -E 와 같음)
s_client	openssl의 TLS 클라이언트 모드	"SSL|TLS|subject|expire|issuer"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: SSL|TLS|subject|expire|issuer

6-2 인증서 만료 점검

판단 -checkend 는 종료 코드로 판정 (모니터링에 활용)

```
openssl x509 -in $P_CRT -noout -enddate
openssl x509 -in $P_CRT -noout -checkend 86400 && echo "24h 이상 유효" || echo "24h 내 만료"
openssl x509 -in $P_CRT -noout -checkend 0 && echo "유효" || echo "만료됨"
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | openssl x509 -noou
t -enddate
for h in a.com b.com; do echo -n "$h "; echo | openssl s_client -connect $h:443 -servername $h 2>/dev/nul
l | openssl x509 -noout -enddate; done
```

단어	뜻	단어	뜻
openssl	암호·인증서·TLS 도구	-connect	접속할 host:port
x509	X.509 인증서 처리 모드	example.com:443	호스트:포트 — 도메인 example.com 의 443번 포트 (HTTPS)

`-in`	입력 파일	`-servername`	SNI로 보낼 호스트 이름
`$P_CRT`	인증서 파일 경로	`example.com`	예시 도메인 이름(실제 주소로 교체)
`-noout`	인코딩된 본문은 출력하지 않음	`2>`	오류 출력(2번)을 파일로 보냄
`-enddate`	만료일 출력	`/dev/null`	출력을 버리는 특수 파일
`-checkend`	N초 뒤에 만료되는지 검사	`for`	목록의 각 값마다 반복 시작
`86400`	숫자 값 86400	`h`	변수 이름(도메인 목록 반복 변수)
`&&`	앞 명령이 성공하면 뒤 명령 실행	`in`	for 반복에서 값 목록 앞에 붙는 말
`echo`	인자를 그대로 출력	`a.com`	예시 도메인
`"24h 이상 유효"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 24h 이상 유효	`b.com`	예시 도메인
`||`	앞 명령이 실패하면 뒤 명령 실행	`;`	명령 구분: 앞 명령이 끝나면 다음 명령 실행
`"24h 내 만료"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 24h 내 만료	`do`	반복문 본문 시작
`0`	숫자 값 0	`-n`	끝 줄바꿈 생략
`"유효"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 유효	`"$h "`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: $h
`"만료됨"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 만료됨	`$h:443`	조합 인자 → $h · 443(번호)
`|`	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	`$h`	셸 변수 h의 값
`s_client`	openssl의 TLS 클라이언트 모드	`done`	반복문 끝

6-3 프로토콜/암호군 불일치

판단 handshake failure / no protocols available

```
C openssl s_client -connect $P_S_IP:$P_TLS_PORT -tls1_2 </dev/null 2>&1 | grep -E "Protocol|Cipher"
C openssl s_client -connect $P_S_IP:$P_TLS_PORT -tls1_3 </dev/null 2>&1 | grep -E "Protocol|Cipher"
C openssl s_client -connect $P_S_IP:$P_TLS_PORT -tls1_1 </dev/null 2>&1 | tail -5
openssl ciphers -v 'HIGH:!aNULL' | head
nmap --script ssl-enum-ciphers -p 443 <IP>
curl --tlsv1.2 --tls-max 1.2 -vI https://<호스트>
```

단어	뜻	단어	뜻
`C`	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	`'HIGH:!aNULL'`	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: HIGH:!aNULL
`openssl`	암호·인증서·TLS 도구	`head`	앞쪽 N줄만 출력
`s_client`	openssl의 TLS 클라이언트 모드	`nmap`	포트·서비스 스캐너
`-connect`	접속할 host:port	`--script`	NSE 스크립트 실행
`$P_S_IP:$P_TLS_PORT`	조합 인자 → $P_S_IP(server 쪽 IP) · $P_TLS_PORT(실습 TLS 포트(고른 값))	`ssl-enum-ciphers`	nmap 스크립트: 지원하는 TLS 암호 나열
`-tls1_2`	TLS 1.2 강제	`-p`	검사할 포트
`<`	파일을 입력으로 사용	`443`	숫자 값 443
`/dev/null`	출력을 버리는 특수 파일	`<`	파일을 입력으로 사용
`2>&1`	오류 출력을 표준 출력과 합침	`IP`	자리표시자: 실제 IP 주소로 바꿔 넣음
`|`	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	`>`	출력을 파일로 보냄(덮어씀)
`grep`	패턴에 맞는 줄만 출력	`curl`	URL로 요청을 보내 응답을 받는 HTTP 클라이언트
`-E`	확장 정규식	`--tlsv1.2`	TLS 1.2 이상 사용
`"Protocol|Cipher"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: Protocol|Cipher	`--tls-max`	사용할 TLS 최대 버전
`-tls1_3`	TLS 1.3 강제	`1.2`	숫자·단위 값 1.2
`-tls1_1`	TLS 1.1 강제	`-vI`	자세히 + 헤더만(-v -I)
`tail`	뒤쪽 N줄 출력	`https://`	https 주소: 방식://호스트[:포트]/경로 →
`-5`	마지막 5줄	`<`	파일을 입력으로 사용
`ciphers`	지원 암호 목록 모드	호스트	자리표시자: 호스트 이름
`-v`	상세 출력	`>`	출력을 파일로 보냄(덮어씀)

6-4 신뢰 체인 · 호스트명 불일치 · 시스템 CA

판단 (60) SSL certificate problem

```
C curl -v https://$P_S_IP:$P_TLS_PORT/ 2>&1 | grep -iE "certificate|verify|SSL"
C curl --cacert $P_CRT --resolve $P_TLS_CN:$P_TLS_PORT:$P_S_IP https://$P_TLS_CN:$P_TLS_PORT/ -o /dev/nul
l -w "%{http_code}\n"
openssl verify -CAfile ca.pem server.pem
```

```
openssl s_client -connect <호스트>:443 -showcerts </dev/null    # 중간 인증서 누락 확인
update-ca-certificates               # Debian/Ubuntu
update-ca-trust extract              # RHEL 계열
curl -v --cacert /etc/ssl/certs/ca-certificates.crt https://<호스트>
date; timedatectl                    # 시간 오차도 인증서 오류의 흔한 원인
```

단어	뜻	단어	뜻
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	ca.pem	CA 인증서 파일
curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트	server.pem	서버 인증서 파일
-v	요청·응답·TLS 과정을 자세히 표시(verbose)	s_client	openssl의 TLS 클라이언트 모드
https://$P_S_IP:$P_TLS_PORT/	https 주소: 방식://호스트[:포트]/경로 → $P_S_IP(server 쪽 IP) · $P_TLS_PORT(실습 TLS 포트(고른 값))	-connect	접속할 host:port
2>&1	오류 출력을 표준 출력과 합침	<	파일을 입력으로 사용
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	호스트	자리표시자: 호스트 이름
grep	패턴에 맞는 줄만 출력	>	출력을 파일로 보냄(덮어씀)
-iE	대소문자 무시 + 확장 정규식	:443	포트 443 (호스트 생략)
"certificate|verify|SSL"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: certificate|verify|SSL	-showcerts	서버가 보낸 인증서 체인 전체 표시
--cacert	검증에 쓸 CA 인증서 파일	/dev/null	출력을 버리는 특수 파일
$P_CRT	인증서 파일 경로	update-ca-certificates	Debian 계열 신뢰 CA 목록 갱신
--resolve	호스트:포트를 지정한 IP로 강제 연결	update-ca-trust	RHEL 계열 신뢰 CA 목록 갱신
$P_TLS_CN:$P_TLS_PORT:$P_S_IP	조합 인자 → $P_TLS_CN(인증서의 공통 이름(도메인)) · $P_TLS_PORT(실습 TLS 포트(고른 값)) · $P_S_IP(server 쪽 IP)	extract	값 추출
https://$P_TLS_CN:$P_TLS_PORT/	https 주소: 방식://호스트[:포트]/경로 → $P_TLS_CN(인증서의 공통 이름(도메인)) · $P_TLS_PORT(실습 TLS 포트(고른 값))	/etc/ssl/certs/ca-certificates.crt	설정 파일 경로(/etc)
-o	본문을 파일로 저장(/dev/null이면 버림)	https://	https 주소: 방식://호스트[:포트]/경로 →
/dev/null	출력을 버리는 특수 파일	<	파일을 입력으로 사용
-w	끝나고 지정한 형식으로 측정값 출력(예: 단계별 시간)	호스트	자리표시자: 호스트 이름
"%{http_code}\n"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: %{http_code}\n	>	출력을 파일로 보냄(덮어씀)
openssl	암호·인증서·TLS 도구	date	현재 날짜·시각 출력
verify	인증서 체인 검증 모드	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
-CAfile	신뢰할 CA 파일	timedatectl	시간대·시간 동기화 상태 조회

6-5 SSH 호스트키 · 키 형식 문제

판단 REMOTE HOST IDENTIFICATION HAS CHANGED

```
ssh-keyscan -t ed25519,rsa <호스트>
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub     # 핑거프린트
ssh-keygen -R <호스트>               # known_hosts 에서 제거
ssh -vvv -o PreferredAuthentications=publickey user@host
ssh-keygen -y -f ~/.ssh/id_ed25519   # 개인키로 공개키 복원
ssh -Q cipher; ssh -Q kex            # 지원 알고리즘
```

단어	뜻	단어	뜻
ssh-keyscan	서버의 SSH 호스트 키 수집	ssh	원격 접속
-t	키 종류	-vvv	매우 상세한 디버그
ed25519,rsa	SSH 키 종류 목록	-o	설정 옵션을 한 줄로 지정
<	파일을 입력으로 사용	PreferredAuthentications=publickey	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)
호스트	자리표시자: 호스트 이름	user@host	자리표시자: 사용자@호스트
>	출력을 파일로 보냄(덮어씀)	-y	개인 키에서 공개 키 추출
ssh-keygen	SSH 키 생성·관리	-f	키 파일 지정
-lf	키 지문 표시(-l -f)	~/.ssh/id_ed25519	사용자 SSH 설정 디렉터리
/etc/ssh/ssh_host_ed25519_key.pub	설정 파일 경로(/etc)	-Q	지원 알고리즘 조회
-R	known_hosts에서 호스트 삭제	cipher	암호 알고리즘
<	파일을 입력으로 사용	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행

호스트	자리표시자: 호스트 이름	kex	키 교환 알고리즘
>	출력을 파일로 보냄(덮어씀)		

6-6 문자셋/인코딩 깨짐

판단 한글이 ???, 깨짐 → locale/파일 인코딩 확인

```
locale; locale -a | egrep -i "utf|euc"
echo "안녕하세요" > /tmp/utf8.txt
iconv -f UTF-8 -t EUC-KR /tmp/utf8.txt > /tmp/euckr.txt
file -i /tmp/utf8.txt /tmp/euckr.txt
cat /tmp/euckr.txt                    # 깨짐
iconv -f EUC-KR -t UTF-8 /tmp/euckr.txt
localectl status; localectl set-locale LANG=ko_KR.UTF-8
export LANG=en_US.UTF-8 LC_ALL=en_US.UTF-8
convmv -f euc-kr -t utf-8 -r --notest .        # 파일명 인코딩 변환
curl -sI <URL> | grep -i content-type          # charset 확인
```

단어	뜻	단어	뜻
locale	현재 언어·문자 설정 출력	/tmp/euckr.txt	임시 파일 경로(/tmp)
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	localectl	systemd 로캘·키보드 설정 조회
-a	설치된 로캘 목록	status	서비스 상태 조회
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	set-locale	시스템 로캘 설정
egrep	확장 정규식으로 줄 검색(grep -E 와 같음)	LANG=ko_KR.UTF-8	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)
-i	대소문자 무시	export	변수를 자식 프로세스에도 전달되게 함
"utf|euc"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: utf|euc	LANG=en_US.UTF-8	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)
echo	인자를 그대로 출력	LC_ALL=en_US.UTF-8	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)
"안녕하세요"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 안녕하세요	convmv	파일 이름의 문자 인코딩 변환
>	출력을 파일로 보냄(덮어씀)	-f	원래 인코딩
/tmp/utf8.txt	임시 파일 경로(/tmp)	euc-kr	한글 완성형 구형 인코딩
iconv	문자 인코딩 변환	-t	바꿀 인코딩
-f	원래 인코딩	utf-8	유니코드 인코딩
UTF-8	유니코드 인코딩	-r	하위 디렉터리 포함
-t	바꿀 인코딩	--notest	실제로 이름을 바꿈(생략하면 미리보기만)
EUC-KR	한글 완성형 구형 인코딩	.	현재 위치 또는 임의 한 글자
/tmp/utf8.txt	임시 파일 경로(/tmp)	curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트
>	출력을 파일로 보냄(덮어씀)	-sI	조용히 + 헤더만(-s -I)
/tmp/euckr.txt	임시 파일 경로(/tmp)	<	파일을 입력으로 사용
file	파일 종류·문자 인코딩 판별	URL	자리표시자: 주소(URL)
-i	MIME 종류와 문자 인코딩 표시	>	출력을 파일로 보냄(덮어씀)
/tmp/utf8.txt	임시 파일 경로(/tmp)	grep	패턴에 맞는 줄만 출력
/tmp/euckr.txt	임시 파일 경로(/tmp)	-i	대소문자 무시
cat	파일 내용을 그대로 출력	content-type	HTTP 헤더: 본문 종류

6-7 압축 · 직렬화 · 데이터 형식 검증

판단 Content-Encoding, JSON/XML 형식 오류

```
curl -sI -H "Accept-Encoding: gzip" <URL> | grep -i content-encoding
curl -s --compressed <URL> | head
echo '{"a":1' | python3 -m json.tool          # 문법 오류 확인
jq . file.json; xmllint --noout file.xml
gzip -t file.gz; tar -tzf file.tgz >/dev/null
base64 -d <<< "aGVsbG8="; file unknown.bin
```

단어	뜻	단어	뜻
curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트	.	현재 위치 또는 임의 한 글자
-sI	조용히 + 헤더만(-s -I)	file.json	예시 JSON 파일
-H	요청 헤더 추가	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
"Accept-Encoding: gzip"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: Accept-Encoding: gzip	xmllint	XML 검사·파싱
<	파일을 입력으로 사용	--noout	결과 출력 없이 문법만 검사
URL	자리표시자: 주소(URL)	file.xml	예시 XML 파일

`>`	출력을 파일로 보냄(덮어씀)	`gzip`	gzip 압축·검사
`|`	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	`-t`	압축 파일 무결성 검사
`grep`	패턴에 맞는 줄만 출력	`file.gz`	예시 gzip 파일
`-i`	대소문자 무시	`tar`	아카이브 묶기·풀기
`content-encoding`	HTTP 헤더: 본문 압축 방식	`-tzf`	gzip tar의 내용 목록(-t -z -f)
`-s`	진행 표시 숨김(silent)	`file.tgz`	예시 tar.gz 파일
`--compressed`	압축 응답 요청·자동 해제	`>`	출력을 파일로 보냄(덮어씀)
`head`	앞쪽 N줄만 출력	`/dev/null`	출력을 버리는 특수 파일
`echo`	인자를 그대로 출력	`base64`	base64 인코딩·디코딩
`'{"a":1'`	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: {"a":1	`-d`	디코딩
`python3`	Python 3 인터프리터	`<<<`	문자열을 표준 입력으로 넘김(here-string)
`-m`	모듈을 프로그램으로 실행(예: http.server)	`"aGVsbG8="`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: aGVsbG8=
`json.tool`	파이썬 JSON 정렬 출력 모듈	`file`	파일 종류·문자 인코딩 판별
`jq`	JSON 가공·추출	`unknown.bin`	형식을 알 수 없는 예시 파일

7계층: 응용 계층 (Application)

HTTP, DNS, SSH, 메일, DB, 웹서버/프록시/LB, 서비스 프로세스와 로그.

7-1 HTTP 상태 코드 · 응답시간 분해

판단 느림의 원인을 DNS/연결/TLS/서버처리/전송으로 분리

```
C curl -sI http://$P_S_IP:$P_PORT/ | head -1        # 200
C curl -sI http://$P_S_IP:$P_PORT/nope | head -1     # 404
C curl -o /dev/null -s -w "dns:%{time_namelookup} tcp:%{time_connect} tls:%{time_appconnect} ttfb:%{time_
starttransfer} total:%{time_total} code:%{http_code}\n" http://$P_S_IP:$P_PORT/
curl -v https://<호스트>/ 2>&1 | head -40
curl -L -I https://<호스트>/                       # 리다이렉트 추적
curl -H "Host: a.example.com" http://<IP>/       # 가상호스트 테스트
curl -X POST -d '{"a":1}' -H 'Content-Type: application/json' <URL>
curl --max-time 5 --retry 3 <URL>
chmod 000 $P_WEBROOT/index.html; C curl -sI http://$P_S_IP:$P_PORT/ | head -1; chmod 644 $P_WEBROOT/inde
x.html
```

단어	뜻	단어	뜻
C	랩의 client 네임스페이스에서 실행하는 별칭(ip netns exec ns_c)	-L	리다이렉트를 따라감
curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트	-I	응답 헤더만 요청(HEAD)
-sI	조용히 + 헤더만(-s -I)	-H	요청 헤더 추가
http://$P_S_IP:$P_PORT/	http 주소: 방식://호스트[:포트]/경로 → $P_S_IP(server 쪽 IP) · $P_PORT(실습 HTTP 포트(고른 값, 기본 8080))	"Host: a.example.com"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: Host: a.example.com
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	http://	http 주소: 방식://호스트[:포트]/경로 →
head	앞쪽 N줄만 출력	IP	자리표시자: 실제 IP 주소로 바꿔 넣음
-1	숫자 값 -1	-X	요청 메서드 지정
http://$P_S_IP:$P_PORT/nope	http 주소: 방식://호스트[:포트]/경로 → $P_S_IP(server 쪽 IP) · $P_PORT(실습 HTTP 포트(고른 값, 기본 8080)) · nope	POST	HTTP POST 메서드
-o	본문을 파일로 저장(/dev/null이면 버림)	-d	요청 본문 데이터
/dev/null	출력을 버리는 특수 파일	'{"a":1}'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: {"a":1}
-s	진행 표시 숨김(silent)	'Content-Type: application/json'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: Content-Type: application/json
-w	끝나고 지정한 형식으로 측정값 출력(예: 단계별 시간)	URL	자리표시자: 주소(URL)
"dns:%{time_namelookup} tcp:%{time_connect} tls:%{time_appconnect} ttfb:%{time_starttransfer} total:%{time_total} code:%{http_code}\n"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: dns:%{time_namelookup} tcp:%{time_connec	--max-time	전체 제한 시간(초)
-v	요청·응답·TLS 과정을 자세히 표시(verbose)	5	숫자 값 5
https://	https 주소: 방식://호스트[:포트]/경로 →	--retry	실패 시 재시도 횟수
<	파일을 입력으로 사용	3	숫자 값 3
호스트	자리표시자: 호스트 이름	chmod	파일 권한 변경
>	출력을 파일로 보냄(덮어씀)	000	숫자 값 000
/	파일·디렉터리 경로	$P_WEBROOT/index.html	조합 인자 → $P_WEBROOT(웹 문서 디렉터리) · index.html
2>&1	오류 출력을 표준 출력과 합침	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
-40	숫자 값 -40	644	숫자 값 644

7-2 상태 코드별 대응 가이드

판단 코드에 따른 1차 조치

```
# 301/302: 리다이렉트 루프 → curl -L -v 로 Location 확인
# 400: 요청 형식/헤더 크기 / 401·403: 인증·권한·ACL·SELinux
# 404: 경로·DocumentRoot / 408·504: 타임아웃(프록시↔업스트림)
```

```
# 413: 업로드 크기 제한(client_max_body_size) / 429: rate limit
# 500: 앱 오류 → 앱 로그 / 502: 업스트림 다운·응답 이상 / 503: 과부하·점검
getenforce; ausearch -m avc -ts recent             # SELinux 거부
ls -ldZ /var/www/html; restorecon -Rv /var/www/html
namei -l /var/www/html/index.html                  # 경로상 권한 확인
nginx -T | grep -E "client_max_body_size|proxy_read_timeout|upstream"
```

단어	뜻	단어	뜻
getenforce	SELinux 동작 모드 출력	-Rv	하위까지 복원(R) + 변경 내용 표시(v)
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	/var/www/html	파일·디렉터리 경로
ausearch	감사 로그에서 조건에 맞는 기록 검색	namei	경로 구성요소별 권한 확인
-m	메시지 종류로 필터	-l	경로 각 단계의 권한 표시
avc	SELinux 접근 거부 기록 종류	/var/www/html/index.html	파일·디렉터리 경로
-ts	이 시각 이후만	nginx	Nginx 웹 서버 제어·설정 검사
recent	최근 항목	-T	설정을 검사하고 전체 출력
ls	디렉터리 내용 나열	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
-ldZ	디렉터리 자체 + SELinux 라벨	grep	패턴에 맞는 줄만 출력
/var/www/html	파일·디렉터리 경로	-E	확장 정규식
restorecon	SELinux 파일 라벨 복원	"client_max_body_size|proxy_read_timeout|upstream"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: client_max_body_size|proxy_read_timeout|

7-3 DNS 장애 진단

판단 ping 8.8.8.8 OK + ping 도메인 FAIL = DNS 문제

```
cat /etc/resolv.conf; resolvectl status | head -20
getent hosts example.com              # 시스템 리졸버 경로 그대로(nsswitch 포함)
dig example.com; dig +short example.com
dig @8.8.8.8 example.com              # 서버 비교
dig +trace example.com                # 루트부터 위임 추적
dig example.com MX / NS / TXT / AAAA / CNAME / SOA
dig -x 8.8.8.8                        # 역방향
dig +tcp example.com; dig +dnssec example.com
dig @10.255.255.1 example.com +time=2 +tries=1   # 타임아웃 재현
nslookup example.com 8.8.8.8
host -a example.com
grep hosts /etc/nsswitch.conf; cat /etc/hosts
resolvectl query example.com; resolvectl flush-caches
systemd-resolve --statistics
tcpdump -i eth0 -n port 53
time getent hosts example.com         # 해석 지연 측정
```

단어	뜻	단어	뜻
cat	파일 내용을 그대로 출력	+dnssec	DNSSEC 서명 포함
/etc/resolv.conf	설정 파일 경로(/etc)	@10.255.255.1	질의할 DNS 서버: 10.255.255.1
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	+time=2	질의 제한 시간(초) (값 2)
resolvectl	systemd-resolved DNS 상태 조회	+tries=1	재시도 횟수 (값 1)
status	서비스 상태 조회	nslookup	DNS 조회
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	example.com	예시 도메인 이름(실제 주소로 교체)
head	앞쪽 N줄만 출력	8.8.8.8	IPv4 주소
-20	숫자 값 -20	host	간단한 DNS 조회
getent	이름 서비스(hosts, passwd 등) 조회	-a	모든 레코드 종류 질의
hosts	호스트 이름 데이터베이스(/etc/hosts)	example.com	예시 도메인 이름(실제 주소로 교체)
example.com	예시 도메인 이름(실제 주소로 교체)	grep	패턴에 맞는 줄만 출력
dig	DNS 서버에 질의해 상세 응답을 보여 주는 도구	hosts	호스트 이름 데이터베이스(/etc/hosts)
example.com	예시 도메인 이름(실제 주소로 교체)	/etc/nsswitch.conf	설정 파일 경로(/etc)
+short	답만 간단히	/etc/hosts	설정 파일 경로(/etc)
@8.8.8.8	질의할 DNS 서버: 8.8.8.8	query	조회
+trace	루트부터 위임 경로를 따라 질의	example.com	예시 도메인 이름(실제 주소로 교체)
MX	DNS 질의 종류: 메일 서버 레코드	flush-caches	DNS 캐시 비움

/	파일·디렉터리 경로	systemd-resolve	systemd DNS 해석기 조회(구형 이름)
NS	DNS 질의 종류: 네임서버 레코드	--statistics	해석기 통계
TXT	DNS 질의 종류: 텍스트 레코드	tcpdump	패킷 캡처·표시
AAAA	DNS 질의 종류: IPv6 주소 레코드	-i	캡처할 인터페이스
CNAME	DNS 질의 종류: 별칭 레코드	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
SOA	DNS 질의 종류: 존 권한 시작 레코드	-n	이름 변환 안 함
-x	IP로 이름을 거꾸로 조회(PTR)	port	포트 번호 조건
8.8.8.8	IPv4 주소	53	숫자 값 53
+tcp	TCP로 질의	time	명령 실행 시간 측정

7-4 서비스 프로세스 (systemd)

판단 서비스 다운이면 Connection refused 로 4계층처럼 보임 → ss 로 리스너 확인

```
systemctl status nginx; systemctl is-active nginx; systemctl is-enabled nginx
systemctl --failed
systemctl restart nginx; systemctl reload nginx
journalctl -u nginx -n 50 --no-pager
journalctl -u nginx --since "10 min ago" -p err
journalctl -xe
nginx -t; apachectl configtest; sshd -t; named-checkconf; haproxy -c -f /etc/haproxy/haproxy.cfg
systemctl cat nginx; systemctl list-dependencies nginx
systemctl stop nginx; curl -sI http://localhost | head -1; ss -tlnp | grep ':80 '; systemctl start nginx
ps aux --sort=-%cpu | head; ps -eo pid,ppid,stat,cmd | grep -w Z   # 좀비
dmesg | grep -i -E "oom|killed process"
```

단어	뜻	단어	뜻
systemctl	systemd 서비스 제어·상태 조회	-c	설정 검사만
status	서비스 상태 조회	-f	설정 파일 지정
nginx	Nginx (서비스·대상 이름)	/etc/haproxy/haproxy.cfg	설정 파일 경로(/etc)
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	cat	설정·유닛 파일 내용 표시
is-active	서비스가 실행 중인지	list-dependencies	의존 관계 나열
is-enabled	부팅 시 자동 시작 설정 여부	stop	중지
--failed	실패한 서비스 목록	curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트
restart	재시작	-sI	조용히 + 헤더만(-s -I)
reload	설정 다시 읽기	http://localhost	http 주소: 방식://호스트[:포트]/경로 → localhost
journalctl	systemd 저널 로그 조회	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
-u	특정 서비스(unit)만	head	앞쪽 N줄만 출력
nginx	Nginx (서비스·대상 이름)	-1	숫자 값 -1
-n	마지막 N줄	ss	소켓 상태 조회(netstat의 후속)
50	숫자 값 50	-tlnp	TCP LISTEN + 숫자 + 프로세스
--no-pager	페이지 넘김 없이 전부 출력	grep	패턴에 맞는 줄만 출력
--since	이 시각 이후	':80 '	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: :80
"10 min ago"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 10 min ago	start	시작
-p	우선순위 이상만(err, warning …)	ps	프로세스 목록
err	오류 수준	aux	모든 사용자의 모든 프로세스(BSD 형식)
-xe	끝부분과 설명을 함께 표시	--sort=-%cpu	정렬 기준(앞의 -는 내림차순) → 값 -%cpu
nginx	Nginx 웹 서버 제어·설정 검사	-eo	전체 프로세스에서 지정한 열만
-t	설정 문법 검사	pid,ppid,stat,cmd	출력할 열: PID, 부모 PID, 상태, 명령
apachectl	Apache 웹 서버 제어·설정 검사	-w	단어 단위로 일치
configtest	설정 검사	Z	좀비 프로세스 상태
sshd	SSH 서버 데몬	dmesg	커널 메시지 버퍼 출력
-t	설정 문법 검사	-i	대소문자 무시
named-checkconf	BIND DNS 설정 문법 검사	-E	확장 정규식
haproxy	HAProxy 로드밸런서 실행·설정 검사	"oom|killed process"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: oom|killed process

7-5 로그 분석

판단 상태코드 분포, 상위 IP, 에러 패턴, 시간대별 급증

```
tail -f /var/log/nginx/error.log
awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn
grep -c ' 5[0-9][0-9] ' /var/log/nginx/access.log
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head
awk '{print substr($4,2,14)}' /var/log/nginx/access.log | uniq -c | sort -rn | head    # 분 단위 요청량
awk '$NF > 2 {print $7,$NF}' access.log | sort -k2 -nr | head                          # 느린 요청(응답시간
필드)
grep -iE "error|fail|denied|timeout|refused" /var/log/syslog | tail -20
journalctl -p err -b --no-pager | tail -30
journalctl --since "2026-11-11 00:00" --until "2026-11-11 01:00"
zgrep "ERROR" /var/log/app/*.gz
lnav /var/log/syslog                  # 설치되어 있는 경우
```

단어	뜻	단어	뜻
tail	뒤쪽 N줄 출력	-nr	숫자 큰 순
-f	새 줄을 계속 따라감	-iE	대소문자 무시 + 확장 정규식
/var/log/nginx/error.log	로그 파일 경로(/var/log)	"error|fail|denied|timeout|refused"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: error|fail|denied|timeout|refused
awk	열(필드) 단위로 줄을 가공하는 텍스트 처리기	/var/log/syslog	로그 파일 경로(/var/log)
'{print $9}'	awk 프로그램: 9번째 열 출력	-20	마지막 20줄
/var/log/nginx/access.log	로그 파일 경로(/var/log)	journalctl	systemd 저널 로그 조회
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	-p	우선순위 이상만(err, warning …)
sort	줄 정렬	err	오류 수준
uniq	이웃한 같은 줄을 합침	-b	이번 부팅 기록만
-c	개수 함께 표시	--no-pager	페이지 넘김 없이 전부 출력
-rn	숫자 내림차순	-30	마지막 30줄
grep	패턴에 맞는 줄만 출력	--since	이 시각 이후
-c	일치한 줄 수만 출력	"2026-11-11 00:00"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 2026-11-11 00:00
' 5[0-9][0-9] '	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 5[0-9][0-9]	--until	이 시각까지
/var/log/nginx/access.log	로그 파일 경로(/var/log)	"2026-11-11 01:00"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 2026-11-11 01:00
'{print $1}'	awk 프로그램: 1번째 열 출력	zgrep	압축 파일 안을 grep
head	앞쪽 N줄만 출력	"ERROR"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: ERROR
'{print substr($4, 2,14)}'	조합 인자 → '{print substr($4 · 2(번호) · 14)}'	/var/log/app/*.gz	로그 파일 경로(/var/log)
'$NF > 2 {print $7,$NF}'	조합 인자 → '$NF > 2 {print $7 · $NF}'	lnav	로그 파일 뷰어
access.log	접근 로그 파일	/var/log/syslog	로그 파일 경로(/var/log)
-k2	2번째 열 기준		

7-6 SSH 접속 장애

판단 단계: 도달성 → 포트 → 핸드셰이크 → 인증

```
ssh -vvv user@host
nc -zv host 22; ss -tlnp | grep sshd
sshd -T | egrep "^(port|permitrootlogin|passwordauthentication|pubkeyauthentication|allowusers|allowgroup
s|maxauthtries)"
journalctl -u sshd -n 30 --no-pager; tail /var/log/auth.log /var/log/secure
lastb | head; last | head
ls -ld ~ ~/.ssh ~/.ssh/authorized_keys     # 권한: 700 / 600
chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys
restorecon -Rv ~/.ssh                     # SELinux
ssh -o StrictHostKeyChecking=no -o ConnectTimeout=5 user@host true
fail2ban-client status sshd; fail2ban-client set sshd unbanip <IP>
ssh -J jump user@target; ssh -L 8080:localhost:80 user@host
```

단어	뜻	단어	뜻
ssh	원격 접속	-ld	디렉터리 자체를 자세히
-vvv	매우 상세한 디버그	~	사용자 홈 디렉터리
user@host	자리표시자: 사용자@호스트	~/.ssh	사용자 SSH 설정 디렉터리
nc	TCP/UDP 연결을 직접 열어 보는 도구(netcat)	~/.ssh/authorized_keys	사용자 SSH 설정 디렉터리
-zv	데이터 없이 포트만 확인(z) + 상세(v)	chmod	파일 권한 변경
host	호스트 조건 (tcpdump: 이 주소와 오가는 패킷)	700	숫자 값 700
22	숫자 값 22	~/.ssh	사용자 SSH 설정 디렉터리
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	600	숫자 값 600
ss	소켓 상태 조회(netstat의 후속)	~/.ssh/authorized_keys	사용자 SSH 설정 디렉터리
-tlnp	TCP LISTEN + 숫자 + 프로세스	restorecon	SELinux 파일 라벨 복원
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	-Rv	하위까지 복원(R) + 변경 내용 표시(v)
grep	패턴에 맞는 줄만 출력	~/.ssh	사용자 SSH 설정 디렉터리
sshd	SSH 서버 서비스·프로세스	-o	설정 옵션을 한 줄로 지정
sshd	SSH 서버 데몬	StrictHostKeyChecking=no	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)
-T	적용 중인 설정 전체 출력	ConnectTimeout=5	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)
egrep	확장 정규식으로 줄 검색(grep -E 와 같음)	true	항상 성공
"^(port|permitrootlogin|passwordauthentication|pubkeyauthentication|allowusers|allowgroups|maxauthtries)"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: ^(port|permitrootlogin|passwordauthentic	fail2ban-client	fail2ban 차단 상태 조회
journalctl	systemd 저널 로그 조회	status	서비스 상태 조회
-u	특정 서비스(unit)만	sshd	SSH 서버 서비스·프로세스
sshd	SSH 서버 서비스·프로세스	set	속성 변경 (예: up/down, mtu)
-n	마지막 N줄	unbanip	fail2ban 차단 해제
30	숫자 값 30	<	파일을 입력으로 사용
--no-pager	페이지 넘김 없이 전부 출력	IP	자리표시자: 실제 IP 주소로 바꿔 넣음
tail	뒤쪽 N줄 출력	>	출력을 파일로 보냄(덮어씀)
/var/log/auth.log	로그 파일 경로(/var/log)	-J	점프 호스트 경유
/var/log/secure	로그 파일 경로(/var/log)	jump	점프(경유) 호스트 자리표시자
lastb	로그인 실패 기록	user@target	자리표시자: 사용자@대상
head	앞쪽 N줄만 출력	-L	로컬 포트 포워딩
last	로그인 기록	8080:localhost:80	로컬 8080 포트 → 원격에서 본 localhost의 80 포트로 전달
ls	디렉터리 내용 나열		

7-7 웹서버 · 리버스 프록시 · LB 점검

판단 502/504 는 프록시와 업스트림 사이 문제

```
nginx -T | head -100; nginx -s reload
curl -sI http://127.0.0.1:<업스트림포트>/           # 업스트림 직접 호출
tail -f /var/log/nginx/error.log | grep -i upstream
apachectl -S; apachectl -t -D DUMP_MODULES
echo "show stat" | socat stdio /var/run/haproxy.sock | cut -d, -f1,2,18 | column -s, -t
ab -n 1000 -c 50 http://<호스트>/                    # 부하 테스트
wrk -t4 -c100 -d30s http://<호스트>/
curl -w "@curl-format.txt" -o /dev/null -s <URL>
```

단어	뜻	단어	뜻
nginx	Nginx 웹 서버 제어·설정 검사	cut	줄에서 지정한 열만 잘라냄
-T	설정을 검사하고 전체 출력	-d,	구분 문자 지정 → 값 ,
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	-f1,2,18	뽑을 열 번호 → 값 1,2,18
head	앞쪽 N줄만 출력	column	출력을 열 맞춤 표로 정렬
-100	숫자 값 -100	-s,	입력 구분 문자 지정 → 값 ,
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-t	열 정렬 표로 출력

-s	신호 전송(reload 등)	ab	Apache 서버에 부하를 주어 초당 처리량을 재는 벤치마크
reload	설정 다시 읽기	-n	전체 요청 수
curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트	1000	숫자 값 1000
-sI	조용히 + 헤더만(-s -I)	-c	동시 요청 수
http://127.0.0.1:	http 주소: 방식://호스트[:포트]/경로 → 127.0.0.1	50	숫자 값 50
<	파일을 입력으로 사용	http://	http 주소: 방식://호스트[:포트]/경로 →
업스트림포트	자리표시자: 상위 서버 포트	<	파일을 입력으로 사용
>	출력을 파일로 보냄(덮어씀)	호스트	자리표시자: 호스트 이름
/	파일·디렉터리 경로	>	출력을 파일로 보냄(덮어씀)
tail	뒤쪽 N줄 출력	/	파일·디렉터리 경로
-f	새 줄을 계속 따라감	wrk	HTTP 부하 생성기
/var/log/nginx/error.log	로그 파일 경로(/var/log)	-t4	스레드 4개
grep	패턴에 맞는 줄만 출력	-c100	연결 100개 유지
-i	대소문자 무시	-d30s	30초 동안 측정
upstream	상위(백엔드) 서버	http://	http 주소: 방식://호스트[:포트]/경로 →
apachectl	Apache 웹 서버 제어·설정 검사	<	파일을 입력으로 사용
-S	가상호스트·포트 설정 요약	호스트	자리표시자: 호스트 이름
-t	설정 문법만 검사	>	출력을 파일로 보냄(덮어씀)
-D	설정 파일의 정의(define) 목록 출력 (예: DUMP_VHOSTS)	/	파일·디렉터리 경로
DUMP_MODULES	로드된 모듈 목록 출력 정의	-w	끝나고 지정한 형식으로 측정값 출력(예: 단계별 시간)
echo	인자를 그대로 출력	"@curl-format.txt"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: @curl-format.txt
"show stat"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: show stat	-o	본문을 파일로 저장(/dev/null이면 버림)
socat	두 데이터 흐름을 잇는 만능 중계기	/dev/null	출력을 버리는 특수 파일
stdio	표준 입출력	-s	진행 표시 숨김(silent)
/var/run/haproxy.sock	파일·디렉터리 경로	URL	자리표시자: 주소(URL)

7-8 DB · 메일 · 기타 서비스

판단 포트 → 인증 → 쿼리 순서로

```
mysqladmin -h <호스트> -u root -p ping; mysql -h <호스트> -e "select 1"
pg_isready -h <호스트> -p 5432
redis-cli -h <호스트> ping; redis-cli info clients
nc -zv <호스트> 25 587 993
openssl s_client -starttls smtp -connect <호스트>:587
swaks --to a@b.com --server <호스트>
postqueue -p; tail /var/log/mail.log
mongosh --host <호스트> --eval 'db.runCommand({ping:1})'
curl -s localhost:9200/_cluster/health?pretty      # Elasticsearch
ntpq -p; chronyc tracking; timedatectl             # 시간 동기화
```

단어	뜻	단어	뜻
mysqladmin	MySQL 관리 명령	587	숫자 값 587
-h	서버 호스트	993	숫자 값 993
<	파일을 입력으로 사용	openssl	암호·인증서·TLS 도구
호스트	자리표시자: 호스트 이름	s_client	openssl의 TLS 클라이언트 모드
>	출력을 파일로 보냄(덮어씀)	-starttls	평문으로 시작해 TLS로 전환(smtp 등)
-u	사용자 이름	smtp	메일 전송 프로토콜
root	최상위(루트) 위치	-connect	접속할 host:port
-p	비밀번호 입력	<	파일을 입력으로 사용
ping	ping 대상 이름	호스트	자리표시자: 호스트 이름
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	>	출력을 파일로 보냄(덮어씀)
mysql	MySQL 클라이언트	:587	포트 587 (호스트 생략)
-h	서버 호스트	swaks	SMTP 테스트 메일 전송
<	파일을 입력으로 사용	--to	수신자
호스트	자리표시자: 호스트 이름	a@b.com	예시 메일 주소

>	출력을 파일로 보냄(덮어씀)	--server	SMTP 서버
-e	SQL을 바로 실행	<	파일을 입력으로 사용
"select 1"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: select 1	호스트	자리표시자: 호스트 이름
pg_isready	PostgreSQL 연결 가능 여부 확인	>	출력을 파일로 보냄(덮어씀)
-h	호스트	postqueue	Postfix 메일 대기열 조회
<	파일을 입력으로 사용	-p	대기 중인 메일 목록
호스트	자리표시자: 호스트 이름	tail	뒤쪽 N줄 출력
>	출력을 파일로 보냄(덮어씀)	/var/log/mail.log	로그 파일 경로(/var/log)
-p	포트	mongosh	MongoDB 셸
5432	숫자 값 5432	--host	접속할 호스트
redis-cli	Redis 클라이언트	<	파일을 입력으로 사용
-h	호스트	호스트	자리표시자: 호스트 이름
<	파일을 입력으로 사용	>	출력을 파일로 보냄(덮어씀)
호스트	자리표시자: 호스트 이름	--eval	JS 식을 바로 실행
>	출력을 파일로 보냄(덮어씀)	'db.runCommand({ping:1})'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: db.runCommand({ping:1})
ping	ping 대상 이름	curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트
info	정보	-s	진행 표시 숨김(silent)
clients	접속한 클라이언트 목록	localhost:9200/_cluster/health?pretty	로컬 9200 포트의 클러스터 상태 API 주소(?pretty는 보기 좋게 출력)
nc	TCP/UDP 연결을 직접 열어 보는 도구 (netcat)	ntpq	NTP 서버 동기화 상태 조회
-zv	데이터 없이 포트만 확인(z) + 상세(v)	-p	동기화 대상(peer) 목록
<	파일을 입력으로 사용	chronyc	chrony 시간 동기화 상태 조회
호스트	자리표시자: 호스트 이름	tracking	연결 추적
>	출력을 파일로 보냄(덮어씀)	timedatectl	시간대·시간 동기화 상태 조회
25	숫자 값 25		

7-9 시간 동기화 · 로그 시각 불일치 (자주 간과)

판단 인증서·Kerberos·로그 상관분석에 영향

```
timedatectl status
chronyc sources -v; chronyc tracking
ntpq -pn
hwclock --show
timedatectl set-ntp true
```

단어	뜻	단어	뜻
timedatectl	시간대·시간 동기화 상태 조회	ntpq	NTP 서버 동기화 상태 조회
status	서비스 상태 조회	-pn	peer 목록 + 숫자 주소
chronyc	chrony 시간 동기화 상태 조회	hwclock	하드웨어 시계 조회
sources	시간 소스 목록	--show	하드웨어 시계 값 표시
-v	상세 출력	set-ntp	시간 자동 동기화 켜기·끄기
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	true	항상 성공
tracking	연결 추적		

부록 A. 시스템 자원 병목 (네트워크처럼 보이는 장애)

CPU, 메모리, 디스크, 파일 디스크립터 문제가 네트워크 지연·타임아웃으로 나타나는 경우가 많습니다.

A-1 CPU · 부하 · 인터럽트

판단 softirq 가 한 코어에 몰리면 패킷 드롭

```
uptime; top -bn1 | head -15; mpstat -P ALL 1 3; vmstat 1 5
pidstat 1 3
cat /proc/softirqs | egrep "NET_RX|NET_TX"
cat /proc/interrupts | grep eth0
cat /proc/net/softnet_stat           # 두 번째 열(dropped) 증가 여부
ethtool -l eth0; ethtool -g eth0     # 큐 수 / 링버퍼 크기
ethtool -G eth0 rx 4096              # 링버퍼 확대
sysctl net.core.netdev_max_backlog net.core.rmem_max net.core.wmem_max
```

단어	뜻	단어	뜻
uptime	가동 시간과 부하 평균	/proc/softirqs	커널이 보여 주는 가상 파일(/proc)
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	egrep	확장 정규식으로 줄 검색(grep -E 와 같음)
top	프로세스 자원 사용 실시간 표시	"NET_RX|NET_TX"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: NET_RX|NET_TX
-bn1	배치 모드(b)로 1번(n1) 출력	/proc/interrupts	커널이 보여 주는 가상 파일(/proc)
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	grep	패턴에 맞는 줄만 출력
head	앞쪽 N줄만 출력	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
-15	숫자 값 -15	/proc/net/softnet_stat	커널이 보여 주는 가상 파일(/proc)
mpstat	CPU 코어별 사용률	ethtool	네트워크 카드(NIC) 속도·링크·통계 조회와 설정
-P	코어 지정(ALL이면 전체)	-l	채널(큐) 수
ALL	전체	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
1	숫자 1	-g	링 버퍼 크기 조회
3	숫자 값 3	-G	링 버퍼 크기 설정
vmstat	메모리·CPU·IO 요약 통계	rx	수신 방향
1	숫자 1	4096	숫자 값 4096
5	숫자 값 5	sysctl	커널 파라미터 조회·설정
pidstat	프로세스별 자원 사용 통계	net.core.netdev_max_backlog	커널 파라미터 net.core.netdev_max_backlog: NIC 수신 큐 크기
1	숫자 1	net.core.rmem_max	커널 파라미터 net.core.rmem_max: 수신 버퍼 최대
3	숫자 값 3	net.core.wmem_max	커널 파라미터 net.core.wmem_max: 송신 버퍼 최대
cat	파일 내용을 그대로 출력		

A-2 메모리 · OOM

판단 OOM 으로 서비스가 죽으면 갑자기 refused

```
free -h; vmstat 1 5; cat /proc/meminfo | head -20
ps aux --sort=-%mem | head
dmesg -T | grep -i -E "out of memory|oom-killer|killed process"
journalctl -k | grep -i oom
slabtop -o | head; sysctl vm.swappiness vm.overcommit_memory
smem -tk 2>/dev/null | tail
```

단어	뜻	단어	뜻
free	메모리 사용량 표시	-i	대소문자 무시
-h	읽기 쉬운 단위	-E	확장 정규식
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	"out of memory|oom-killer|killed process"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: out of memory|oom-killer|killed process
vmstat	메모리·CPU·IO 요약 통계	journalctl	systemd 저널 로그 조회
1	숫자 1	-k	커널 메시지만
5	숫자 값 5	oom	메모리 부족 강제 종료 기록

cat	파일 내용을 그대로 출력	slabtop	커널 슬랩 메모리 사용 표시
/proc/meminfo	커널이 보여 주는 가상 파일(/proc)	-o	한 번만 출력
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	sysctl	커널 파라미터 조회·설정
head	앞쪽 N줄만 출력	vm.swappiness	커널 파라미터 vm.swappiness: 스왑 사용 성향
-20	숫자 값 -20	vm.overcommit_memory	커널 파라미터 vm.overcommit_memory: 메모리 초과 할당 정책
ps	프로세스 목록	smem	프로세스별 실제 메모리 사용량
aux	모든 사용자의 모든 프로세스(BSD 형식)	-tk	합계(t) + KB 등 읽기 쉬운 단위(k)
--sort=-%mem	정렬 기준(앞의 -는 내림차순) → 값 -%mem	2>	오류 출력(2번)을 파일로 보냄
dmesg	커널 메시지 버퍼 출력	/dev/null	출력을 버리는 특수 파일
-T	사람이 읽는 시각으로 표시	tail	뒤쪽 N줄 출력
grep	패턴에 맞는 줄만 출력		

A-3 디스크 · I/O · inode

판단 디스크 가득 → 로그/세션/소켓 파일 생성 실패

```
df -h; df -i
du -xh / --max-depth=2 2>/dev/null | sort -rh | head
lsof +L1                              # 삭제됐지만 열려 있는 파일
iostat -xz 1 3; iotop -oPa
lsblk; mount | grep -E "ro,|errors"
dmesg | grep -i -E "i/o error|ext4|xfs|read-only"
```

단어	뜻	단어	뜻
df	파일시스템별 사용량 표시	+L1	링크 수 0(삭제됐지만 열려 있는) 파일만
-h	사람이 읽기 쉬운 단위(K,M,G)	iostat	디스크 입출력 통계
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-xz	확장 통계(x) + 활동 없는 장치 숨김(z)
-i	용량 대신 inode 사용량	1	숫자 1
du	디렉터리별 디스크 사용량 합계	3	숫자 값 3
-xh	다른 파일시스템 제외(x) + 읽기 쉬운 단위(h)	iotop	프로세스별 디스크 입출력 표시
/	파일·디렉터리 경로	-oPa	입출력 중인 것만(o), 프로세스 단위(P), 누적(a)
--max-depth=2	내려갈 디렉터리 깊이 → 값 2	lsblk	블록 장치(디스크) 목록
2>	오류 출력(2번)을 파일로 보냄	mount	파일시스템 마운트·조회
/dev/null	출력을 버리는 특수 파일	grep	패턴에 맞는 줄만 출력
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	-E	확장 정규식
sort	줄 정렬	"ro,|errors"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: ro,|errors
-rh	사람이 읽는 단위 큰 순	dmesg	커널 메시지 버퍼 출력
head	앞쪽 N줄만 출력	-i	대소문자 무시
lsof	열린 파일·소켓 목록	"i/o error|ext4|xfs|read-only"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: i/o error|ext4|xfs|read-only

A-4 프로세스 · 파일 디스크립터 · 스레드

판단 Too many open files

```
ulimit -a; cat /proc/<PID>/limits
ls /proc/<PID>/fd | wc -l
lsof -p <PID> | wc -l; lsof -i -nP | awk '{print $1}' | sort | uniq -c | sort -rn
cat /proc/sys/fs/file-nr
strace -f -p <PID> -e trace=network -tt   # 시스템콜 추적
perf top; ltrace -p <PID>
systemctl show nginx -p LimitNOFILE
```

단어	뜻	단어	뜻
ulimit	프로세스 자원 한도 조회·설정	sort	줄 정렬
-a	모든 한도 표시	uniq	이웃한 같은 줄을 합침
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-c	개수 함께 표시
cat	파일 내용을 그대로 출력	-rn	숫자 내림차순

/proc/	커널이 보여 주는 가상 파일(/proc)	/proc/sys/fs/file-nr	커널이 보여 주는 가상 파일(/proc)
<	파일을 입력으로 사용	strace	시스템 호출 추적
PID	자리표시자: 프로세스 번호	-f	자식 프로세스도 추적
>	출력을 파일로 보냄(덮어씀)	-p	PID에 붙음
/limits	파일·디렉터리 경로	<	파일을 입력으로 사용
ls	디렉터리 내용 나열	PID	자리표시자: 프로세스 번호
/proc/	커널이 보여 주는 가상 파일(/proc)	>	출력을 파일로 보냄(덮어씀)
<	파일을 입력으로 사용	-e	추적할 시스템 호출 종류 지정
PID	자리표시자: 프로세스 번호	trace=network	변수 대입 이름=값 (대입 후 이 이름으로 값을 읽음)
>	출력을 파일로 보냄(덮어씀)	-tt	마이크로초 단위 시각 표시
/fd	파일·디렉터리 경로	perf	성능 프로파일러
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	top	상위 항목(요약 보기)
wc	줄·단어·바이트 수 세기	ltrace	라이브러리 함수 호출 추적
-l	줄 수	-p	지정한 PID에 붙음
lsof	열린 파일·소켓 목록	<	파일을 입력으로 사용
-p	지정한 PID의 열린 파일	PID	자리표시자: 프로세스 번호
<	파일을 입력으로 사용	>	출력을 파일로 보냄(덮어씀)
PID	자리표시자: 프로세스 번호	systemctl	systemd 서비스 제어·상태 조회
>	출력을 파일로 보냄(덮어씀)	show	조회(표시)
-i	네트워크 연결만	nginx	Nginx (서비스·대상 이름)
-nP	이름·포트 변환 없이 숫자로	-p	표시할 속성
awk	열(필드) 단위로 줄을 가공하는 텍스트 처리기	LimitNOFILE	열 수 있는 파일 수 한도(systemd 항목)
'{print $1}'	awk 프로그램: 1번째 열 출력		

A-5 커널 네트워크 파라미터 일괄 점검

판단 튜닝 전 현재값 기록

```
sysctl -a 2>/dev/null | egrep "net\.(core|ipv4\.tcp|ipv4\.ip_local|netfilter)" | head -80
sysctl net.ipv4.tcp_congestion_control net.core.default_qdisc
sysctl net.ipv4.tcp_rmem net.ipv4.tcp_wmem
sysctl -p /etc/sysctl.conf; sysctl --system
nstat -az | head -50; cat /proc/net/snmp; cat /proc/net/netstat
```

단어	뜻	단어	뜻
sysctl	커널 파라미터 조회·설정	net.ipv4.tcp_wmem	커널 파라미터 net.ipv4.tcp_wmem: TCP 송신 버퍼 최소·기본·최대
-a	모든 커널 파라미터	-p	설정 파일을 읽어 적용
2>	오류 출력(2번)을 파일로 보냄	/etc/sysctl.conf	설정 파일 경로(/etc)
/dev/null	출력을 버리는 특수 파일	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	--system	모든 설정 파일을 다시 적용
egrep	확장 정규식으로 줄 검색(grep -E 와 같음)	nstat	커널 네트워크 카운터 증가량 표시
"net\.(core|ipv4\.tcp|ipv4\.ip_local|netfilter)"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: net\.(core|ipv4\.tcp|ipv4\.ip_local|netf	-az	0인 카운터도 포함한 절대값(-a -z)
head	앞쪽 N줄만 출력	-50	숫자 값 -50
-80	숫자 값 -80	cat	파일 내용을 그대로 출력
net.ipv4.tcp_congestion_control	커널 파라미터 net.ipv4.tcp_congestion_control: TCP 혼잡 제어 알고리즘	/proc/net/snmp	커널이 보여 주는 가상 파일(/proc)
net.core.default_qdisc	커널 파라미터 net.core.default_qdisc: 기본 큐잉 방식	/proc/net/netstat	커널이 보여 주는 가상 파일(/proc)
net.ipv4.tcp_rmem	커널 파라미터 net.ipv4.tcp_rmem: TCP 수신 버퍼 최소·기본·최대		

부록 B. 패킷 캡처 · 분석 종합

어느 계층 문제인지 불확실할 때 가장 확실한 방법은 패킷을 직접 보는 것입니다.

B-1 tcpdump 필터 모음

판단 캡처 후 Wireshark 로 분석

```
tcpdump -i any -nn host $P_S_IP
tcpdump -i eth0 -nn port 443 and host <IP>
tcpdump -i eth0 -nn net $P_NET
tcpdump -i eth0 -nn 'tcp port 80 and (tcp[tcpflags] & tcp-syn != 0)'
tcpdump -i eth0 -nn -A 'tcp port 80' -c 20          # ASCII 출력(평문 HTTP)
tcpdump -i eth0 -nn -XX -c 5                        # 헥사 덤프
tcpdump -i eth0 -nn not port 22                     # SSH 제외
tcpdump -i eth0 -nn 'icmp or arp'
tcpdump -i eth0 -nn -s0 -w /tmp/cap.pcap -C 100 -W 5   # 100MB x 5개 순환
tcpdump -r /tmp/cap.pcap -nn | head
tcpdump -i eth0 -nn 'ip[8] < 5'                     # TTL 낮은 패킷
tcpdump -i eth0 -nn 'greater 1400'                  # 큰 패킷
```

단어	뜻	단어	뜻
tcpdump	패킷 캡처·표시	-c	N개 잡고 종료
-i	캡처할 인터페이스	20	숫자 값 20
any	모든 인터페이스·주소 (임의)	-XX	링크 헤더 포함 16진수 + ASCII
-nn	호스트·포트 이름 변환 안 함	5	숫자 값 5
host	호스트 조건 (tcpdump: 이 주소와 오가는 패킷)	not	조건 부정
$P_S_IP	server 쪽 IP	22	숫자 값 22
eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)	'icmp or arp'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: icmp or arp
port	포트 번호 조건	-s0	패킷 전체 길이 캡처
443	숫자 값 443	-w	pcap 파일로 저장
and	조건 둘 다 만족	/tmp/cap.pcap	임시 파일 경로(/tmp)
<	파일을 입력으로 사용	-C	파일 크기(MB) 단위로 분할
IP	자리표시자: 실제 IP 주소로 바꿔 넣음	100	숫자 값 100
>	출력을 파일로 보냄(덮어씀)	-W	파일 개수 제한
net	네트워크 대역 조건	-r	저장된 pcap 파일 읽기
$P_NET	랩 네트워크 대역	|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
'tcp port 80 and (tcp[tcpflags] & tcp-syn != 0)'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: tcp port 80 and (tcp[tcpflags] & tcp-syn	head	앞쪽 N줄만 출력
-A	내용을 ASCII로 출력	'ip[8] < 5'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: ip[8] < 5
'tcp port 80'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: tcp port 80	'greater 1400'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: greater 1400

B-2 tshark / 기타 분석 도구

판단 통계·대화 단위 분석

```
tshark -r /tmp/cap.pcap -q -z conv,tcp
tshark -r /tmp/cap.pcap -Y "tcp.analysis.retransmission" | head
tshark -r /tmp/cap.pcap -Y "http.request" -T fields -e ip.src -e http.host -e http.request.uri
tshark -r /tmp/cap.pcap -Y "dns" -T fields -e dns.qry.name -e dns.flags.rcode
tshark -r /tmp/cap.pcap -q -z io,stat,1
tshark -r /tmp/cap.pcap -Y "tls.handshake.type==1" -T fields -e tls.handshake.extensions_server_name
ngrep -d eth0 -q "GET" port 80
iftop -i eth0; nload eth0; iptraf-ng
nethogs eth0                          # 프로세스별 대역폭
bmon; vnstat -l
```

단어	뜻	단어	뜻

`tshark`	터미널 Wireshark	`"tls.handshake.type==1"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: tls.handshake.type==1
`-r`	pcap 파일 읽기	`tls.handshake.extensions_server_name`	필드: TLS 핸드셰이크의 SNI 이름
`/tmp/cap.pcap`	임시 파일 경로(/tmp)	`ngrep`	패킷 내용을 grep처럼 검색
`-q`	조용히(통계만)	`-d`	감시할 인터페이스
`-z`	통계 항목(예: conv,tcp)	`eth0`	네트워크 인터페이스 이름(일반 이더넷 NIC)
`conv,tcp`	통계: TCP 대화(연결)별 합계	`-q`	조용히(요약 생략)
`-Y`	표시 필터	`"GET"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: GET
`"tcp.analysis.retransmission"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: tcp.analysis.retransmission	`port`	포트 번호 조건
`|`	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	`80`	숫자 값 80
`head`	앞쪽 N줄만 출력	`iftop`	연결별 대역폭 실시간 표시
`"http.request"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: http.request	`-i`	감시할 인터페이스
`-T`	출력 형식(fields 등)	`eth0`	네트워크 인터페이스 이름(일반 이더넷 NIC)
`fields`	출력 형식: 지정한 필드만	`;`	명령 구분: 앞 명령이 끝나면 다음 명령 실행
`-e`	출력할 필드	`nload`	인터페이스 트래픽 그래프
`ip.src`	필드: 출발지 IP	`eth0`	네트워크 인터페이스 이름(일반 이더넷 NIC)
`http.host`	필드: HTTP Host 헤더	`iptraf-ng`	터미널 네트워크 모니터
`http.request.uri`	필드: HTTP 요청 경로	`nethogs`	프로세스별 네트워크 사용량
`"dns"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: dns	`eth0`	네트워크 인터페이스 이름(일반 이더넷 NIC)
`dns.qry.name`	필드: DNS 질의 이름	`bmon`	인터페이스별 대역폭을 실시간으로 보여 주는 도구
`dns.flags.rcode`	필드: DNS 응답 코드	`vnstat`	누적 네트워크 트래픽 통계
`io,stat,1`	통계: 1초 단위 입출력량	`-l`	실시간 보기

B-3 실시간 트래픽 · 대역폭

판단 누가 대역을 쓰는가

```
sar -n DEV 1 5; sar -n TCP,ETCP 1 5; sar -n SOCK 1 3
ip -s link show eth0
watch -n1 "cat /proc/net/dev"
ss -tin | grep -E "bytes_sent|bytes_acked"
cat /sys/class/net/eth0/statistics/{rx_bytes,tx_bytes,rx_dropped,tx_dropped,rx_errors,tx_errors}
dstat -n 1 5
```

단어	뜻	단어	뜻
`sar`	시스템 활동 통계(과거 기록 포함)	`"cat /proc/net/dev"`	따옴표로 묶은 명령 문자열: 다른 명령(예: C, S, bash -c)이 한 덩어리로 받아 실행
`-n`	네트워크 통계 종류 지정(DEV, TCP …)	`ss`	소켓 상태 조회(netstat의 후속)
`DEV`	인터페이스별 통계	`-tin`	TCP + 내부 정보 + 숫자
`1`	숫자 1	`|`	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)
`5`	숫자 값 5	`grep`	패턴에 맞는 줄만 출력
`;`	명령 구분: 앞 명령이 끝나면 다음 명령 실행	`-E`	확장 정규식
`TCP,ETCP`	TCP 통계와 TCP 오류 통계	`"bytes_sent|bytes_acked"`	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: bytes_sent|bytes_acked
`SOCK`	소켓 통계	`cat`	파일 내용을 그대로 출력
`3`	숫자 값 3	`/sys/class/net/eth0/statistics/`	커널 장치 정보 가상 파일(/sys)
`ip`	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	`{`	명령 묶음 시작(현재 셸)
`-s`	통계 카운터 포함	`rx_bytes,tx_bytes,rx_dropped,tx_dropped,rx_errors,tx_errors`	읽을 통계 이름(수신/송신 바이트·드롭·오류)
`link`	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	`}`	명령 묶음 끝
`show`	조회(표시)	`dstat`	CPU·디스크·네트워크 통계를 한 줄씩 보여 줌
`eth0`	네트워크 인터페이스 이름(일반 이더넷 NIC)	`-n`	네트워크 통계 포함
`watch`	명령을 주기적으로 반복 실행해 화면 갱신	`1`	숫자 1
`-n1`	갱신 간격(초) → 값 1	`5`	숫자 값 5

부록 C. 컨테이너 · 가상화 · 클라우드 환경

Docker/Kubernetes/클라우드에서는 L2~L4 가 오버레이·iptables·보안그룹으로 추상화됩니다.

C-1 Docker

판단 컨테이너 네트워크 확인

```
docker ps; docker network ls; docker network inspect bridge
docker inspect -f '{{.NetworkSettings.IPAddress}}' <컨테이너>
docker exec -it <컨테이너> sh -c "ip a; ip r; cat /etc/resolv.conf; ss -tlnp"
docker logs --tail 100 -f <컨테이너>
docker port <컨테이너>
nsenter -t $(docker inspect -f '{{.State.Pid}}' <컨테이너>) -n ss -tlnp   # 호스트 도구로 컨테이너 네트워크 확인
iptables -t nat -L DOCKER -n -v
docker stats --no-stream; docker events --since 10m
```

단어	뜻	단어	뜻
docker	컨테이너 관리 도구	port	포트 번호 조건
ps	프로세스 조회 대상	nsenter	다른 프로세스의 네임스페이스에 들어가 명령 실행
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-t	대상 PID
network	네트워크 종류의 대상	$(docker inspect -f '{{.State.Pid}}' <컨테이너>)	명령 치환: 안쪽 명령을 실행한 출력이 이 자리에 들어감
ls	목록 나열 (s3 ls 등 하위 명령)	'{{.State.Pid}}'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: {{.State.Pid}}
inspect	객체의 상세 정보 출력	-n	네트워크 네임스페이스에 들어감
bridge	브리지	ss	소켓 상태 조회 명령
-f	새 로그를 계속 따라가며 출력	-tlnp	대상 PID 지정(-t)과 이어지는 -l -n -p 의 조합 (명령 줄에서 확인)
'{{.NetworkSettings.IPAddress}}'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: {{.NetworkSettings.IPAddress}}	iptables	netfilter 패킷 필터 규칙 조회·추가·삭제
<	파일을 입력으로 사용	-t	테이블 지정(filter, nat, mangle)
컨테이너	자리표시자: 컨테이너 이름	nat	NAT 테이블
>	출력을 파일로 보냄(덮어씀)	-L	규칙 나열
exec	네임스페이스·컨테이너 안에서 명령 실행	DOCKER	docker 체인(컨테이너 포트 규칙)
-it	터미널 연결(상호작용 + tty)	-n	이름 변환 없이 숫자로 표시
sh	셸	-v	패킷·바이트 카운터 표시
-c	컨테이너 안에서 실행할 명령	stats	통계
"ip a; ip r; cat /etc/resolv.conf; ss -tlnp"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: ip a; ip r; cat /etc/resolv.conf; ss -tl	--no-stream	한 번만 출력하고 끝냄
logs	로그 보기	events	이벤트 목록
--tail	끝에서 N줄	--since	이 시점 이후 로그
100	숫자 값 100	10m	숫자·단위 값 10m

C-2 Kubernetes

판단 Service → Endpoint → Pod → DNS 순으로

```
kubectl get pods -A -o wide; kubectl describe pod <파드>
kubectl logs <파드> --previous; kubectl get events --sort-by=.lastTimestamp
kubectl get svc,ep,ingress -A
kubectl exec -it <파드> -- sh -c "nslookup kubernetes.default; curl -sI http://<svc>:<port>"
kubectl run tmp --rm -it --image=nicolaka/netshoot -- bash     # 디버그 파드
kubectl get networkpolicy -A
kubectl -n kube-system logs -l k8s-app=kube-dns
kubectl top nodes; kubectl top pods
kubectl port-forward svc/<svc> 8080:80
kubectl debug node/<노드> -it --image=busybox
```

단어	뜻	단어	뜻

kubectl	쿠버네티스 클러스터 제어	"nslookup kubernetes.default; curl -sI http://<svc>:<port>"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: nslookup kubernetes.default; curl -sI ht
get	하나를 조회·계산해서 보여 줌	run	실행(파드 생성)
pods	쿠버네티스 파드 목록	tmp	임시 이름
-A	모든 네임스페이스	--rm	끝나면 파드 삭제
-o	출력 형식(wide, yaml, json 등)	--image=nicolaka/netshoot	사용할 컨테이너 이미지 → 값 nicolaka/netshoot
wide	열을 넓게(추가 정보)	bash	bash 셸
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	networkpolicy	네트워크 정책 객체
describe	상세 설명	-n	네임스페이스
pod	쿠버네티스 파드	kube-system	쿠버네티스 시스템 네임스페이스
<	파일을 입력으로 사용	-l	라벨 선택자
파드	자리표시자: 쿠버네티스 파드 이름	k8s-app=kube-dns	라벨: DNS 파드
>	출력을 파일로 보냄(덮어씀)	top	상위 항목(요약 보기)
logs	로그 보기	nodes	노드 목록
--previous	직전 종료된 컨테이너의 로그	port-forward	로컬 포트를 파드·서비스로 전달
events	이벤트 목록	svc/	서비스를 가리키는 접두어
--sort-by=.lastTimestamp	정렬 기준 필드 → 값 .lastTimestamp	svc	쿠버네티스 서비스
svc,ep,ingress	서비스·엔드포인트·인그레스	8080:80	로컬 8080 포트 → 대상의 80 포트
exec	네임스페이스·컨테이너 안에서 명령 실행	debug	디버그 파드 실행
-it	터미널 연결	node/	노드를 가리키는 접두어
--	이후는 컨테이너 안에서 실행할 명령	노드	자리표시자: 노드 이름
sh	셸	--image=busybox	사용할 컨테이너 이미지 → 값 busybox
-c	컨테이너 이름		

C-3 클라우드 (AWS 예시)

판단 OS 내부 점검 + 보안그룹/NACL/라우팅 테이블

```
curl -s http://169.254.169.254/latest/meta-data/local-ipv4      # IMDS(v1 허용 시)
aws ec2 describe-security-groups --group-ids <sg-id>
aws ec2 describe-route-tables --filters Name=vpc-id,Values=<vpc-id>
aws ec2 describe-network-acls
aws ec2 describe-instance-status --instance-ids <id>
aws ec2 get-console-output --instance-id <id>                   # 부팅 로그
# 4계층 도달 확인: OS 방화벽 + 보안그룹 + NACL 세 곳을 모두 점검
VPC Reachability Analyzer 사용(콘솔/CLI)
```

단어	뜻	단어	뜻
curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트	describe-network-acls	AWS: 네트워크 ACL 조회
-s	진행 표시 숨김(silent)	describe-instance-status	AWS: 인스턴스 상태 조회
http://169.254.169.254/latest/meta-data/local-ipv4	http 주소: 방식://호스트[:포트]/경로 → 169.254.169.254 · latest · meta-data · local-ipv4	--instance-ids	인스턴스 ID 목록 지정
aws	AWS 클라우드 명령줄 도구	id	VLAN/장치 식별 번호
ec2	AWS 가상 서버(EC2) 서비스	get-console-output	AWS: 콘솔 출력 조회
describe-security-groups	AWS: 보안 그룹 조회	--instance-id	인스턴스 ID 지정
--group-ids	보안 그룹 ID 지정	VPC	AWS 가상 네트워크
<	파일을 입력으로 사용	Reachability	AWS 도달성 분석
sg-id	자리표시자: 보안 그룹 ID	Analyzer	AWS 분석 도구
>	출력을 파일로 보냄(덮어씀)	사용	설명 문구
describe-route-tables	AWS: 라우팅 표 조회	(	묶음 시작: 서브셸(별도 셸)에서 실행
--filters	조회 결과 필터 조건	콘솔/CLI	설명 문구: 콘솔 또는 CLI

Name=vpc-id,Values=	AWS 필터: 이름=vpc-id 이고 값은 뒤에 오는 값	)	묶음 끝
vpc-id	자리표시자: VPC ID		

C-4 VPN · 터널 · 오버레이

판단 MTU 와 시계열 핸드셰이크 확인

```
wg show; ip -d link show wg0
ipsec statusall; ip xfrm state; ip xfrm policy
ip tunnel show; ip -d link show type vxlan
ping -M do -s 1372 <터널건너편IP>      # 터널 MTU 확인
openvpn --config client.ovpn --verb 4
```

단어	뜻	단어	뜻
wg	WireGuard VPN 상태 조회	vxlan	VXLAN 터널 종류
show	조회(표시)	ping	ICMP 에코 요청으로 도달성·지연 확인
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-M	MTU 탐색 방식(do이면 단편화 금지)
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	do	반복문 본문 시작
-d	상세(details)	-s	데이터 크기(바이트)
link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	1372	숫자 값 1372
show	조회(표시)	<	파일을 입력으로 사용
wg0	WireGuard 인터페이스	터널건너편IP	자리표시자: 터널 반대편 IP
ipsec	IPsec VPN 상태 조회	>	출력을 파일로 보냄(덮어씀)
statusall	IPsec 상세 상태	openvpn	OpenVPN 실행
xfrm	IPsec 변환 정책·상태	--config	설정 파일
state	상태로 필터	client.ovpn	OpenVPN 설정 파일
policy	정책	--verb	로그 상세도
tunnel	터널	4	숫자 값 4
type	인터페이스 종류 지정		

부록 D. 보안 관점 점검 (침입·공격 징후)

DDoS, 무차별 대입, 의심 연결, 비인가 포트 등. 자신이 관리하는 시스템에만 사용하세요.

D-1 의심 연결 · 비인가 포트

판단 알 수 없는 리슨 포트/외부 연결 확인

```
ss -tulnp; ss -tnp state established
lsof -i -nP | grep -v 127.0.0.1
netstat -tunap | grep -v LISTEN | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn | head
ps auxf; ls -la /tmp /var/tmp /dev/shm
crontab -l; ls /etc/cron.*; systemctl list-timers
find / -xdev -mtime -1 -type f 2>/dev/null | head
rkhunter --check; chkrootkit
```

단어	뜻	단어	뜻
ss	소켓 상태 조회(netstat의 후속)	head	앞쪽 N줄만 출력
-tulnp	TCP·UDP LISTEN + 숫자 + 프로세스	ps	프로세스 목록
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	auxf	프로세스를 트리 형태로
-tnp	TCP + 숫자 + 프로세스	ls	디렉터리 내용 나열
state	상태로 필터	-la	숨김 포함 자세히
established	연결이 맺어진 상태	/tmp	파일·디렉터리 경로
lsof	열린 파일·소켓 목록	/var/tmp	파일·디렉터리 경로
-i	네트워크 연결만	/dev/shm	장치 파일(/dev)
-nP	이름·포트 변환 없이 숫자로	crontab	사용자 예약 작업 표 조회·편집
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	-l	등록된 예약 작업 목록
grep	패턴에 맞는 줄만 출력	/etc/cron.*	설정 파일 경로(/etc)
-v	일치하지 않는 줄만	systemctl	systemd 서비스 제어·상태 조회
127.0.0.1	IPv4 주소	list-timers	systemd 타이머 목록
netstat	소켓·라우팅·통계 조회(구형 도구)	find	조건에 맞는 파일 탐색
-tunap	TCP·UDP 전체, 숫자, 프로세스(-t -u -n -a -p)	/	파일·디렉터리 경로
LISTEN	연결 대기 중 상태	-xdev	다른 파일시스템으로 넘어가지 않음
awk	열(필드) 단위로 줄을 가공하는 텍스트 처리기	-mtime	수정 시점(일) 조건
'{print $5}'	awk 프로그램: 5번째 열 출력	-1	숫자 값 -1
cut	줄에서 지정한 열만 잘라냄	-type	파일 종류 조건
-d:	구분 문자 지정 → 값 :	f	변수·옵션 이름
-f1	뽑을 열 번호 → 값 1	2>	오류 출력(2번)을 파일로 보냄
sort	줄 정렬	/dev/null	출력을 버리는 특수 파일
uniq	이웃한 같은 줄을 합침	rkhunter	루트킷 검사
-c	개수 함께 표시	--check	검사 실행
-rn	숫자 내림차순	chkrootkit	루트킷 흔적 검사

D-2 무차별 대입 · 로그인 시도 분석

판단 공격 IP 추출 및 차단

```
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head
lastb | awk '{print $3}' | sort | uniq -c | sort -rn | head
journalctl -u sshd | grep -i "invalid user" | tail
iptables -I INPUT -s <공격IP> -j DROP
nft add rule inet filter input ip saddr <공격IP> drop
fail2ban-client status; fail2ban-client status sshd
ipset create block hash:ip; ipset add block <IP>; iptables -I INPUT -m set --match-set block src -j DROP
```

단어	뜻	단어	뜻
grep	패턴에 맞는 줄만 출력	add	추가
"Failed password"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: Failed password	rule	정책 라우팅 규칙
/var/log/auth.log	로그 파일 경로(/var/log)	inet	IPv4 계열 테이블
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	filter	기본 필터 테이블
awk	열(필드) 단위로 줄을 가공하는 텍스트 처리기	input	입력 체인

'{print $(NF-3)}'	작은따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: {print $(NF-3)}	ip	IP 프로토콜 계열
sort	줄 정렬	saddr	출발지 주소 (nft)
uniq	이웃한 같은 줄을 합침	<	파일을 입력으로 사용
-c	개수 함께 표시	공격IP	자리표시자: 공격 출발지 IP
-rn	숫자 내림차순	>	출력을 파일로 보냄(덮어씀)
head	앞쪽 N줄만 출력	drop	버림
lastb	로그인 실패 기록	fail2ban-client	fail2ban 차단 상태 조회
'{print $3}'	awk 프로그램: 3번째 열 출력	status	서비스 상태 조회
journalctl	systemd 저널 로그 조회	;	명령 구분: 앞 명령이 끝나면 다음 명령 실행
-u	특정 서비스(unit)만	sshd	SSH 서버 서비스·프로세스
sshd	SSH 서버 서비스·프로세스	ipset	IP 집합 관리(방화벽 규칙용)
-i	대소문자 무시	create	만들기
"invalid user"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: invalid user	block	차단
tail	뒤쪽 N줄 출력	hash:ip	IP 해시 집합 종류
iptables	netfilter 패킷 필터 규칙 조회·추가·삭제	add	추가
-I	규칙을 체인 앞쪽에 삽입	<	파일을 입력으로 사용
INPUT	이 호스트로 들어오는 패킷 체인	IP	자리표시자: 실제 IP 주소로 바꿔 넣음
-s	출발지 주소	>	출력을 파일로 보냄(덮어씀)
<	파일을 입력으로 사용	-m	확장 모듈 사용
공격IP	자리표시자: 공격 출발지 IP	set	속성 변경 (예: up/down, mtu)
>	출력을 파일로 보냄(덮어씀)	--match-set	ipset 집합과 비교
-j	일치 시 동작(DROP, ACCEPT, REJECT, LOG)	block	차단
DROP	응답 없이 조용히 버림	src	출발지 주소 조건
nft	nftables 규칙 조회·설정		

D-3 DDoS / 트래픽 폭주 대응

판단 SYN flood, 연결 수 제한, 레이트 리밋

```
ss -tan state syn-recv | wc -l
ss -tan | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn | head
sysctl -w net.ipv4.tcp_syncookies=1
iptables -A INPUT -p tcp --syn -m limit --limit 50/s --limit-burst 100 -j ACCEPT
iptables -A INPUT -p tcp --syn -j DROP
iptables -A INPUT -p tcp --dport 80 -m connlimit --connlimit-above 50 -j REJECT
iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 5/s -j ACCEPT
tcpdump -i eth0 -nn -c 1000 | awk '{print $3}' | cut -d. -f1-4 | sort | uniq -c | sort -rn | head
```

단어	뜻	단어	뜻
ss	소켓 상태 조회(netstat의 후속)	limit	속도 제한 모듈
-tan	TCP 전체 상태 + 숫자	--limit	단위 시간당 허용 횟수
state	상태로 필터	50/s	값 50 (단위 /s)
syn-recv	SYN을 받고 3-way 완료 대기 중	--limit-burst	한 번에 허용할 최대 횟수
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	100	숫자 값 100
wc	줄·단어·바이트 수 세기	-j	일치 시 동작(DROP, ACCEPT, REJECT, LOG)
-l	줄 수	ACCEPT	허용
awk	열(필드) 단위로 줄을 가공하는 텍스트 처리기	DROP	응답 없이 조용히 버림
'{print $5}'	awk 프로그램: 5번째 열 출력	--dport	목적지 포트
cut	줄에서 지정한 열만 잘라냄	80	숫자 값 80
-d:	구분 문자 지정 → 값 :	connlimit	동시 연결 수 제한 모듈
-f1	뽑을 열 번호 → 값 1	--connlimit-above	동시 연결이 이 수를 넘으면 일치
sort	줄 정렬	50	숫자 값 50
uniq	이웃한 같은 줄을 합침	REJECT	거부하고 거부 사실을 통지
-c	개수 함께 표시	icmp	ICMP 프로토콜
-rn	숫자 내림차순	--icmp-type	ICMP 종류
head	앞쪽 N줄만 출력	echo-request	ICMP 에코 요청(ping)
sysctl	커널 파라미터 조회·설정	5/s	값 5 (단위 /s)
-w	값을 지금 바꿈	tcpdump	패킷 캡처·표시
net.ipv4.tcp_syncookies=1	커널 파라미터 net.ipv4.tcp_syncookies: SYN 폭주 시 쿠키로 방어 → 값 1로 설정	-i	캡처할 인터페이스

iptables	netfilter 패킷 필터 규칙 조회·추가·삭제	eth0	네트워크 인터페이스 이름(일반 이더넷 NIC)
-A	규칙을 체인 끝에 추가	-nn	호스트·포트 이름 변환 안 함
INPUT	이 호스트로 들어오는 패킷 체인	-c	N개 잡고 종료
-p	프로토콜(tcp, udp, icmp)	1000	숫자 값 1000
tcp	TCP 프로토콜	'{print $3}'	awk 프로그램: 3번째 열 출력
--syn	SYN 패킷만(연결 시작)	-d.	구분 문자 지정 → 값 .
-m	확장 모듈 사용	-f1-4	뽑을 열 번호 → 값 1-4

D-4 감사 · 무결성

판단 파일 변경과 접근 기록

```
auditctl -l; ausearch -ts today -m USER_LOGIN
aide --check; rpm -Va | head; debsums -c
last -F | head; lastlog | head
getent passwd | awk -F: '$3==0'      # UID 0 계정
awk -F: '($2=="") {print $1}' /etc/shadow   # 빈 비밀번호
```

단어	뜻	단어	뜻
auditctl	리눅스 감사(audit) 규칙 조회·설정	head	앞쪽 N줄만 출력
-l	현재 감사 규칙 목록	debsums	설치된 deb 패키지 파일의 체크섬 검증
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-c	바뀐 파일만 표시
ausearch	감사 로그에서 조건에 맞는 기록 검색	last	로그인 기록
-ts	이 시각 이후만	-F	전체 날짜·시각 표시
today	오늘	lastlog	계정별 마지막 로그인 시각
-m	메시지 종류로 필터	getent	이름 서비스(hosts, passwd 등) 조회
USER_LOGIN	감사 기록 종류: 로그인	passwd	계정 파일 항목
aide	파일 해시 데이터베이스와 현재 파일을 비교해 변조를 찾는 무결성 검사기	awk	열(필드) 단위로 줄을 가공하는 텍스트 처리기
--check	저장된 DB와 현재 파일을 비교	-F:	필드 구분 문자 지정 → 값 :
rpm	RPM 패키지 조회·검증	'$3==0'	조합 인자 → '$3==0'
-Va	설치된 모든 패키지 파일 검증	'($2=="") {print $1}'	조합 인자 → '($2=="") {print $1}'
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	/etc/shadow	설정 파일 경로(/etc)

부록 E. 종합 진단 자동화 스크립트

아래에서 위로(L1→L7) 점검하는 한 방 스크립트와, 로그를 수집해 보고서로 남기는 스크립트입니다.

E-1 diag.sh: 계층별 자동 점검

판단 사용: ./diag.sh <대상IP> <포트> [도메인]

```
#!/bin/bash
IF=${IF:-eth0}; GW=$(ip route | awk '/default/{print $3; exit}')
T=$1; P=$2; D=$3
echo "[L1] 링크:";   ip -br link show $IF; ethtool $IF 2>/dev/null | egrep "Link detected|Speed|Duplex"
echo "[L2] ARP:";    ip neigh show | head -5; ip -s link show $IF | grep -A1 -E "RX:|TX:"
echo "[L3] IP/GW:";  ip -br addr show $IF; ip route show default
echo -n "[L3] GW ping: ";  ping -c 2 -W 1 $GW >/dev/null && echo OK || echo FAIL
echo -n "[L3] 대상 ping: "; ping -c 2 -W 1 $T >/dev/null && echo OK || echo "FAIL (ICMP 차단 가능성)"
echo -n "[L4] 포트 $P: ";   nc -zv -w 3 $T $P 2>&1 | tail -1
[ -n "$D" ] && { echo "[L7] DNS:"; dig +short $D | head -3; }
[ "$P" = "443" ] && { echo "[L6] TLS:"; echo | openssl s_client -connect $T:443 ${D:+-servername $D} 2>/d
ev/null | openssl x509 -noout -dates; }
echo "[L7] HTTP:";   curl -sI -m 5 ${D:+--resolve $D:$P:$T} "http://${D:-$T}:$P/" | head -1
```

단어	뜻	단어	뜻
IF=$	변수 대입 이름=값	&&	앞 명령이 성공하면 뒤 명령 실행
{	명령 묶음 시작(현재 셸)	OK	성공 문구
IF:-eth0	변수 IF가 비었으면 eth0 사용	||	앞 명령이 실패하면 뒤 명령 실행
}	명령 묶음 끝	FAIL	실패 문구
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	"[L3] 대상 ping: "	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: [L3] 대상 ping:
GW=$	변수 대입 이름=값	$T	셸 변수 T의 값
(	묶음 시작: 서브셸(별도 셸)에서 실행	"FAIL (ICMP 차단 가능성)"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: FAIL (ICMP 차단 가능성)
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	"[L4] 포트 $P: "	조합 인자 → "[L4] 포트 $P · "
route	라우팅 표 대상	nc	TCP/UDP 연결을 직접 열어 보는 도구(netcat)
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	-zv	데이터 없이 포트만 확인(z) + 상세(v)
awk	열(필드) 단위로 줄을 가공하는 텍스트 처리기	-w	연결 제한 시간(초)
'/default/{print $3; exit}'	조합 인자 → ' · default · {print $3; exit}'	3	숫자 값 3
)	묶음 끝	$T	셸 변수 T의 값
T=$1	변수 대입 이름=값	$P	셸 변수 P의 값
P=$2	변수 대입 이름=값	2>&1	오류 출력을 표준 출력과 합침
D=$3	변수 대입 이름=값	tail	뒤쪽 N줄 출력
echo	인자를 그대로 출력	-1	마지막 1줄
"[L1] 링크:"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: [L1] 링크:	[	조건식 시작(test)
-br	한 줄 요약(brief)	-n	문자열이 비어 있지 않으면 참
link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	"$D"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: $D
show	조회(표시)	]	조건식 끝
$IF	셸 변수 IF의 값	"[L7] DNS:"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: [L7] DNS:
ethtool	네트워크 카드(NIC) 속도·링크·통계 조회와 설정	dig	DNS 서버에 질의해 상세 응답을 보여 주는 도구
$IF	셸 변수 IF의 값	+short	답만 간단히
2>	오류 출력(2번)을 파일로 보냄	$D	셸 변수 D의 값
/dev/null	출력을 버리는 특수 파일	-3	숫자 값 -3
egrep	확장 정규식으로 줄 검색(grep -E 와 같음)	}	명령 묶음 끝
"Link detected|Speed|Duplex"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: Link detected|Speed|Duplex	"$P"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: $P
"[L2] ARP:"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: [L2] ARP:	=	같다(문자열 비교)
neigh	이웃(ARP/NDP) 표 대상	"443"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 443
head	앞쪽 N줄만 출력	"[L6] TLS:"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: [L6] TLS:

-5	숫자 값 -5	openssl	암호·인증서·TLS 도구
-s	통계 카운터 포함	s_client	openssl의 TLS 클라이언트 모드
grep	패턴에 맞는 줄만 출력	-connect	접속할 host:port
-A1	일치한 줄 뒤 1줄도 출력	$T:443	조합 인자 → $T · 443(번호)
-E	확장 정규식	${D:+-servername $D}	조합 인자 → ${D · +-servername $D}
"RX:|TX:"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: RX:|TX:	2>	오류 출력(2번)을 파일로 보냄
"[L3] IP/GW:"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: [L3] IP/GW:	/dev/null	출력을 버리는 특수 파일
addr	IP 주소 대상	x509	X.509 인증서 처리 모드
default	기본 경로(0.0.0.0/0)	-noout	인코딩된 본문은 출력하지 않음
-n	끝 줄바꿈 생략	-dates	유효 시작·종료일 출력
"[L3] GW ping: "	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: [L3] GW ping:	"[L7] HTTP:"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: [L7] HTTP:
ping	ICMP 에코 요청으로 도달성·지연 확인	curl	URL로 요청을 보내 응답을 받는 HTTP 클라이언트
-c	보낼 횟수	-sI	조용히 + 헤더만(-s -I)
2	숫자 값 2	-m	전체 제한 시간(초)
-W	응답 대기 시간(초)	5	숫자 값 5
1	숫자 1	${D:+--resolve $D:$P:$T}	조합 인자 → ${D · +--resolve $D · $P · $T}
$GW	셸 변수 GW의 값	"http://${D:-$T}:$P/"	조합 인자 → "http · ${D · -$T} · $P · "
>	출력을 파일로 보냄(덮어씀)	-1	숫자 값 -1
/dev/null	출력을 버리는 특수 파일		

E-2 장애 증거 일괄 수집

판단 장애 발생 직후 실행해 두면 사후 분석에 유용

```
#!/bin/bash
OUT=/tmp/incident-$(hostname)-$(date +%Y%m%d-%H%M%S); mkdir -p $OUT
{ date; uptime; ip -br addr; ip route; ip neigh; ip -s link; ss -s; ss -tanp; } > $OUT/net.txt 2>&1
{ free -h; df -h; df -i; vmstat 1 3; top -bn1 | head -30; } > $OUT/res.txt 2>&1
{ nstat -az; netstat -s; cat /proc/net/softnet_stat; } > $OUT/stats.txt 2>&1
dmesg -T | tail -300 > $OUT/dmesg.txt
journalctl -p warning --since "1 hour ago" --no-pager > $OUT/journal.txt 2>&1
iptables-save > $OUT/iptables.txt 2>&1; nft list ruleset > $OUT/nft.txt 2>&1
timeout 30 tcpdump -i any -nn -c 2000 -w $OUT/cap.pcap 2>/dev/null
tar czf $OUT.tgz -C /tmp $(basename $OUT); echo "수집 완료: $OUT.tgz"
```

단어	뜻	단어	뜻
OUT=/tmp/incident-$	변수 대입 이름=값	dmesg	커널 메시지 버퍼 출력
(	묶음 시작: 서브셸(별도 셸)에서 실행	-T	사람이 읽는 시각으로 표시
hostname	호스트 이름 출력	tail	뒤쪽 N줄 출력
)	묶음 끝	-300	마지막 300줄
-$	앞 값과 이어지는 인자 조각	>	출력을 파일로 보냄(덮어씀)
date	현재 날짜·시각 출력	$OUT/dmesg.txt	조합 인자 → $OUT · dmesg.txt
+%Y%m%d-%H%M%S	출력 형식: 연월일-시분초	journalctl	systemd 저널 로그 조회
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	-p	우선순위 이상만(err, warning …)
mkdir	디렉터리 생성	warning	경고 수준
-p	중간 경로도 만들고, 이미 있어도 오류 없음	--since	이 시각 이후
$OUT	셸 변수 OUT의 값	"1 hour ago"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: 1 hour ago
{	명령 묶음 시작(현재 셸)	--no-pager	페이지 넘김 없이 전부 출력
uptime	가동 시간과 부하 평균	>	출력을 파일로 보냄(덮어씀)
ip	리눅스 네트워크(주소·경로·링크·이웃) 조회와 설정의 표준 도구	$OUT/journal.txt	조합 인자 → $OUT · journal.txt
-br	한 줄 요약(brief)	2>&1	오류 출력을 표준 출력과 합침
addr	IP 주소 대상	iptables-save	iptables 규칙 전체를 텍스트로 출력
route	라우팅 표 대상	>	출력을 파일로 보냄(덮어씀)
neigh	이웃(ARP/NDP) 표 대상	$OUT/iptables.txt	조합 인자 → $OUT · iptables.txt
-s	통계 카운터 포함	2>&1	오류 출력을 표준 출력과 합침

link	링크(인터페이스) 계층 대상 — ip link: 장치의 L2 속성	nft	nftables 규칙 조회·설정
ss	소켓 상태 조회(netstat의 후속)	list	목록 조회
-s	소켓 통계 요약	ruleset	규칙 전체
-tanp	TCP 전체 + 숫자 + 프로세스	>	출력을 파일로 보냄(덮어씀)
}	명령 묶음 끝	$OUT/nft.txt	조합 인자 → $OUT · nft.txt
>	출력을 파일로 보냄(덮어씀)	2>&1	오류 출력을 표준 출력과 합침
$OUT/net.txt	조합 인자 → $OUT · net.txt	timeout	지정한 시간 안에 안 끝나면 명령을 중단
2>&1	오류 출력을 표준 출력과 합침	30	30초 제한
free	메모리 사용량 표시	tcpdump	패킷 캡처·표시
-h	읽기 쉬운 단위	-i	캡처할 인터페이스
df	파일시스템별 사용량 표시	any	모든 인터페이스·주소 (임의)
-h	사람이 읽기 쉬운 단위(K,M,G)	-nn	호스트·포트 이름 변환 안 함
-i	용량 대신 inode 사용량	-c	N개 잡고 종료
vmstat	메모리·CPU·IO 요약 통계	2000	숫자 값 2000
1	숫자 1	-w	pcap 파일로 저장
3	숫자 값 3	$OUT/cap.pcap	조합 인자 → $OUT · cap.pcap
top	프로세스 자원 사용 실시간 표시	2>	오류 출력(2번)을 파일로 보냄
-bn1	배치 모드(b)로 1번(n1) 출력	/dev/null	출력을 버리는 특수 파일
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	tar	아카이브 묶기·풀기
head	앞쪽 N줄만 출력	czf	gzip 압축(c: 만들기, z: gzip, f: 파일 이름)
-30	숫자 값 -30	$OUT.tgz	변수 OUT 뒤에 .tgz 를 붙인 아카이브 파일 이름
$OUT/res.txt	조합 인자 → $OUT · res.txt	-C	이 디렉터리로 이동해서 처리
nstat	커널 네트워크 카운터 증가량 표시	/tmp	파일·디렉터리 경로
-az	0인 카운터도 포함한 절대값(-a -z)	$(basename $OUT)	명령 치환: 안쪽 명령을 실행한 출력이 이 자리에 들어감
netstat	소켓·라우팅·통계 조회(구형 도구)	basename	경로에서 마지막 이름만
-s	프로토콜별 통계	$OUT	셸 변수 OUT의 값
cat	파일 내용을 그대로 출력	echo	인자를 그대로 출력
/proc/net/softnet_stat	커널이 보여 주는 가상 파일(/proc)	"수집 완료: $OUT.tgz"	조합 인자 → "수집 완료 · $OUT.tgz"
$OUT/stats.txt	조합 인자 → $OUT · stats.txt		

E-3 지속 모니터링 한 줄 명령

판단 간헐적 장애 포착용

```
while true; do echo "$(date +%T) $(ping -c1 -W1 8.8.8.8 >/dev/null && echo up || echo DOWN)"; sleep 1; do
ne | tee ping.log
watch -n1 'ss -s; echo; nstat -az | egrep "Retrans|Drop"'
while sleep 5; do curl -o /dev/null -s -w "%{time_total} %{http_code}\n" http://<호스트>/; done
journalctl -f -p warning
ping -D -i 0.5 <IP> | awk '{print strftime("%T"), $0}' >> ping.log
```

단어	뜻	단어	뜻
while	조건이 참인 동안 반복	"%{time_total} %{http_code}\n"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: %{time_total} %{http_code}\n
true	항상 성공(종료 상태 0)	http://	http 주소: 방식://호스트[:포트]/경로 →
;	명령 구분: 앞 명령이 끝나면 다음 명령 실행	<	파일을 입력으로 사용
do	반복문 본문 시작	호스트	자리표시자: 호스트 이름
echo	인자를 그대로 출력	>	출력을 파일로 보냄(덮어씀)
"$(date +%T) $(ping -c1 -W1 8.8.8.8 >/dev/null && echo up || echo DOWN)"	큰따옴표로 묶은 한 덩어리 인자 → 안쪽 내용: $(date +%T) $(ping -c1 -W1 8.8.8.8 >/dev	/	파일·디렉터리 경로
sleep	지정한 초만큼 대기	journalctl	systemd 저널 로그 조회
1	숫자 1	-f	새 기록 계속 따라감
done	반복문 끝	-p	우선순위 이상만(err, warning …)
|	앞 명령의 출력을 뒤 명령의 입력으로 넘김(파이프)	warning	경고 수준
tee	출력을 화면과 파일 양쪽에 보냄	ping	ICMP 에코 요청으로 도달성·지연 확인
ping.log	ping 결과 파일	-D	각 줄 앞에 타임스탬프
watch	명령을 주기적으로 반복 실행해 화면 갱신	-i	보내는 간격(초)

`-n1`	갱신 간격(초) → 값 1	`0.5`	숫자·단위 값 0.5
`'ss -s; echo; nstat -az | egrep "Retrans|Drop"'`	따옴표로 묶은 명령 문자열: 다른 명령(예: C, S, bash -c)이 한 덩어리로 받아 실행	`<`	파일을 입력으로 사용
`5`	숫자 값 5	`IP`	자리표시자: 실제 IP 주소로 바꿔 넣음
`curl`	URL로 요청을 보내 응답을 받는 HTTP 클라이언트	`>`	출력을 파일로 보냄(덮어씀)
`-o`	본문을 파일로 저장(/dev/null이면 버림)	`awk`	열(필드) 단위로 줄을 가공하는 텍스트 처리기
`/dev/null`	출력을 버리는 특수 파일	`'{print strftime("%T"), $0}'`	조합 인자 → '{print strftime("%T") · $0}'
`-s`	진행 표시 숨김(silent)	`>>`	출력을 파일 끝에 덧붙임
`-w`	끝나고 지정한 형식으로 측정값 출력(예: 단계별 시간)	`ping.log`	ping 결과 파일

부록 F. 증상 → 계층 빠른 판별표

증상	의심 계층	확인할 것
ethtool: Link detected: no	L1	케이블/포트/NIC, ethtool, dmesg
같은 대역인데 ping 불가, neigh FAILED/INCOMPLETE	L2	ip neigh, arping, VLAN, 스위치
ip neigh 에 중복 MAC / arping -D 응답	L2	IP·MAC 충돌, 브리지 루프
Network is unreachable	L3	ip addr, ip route, 서브넷·게이트웨이
ping 8.8.8.8 OK, ping 도메인 FAIL	L7	DNS: dig, resolv.conf, nsswitch
작은 패킷 OK, 큰 전송 중 멈춤	L2/L3	MTU: ping -M do -s, tracepath
즉시 Connection refused	L4/L7	ss -tlnp, systemctl status (서비스 다운)
30초 뒤 timed out	L3/L4	iptables DROP, 보안그룹, 경로, tcpdump
SYN 만 가고 SYN-ACK 없음	L3/L4	방화벽, 서버 다운, 경로
CLOSE_WAIT 수천 개	L4/L7	앱이 close() 미호출, 앱 로그/덤프
SYN_RECV 급증	L4	SYN flood, backlog, syncookies
nf_conntrack: table full	L3/L4	conntrack_max 증설, 타임아웃 조정
Cannot assign requested address	L4	포트 고갈, ip_local_port_range, tw_reuse
유휴 후 SSH/DB 연결 끊김	L5	keepalive, 방화벽/LB idle timeout
NFS stale file handle / 응답 없음	L5	rpcinfo, nfsstat, 마운트 옵션
SSL certificate problem (60)	L6	openssl s_client, 만료/체인/호스트명/시간
handshake failure	L6	프로토콜/암호군 불일치, SNI
한글이 ??? 로 표시	L6	locale, file -i, iconv
HTTP 502 Bad Gateway	L7	업스트림 다운, 프록시 로그
HTTP 504 Gateway Timeout	L7	업스트림 지연, proxy_read_timeout
HTTP 403	L7	권한/ACL/SELinux, namei -l
서비스 간헐적 느림	전계층	curl -w 분해, mtr, softnet_stat, 자원 병목
패킷 드롭 증가 (softnet dropped)	L1~L3	링버퍼, IRQ 분산, netdev_max_backlog

부록 G. 계층별 핵심 명령어 한 장 요약

계층	핵심 명령어
L1	`ip -br link | ethtool eth0 | ethtool -S eth0 | dmesg -T | grep -i link`
L2	`ip neigh | arping -I eth0 <IP> | ip -s link | tcpdump -e arp | bridge fdb`
L3	`ip addr | ip route get <IP> | ping | traceroute -n | mtr -rwzbc 100 | ping -M do -s`
L4	`ss -tulnp | ss -s | nc -zv <IP> <PORT> | iptables -L -n -v | nft list ruleset | nstat -az`
L5	`ss -o state established | ss -tnoi | sysctl net.ipv4.tcp_keepalive_* | nfsstat | who -a`
L6	`openssl s_client -connect H:443 | openssl x509 -checkend | locale | iconv | file -i`
L7	`curl -v / -w | dig +trace | systemctl status | journalctl -u | tail -f /var/log/...`
전체	`tcpdump -w cap.pcap | tshark -z conv,tcp | sar -n DEV,TCP | vmstat | iostat | dmesg`

진단 원칙

1. 증상 재현 및 범위 파악 (누가, 어디서, 언제부터, 무엇이 안 되는가)
2. L1→L7 순서로 정상/비정상 경계를 찾는다
3. 가설을 세우고 한 번에 하나만 변경한다
4. 변경 전 설정을 백업하고, 복구 방법을 먼저 정한다
5. 원인과 조치를 기록해 재발을 방지한다

부록 H. 일방향 벡터·패턴·바이트 다리 변수분리 대응 총정리

변수분리(분할, 전역 상수, 매개변수, 불리언 수축, 역방향 탐색)를 OSI 7계층과 리눅스 명령에 대응시킨 내용을 한데 모은 부록입니다. 철학의 근거는 **검증기_철학_취합2.pdf**(새 폴더.zip)에서 찾았고, 모든 코드와 수치는 작성 환경에서 **실제로 실행한 결과**입니다. 틈을 메운 정의는 H-10에 적었습니다.

요약

1. **틀:** 접두사 `N_` (분할) `K_` (전역 상수, 조합론 점화식으로 계산한 개수) `P_` (매개변수, 고른 값) `v_` (검증기, 참거짓으로 수축) `r_` (역방향 탐색).
2. **근거:** 구간 적분 결과는 유리수 계수 다항식이고 e, π는 초월수라서 조립제법의 나머지가 0이 되지 않는다. 그래서 계산은 전역 상수와 정확한 등식으로 만나지 못하고 부등식 구간에서만 만난다.
3. **패턴:** 오일러 패턴표의 다섯 종류(성장·감쇠, 회전, 주기, 다항식 배수, 기준 값)를 OSI·리눅스에서 관측되는 현상에 대응시켰다.
4. **다리:** ASCII(L6~L7) → C 구조체(L2~L4) → 컴파일러·어셈블리(엔디언, 다대일) → 격자점·점화식(L1 디지털 부분).
5. **틈 메우기:** 구조적 틈은 오일러 분해 요소(e, i, π, 0·1, t^k)로 정의해 대응시켰고(H-10), P≠NP로 가는 길은 대응 가정으로 H-11에 분리했다.
6. **1번 문서 전체 대응:** ORacle 문서의 개념 39개를 0장, 1~7계층, 부록 A~H 16곳에 대응시켰다(H-12).
7. **수학·알고리즘 적용:** 전역 상수는 조합론 점화식으로 계산한 개수만 `K_` 로 남기고 고른 값은 `P_` 로 개정했으며, 1번 문서의 알고리즘 15종을 구현·실행했다(H-14).

H-0. 구축 방향과 효과

구축 방향. 변수분리를 다음 순서로 세우면 OSI 7계층과 리눅스 명령이 하나의 틀로 묶입니다.

1. **이름 규칙을 고정합니다.** `lab.env` 한 곳에 전역 상수 `K_` (파스칼 점화식으로 계산한 개수, 읽기 전용), 매개변수 `P_` (이름·IP·포트·경로·표준 값처럼 고른 값), 영역 이름 `N_` 을 선언합니다.
2. **계층마다 검증기와 역탐색을 둡니다.** `v_l1` ~ `v_l7` 이 참거짓 하나를 돌려주고, `r_trace` 가 L1부터 돌려 첫 실패 계층을 원인 후보로 돌려줍니다.
3. **바이트를 바닥 표현으로 삼습니다.** L6~L7은 ASCII 코드표와 제어 코드, L2~L4는 C 구조체의 `sizeof` · `offsetof` · `_Static_assert` 로 헤더 필드를 컴파일 타임 상수로 고정하고, 엔디언은 `htons` 로 전선 순서와 호스트 순서를 분리합니다.
4. **패턴을 허용 구간으로 판정합니다.** 성장·감쇠, 회전, 주기, 다항식 배수, 기준 값 다섯 패턴마다 `P_` 에 허용치를 두고 `v_pat_*` 로 구간 판정합니다.
5. **L1은 격자점과 점화식으로 구성합니다.** 심볼당 격자점을 정해 `for` 로 구간을 평균하고 `if` 로 문턱을 비교해 비트를 복원하며, CRC는 비트 단위 점화식으로 계산합니다.
6. **실행은 한 흐름입니다.** `source lab.env` → `P_` 지정 → `v_l1` ⋯ `v_l7` → `r_trace` → 원인 계층 보고.

```
layers.env   계층별 P_ / v_ / r_ (K_ 는 개수만)
ascii.env    L6~L7 바이트 층
pat.env      패턴 검증기
hdr.c        L2~L4 구조체 상수와 체크섬
l1.py        L1 격자점 모델, CRC 점화식
mathlab.py   1번 문서의 수학·알고리즘 적용 (H-14)
oracle.env   판정 장치 (H-12-4)
```

구축했을 때의 효과.

효과	구조에서 나오는 이유	실행으로 확인된 것
원인 계층을 바로 짚음	계층마다 검증기가 있고 `r_trace`가 L1부터 돌려 첫 실패에서 멈춤	닫힌 포트는 L4, 없는 인터페이스는 L1로 즉시 판정
오탐과 진짜 장애를 구분	도구가 없으면 "측정 불가"로 건너뜀	`ping`이 없는 환경에서 L3을 장애로 오판하지 않음
고칠 곳이 한 군데	IP, 포트, 이름, 경로를 `lab.env`의 매개변수(`P_`)로 분리	값 14개를 한 파일에서 바꾸면 문서 전체 명령에 반영
실수로 값이 바뀌지 않음	계산한 `K_`는 `readonly`	`K_N_PORT=1`이 막히고 65536 유지
호출이 서로 간섭하지 않음	매개변수는 복사본	함수 안에서 바꿔도 바깥 값은 그대로
노이즈에 흔들리지 않음	등식이 아니라 허용 구간으로 판정	백오프 `1.1 1.9 4.2 7.8` 통과, 비트오류율이 이론 구간과 일치
1바이트 변조도 탐지	체크섬·CRC가 기준 값에 닫힘	IP 헤더 접은 합 `0xffff`가 TTL 1바이트 변경으로 `0x1000`이 됨, CRC32도 불일치
어긋난 위치까지 지정	바이트 수준 역탐색 `r_firstdiff`	`cmp`가 어긋난 바이트를 "char 3"으로 보고
규약 위반을 빌드 때 차단	구조체 크기·오프셋을 `_Static_assert`로 고정	이더넷 14, IP 20, TCP 20과 필드 오프셋이 컴파일 시점에 검증
재현과 초기화가 쉬움	네임스페이스 랩과 일방향 장애 실습	명령 몇 줄로 생성, 두 줄로 삭제
자동화에 바로 연결	검증기가 종료 코드(0/1)를 돌려줌	cron, CI, 모니터링 스크립트에 그대로 연동
전 계층을 같은 방식으로 다룸	ASCII, 구조체, 격자점이 같은 접두사 규칙 아래	L1~L7을 `K_`·`P_`·`v_`·`r_` 하나의 틀로 점검

한 줄 요약: 상수와 매개변수를 나누고 검증기를 계층마다 세우면, 장애 지점을 빠르게 찾고 설정을 한 곳에서 관리하며 점검을 자동화할 수 있습니다.

H-1. 변수분리 다섯 원리와 이름 규칙

원리	뜻	접두사	리눅스에서의 모습
분할	영역은 서로소로 나뉜다	`N_`	네임스페이스, 프로세스·마운트 격리
전역 상수	조합론의 점화식(파스칼 삼각형)에서 계산한 개수. 고른 값이 아니며 안쪽에서 바꿀 수 없다	`K_`	`declare -rx`, 읽기 전용 마운트
매개변수	고른 값 또는 표준이 정한 값. 호출 때 확정되는 복사본	`P_`	함수 인자, 자식 프로세스가 물려받는 환경 변수
불리언 수축	결과를 참거짓 하나로 모아 원인 정보가 사라진다	`v_`	종료 코드 `$?`, `grep -q`, `nc -z`
역방향 탐색	수축된 결과에서 원인을 되짚는다	`r_`	`journalctl`, `tcpdump`, `cmp`, `strace`

H-2. 철학의 근거: 왜 계산과 전역 상수는 만나지 못하는가

1. 매개변수(격자 간격 h, 걸음 τ, 걸음 수, 계수, 요소 모양)는 정확 산술에서 **유리수**다.
2. 구간마다 적분한 결과는 **유리수 계수 다항식 p**다.
3. 전역 상수 e, π는 **초월수**(에르미트 1873, 린데만 1882)이므로 p(e) ≠ 0, p(π) ≠ 0이다. 조립제법의 나머지가 0이 되지 않는다.

4. 참 해의 적분 $(e^{\lambda\tau}-1)/\lambda$도 $\lambda\tau$가 0이 아닌 대수적 수이면 초월수(린데만–바이어슈트라스)라서 유리수와 같아질 수 없다.
5. 그래서 적분 항등식은 없고 **부등식 구간**만 남는다. 계산과 전역 상수는 비교의 자리에서만 만난다.
6. 항등식이 살아남는 곳은 한 바퀴를 온전히 도는 경우($\int_0^{2\pi} e^{it}dt = 0$, $e^{i\pi}+1 = 0$)뿐인데, τ가 유리수라 π의 정수배에 닿지 못한다. 오일러 항등식은 계산 안의 항등식이 아니라 **판정의 기준점**으로만 선다.

> **"만난다"의 정의를 앞에 둘 것.** 이 문서에서 "만난다"는 **정확한 등식으로 일치함**을 뜻합니다. 그 수준에서 위 정리가 불가능함을 증명하므로, 정의를 결론에 맞춰 고른 것이 아닙니다. 호출 지점의 값 전달(복사)은 이 정의의 만남이 아닙니다. 적용 범위는 상수 계수 선형계와 유리수 입력이며, 비선형계는 구간마다 야코비안을 얼린다는 가정(문서 1.7절)이 추가로 필요합니다.

찾은 곳	내용
검증기_철학_취합2 1절	상수는 계산에서 교집합이 없는 일방향 벡터. 일방향으로 흘러온 계산은 오일러 분해의 패턴에 대어 판정한다.
검증기_철학_취합2 2절	일방향 벡터 표, 조립제법 논증, 오일러 분해의 패턴표, 내적·외적(취합2에서 추가된 절)
검증기_철학_취합2 4절	확인은 앞으로 가서 닫히고 찾기는 거꾸로 가서 열려 있다. "문서의 명제가 아니라 저자의 입장."
검증기_철학_취합2 5절	"패턴 자체가 초월적이기 때문에 적분 항등식이 없다." 항등식이 사는 한 곳(한 바퀴)
본문 / 본문(작은거)	추출 텍스트가 완전히 동일. 본문에는 '일방향'과 '조립제법'이 0회 등장(철학 단락을 본문에 올리는 일이 남아 있음)
유지보수_261103_032620 (1)	텍스트가 추출되지 않아 읽지 못함(이미지로 보임)

H-3. '패턴'이라는 단어의 세 가지 쓰임

구분	출처	가리키는 것	성질
오일러 패턴	검증기_철학_취합(2) 1·2·5절	$t^k e^{\lambda t}$의 분해(성장·감쇠, 회전, 주기, 다항식 배수, 기준 값)	연속적·초월적
활성 패턴	본문 20.4 근처, 코드10 `pattern = []`	ReLU 뉴런의 켜짐/꺼짐 조합(은닉 폭 H이면 최대 2^H)	이산적·유한, 결정 열거 가능
디자인 패턴	세션_보안_파이프라인_전체	프론트 시간 이산화 값에서 읽는 선형 추세	관측된 경향

세 쓰임의 성질이 정반대라서 "패턴에 대어 판정한다"가 어느 쪽인지 독자가 가릴 수 없게 될 수 있습니다. 용어표를 두고 "오일러 패턴 / 활성 패턴 / 디자인 패턴"으로 접두어를 붙여 구분하세요. 이 부록의 H-4 이후에서 말하는 패턴은 **오일러 패턴**입니다.

H-4. 패턴 종류별 대응 (오일러 패턴표 기준)

패턴	OSI·리눅스에서 나타나는 곳	관측 명령	이상이면 의심
성장·감쇠	TCP 재전송 백오프, SYN 재전송(1·2·4·8초…), 혼잡 윈도우 초기 증가(RTT마다 2배), load average(1·5·15분 지수 감쇠 이동평균)	`ss -ti`, `tcpdump -ttt`, `nstat`, `uptime`	간격이 일정하면 백오프 미작동(L4), 감쇠 없이 상승하면 부하 누적
회전	ARP 이웃 상태 순환 (REACHABLE→STALE→DELAY→PROBE), TCP 상태 머신, 링크 플래핑	`ip neigh`, `ss -tan`, `carrier_changes`	한 상태에 고착(INCOMPLETE), 순환이 멈추지 않음(L1·L2)
주기·바퀴수	TCP keepalive(7200초 후 75초 간격 9회), DHCP 갱신(수명의 50%, 87.5%), TIME_WAIT 60초, ntp 폴링(64~1024초)	`sysctl net.ipv4.tcp_keepalive_*`, `journalctl`	주기·횟수가 틀어짐(설정 오류, 중간 장비, L5)
다항식 배수	TTL이 홉마다 1씩 감소, 시퀀스 번호·바이트 카운터의 선형 증가, DNS 캐시 TTL 카운트다운	`traceroute`, `ip -s link`, `dig`	기울기 급변, 카운터 정체(L2·L3)
기준 값	종료 코드 0, 체크섬 검증 시 정해진 값, ping 손실 0%, 인터페이스 UP	`$?`, `ping -c`, `ip -br link`	기준 값에서 벗어남(판정이 refuted인 지점)

주의: 오일러 패턴과의 대응은 **모양의 유비**입니다. "회전"은 복소수 회전이 아니라 상태가 순환한다는 뜻이고, TTL 감소는 지수가 아니라 선형(t^1)입니다. 혼잡 윈도우는 지수로 늘다가 임계점 이후 선형으로 바뀌어 한 현상 안에 두 패턴이 이어 붙습니다.

관측값에는 노이즈가 있으므로 패턴 검증기는 등식이 아니라 **허용 구간**으로 판정합니다. 지수 백오프 검증기(연속 간격의 비가 2±25% 안인가)의 실행 결과:

입력(초)	판정
1 2 4 8 16 32	O
1.1 1.9 4.2 7.8 (노이즈 포함)	O
1 1 1 1 (간격 일정)	X
1 2 4 4 (중간에 멈춤)	X
3 6 12	O

```
: "${P_PAT_BACKOFF_BASE:=2}" "${P_PAT_TOL:=0.25}"   # 고른 값(매개변수)
# 연속 간격의 비가 base±tol 구간 안인가 (등식이 아니라 구간으로 판정)
v_pat_backoff() {
  awk -v b="$P_PAT_BACKOFF_BASE" -v t="$P_PAT_TOL" 'BEGIN{ok=1}
    NR>1{ r=$1/prev; if (r<b*(1-t) || r>b*(1+t)) ok=0 } {prev=$1}
    END{exit !(ok && NR>1)}' <(printf '%s\n' "$@")
}
```

H-5. OSI 7계층 × 변수 접두사 (layers.env)

계층	전역 상수 `K_` (계산한 개수) · 매개변수 기본값 `P_` (고른 값)	검증기 `v_`	호출 인자 `P_`
L1	(없음)	`v_l1` 링크 UP	`P_IF`
L2	`K_N_MAC` / `P_L2_ETHERTYPE_ARP` (0x0806), `P_L2_MTU` (1500)	`v_l2` ARP 실패 없음	`P_IF`
L3	`K_N_TTL` / `P_L3_PROTO_ICMP` (1), `P_L3_TTL` (64)	`v_l3` ping	`P_TARGET`
L4	`K_N_PORT` (포트 범위 검사) / `P_L4_PROTO_TCP` (6), 포트 22·80·443	`v_l4` 포트 연결	`P_TARGET`, `P_PORT`
L5	`P_L5_KEEPALIVE_SEC` (7200)	`v_l5` keepalive 설정	(없음)
L6	`P_L6_CERT_WARN_SEC` (86400), `P_L6_CHARSET`	`v_l6` 인증서 만료	`P_HOST`, `P_PORT`
L7	`P_L7_HTTP_OK` (200), `P_L7_PORT_DNS` (53)	`v_l7` HTTP 200	`P_URL`

실행 결과 (루프백 환경):

상황	결과
정상 서비스	전 계층 정상 (이 환경에 ping이 없어 L3은 "측정 불가"로 건너뜀)
닫힌 포트	원인 후보: L4
없는 인터페이스	원인 후보: L1
상수 변경 시도 `K_N_PORT=1`	`readonly variable` 오류, 값은 65536으로 유지
매개변수 변경	함수 안에서만 바뀌고 바깥 상수는 그대로(복사본)

수축으로 정보가 사라지는 일이 실제로 있었습니다. 처음에는 L3 검증기가 FAIL을 냈는데 원인은 망 장애가 아니라 `ping` 명령이 없었기 때문이었습니다. FAIL 하나로는 "도구 없음"과 "망 장애"를 구분할 수 없어서, 도구가 없으면 건너뛰도록 고쳤습니다.

```
#!/usr/bin/env bash
# layers.env — OSI 7계층 × 변수 분리
#   N_ = 분할(영역 이름)      K_ = 전역 상수(조합론 점화식으로 계산한 값, 읽기 전용)
#   P_ = 매개변수(고른 값·표준이 정한 값, 호출 때 확정)   v_ = 검증기(결과를 참/거짓 하나로 수축)
#   r_ = 역탐색(수축된 결과에서 원인 계층을 되짚음)

# ── N_: 분할 ──────────────────────────────────────────
declare -rx N_C=client N_S=server

# ── K_: 전역 상수 (파스칼 점화식으로 계산한 개수. lab.env 의 _ROW/_SUM/_C/_K 를 먼저 source) ──
_K K_N_PORT $(_SUM 16)    # 포트 개수          sum C(16,r)
_K K_N_TTL  $(_SUM 8)     # TTL 값 개수         sum C(8,r)
_K K_N_MAC  $(_SUM 48)    # MAC 주소 개수      sum C(48,r)

# ── P_: 매개변수 (표준·관습이 정한 값. 환경 변수로 덮어쓸 수 있음) ──
: "${P_L2_ETHERTYPE_ARP:=0x0806}" "${P_L2_MTU:=1500}"
: "${P_L3_PROTO_ICMP:=1}" "${P_L3_TTL:=64}"
: "${P_L4_PROTO_TCP:=6}" "${P_L4_PORT_SSH:=22}" "${P_L4_PORT_HTTP:=80}" "${P_L4_PORT_HTTPS:=443}"
: "${P_L5_KEEPALIVE_SEC:=7200}"
: "${P_L6_CERT_WARN_SEC:=86400}" "${P_L6_CHARSET:=UTF-8}"
: "${P_L7_HTTP_OK:=200}" "${P_L7_PORT_DNS:=53}"

# ── v_: 검증기 (순방향 확인. 반환값은 0/1 하나뿐 = 원인 정보가 사라짐) ──
```

```
v_l1() { local P_IF=$1;  [[ $(cat "/sys/class/net/$P_IF/operstate" 2>/dev/null) =~ ^(up|unknown)$ ]]; }
v_l2() { local P_IF=$1;  ! ip neigh show dev "$P_IF" 2>/dev/null | grep -qE 'FAILED|INCOMPLETE'; }
v_l3() { local P_TARGET=$1; ping -c1 -W1 "$P_TARGET" &>/dev/null; }
v_l4() { local P_TARGET=$1 P_PORT=$2; (( P_PORT >= 0 && P_PORT < K_N_PORT )) || return 2; timeout 3 bash
-c ">/dev/tcp/$P_TARGET/$P_PORT" 2>/dev/null; }
v_l5() { [[ $(sysctl -n net.ipv4.tcp_keepalive_time 2>/dev/null || echo 99999) -le $P_L5_KEEPALIVE_SEC
]]; }
v_l6() { local P_HOST=$1 P_PORT=$2
         echo | timeout 5 openssl s_client -connect "$P_HOST:$P_PORT" -servername "$P_HOST" 2>/dev/null \
           | openssl x509 -noout -checkend "$P_L6_CERT_WARN_SEC" &>/dev/null; }
v_l7() { local P_URL=$1; [[ $(curl -s -o /dev/null -m5 -w '%{http_code}' "$P_URL") == "$P_L7_HTTP_OK" ]];
}

# ── r_: 역탐색 (L1→L7 순서로 확인해 첫 실패 계층을 원인 후보로 반환) ──
r_trace() { local P_IF=$1 P_TARGET=$2 P_PORT=$3 P_URL=$4 L
  for L in 1 2 3 4 5 6 7; do
    case $L in
      1) v_l1 "$P_IF" ;;
      2) v_l2 "$P_IF" ;;
      3) command -v ping >/dev/null || { echo "L3 건너뜀: ping 없음(측정 불가 ≠ 장애)"; continue; }
         v_l3 "$P_TARGET" ;;
      4) v_l4 "$P_TARGET" "$P_PORT" ;;
      5) v_l5 ;;
      6) [[ $P_PORT != "$P_L4_PORT_HTTPS" ]] || v_l6 "$P_TARGET" "$P_PORT" ;;
      7) v_l7 "$P_URL" ;;
    esac || { echo "원인 후보: L$L"; return "$L"; }
  done
  echo "전 계층 정상"
}
```

H-6. ASCII 다리: L6~L7 텍스트 층

모든 계층의 헤더는 바이트이고, HTTP·SMTP·FTP는 ASCII 텍스트이며, 리눅스 파이프는 바이트를 흘립니다. 그래서 바이트·문자 수준에서 변수를 정의하면 OSI와 리눅스가 같은 변수로 묶입니다.

ASCII의 요소	변수분리의 역할	OSI에서의 자리	리눅스에서 보는 법
128개 코드표(7비트)	전역 상수 `K_ASCII_SIZE`	L6 표현	`man ascii`, `locale charmap`
문자·바이트 값	매개변수 `P_BYTE`	L7 페이로드	`od -c`, `tcpdump -A`
제어 코드 ACK(6), NAK(21), SYN(22), EOT(4)	고른 코드값(`P_ASCII_*`)	초기 전송 프로토콜(BSC 등)의 제어, L2·L5의 확인·동기 개념	`cat -v`, `od -c`
패리티 비트	불리언 수축(1비트, 다대일)	L1·L2 오류 검출	`parity_even`
대소문자 차이 32, 숫자 '0'+n	선형 패턴(다항식 배수)	L7 텍스트 파싱	`tr 'a-z' 'A-Z'`
ROT13	회전·주기(두 번 적용하면 항등)	(유비)	`tr 'A-Za-z' 'N-ZA-Mn-za-m'`
CRLF(13,10)와 LF(10)	분할 경계의 불일치	L7 HTTP(CRLF) ↔ 리눅스(LF)	`cat -A`, `dos2unix`
7비트 밖(UTF-8 다바이트)	`v_ascii` 거짓	L6 인코딩	`iconv`, `file -i`
첫 번째로 어긋난 바이트	역방향 탐색 `r_firstdiff`	전 계층	`cmp`, `diff`, `od`

실행 결과: A=65, a=97(차이 32), Ctrl+A=0x01(= A & 31), `v_ascii 65` 참·`v_ascii 200` 거짓, 짝수 패리티 비트 A=0·C=1, ROT13 `Hello → Uryyb → Hello`, HTTP 요청의 CR·LF가 각 3개, `cmp` 가 `ABCD` 와 `ABXD` 의 어긋난 위치를 "char 3"으로 보고, 한(韓)은 `ed 95 9c` (3바이트, 모두 127 초과).

```
# K_: 전역 상수(계산한 개수)  P_: 매개변수(코드표가 정한 값, 검사할 바이트/문자열)  v_: 검증기  r_: 역탐색
_K K_ASCII_SIZE $(_SUM 7)     # 7비트 코드표의 문자 개수 sum C(7,r) = 128
: "${P_ASCII_CASE_BIT:=32}" "${P_ASCII_CTRL_MASK:=31}"
: "${P_ASCII_ACK:=6}" "${P_ASCII_NAK:=21}" "${P_ASCII_SYN:=22}" "${P_ASCII_EOT:=4}" "${P_ASCII_LF:=10}"
"${P_ASCII_CR:=13}"

v_ascii()  { local P_BYTE=$1; (( P_BYTE >= 0 && P_BYTE < K_ASCII_SIZE )); }           # 7비트 범위인가
parity_even() { local P_BYTE=$1 n=0 i; for ((i=0;i<7;i++)); do (( n += (P_BYTE>>i)&1 )); done; echo $((n%
2)); }
v_parity() { local P_BYTE=$1 P_BIT=$2; [[ $(parity_even "$P_BYTE") == "$P_BIT" ]]; }  # 1비트로 수축되는 검사
r_firstdiff() { local P_A=$1 P_B=$2; cmp "$P_A" "$P_B" 2>&1 | head -1; }              # 어긋난 첫 바이트 위치
```

H-7. 구조체·컴파일러·어셈블리 다리: L2~L4 바이너리 헤더

L1~L4의 헤더는 텍스트가 아니라 바이너리라서 ASCII로 닿지 못합니다. C 구조체와 컴파일러가 그 틈을 메웁니다.

변수분리	C 컴파일러 쪽	실행으로 확인된 것
표준이 정한 값 `P_`	컴파일 타임에 고정하는 값: `sizeof`, `offsetof`, `#define`, `_Static_assert`	이더넷 14, IP 20, TCP 20바이트와 필드 오프셋(EtherType 12, IP protocol 9, IP check 10, TCP window 14)이 모두 통과. 어긋나면 컴파일 자체가 실패
런타임 매개변수 `P_`	런타임에 구조체 필드에 채우는 값	MAC, IP(10.0.0.1→10.0.0.2), 포트(40000→8080), TTL 64, SYN 플래그
불리언 수축 `v_`	체크섬 검증	정상 헤더의 접은 합이 `0xffff`로 닫힘, TTL 1바이트만 바꾸면 `0x1000`으로 어긋남
역탐색 `r_`	어긋난 바이트 위치 찾기	`cmp`, `od`, `tcpdump -XX`

전선 위의 SYN 패킷 (54바이트)

```
ff ff ff ff ff ff 02 00 00 00 00 01 08 00          ← 이더넷(14) L2: 목적지·출발지 MAC, EtherType 0x0800
45 00 00 28 00 00 00 00 40 06 66 ce 0a 00 00 01 0a 00 00 02   ← IP(20) L3: 버전4·길이5, TTL 64, 프로토콜 6, 체크섬
9c 40 1f 90 00 00 00 00 00 00 00 00 50 02 ff ff 00 00 00 00   ← TCP(20) L4: 포트 40000→8080, 플래그 0x02(SYN)
```

엔디언과 어셈블리

전선에는 EtherType이 `08 00`으로 실리지만 리틀 엔디언 호스트가 그대로 읽으면 `0x0008`이고, `ntohs`를 거쳐야 `0x0800`이 됩니다. 이 변환은 어셈블리에서 `rolw $8, %ax`(16비트를 8칸 회전) 한 줄입니다. 바이트 순서 교환은 8비트 회전이고 두 번 적용하면 항등입니다(ROT13과 같은 구조). 네트워크 바이트 순서(전선)와 호스트 바이트 순서(메모리)를 분리해 두는 것이 변수분리의 가장 낮은 층 구현입니다.

컴파일은 일방향(다대일 수축)

`x*2`, `x+x`, `x<<1`은 소스가 서로 다르지만 **컴파일 결과가 3개 모두 같은 기계어**였습니다. 기계어에서 원래 소스를 정확히 복원할 수 없습니다(변수명, 주석, 쓰는 방식이 사라짐). 불리언 수축·노름과 같은 구조이며 "찾기는 열려 있다"는 문서의 문장과 맞습니다.

한계: 비트필드의 배치는 컴파일러와 아키텍처가 정합니다. 리눅스 헤더(`linux/ip.h`)가 `__LITTLE_ENDIAN_BITFIELD`와 `__BIG_ENDIAN_BITFIELD`로 구조체 정의를 둘로 나누는 것이 그 증거이며, 이식 가능한 코드는 비트필드 대신 시프트와 마스크를 씁니다. 규약의 일부를 컴파일러가 쥐고 있어서 완전히 닫히지는 않습니다.

```c
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>
#include <string.h>
#include <arpa/inet.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>

/* K_: 컴파일 타임 상수. 규약과 어긋나면 컴파일 자체가 실패한다 */
_Static_assert(sizeof(struct ethhdr) == 14, "eth");
_Static_assert(offsetof(struct ethhdr, h_proto) == 12, "eth.proto");
_Static_assert(sizeof(struct iphdr) == 20, "ip");
_Static_assert(offsetof(struct iphdr, protocol) == 9, "ip.proto");
_Static_assert(offsetof(struct iphdr, check) == 10, "ip.check");
_Static_assert(sizeof(struct tcphdr) == 20, "tcp");
_Static_assert(offsetof(struct tcphdr, window) == 14, "tcp.win");

/* 빅엔디언으로 16비트 워드를 읽어 접는 합 (호스트 엔디언과 무관) */
```

```
static uint16_t fold(const void *p, size_t n) {
  const uint8_t *b = p; uint32_t s = 0;
  for (size_t i = 0; i + 1 < n; i += 2) s += (b[i] << 8) | b[i + 1];
  while (s >> 16) s = (s & 0xffff) + (s >> 16);
  return (uint16_t)s;
}

int main(void) {
  uint8_t pkt[sizeof(struct ethhdr) + sizeof(struct iphdr) + sizeof(struct tcphdr)];
  memset(pkt, 0, sizeof pkt);
  struct ethhdr *e = (void *)pkt; struct iphdr *ip = (void *)(e + 1); struct tcphdr *t = (void *)(ip +
1);

  /* P_: 런타임 매개변수 (호출마다 달라지는 값) */
  memcpy(e->h_dest, "\xff\xff\xff\xff\xff\xff", 6); memcpy(e->h_source, "\x02\x00\x00\x00\x00\x01", 6);
  e->h_proto = htons(ETH_P_IP);
  ip->version = 4; ip->ihl = 5; ip->ttl = 64; ip->protocol = IPPROTO_TCP;
  ip->tot_len = htons(sizeof *ip + sizeof *t);
  ip->saddr = htonl(0x0a000001); ip->daddr = htonl(0x0a000002);
  ip->check = 0; ip->check = htons((uint16_t)~fold(ip, sizeof *ip));
  t->source = htons(40000); t->dest = htons(8080); t->doff = 5; t->syn = 1; t->window = htons(65535);

  printf("[구조체] eth=%zu ip=%zu tcp=%zu  (모두 _Static_assert 통과)\n", sizeof(struct ethhdr), sizeof(stru
ct iphdr), sizeof(struct tcphdr));
  printf("[전선 순서] ");
  for (size_t i = 0; i < 14 + 20 + 20; i++) printf("%02x%s", pkt[i], (i == 13 || i == 33) ? " | " : " ");
  printf("\n[엔디언] 전선의 EtherType 바이트=%02x %02x, 호스트가 그대로 읽은 값=0x%04x, ntohs 후=0x%04x\n", pkt[1
2], pkt[13], e->h_proto, ntohs(e->h_proto));
  printf("[기준 값 닫힘] IP 헤더 접은 합(체크섬 포함)=0x%04x (0xffff면 정상)\n", fold(ip, sizeof *ip));
  printf("[플래그] TCP 13번째 바이트=0x%02x (SYN=0x02)\n", pkt[14 + 20 + 13]);
  ip->ttl ^= 0x10;
  printf("[1바이트 변조 후] 접은 합=0x%04x → %s\n", fold(ip, sizeof *ip), fold(ip, sizeof *ip) == 0xffff ? "정
상" : "불일치(검증기 FAIL)");
  return 0;
}
```

H-8. 격자점과 점화식: L1의 디지털 부분

L1의 연속 신호를 시간 격자점(한 심볼에 8점, 격자 간격 h = T/8)으로 쪼개면 수신기의 일이 그대로 `for`(구간 평균)와 `if`(문턱 비교)가 됩니다. 잡음 섞인 NRZ 파형으로 4만 비트를 복원한 결과, 시뮬레이션과 이론이 허용 구간 안에서 일치했습니다.

잡음 σ	시뮬레이션 비트오류율	이론값	구간 안
0.5	0.0000	0.0000	O
1.0	0.0024	0.0023	O
1.5	0.0285	0.0297	O
2.0	0.0778	0.0786	O

점화식은 L2에서도 이미 쓰입니다. 이더넷 FCS인 CRC32를 비트 단위 시프트 재귀로 계산한 값이 표준 확인값 `0xCBF43926`(입력 "123456789")과 일치했고 `zlib`과도 일치했습니다. 글자 하나를 바꾸면 `0xB2288182`로 달라져 검증기가 FAIL을 냅니다.

경계: 위 코드는 L1을 **모의한 것**이지 실제 케이블의 전압을 `for` 문이 움직이는 것은 아닙니다. 리눅스에서 실제 PHY에 닿는 명령은 `ethtool`(속도·듀플렉스·오토네고)과 드라이버가 노출한 레지스터 정도입니다. 또 아날로그→디지털 경계(표본화·양자화)가 곧 다대일 일방향 수축이며, 격자 근사의 오차는 구간 부등식으로 따로 가둬야 합니다.

```
import random, math, zlib
# K_: 전역 상수
K_SPS = 8                 # 한 심볼당 격자점 수 (격자 간격 h = T/SPS)
K_THR = 0.0               # 판정 문턱
K_CRC32_POLY = 0xEDB88320 # 이더넷 FCS 다항식(반사형)
K_TOL = 0.15              # 이론값과의 허용 상대 구간

def tx(bits):                         # 비트 -> 격자점 위의 파형 (NRZ: 1=+1, 0=-1)
    return [(1.0 if b else -1.0) for b in bits for _ in range(K_SPS)]

def rx(samples):                      # for/if 로 비트 복원 (구간 평균 후 문턱 비교)
    out = []
    for k in range(0, len(samples), K_SPS):
        m = sum(samples[k:k+K_SPS]) / K_SPS
        out.append(1 if m > K_THR else 0)
    return out

def ber(sigma, n=40000, seed=7):      # P_: 잡음 세기 sigma
    r = random.Random(seed)
    bits = [r.getrandbits(1) for _ in range(n)]
    wave = [v + r.gauss(0, sigma) for v in tx(bits)]
    got = rx(wave)
    return sum(a != b for a, b in zip(bits, got)) / n

def ber_theory(sigma):                # 구간 평균 뒤 잡음 표준편차 = sigma/sqrt(SPS)
    return 0.5 * math.erfc((1.0 / (sigma / math.sqrt(K_SPS))) / math.sqrt(2))

print("[L1 모의] 격자점+for/if 로 비트 복원, 시뮬레이션 vs 이론 (구간 판정)")
for s in (0.5, 1.0, 1.5, 2.0):
    a, b = ber(s), ber_theory(s)
    ok = abs(a - b) <= K_TOL * b + 3e-4
    print(f"  sigma={s:<3}  모의 BER={a:.4f}  이론={b:.4f}  구간 안={'O' if ok else 'X'}")

def crc32_bitwise(data):              # 점화식: 비트마다 if 로 갈라지는 시프트 재귀
    crc = 0xFFFFFFFF
    for byte in data:
        crc ^= byte
        for _ in range(8):
            crc = (crc >> 1) ^ K_CRC32_POLY if crc & 1 else crc >> 1
```

```
    return crc ^ 0xFFFFFFFF

v = crc32_bitwise(b"123456789")
print(f"[L2 FCS] 점화식 CRC32('123456789')=0x{v:08X}  표준 확인값 0xCBF43926 일치={v==0xCBF43926}  zlib 일치={v
==zlib.crc32(b'123456789')}")
m = crc32_bitwise(b"123456780")
print(f"[1글자 변조] 0x{m:08X} → 불일치로 FAIL (다대일 수축 중 하나가 걸러짐)")
```

H-9. 층별 요약

층	다리	닿는 계층	상태
ASCII	코드표, 제어 문자, 텍스트 프로토콜	L6~L7	메워짐
C 구조체	헤더 필드 오프셋·크기·상수	L2~L4	메워짐(비트필드 순서는 컴파일러 의존)
컴파일러·어셈블리	컴파일 타임 상수와 런타임 값의 분리, 엔디언, 다대일 수축	구현 전반	메워짐
격자점·점화식	`for` / `if` 로 심볼 복원, CRC 재귀	L1 디지털, L2	메워짐(모의)
L5 세션	keepalive 설정 검사 정도	L5	회전(i)·주기(π) 요소로 정의(H-10)
L1 아날로그 매질	양자화·격자 오차의 상한 구간	L1	구간 부등식으로 가둠(H-10)

H-10. 오일러 분해 요소로 틈 메우기

문서의 오일러 분해 $t^k e^{\lambda t} = t^k e^{at}(\cos bt + i \sin bt)$와 $e^{i\pi}+1 = 0$의 요소(e: 길이·성장·감쇠, i: 회전, π: 주기·각도·바퀴수, 0·1: 닫힘의 기준, t^k: 다항식 배수)로 OSI·리눅스에 남아 있던 틈을 **정의**했습니다. 정의는 판정보다 앞에서 선언하는 설계 결정입니다.

판정 형태의 이분법 (문서 5절을 그대로 옮김)

양쪽의 값	OSI·리눅스에서	판정
정확한 값 ↔ 정확한 값	바이트, 헤더 필드, 상태값, 포트, 체크섬	**정확한 등식** (`cmp` , `==` , 접은 합 `0xffff`)
정확한 값 ↔ 연속 참값	지연, 손실률, 재전송 간격, 신호 전압, 부하	**부등식 구간** (허용 오차 `P_ORA_TOL_MS` , 오차 상한)

오일러 요소별 대응

틀	오일러 요소	메우는 정의
관측값 노이즈, L1 아날로그	e (길이, 나머지 사다리)	값은 구간 [L, U]로 가둔다. 양자화 오차 $\|x-q(x)\| \le \Delta/2$, 격자 간격 h의 사다리꼴 적분 오차 $\le (b-a)h^2/12 \cdot \max\|f''\|$. 상한이 없는 잡음은 확률 구간(비트오류율의 이론 구간)으로 가둔다.
데이터 경로의 양방향 (캡슐화 ↔ 역캡슐화)	한 바퀴 돌 때만 사는 항등식 ($\int_0^{2\pi} e^{it}dt = 0$, $e^{i\pi}+1 = 0$)	계층을 내려갔다 올라오는 한 바퀴는 정확한 등식(항등)으로 판정한다. 문서에서 항등식이 사는 한 곳과 같은 자리이고, 그 바깥의 측정값은 구간으로 판정한다.
엔디언, 비트필드 배치	π (반 바퀴)	바이트 순서 교환 `rolw $8` 은 16비트 고리의 반 바퀴이고 두 번 적용하면 항등이다. 비트 배치는 시프트와 마스크로 한 방향을 `P_` 값으로 고정한다.
L5 세션 (독립 계층 없음)	i (회전) + π (주기·바퀴 수)	`v_l5` := 세션 상태 순환(수립 → 유지 → 종료)이 `P_` 주기(keepalive 7200·75·9 등) 안에 한 바퀴를 닫는가.
교차 계층 (`iptables` , `conntrack` , MTU, DNS)	전역 상수 (모든 영역이 읽는 e·π·i)	여러 영역에 걸치는 규약 중 개수로 계산되는 값(포트·MAC·TTL 개수)은 `K_` 읽기 전용 교집합으로 두고, 고른 값(MTU·포트 번호)은 영역마다 `P_` 복사본으로 전달한다. 영역은 서로소, 교집합은 상수뿐이다.
명령 하나가 여러 계층을 봄	오일러 분해 (성분의 합)	한 관측을 성분으로 분해해 계층에 배분한다. 예: `curl -w` 의 `time_namelookup` (DNS), `time_connect` (L4), `time_appconnect` (L6 TLS), `time_starttransfer` (L7).
패턴 판정	e^{at}, i, π, t^k, 0·1	H-4의 다섯 패턴표 그대로 `v_pat_*` 로 구간 판정한다.

이 대응은 정의(설계 결정)입니다. 판정에 앞서 선언해 두면 결론에 맞춰 고른 것으로 읽히지 않습니다. P≠NP로 가는 연결은 여기서 정의한 대응이 아니라 H-11의 대응 가정으로 따로 둡니다.

H-11. P≠NP 서술의 위치와 확인이 필요한 간극

문서는 P vs NP를 "2026년 11월 현재 미해결"이라 쓰고 해당 절을 "문서의 명제가 아니라 저자의 입장"으로 두었습니다. 이 표기를 유지하는 것이 적절합니다. 이 부록의 대응으로 말할 수 있는 최대치는 다음 두 문장입니다.

구조에 대한 서술(사실). 이 구조에서 실행 범위들은 서로소인 영역으로 분할된다. 전역 상수는 모든 영역이 읽을 수만 있는 값이고, 매개변수는 영역 사이를 한 방향으로만 건너는 값이다. 둘 모두 안쪽에서 바깥의 원본을 변경하는 경로가 없으므로, 이 구조의 값 이동은 쓰기 방향의 역연산을 갖지 않는다.

관점(은유). 정보가 한 방향으로만 흐르고 참거짓으로 수축되는 구조에서는 순방향 확인과 역방향 복원이 대칭이 아니며, 이 비대칭은 P와 NP의 구분이 가진 구조와 닮아 있다. 본 부록은 이를 P≠NP의 구조적 은유로 제시한다.

간극	내용
1. 체(field)가 다름	MQ(다변수 이차 연립방정식)가 NP-완전인 것은 **유한체** 위에서다. 계산은 유리수·실수 위에서 일어나므로, 임의의 MQ 사례를 한 걸음의 역풀이로 심는 구성(환원)이 필요하다.
2. 최악 경우 vs 일방향 함수	NP-완전성은 **최악 경우**의 어려움이다. 일방향 함수는 평균적으로 거의 모든 입력에서 거꾸로 구하기 어려운 함수이며, 그 존재가 P≠NP를 함의한다(역은 알려져 있지 않음). MQ가 NP-완전하다는 것만으로는 일방향 함수의 존재가 따라 나오지 않는다.
3. 접근 불가 ≠ 계산 난이도	"되돌아가는 경로가 없다(쓰기 불가)"와 "되돌리는 데 시간이 많이 든다"는 다른 성질이다. 앞의 대응에서 '대응 가정'을 별도 문단으로 분리해 두어야 한다.
4. 독립변수(키)의 도입	키 같은 독립변수를 넣으면 키를 가진 쪽에는 역연산이 생긴다(함정문). "역연산이 없다"는 말은 관찰자가 가진 정보에 상대적인 성질로 고쳐 써야 하고, 암호는 P≠NP를 증명해 주는 것이 아니라 그 가정 위에 서 있다.

H-12. 1번 문서(ORacle) 개념 ↔ 2번 문서 전체 대응

1번 문서는 `ORacle-main.zip` 이고, 본문(102쪽: 검증기, 오라클 문제 일반화, 블록체인·ML 오라클), 코드 10권, 철학 취합 2종, 점화식 취합, 세션 보안 파이프라인, 개념 메모 5종, Readme로 이루어져 있습니다. 개념 39개를 뽑아 2번 문서(이 문서)의 16곳에 대응시켰습니다. 대응은 정의(설계 결정)이며, H-10과 같은 원칙으로 판정에 앞서 선언합니다.

대응의 큰 줄기

1. 1번 문서의 **일방향 벡터**(매개변수 → 계산, 전역상수 → 판정)는 2번 문서의 `P_` / `K_` 분리다.
2. 1번 문서의 **검사 순서 공간 → 시간 → 정확도**는 2번 문서의 L1~L3 → L4~L5 → L6~L7 진단 순서다.
3. 1번 문서의 **proved / refuted / 정의**는 2번 문서 검증기 `v_*` 의 0 / 1 / 범위 밖이다.
4. 1번 문서의 **육하원칙 보고**는 2번 문서 E-2 증거 수집이다.
5. 1번 문서의 **세션 보안 9단계**는 2번 문서의 L1~L7 흐름 위에 놓인다.

H-12-1. 개념 목록과 2번 문서에서의 대응

A. 철학·수학 기반

번호	1번 문서의 개념	출처	2번 문서에서의 대응
C1	구간 근사: 극한 대신 구간을 쪼갠다	철학 취합 §2, 본문 3·7	측정값을 점이 아니라 구간으로 판정한다(`K_ORA_TOL_MS` , 손실률·지연 임계). 극한 한 점 대신 양쪽 끝이 있는 부등식만 요구한다.
C2	일방향 벡터: 매개변수는 계산으로, 전역상수는 판정으로	철학 취합 §2·§4, 매개변수·P np 메모	`P_` 지역 복사본(`local P_X=$1`)과 `K_` 읽기 전용(`declare -gr` , 계산한 개수만). 값은 상수에서 매개변수로만 흐른다(H-1).
C3	서로소 분할과 변수분리	철학 취합 §3, 본문 1.2·1.7	`N_C` , `N_S` 네임스페이스 분할과 접두사 5종. 영역은 서로소이고 교집합은 `K_` 상수뿐이다.
C4	전역상수: 조합론적 이항계수, 세 조건(기원·불변·보편)	전역상수·ㅇㅇㅇㅇㅇ 메모	`lab.env` 의 `_SUM` · `_C` (파스칼 점화식)로 `K_N_PORT` , `K_N_IPV4` , `K_N_MAC` , `K_N_TTL` , `K_LINKS_LAB` 를 계산한다. MTU·포트 번호·IP처럼 고른 값은 `K_` 가 아니라 `P_` 다(H-14).
C5	매개변수: 확률과 무관한 경우의 수(정수형·실수형)	매개변수 메모	`P_PORT` (정수형), `P_TARGET` , 허용치(실수형은 정수 ms로 내려씀). 호출 때 값이 확정되어 불확실성이 없다.
C6	종속변수: 결과값은 전역상수와 서로소	종속변수 메모	명령의 출력(종료 코드, 지연 ms, HTTP 상태 코드)은 종속변수이고 `K_` 가 아니다. 틀 `K_` / 입력 `P_` / 결과 3층으로 나눈다.
C7	확률은 연분수 꼴의 독립시행	매개변수 메모	계층별 검증기 `v_l1` ~ `v_l7` 이 서로의 상태를 바꾸지 않는 독립 시행이라는 가정. 손실 패킷도 독립 시행으로 둔다(C32).
C8	오일러 분해 패턴표: e, i, π, t^k, 0·1	철학 취합 §2	성장·감쇠(백오프), 회전(ARP·TCP 상태 순환), 주기(keepalive), 다항식 배수(TTL), 기준값(종료 코드 0)(H-4, H-10).

B. 구간·등식 판정 장치

번호	1번 문서의 개념	출처	2번 문서에서의 대응
C9	나머지 사다리: $a_n = n \cdot a_{n-1} + 1$, $a_n < n! \cdot e < a_n + 1/n$	점화식 취합 A, 본문 5.1	정수 점화식으로 계산하고 구간에서만 비교한다. `oracle.env` 의 `ladder_a` , `v_ladder` . 재전송 간격($R_n = 2 \cdot R_{n-1}$)도 같은 꼴이다.
C10	적분 점화식: $I_n = I_{n-1} + c_n$	점화식 취합 B, 본문 7	누적 합 점화식: 인터페이스 바이트 카운터 누적(`ip -s link`), 인터넷 체크섬의 접은 합. 합은 정확 정수라 등식으로 판정한다.

C11	정확한 등식 ↔ 부등식 구간 (유리수 ↔ 유리수, 유리수 ↔ 초월수)	철학 취합 §5	정확한 값(MAC, 포트, 헤더 필드)은 `v_exact`, 연속 참값(지연, 손실률)은 `v_interval` (H-10).
C12	한 바퀴에서만 사는 항등식 ($\int e^{it}dt = 0$)	철학 취합 §5	캡슐화에서 역캡슐화까지 한 바퀴는 정확한 등식으로 판정한다(H-10).
C13	조립제법의 나머지는 0이 되지 않는다 (거슬러 닿지 못함)	철학 취합 §2	되돌릴 수 없는 단계: 체크섬, 해시, DROP, 컴파일(다대일 수축). 결과에서 원인을 복원하지 못한다(H-7).
C14	후보 하나와 검은 상자 (내부를 보지 않음)	본문 1.1	서비스 내부를 열지 않고 외부 출력만 검사: `curl`, `nc`, `openssl s_client`, `ping`.
C15	두 경로: A 고정 이산 공간, B 연속 공간	본문 1.2	경로 A = 랩(고정 격자 10.0.0.0/24, veth 두 개), 경로 B = 실망(가변 토폴로지, 클라우드).
C16	proved / refuted 두 불리언, 증명·반례·정의, 보류 없음	본문 1.4	`v_*`의 종료 코드 0/1 + 반례 출력. 측정할 수 없으면 "정의(범위 밖)"로 적는다(`ping` 없음 → L3 건너뜀, `v_ladder`의 n>10 → 2).
C17	증명의 근거는 적분과 연쇄곱뿐, 미분·선형화는 원인 찾기 보조	본문 1.5	합격 근거는 카운터 누적합과 정확 비교뿐. 부하 평균(`uptime`)과 추세는 원인을 찾는 보조로만 쓴다.
C18	반올림 엄밀성	본문 11	초 단위 부동소수는 정수 ms로 내려 비교한다(`curl -w` 값을 ms 정수로). 허용치에 반올림 여유를 포함한다.

C. 보고·국소화·역추적

번호	1번 문서의 개념	출처	2번 문서에서의 대응
C19	육하원칙 보고: 누가·무엇을·언제·어디서·왜·어떻게	본문 13.4	`oracle.env`의 `r_6w`. 누가 = 장비·계층, 무엇 = 증상, 언제 = 타임스탬프, 어디서 = NIC·포트, 왜 = 원인 계층, 어떻게 = 복구 명령(E-2).
C20	국소화: 몫의 합 = 한도 (항등식)	본문 13	`curl -w`의 성분(DNS, 연결, TLS, 첫 바이트)을 계층에 배분하면 합이 `time_total`과 같다(H-10).
C21	역추적과 처방	본문 10	`r_trace`(L1에서 L7까지 첫 실패 계층)와 0-4 정리·복구 명령.
C22	얼린 야코비안: 구간마다 선형으로 읽기	본문 1.7, 6.5	비선형 부하를 구간별로 얼려 읽는다: `sar` 구간 평균, TCP 슬로 스타트 구간의 증폭(e^{at}).
C23	동결 전파자 연쇄곱, 그린월형 누적	본문 9	계층 연쇄: 종단 성공 = 계층별 성공의 곱, 종단 지연 상한 = 계층별 상한의 합(`traceroute` 홉별 상한 누적).
C24	그래프 상수 L = (V, T), 수식과 그림의 왕복	본문 4.4	토폴로지 그래프: 마디 = 네임스페이스·호스트, 변 = veth·링크. `ip -br link`, `ip route`, `traceroute` 경로가 그래프 경로다.
C25	검사 순서: 공간 → 시간 → 정확도	본문 1.7	공간 = L1~L3(연결), 시간 = L4~L5(타이머·세션), 정확도 = L6~L7(내용·응답). `r_trace`의 L1→L7 순서와 같다.
C26	PINN: 출력이 명세를 어기는 정도를 상수로 명시	본문 17	서비스 출력이 SLO를 어기는 정도를 상수로 적는다: 응답 시간 상한, `P_L7_HTTP_OK=200`.

D. 오라클 문제의 확장 (제5·6부)

번호	1번 문서의 개념	출처	2번 문서에서의 대응
C27	오라클 문제 5층: 구현·공식·정확도·입력 일반성·주장 부재	본문 20.1	구현 = 설정이 의도대로인가(`ip addr` vs `P_`), 공식 = 정책이 주장대로인가(`iptables -S`), 정확도 = 응답이 기준 안인가, 입력 일반성 = 여러 포트·호스트로 반복, 주장 부재 = 명세 없는 서비스(부록 F).
C28	정확 재실행·기호 실행(S)·유한체 실행(G)·정수 추적(T), 슈바르츠–지펠 오판 상한	본문 20.2–20.3	정수 추적의 대표 크기 ↔ 경계 대표 포트(1, 1023, 1024, 65535). 오판 상한 ↔ 반복 시행: 손실률 p에서 `ping -c N` 이 놓칠 확률 $(1-p)^N$.
C29	값 분기: 결정 열거, 범위 밖(정의)	본문 20.4	`case` 로 분기별 열거: HTTP 상태 코드별 대응(7-2), 같음 분기의 특수 위치(TTL=1, 포트 경계)는 층마다 입력.
C30	변이 분석: 변이체를 검증기가 반례로 잡는가	본문 20.6	장애 주입 = 변이체: `iptables` DROP/REJECT, `netem` , MTU, `rp_filter` . 랩 전체가 변이 분석이다.
C31	판정 지도: 분야가 아니라 명세의 종류로 갈린다	본문 21.5	부록 F의 증상 → 계층표: 대수 등식(설정·헤더), 확률(손실률), 순서(상태 머신), 극한(백오프 수렴), 초월(시간), 반올림(계량).
C32	확률 명세: 큰수의 법칙, 비율 편차, δ-닫힘	본문 21.4	손실률 판정: `ping -c200` 의 손실 비율을 구간 [p̂−ε, p̂+ε]로 가두고 `netem loss 10%` 를 검출한다.
C33	블록체인 오라클 3층: 계산·집계·진실	본문 26	계산 층 = 체크섬·헤더 정확 계산, 집계 층 = 다중 DNS 리졸버·NTP 서버 다수결, 진실 층 = 체인 밖 실제 값과 응답의 대조.
C34	ML 오라클: 세계 증인, 분포 이동 검출	본문 28	일방향 장애 A(INPUT DROP)와 B(OUTPUT DROP)는 클라이언트 관측이 같다. 입력이 같은데 원인이 다른 두 세계가 곧 세계 증인이다(`v_world`). 분포 이동 = 기준선 대비 트래픽 변화(B-3).
C35	새 문제를 만났을 때: 질문 0~13 트리	본문 32	부록 F 판별표와 E-1 `diag.sh` : 질문 순서대로 분기해 계층을 고른다.

E. 세션 보안 파이프라인, 철학 메모, P≠NP

번호	1번 문서의 개념	출처	2번 문서에서의 대응
C36	9단계 흐름: 격자점 → 점화식 → 쿠키 → 프록시 → 세션 ID → AES → node → DB 조건문 → RSA	세션 보안 파이프라인, Readme	아래 표 H-12-3에서 L1~L7과 부록에 대응시킨다.
C37	참과 참이 만나면 거짓이 될 수 있고, 거짓과 거짓은 거짓	철학 메모	각각 올바른 두 설정이 합쳐져 장애: MTU 1500과 1400, ACCEPT와 DROP 규칙 순서, IP 중복(2-3). 두 장애가 겹치면 거짓이 거짓 위에 쌓인다.
C38	P≠NP 서술: 네 가지 이유와 불리언 수축 전제	P np 메모	H-11에 대응 가정으로 분리해 두었다. 이 절의 대응은 그 가정을 쓰지 않는다.
C39	오일러 항등식 $e^{i\pi}+1=0$, 0과 1의 닫힘	철학 취합 §2	기준값: 종료 코드 0, 링크 UP, HTTP 200, 체크섬 접은 합 `0xffff` .

H-12-2. 2번 문서의 장·부록 16곳에 대응시키기

각 곳에서 쓰는 개념 번호와 그 자리의 구체적 대응입니다. 0장과 7개 계층, 부록 A~H를 빠짐없이 다룹니다.

0장과 1~7계층

2번 문서의 곳	개념	그 자리의 대응
0. 실습 랩 구성	C2 C3 C4 C15 C24 C30	`lab.env` = `K_`(계산한 개수, 읽기 전용) + `P_`(고른 값). 랩은 경로 A(고정 격자). 마디 = 네임스페이스 2개, 변 = veth 1개의 그래프. 0-4 초기화가 기준점 복원이다.
1계층 물리	C1 C8 C10 C18	1-2 링크 속도는 정확한 값(`P_`), 광 수신 전력·온도는 구간. 1-3 플래핑 = 회전(i)과 주기(π). 오류 카운터는 누적 점화식 I_n.
2계층 데이터 링크	C11 C13 C24 C30 C37	MAC·ARP는 정확한 등식. 브리지 학습·ARP 캐시는 다대일. 2-3 IP/MAC 중복은 "참과 참이 만나 거짓". 2-4 `netem`은 변이체. 2-8·2-7 브리지와 본딩은 그래프.
3계층 네트워크	C8 C16 C23 C24 C34	3-1 서브넷은 정확 계산. 3-4 홉별 지연 상한을 누적(연쇄합). TTL은 다항식 배수(t^K). 3-5 비대칭 경로·`rp_filter`는 일방향. 3-3과 `ping` 없음은 정의(범위 밖).
4계층 전송	C9 C28 C29 C32 C34	4-4 SYN 재전송 1, 2, 4, 8초 = 나머지 사다리 점화식. 4-2 DROP vs REJECT는 값 분기 열거. 포트 경계 대표 입력. 4-8 SYN flood는 비율 편차. INPUT/OUTPUT DROP은 세계 증인.
5계층 세션	C2 C8 C10 C36	5-1 세션 타이머와 5-3 keepalive(7200·75·9) = 주기(π), 수립 → 유지 → 종료 순환(i). 쿠키(과거값)와 세션 ID(현재값)는 발급 방향으로만 흐르는 일방향 값(C36).
6계층 표현	C13 C18 C33 C36	6-2 인증서 만료는 구간(`-checkend`). 6-1 AES·TLS는 미래값을 넣어야 풀리는 단계. 6-4 신뢰 체인은 진실 층. 6-5 호스트키·서명은 RSA 스위칭 점검. 해시·서명은 다대일.
7계층 응용	C16 C19 C20 C29 C36	7-1 응답 시간 분해(`curl -w`)가 국소화, 합 = `time_total`. 7-2 상태 코드별 분기. 7-5 로그 분석이 육하원칙. 7-8 DB 질의의 조건문 점검.

부록 A~H

2번 문서의 곳	개념	그 자리의 대응
부록 A 시스템 자원	C10 C22 C23 C32	A-1 부하는 구간별로 얼린 선형(`sar`). 누적 카운터 I_n. A-2~A-4 한도 상수와 사용량의 비교는 구간 판정.
부록 B 패킷 캡처	C14 C20 C28 C34	캡처는 검은 상자 관찰. B-1 필터 술어가 대표 입력의 칸. B-3 기준선 대비 트래픽 변화 = 분포 이동 검출.
부록 C 컨테이너·클라우드	C3 C15 C24 C33	C-1 네임스페이스·cgroup = 분할. C-2 쿠버네티스 노드·파드는 그래프, 다수결(집계층). C-3 클라우드는 경로 B. C-4 오버레이 그래프.
부록 D 보안 점검	C13 C32 C34 C36 C37	D-2 조건문 주입 탐지(`OR 1=1`, `--`)가 9단계의 8번. D-3 DDoS는 비율 편차. DROP은 다대일. 정상 규칙과 정상 규칙의 충돌.
부록 E 자동화 스크립트	C16 C19 C21 C35	E-1 `diag.sh` = 질문 트리와 `r_trace`. E-2 증거 수집 = 육하원칙 보고(`r_6w`). 판정은 증명·반례·정의 중 하나.
부록 F 증상 → 계층	C25 C31 C35	증상(명세의 종류)으로 계층이 갈린다. 공간 → 시간 → 정확도 순서로 짚는다.
부록 G 한 장 요약	C4 C5 C39	계층별 핵심 명령 = `P_` 기본값 표와 `K_` 개수표. 기준값은 종료 코드 0.
부록 H 대응 총정리	C1~C39	H-0~H-11이 철학 취합의 §1~§6에, H-12가 본문·점화식·세션 파이프라인·메모 전체에 대응한다.

H-12-3. 세션 보안 9단계 ↔ L1~L7

9단계는 프론트에서 시작해 L7을 지나 node(L3~L4)로 내려가므로 계층 번호가 단조롭지 않습니다. 단계 순서대로 적었습니다.

단계	1번 문서의 단계	2번 문서의 자리	쓸 수 있는 명령·변수
1~2	디자인 패턴 → 격자점 → 점화식 이산화	L1·L2 (H-8)	신호·프레임 표본을 격자점으로 보고 1차 점화식을 맞춘다(`fit_recurrence`). `cbridge/l1.py` , `v_pat_backoff` .
3	과거값 → 쿠키 → WAS	L5 (5-5, 7-2)	점화식의 과거 순서 값을 서명해 쿠키로 보낸다. `curl -c jar` (받기), `curl -b jar` (보내기).
4	백엔드 도메인의 프록시 진입	L7 (7-7)	리버스 프록시·LB. `curl -x` , 프록시 접근 로그. 쿠키의 과거 값이 맞아야 통과한다.
5	현재값 순서 → 세션 ID 생성	L5 (5-1)	요청 값에 현재값 순서를 대입. `ss -tn state established` 로 수립된 세션을 본다.
6	미래값 주입 → AES 복호 → response (https/http, LocalC:Local:D)	L6·L7 (6-1, 7-1)	`openssl s_client` 로 핸드셰이크, `curl -w` 로 응답 분해. LocalC·LocalD는 `lab.env` 의 주소 자리.
7	node 접근: tcp/udp 요청 라우팅	L3·L4 (3-4, 4-9)	`ip route` , `ss -lntu` , `traceroute` . TCP와 UDP 둘 다 거친다.
8	DB 점검: query에 조건문이 있는가	L7 (7-8), 부록 D-2	1차 방어는 매개변수화된 질의(값과 명령 분리 = `P_`), 2차 점검은 조건문 토큰 탐지.
9	못 보면 RSA 스위칭 포인트 체크	L6 (6-5), 부록 D	불투명 경로면 비대칭 서명으로 경로를 재확인한다. `ssh-keygen -l` , `openssl dgst -verify` .

H-12-4. 대응 도구: oracle.env

1번 문서의 판정 장치(정확 등식, 구간, 나머지 사다리, 세계 증인, 육하원칙)를 `layers.env` 의 접두사 규칙에 맞춰 얹은 최소 구현입니다. `layers.env` 뒤에 `source` 해서 씁니다.

```
#!/usr/bin/env bash
# oracle.env — 1번 문서(ORacle)의 판정 장치를 2번 문서 변수분리에 얹은 최소 구현
#   K_ = 전역 상수(계산한 값)   P_ = 매개변수(고른 값)   v_ = 검증기(0/1)   r_ = 역탐색·보고
: "${P_ORA_TOL_MS:=50}"          # 정수 ms 단위 허용치 (고른 값, 반올림은 정수로 내려서 피한다)
: "${P_ORA_E_NUM:=271828182845}" "${P_ORA_E_DEN:=100000000000}" "${P_ORA_N_MAX:=10}"   # e를 적힌 그대로의 유리수로 둔다(고른 정밀도). 64비트 정수가 넘치지 않는 n ≤ 10

# 정확한 등식: 정수·문자열·바이트 (유리수 ↔ 유리수 자리)
v_exact()    { [[ "$1" == "$2" ]]; }
# 부등식 구간: 측정값 x가 [L,U] 안인가 (연속 참값 ↔ 정확한 값 자리). 정수 ms만 받는다.
v_interval() { local P_X=$1 P_L=$2 P_U=$3; (( P_X >= P_L && P_X <= P_U )); }

# 나머지 사다리: a_0=1, a_n = n*a_{n-1}+1  (정수 점화식, e는 계산에 들어오지 않는다)
ladder_a()   { local P_N=$1 a=1 k; for ((k=1;k<=P_N;k++)); do a=$((k*a+1)); done; echo "$a"; }
# 비교의 자리에서만 e를 만난다:  a_n < n!*e < a_n + 1/n   (n!*e*den 을 정수로 비교)
v_ladder()   { local P_N=$1 a f=1 i; (( P_N >= 1 && P_N <= P_ORA_N_MAX )) || return 2   # 범위 밖(정의): 2를 돌려준다
  a=$(ladder_a "$P_N")
  for ((i=2;i<=P_N;i++)); do f=$((f*i)); done
  local x=$((f*P_ORA_E_NUM))                          # n!*e*den
  local lo=$((a*P_ORA_E_DEN))                         # a_n*den
  local hi=$((a*P_ORA_E_DEN + P_ORA_E_DEN/P_N))       # (a_n+1/n)*den
  (( x > lo && x < hi )); }
```

```
# 세계 증인: 클라이언트가 본 값이 같은데 원인이 다르면, 클라이언트 판정기는 두 세계를 가르지 못한다(반례).
v_world()    { local P_OBS_A=$1 P_OBS_B=$2; [[ "$P_OBS_A" != "$P_OBS_B" ]]; }

# 육하원칙 보고 (판정은 증명·반례·정의 중 하나, 보류 없음)
r_6w() { local P_VERDICT=$1 P_WHO=$2 P_WHAT=$3 P_WHEN=$4 P_WHERE=$5 P_WHY=$6 P_HOW=$7
  printf '판정 : %s\n누가 : %s\n무엇 : %s\n언제 : %s\n어디서: %s\n왜   : %s\n어떻게: %s\n' \
    "$P_VERDICT" "$P_WHO" "$P_WHAT" "$P_WHEN" "$P_WHERE" "$P_WHY" "$P_HOW"; }
```

실행 결과

호출	결과	의미
`v_ladder n` (n = 1, 2, 3, 4, 5, 8, 10)	모두 구간 안 (0)	a_n = 2, 5, 16, 65, 326, 109601, 9864101이 $a_n < n!\cdot e < a_n+1/n$을 만족
`v_ladder 11`	2 (범위 밖, 정의)	64비트 정수가 넘치므로 판정하지 않고 범위 밖으로 적는다
`v_exact 1500 1400`	1 (불일치)	MTU 정확한 등식의 반례
`v_interval 61 0 50`	1 (구간 밖)	61ms는 [0, 50] 밖
`v_world timeout timeout`	1 (구분 불가)	INPUT DROP과 OUTPUT DROP을 클라이언트 관측만으로는 가르지 못함(세계 증인)

독립 검증: 같은 점화식을 파이썬 `Fraction`(정확 유리수, e는 40항 합)으로 n = 1~10까지 돌려 $a_n < n!\cdot e < a_n+1/n$이 모두 성립함을 확인했습니다. 처음 만든 `v_ladder`는 64비트 정수가 넘쳐 $n \geq 8$에서 우연히 통과하는 오류가 있어, e를 11자리 유리수로 줄이고 $n \leq 10$으로 범위를 정의해 고쳤습니다.

1번 문서의 유지보수 PDF는 이미지 2쪽이며 Readme의 구성 9단계와 같은 내용이었습니다. `ㅇㅇ` 파일은 음식 취향에 관한 대화 기록이라 2번 문서에 대응시킬 개념이 없어 제외했습니다.

H-14. 1번 문서의 수학적 특성·기계적 알고리즘 적용

H-12는 1번 문서의 개념을 표로 대응시켰지만, 알고리즘을 돌려 본 것은 아니었고 전역 상수의 정의(기원·불변·보편)도 지키지 못했습니다. 이 절은 두 가지를 바로잡습니다. 첫째, 전역 상수와 매개변수의 분류를 정의대로 고쳐 `lab.env`, `layers.env`, `ascii.env`, `pat.env`, `oracle.env`에 반영했습니다. 둘째, 1번 문서의 수학적 특성과 기계적 알고리즘 15종을 `mathlab.py`로 구현해 실행했습니다.

H-14-1. 분류 기준 개정

전역 상수의 세 조건 (전역상수·ㅇㅇㅇㅇㅇ 메모)

1. **기원:** 극한이나 물리적 값이 아니라 순수 조합론(a와 b를 몇 개씩 뽑는가, C(n,r))에서 도출된 숫자
2. **불변:** 기준점·간격·함수·위치가 변해도 변하지 않음
3. **보편:** 특정 식에 한정되지 않고 모든 차수에서 같은 규칙으로 작동

매개변수는 확률과 무관하게 고르는 경우의 수이고 정수형이나 실수형으로 호출 때 확정됩니다 (매개변수 메모). 필드 폭 n은 매개변수이고, n이 정해지면 나오는 C(n,r)과 그 합 2^n이 전역 상수입니다.

값	이전 분류 (잘못)	개정
이름·IP·포트·경로(`NS_C`, `C_IP`, `PORT` …)	`readonly` 전역 상수	매개변수 `P_NS_C`, `P_C_IP`, `P_PORT` … (고른 값)
MTU 1500, 포트 22·80·443, TTL 64, keepalive 7200, HTTP 200	`K_L2_MTU` 등 전역 상수	매개변수 `P_L2_MTU` 등 (표준·관습이 정한 값, 환경 변수로 덮어쓰기 가능)
포트·TTL·MAC 개수, 랩의 전체 연결 수	없음	전역 상수 `K_N_PORT`, `K_N_TTL`, `K_N_MAC`, `K_LINKS_LAB` (파스칼 점화식으로 계산)
ASCII 코드표 크기 128	`K_ASCII_SIZE` (적은 값)	`K_ASCII_SIZE` = Σ C(7,r) 로 계산, 제어 코드 값은 `P_ASCII_*`
백오프 기준·허용치	`K_PAT_*`	`P_PAT_*`

8080, 10.0.0.1, MTU 1500처럼 사람이나 표준이 정한 값은 조합론에서 계산되지 않으므로 전역 상수가 아닙니다. `readonly`는 값을 잠그는 도구일 뿐 전역 상수를 만들어 주지 않기 때문에, 이전에는 `readonly`를 붙였다는 이유로 고른 값까지 상수로 분류한 것이 어긋남의 원인이었습니다.

H-14-2. 알고리즘 15종과 실행 결과

모듈	1번 문서의 특성·알고리즘	OSI 문서에서의 적용	실행으로 확인된 것
M1	전역 상수 = 파스칼 점화식 C(n,r)=C(n−1,r−1)+C(n−1,r)에서 나온 이항계수 (전역상수 메모)	`lab.env`의 `_ROW`·`_SUM`·`_C`·`_K`로 `K_N_PORT`, `K_N_IPV4`, `K_N_MAC`, `K_N_TTL`, `K_LINKS_LAB` 생성	이항정리 125회 정확 성립, 계수 불변. 65536, 4294967296, 281474976710656, 256, 연결 수 1
M1b	전역 상수의 세 조건(기원·불변·보편)을 판정하는 분류 알고리즘	값마다 K_ / P_ 분류. 도출 규칙이 없는 값은 P_	11개 중 K_ 5개(포트·IPv4·MAC·TTL 개수, 연결 수), P_ 6개(포트 8080, IP, MTU, 80, TTL 64, keepalive). K_ 변이 20/20 반례
M2	미분의 정체: 전체 변화량 − a^n을 b로 나눈 몫에서 b^1 계수 추출 (전역상수 메모 3절)	누적 카운터의 변화율 = 차분 몫. 고차 꼬리가 허용 구간(= 창 b)	n=2,3,5,8에서 정확한 등식, 꼬리 크기 단조 감소, 누적 t^2 창 b=1/4에서 $2t \le 801/4 \le 2t+b$

M3	나머지 사다리 a_n과 적분 점화식 I_n (점화식 취합 A·B)	지수 성장($e^{t/2}$)의 4구간 누적량을 정확 분수 점화식 + 하한·상한으로 가둠	n=1~12 부등식 통과. I_n 구간 폭 8 → 1.25×10^{-4}, 참값 12.778112 항상 포함
M4	조립제법: 유리수 다항식을 (x−c)로 나눈 나머지. c=e이면 0이 아님 (철학 취합 §2)	이더넷 FCS = GF(2)에서의 같은 나눗셈. 온전한 프레임은 나머지 0	p(e) 구간이 0을 포함하지 않음 4/4, 유리수근은 나머지 0, CRC-16 비트 1개 변조 시 나머지 0x6f45, CRC-32 잔여값 0x2144DF1C
M5	오일러 분해 패턴표(e, i, π, t^k, 0·1), 닫힘 검사(연분수), i^n mod 4 (본문 6.3–6.4)	관측 열을 성장·감쇠·회전·다항식·기준값으로 분류. 재전송 간격, ARP 상태 순환, keepalive	8개 열 분류. 5상태와 7표본 순환은 한 바퀴 5걸음·7걸음으로 닫힘, $\sqrt{2}-1$은 닫히지 않아 "패턴표 밖(정의)"
M6	그래프 상수 L = (V,T), 2계는 L^2, 수식 ↔ 그림 왕복 (본문 4.4)	랩(client–server)과 라우터 경유 토폴로지. 홉 수 = 인접행렬 거듭제곱	L의 행 합 0·대칭·L^2 행 합 0, 홉 수 1과 2, 그림 → 수식 왕복 일치
M7	국소화: 몫의 합 = 한도, 망원 합 (본문 13절)	`curl -w` 누적 시각을 DNS·연결·TLS·준비·서버·전송 몫으로 분해	실제 측정 20회 모두 정수 μs로 몫의 합 = `time_total`
M8	정수 추적과 대표 크기 (명제 20.3)	포트 분류 함수의 술어 칸마다 대표 입력	65536개 포트를 전수 열거한 칸을 대표 9개와 11개가 모두 덮음
M9	유한체 GF(p) 무작위 실행, 슈바르츠–지펠 오판 상한 (본문 20.2)	M1의 계수표를 $p=2^{61}-1$ 위에서 무작위로 검증하고 변이체를 잡음	틀린 계수표 39/39 반례. 비교 1회 오판 상한 $\le 1.73\times10^{-17}$
M10	변이 분석 (본문 20.6)	설정 표(K_, P_)를 일부러 틀리게 해 검증기가 잡는가	K_ 변이 20/20 반례. P_는 도출 규칙이 없어 형태(정수·범위)만 검사해 70000, 0 같은 반례를 잡음
M11	확률 명세: 비율 편차 구간, δ-닫힘 (본문 21.4)	손실률 판정과 ping 횟수 정하기	2000회 중 참 손실률을 덮은 시행 2000. N=200 검출 703/2000, N=1000 검출 2000/2000, 놓칠 확률 1% 아래는 N ≥ 44
M12	육하원칙 보고와 지위(증명·반례·정의) (본문 13.4)	K_ 변이체 하나를 여섯 칸으로 보고	판정 반례, 어떻게 칸은 재판정 통과로 증명
M13	판정 지도와 질문 트리 (본문 21.5, 32절)	관측 항목을 명세 종류로 분류해 판정 장치 선택	10개 항목이 정확 등식·비율 편차·결정 열거·세계 증인 등으로 갈림
M14	세션 파이프라인: 점화식 씨앗으로 과거·현재·미래 값 (세션 보안 문서)	L5 쿠키(과거값)·세션 ID(현재값), L6 AES-GCM(미래값)	1비트 위조 쿠키 거부, 다른 순서 값은 다른 ID, 틀린 미래값은 복호 실패
M15	같은 점화식의 이중 구현(bash, python)	`lab.env` 가 만든 K_ 와 파이썬 도출값 비교	6개 값 정확 일치, `K_N_PORT=1` 은 readonly 오류, `P_PORT` 는 호출이 정한 값 유지

M1b의 분류표(실행 결과)는 아래와 같습니다. 도출 규칙이 있고 적힌 값과 정확히 같은 항목만 `K_` 가 됩니다.

항목	적힌 값	분류	이유
`K_N_PORT` 포트 개수	65536	K_	$\Sigma\, C(16,r) = 2^{16}$
`K_N_IPV4` IPv4 주소 개수	4294967296	K_	$\Sigma\, C(32,r)$
`K_N_MAC` MAC 주소 개수	281474976710656	K_	$\Sigma\, C(48,r)$
`K_N_TTL` TTL 값 개수	256	K_	$\Sigma\, C(8,r)$
`K_LINKS_LAB` 호스트 2대 전체 연결	1	K_	C(2,2) = 1, veth 1개와 일치

`P_PORT` 실습 HTTP 포트	8080	P_	기원 없음 (고른 값)
`P_C_IP` client IP	10.0.0.1	P_	기원 없음 (고른 값)
`P_L2_MTU` 이더넷 MTU	1500	P_	기원 없음 (표준이 정한 값)
`P_L4_PORT_HTTP`	80	P_	기원 없음 (표준이 정한 값)
`P_L3_TTL` 기본 TTL	64	P_	기원 없음 (운영체제 기본값)
`P_L5_KEEPALIVE_SEC`	7200	P_	기원 없음 (관습 값)

H-14-3. 실행 출력 전체

`python3 mathlab.py all`의 출력입니다. 모든 단언이 통과해야 마지막 줄 "모든 단언 통과"가 나옵니다(보류 없음).

```
=== M1 전역 상수 생성기 (파스칼 점화식) ===
보편: n=0..24, 무작위 유리수 a,b 125회에서 이항정리가 정확히 성립
불변: n=12의 모든 b^r 계수가 기준점 a가 5가지로 변해도 같은 C(12,r)
생성한 전역 상수 (K_ = 이항계수의 합 / 이항계수)
  K_N_PORT   = sum_r C(16,r) = 65536
  K_N_IPV4   = sum_r C(32,r) = 4294967296
  K_N_MAC    = sum_r C(48,r) = 281474976710656
  K_N_TTL    = sum_r C(8,r) = 256
  K_N_CKSUM  = sum_r C(16,r) = 65536
  K_LINKS_2   = C(2,2) = 1   (호스트 2대 전체 연결)
  K_LINKS_3   = C(3,2) = 3   (호스트 3대 전체 연결)
  K_LINKS_4   = C(4,2) = 6   (호스트 4대 전체 연결)
  K_LINKS_8   = C(8,2) = 28   (호스트 8대 전체 연결)
랩(호스트 2대)의 전체 연결 수 C(2,2) = 1 → veth 1개와 일치

=== M1b 분류 알고리즘: K_ / P_ ===
  K_N_PORT   포트 개수                    65536 → K_  (기원·불변·보편 통과)
  K_N_IPV4   IPv4 주소 개수          4294967296 → K_  (기원·불변·보편 통과)
  K_N_MAC    MAC 주소 개수         281474976710656 → K_  (기원·불변·보편 통과)
  K_N_TTL    TTL 값 개수                 256 → K_  (기원·불변·보편 통과)
  K_LINKS_LAB 호스트2 전체 연결              1 → K_  (기원·불변·보편 통과)
  P_PORT     실습 HTTP 포트              8080 → P_  (기원 없음(고른 값 또는 표준이 정한 값))
  P_C_IP     client IP            10.0.0.1 → P_  (기원 없음(고른 값 또는 표준이 정한 값))
  P_L2_MTU   이더넷 MTU                 1500 → P_  (기원 없음(고른 값 또는 표준이 정한 값))
  P_L4_PORT_HTTP 표준 포트                80 → P_  (기원 없음(고른 값 또는 표준이 정한 값))
  P_L3_TTL    기본 TTL                  64 → P_  (기원 없음(고른 값 또는 표준이 정한 값))
  P_L5_KEEPALIVE_SEC 초                7200 → P_  (기원 없음(고른 값 또는 표준이 정한 값))
  K_ 변이 20개 중 반례로 잡힌 것 20개

=== M2 차분 몫과 b^1 계수 추출 ===
  n=2: ((a+b)^n-a^n)/b = n·a^(n-1) + (b로 소멸하는 꼬리)  a=3/2, b=1,1/10,1/1000 에서 정확히 성립
  n=3: ((a+b)^n-a^n)/b = n·a^(n-1) + (b로 소멸하는 꼬리)  a=3/2, b=1,1/10,1/1000 에서 정확히 성립
  n=5: ((a+b)^n-a^n)/b = n·a^(n-1) + (b로 소멸하는 꼬리)  a=3/2, b=1,1/10,1/1000 에서 정확히 성립
  n=8: ((a+b)^n-a^n)/b = n·a^(n-1) + (b로 소멸하는 꼬리)  a=3/2, b=1,1/10,1/1000 에서 정확히 성립
  꼬리 크기 b=1,.1,...,1e-5: [12281.0, 962.87801, 94.0003753001, 9.377500375030001, 0.937525000375003, 0.093750250000375]
  적용: 누적량 t^2에서 창 b=1/4로 잰 전송률 801/4 ∈ [2t, 2t+b] = [200, 801/4] (허용 구간 = b)

=== M3 나머지 사다리 a_n 과 적분 점화식 I_n ===
  a_n = n·a_(n-1)+1: n=1..12에서 a_n < n!·e < a_n+1/n (60자리 십진수로 확인)
  λ=1/2, τ=4 (x=2): 참값 (e^x-1)/λ = 12.778112198
   n   I_n(정확 분수)                하한         상한         폭
   0  4                             8.000000  16.000000 8.00e+00
   1  8                            10.666667  13.333333 2.67e+00
   2  32/3                         12.000000  12.888889 8.89e-01
   3  12                           12.533333  12.800000 2.67e-01
   4  188/15                       12.711111  12.782222 7.11e-02
   5  572/45                       12.761905  12.778836 1.69e-02
   6  268/21                       12.774603  12.778231 3.63e-03
   7  4024/315                     12.777425  12.778131 7.05e-04
   8  36224/2835                   12.777989  12.778115 1.25e-04
  적용: 지수 성장(슬로 스타트형 e^(t/2))의 4구간 누적량을 유리수 점화식 + 구간으로 가둔다

=== M4 조립제법과 나머지 ===
  p=x-3          나머지 p(e) ∈ (-0.281718, -0.281718)  0 포함? 아니오 → 나머지 ≠ 0 증명 True
  p=x²-7         나머지 p(e) ∈ (0.389056, 0.389056)  0 포함? 아니오 → 나머지 ≠ 0 증명 True
  p=x³-2x²+x-5   나머지 p(e) ∈ (3.025706, 3.025707)  0 포함? 아니오 → 나머지 ≠ 0 증명 True
  p=x²-x-1       나머지 p(e) ∈ (3.670774, 3.670774)  0 포함? 아니오 → 나머지 ≠ 0 증명 True
  유리수 근 c=2: x²-3x+2 ÷ (x-2) 나머지 0, 몫 x-1 (정확한 등식)
  GF(2) CRC-16: 온전한 프레임 나머지 0, 비트 1개 뒤집으면 나머지 0x6f45 ≠ 0
  이더넷 FCS(CRC-32): crc32(data‖crc) = 0x2144DF1C 상수 (유리수·실수에서는 0이 안 되고, GF(2)에서는 닫힌다)

=== M5 오일러 패턴 분류와 닫힘 ===
  TCP 재전송 간격 1,2,4,8,16          → 성장(e^(at), a>0)
  하향(TTL식 반감) 64,32,16,8         → 감쇠(e^(at), a<0)
  ARP 5상태 순환(코사인 표본)            → 회전(i)·주기(π)  한 바퀴 = 5걸음 (θ/2π=1/5)
  keepalive 주기 7샘플/바퀴           → 회전(i)·주기(π)  한 바퀴 = 7걸음 (θ/2π=1/7)
  누적 카운터 선형 0,5,10,15,20        → 다항식 배수(t^1)
  누적 카운터 2차 0,1,4,9,16          → 다항식 배수(t^2)
  링크 UP 상수열 1,1,1,1              → 기준값(상수열, 0·1 닫힘)
  무작위 3,1,4,1,5,9                → 분류 밖(정의: 패턴표 밖)
  i^n mod 4: 1, i, −1, −i 반복 (4걸음 닫힘). TCP 상태 4개를 한 바퀴 순환시키면 같은 규칙
  √2−1 (무리수) 한 바퀴 비율: 분모 64 이하로는 닫히지 않음 → 패턴표 밖(정의)으로 보고

=== M6 그래프 상수 L ===
  랩 (client–server): L 행 합 0 · 대칭 · L² 행 합 0 확인, client→server 홉 수 = 1 (traceroute 홉과 같아야 함)
   L = [['1', '-1'], ['-1', '1']]
   L² = [['2', '-2'], ['-2', '2']]
```

```
    그림 "client -(1)- server" → 다시 L: 정확히 같음
  라우터 경유 (client–router–server): L 행 합 0 · 대칭 · L² 행 합 0 확인, client→server 홉 수 = 2 (traceroute 홉과 같아야 함)
    L = [['1', '-1', '0'], ['-1', '2', '-1'], ['0', '-1', '1']]
    L² = [['2', '-3', '1'], ['-3', '6', '-3'], ['1', '-3', '2']]
    그림 "client -(1)- router  router -(1)- server" → 다시 L: 정확히 같음

=== M7 지연의 국소화 (망원 합, 실제 curl 측정) ===
  curl 20회 모두 몫의 합 == time_total (정수 μs 정확 등식) 20/20
  마지막 측정 몫(μs): {'DNS(L7)': 15, 'TCP 연결(L4)': 56, 'TLS(L6)': 0, '요청 준비': 16, '서버 처리(L7)': 397, '전송(L7)': 138}

=== M8 정수 추적과 대표 크기 ===
  port_class: 전수 열거 칸 3개, 대표 9개로 모든 칸을 덮음 (M=49152, L=1), 전수 65536개 대 대표 9개
  rtp_even: 전수 열거 칸 2개, 대표 11개로 모든 칸을 덮음 (M=32768, L=2), 전수 65536개 대 대표 11개
  대표 입력만 통과하면 65536개 포트 전체 칸이 통과 — 술어가 아핀(상수 비교)·주기(나머지)뿐일 때

=== M9 GF(p) 무작위 실행 (p = 2^61−1) ===
  이항 계수표가 n=2,8,...,38에서 GF(p) 무작위 a,b 20회씩 통과
  틀린 계수표(변이체) 39개 중 39개가 반례로 잡힘
  오판 확률 상한: 비교 1회 ≤ deg/p = 40/(2^61−1) = 1.73e-17, 독립 2회 ≤ 3.01e-34

=== M10 변이 분석 (설정 표) ===
  K_ 변이체 20개 중 20개 반례 → 변이 점수 20/20
  P_ 6개 항목은 도출 규칙이 없어 판정 기준이 없다 → 값 자체는 호출 때 정하고, 형태(정수인가, 범위인가)만 검사
  P_ 형태 검사 반례: [('P_PORT     실습 HTTP 포트', 70000), ('P_L3_TTL    기본 TTL', 0)]

=== M11 비율 편차 구간 (손실률 판정) ===
  N=200, δ=0.05: ε = 0.0960  → 참 손실률을 덮은 시행 2000/2000 = 1.000 (보증 ≥ 0.95)
  netem loss 10% 검출(하한 > 1%) 703/2000  ← N이 작으면 구간이 넓어 놓칠 수 있다
  N=1000: ε = 0.0429, 검출 2000/2000  (표본이 늘면 같은 보증으로 검출력이 오른다)
  손실률 10%에서 ping -c  10 이 손실을 하나도 못 볼 확률 (1−p)^N = 0.3487
  손실률 10%에서 ping -c  20 이 손실을 하나도 못 볼 확률 (1−p)^N = 0.1216
  손실률 10%에서 ping -c  50 이 손실을 하나도 못 볼 확률 (1−p)^N = 0.0052
  손실률 10%에서 ping -c 100 이 손실을 하나도 못 볼 확률 (1−p)^N = 0.0000
  손실 놓침 확률을 1% 아래로 하려면 ping 횟수 N ≥ 44

=== M12 육하원칙 보고 (M1b 변이체 하나를 보고) ===
  판정  : 반례   [반례]
  누가  : K_ 검증기 classify()   [증명]
  무엇을 : K_N_PORT 적힌 값 65535 ≠ 계산값 65536   [정의]
  언제  : 2026-11-13 13:51:36   [정의]
  어디서 : layers.env / lab.env 의 해당 상수 한 줄   [정의]
  왜   : 적힌 값이 sum C(16,r) = 2^16 과 1 어긋남   [증명]
  어떻게 : 값을 65536로 되돌려 다시 판정 → 통과   [증명]

=== M13 판정 지도 ===
  MTU 설정 일치(1500=1500)           → 정확 등식 (GF 실행·정규형 일치)
  헤더 필드·MAC 일치                  → 정확 등식 (GF 실행·정규형 일치)
  패킷 손실률                         → 비율 편차 δ-닫힘
  TCP 상태 전이 순서                  → 결정 열거(대수 부분) + 실행 검사의 부등식
  재전송 백오프 수렴                  → 증폭률 거듭제곱 상한 + 사다리 허용치
  지연 시간(ms, 참값은 연속)          → 사다리 꼬리 + π 연분수
  curl 시간 초→μs 변환               → 반올림 부등식(정수로 내려 비교)
  외부 DNS 응답이 실제와 같은가       → 블록체인형: 계산 층 사다리, 집계 층 그래프 상수, 진실 층 사이클
  정답 표지 없는 이상 탐지            → 세계 증인(반례) 또는 δ-증명
  정지 문제형 질문                    → 정의 25.2: 자원을 늘려 가는 후보 목록과 육하원칙 처방

=== M14 세션 보안 파이프라인 (L5·L6) ===
  L5 쿠키(과거값): 정상 통과, 1비트 위조는 거부
  L5 세션 ID(현재값) 96cf527f9803e885… , 다른 순서의 값이면 다른 ID
  L6 AES-GCM(미래값): 맞는 미래값으로만 복호, 다른 순서의 값은 복호 실패
  일방향: 쿠키·세션 ID에서 씨앗이나 다른 순서의 값을 복원하는 경로가 없다 (HMAC은 다대일 수축)

=== M15 bash가 만든 K_ 와 파이썬 도출값의 일치 ===
  bash:   ['65536', '4294967296', '281474976710656', '256', '1', '128']
  python: ['65536', '4294967296', '281474976710656', '256', '1', '128']  → 정확히 일치 (같은 점화식을 두 언어로 따로 구현)
  K_N_PORT=1 시도 → readonly variable 오류, 값은 65536 유지
  P_PORT=9090 (호출 때 확정) → source 후에도 9090 유지: 매개변수는 호출이 정한다

모든 단언 통과
```

H-14-4. 전체 코드: mathlab.py

아래가 `mathlab.py` 전체입니다. 이 PDF 안에 환경 파일도 모두 실려 있습니다: `lab.env` (0-1), `layers.env` (H-5), `ascii.env` (H-6), `pat.env` (H-4), `oracle.env` (H-12-4). M15는 `lab.env` 내용을 `labtest.env` 라는 이름으로 같은 폴더에 두면 돕니다.

```
#!/usr/bin/env python3
"""mathlab.py — 1번 문서(ORacle)의 수학적 특성과 기계적 알고리즘을 OSI 7계층 문서(부록 H)에 적용한 실행 코드.
각 함수는 계산 결과를 출력하고, 어긋나면 AssertionError로 멈춘다(보류 없음: 증명·반례·정의).
사용: python3 mathlab.py [m1 m2 ... m15 | all]  (labtest.env, layers.env, ascii.env 가 같은 폴더에 있어야 m15 가 돈다)
"""
import cmath, hashlib, hmac, itertools, math, os, random, struct, subprocess, sys, time, zlib
from fractions import Fraction as Fr
from decimal import Decimal, getcontext
from math import comb, factorial, gcd

getcontext().prec = 60
P61 = (1 << 61) - 1           # 메르센 소수 (유한체 GF(p), 본문 20.2)

def head(t): print(f"\n=== {t} ===")

# ──────────────────────────────────────────────────────────────────────
# M1. 전역 상수 = 파스칼 삼각형(순수 조합론)에서 점화식으로 만든 이항계수
#     (전역상수 메모: 기원·불변·보편 세 조건)
# ──────────────────────────────────────────────────────────────────────
def pascal_row(n):
    """C(n,r) = C(n-1,r-1) + C(n-1,r) 점화식만으로 n행을 만든다(곱셈·나눗셈·극한 없음)."""
    row = [1]
    for _ in range(n):
        row = [1] + [row[i] + row[i + 1] for i in range(len(row) - 1)] + [1]
    return row

# 구조 입력(매개변수): 필드 폭 n (비트). 전역 상수: n이 정해지면 이항계수의 합으로 나오는 개수.
WIDTHS = {"K_N_PORT": 16, "K_N_IPV4": 32, "K_N_MAC": 48, "K_N_TTL": 8, "K_N_CKSUM": 16}
def derive_count(width):                  # 비트를 r개 뽑는 경우의 수를 모두 더한 값
    return sum(pascal_row(width))
def derive_pairs(n):                      # 호스트 n대에서 쌍을 뽑는 경우의 수 = 전체 연결 변 수
    return pascal_row(n)[2] if n >= 2 else 0

def m1():
    head("M1 전역 상수 생성기 (파스칼 점화식)")
    # (a) 보편: 이항정리 (a+b)^n = sum C(n,r) a^(n-r) b^r 이 임의 유리수 a,b에서 정확히 성립
    rnd = random.Random(1)
    for n in range(0, 25):
        row = pascal_row(n)
        assert row == [comb(n, r) for r in range(n + 1)], n          # 독립 계산과 일치
        for _ in range(5):
            a, b = Fr(rnd.randint(-9, 9), rnd.randint(1, 9)), Fr(rnd.randint(-9, 9), rnd.randint(1, 9))
            assert (a + b) ** n == sum(row[r] * a ** (n - r) * b ** r for r in range(n + 1))
    print("보편: n=0..24, 무작위 유리수 a,b 125회에서 이항정리가 정확히 성립")
    # (b) 불변: 전개식의 b^r 항 계수를 a의 값마다 뽑아 a^(n-r)로 나누면 항상 같은 C(n,r)
    n = 12
    for r in range(n + 1):
        vals = set()
        for a in (Fr(1), Fr(2), Fr(-3), Fr(5, 7), Fr(10 ** 6)):
            coef = comb(n, r) * a ** (n - r)       # (a+b)^n 을 b의 다항식으로 전개했을 때 b^r 의 계수
            vals.add(coef / a ** (n - r))
        assert len(vals) == 1
    print("불변: n=12의 모든 b^r 계수가 기준점 a가 5가지로 변해도 같은 C(12,r)")
    # (c) 기원: 값 생성에 조합론(파스칼 점화식) 외의 입력이 없다. 생성한 상수:
    print("생성한 전역 상수 (K_ = 이항계수의 합 / 이항계수)")
    for name, w in WIDTHS.items():
        v = derive_count(w)
        assert v == 2 ** w
        print(f"  {name:10s} = sum_r C({w},r) = {v}")
    for n in (2, 3, 4, 8):
        print(f"  K_LINKS_{n}   = C({n},2) = {derive_pairs(n)}   (호스트 {n}대 전체 연결)")
    assert derive_pairs(2) == 1                    # 랩: client-server 사이 veth 1개
    print("랩(호스트 2대)의 전체 연결 수 C(2,2) = 1 → veth 1개와 일치")

# 분류기: 전역 상수(K_)인가 매개변수(P_)인가
def classify(name, value, derive=None, env_variants=(0, 1, 2)):
    """기원: derive()가 있고 value와 정확히 같다. 불변: 환경 변형(env)에 따라 값이 변하지 않는다. 보편: 모든 n에서 동일 규칙.
    derive가 없으면(고른 값) 기원 조건이 서지 않으므로 매개변수 P_로 분류한다."""
    if derive is None:
        return "P_", "기원 없음(고른 값 또는 표준이 정한 값)"
    outs = {derive(env) for env in env_variants}
    if len(outs) != 1:
        return "P_", "환경에 따라 변함(불변 조건 실패)"
    if outs.pop() != value:
        return "반례", "계산값과 적힌 값이 다름"
    return "K_", "기원·불변·보편 통과"

ITEMS = [  # (이름, 적힌 값, 도출 함수 또는 None)
    ("K_N_PORT   포트 개수",         65536,      lambda env: derive_count(16)),
```

```
    ("K_N_IPV4   IPv4 주소 개수",    4294967296, lambda env: derive_count(32)),
    ("K_N_MAC    MAC 주소 개수",     2 ** 48,    lambda env: derive_count(48)),
    ("K_N_TTL    TTL 값 개수",       256,        lambda env: derive_count(8)),
    ("K_LINKS_LAB 호스트2 전체 연결", 1,          lambda env: derive_pairs(2)),
    ("P_PORT     실습 HTTP 포트",    8080,       None),
    ("P_C_IP     client IP",         "10.0.0.1", None),
    ("P_L2_MTU   이더넷 MTU",        1500,       None),
    ("P_L4_PORT_HTTP 표준 포트",    80,         None),
    ("P_L3_TTL    기본 TTL",         64,         None),
    ("P_L5_KEEPALIVE_SEC 초",     7200,       None),
]
def m1b():
    head("M1b 분류 알고리즘: K_ / P_")
    for name, val, d in ITEMS:
        c, why = classify(name, val, d)
        print(f"  {name:28s} {str(val):>12s} → {c:3s} ({why})")
    # 변이: 적힌 값을 1 틀리게 하면 K_ 검증이 반례로 잡는가
    caught = total = 0
    for name, val, d in ITEMS:
        if d is None: continue
        for mut in (val + 1, val - 1, val * 2, val // 2):
            total += 1
            caught += classify(name, mut, d)[0] == "반례"
    print(f"  K_ 변이 {total}개 중 반례로 잡힌 것 {caught}개")
    assert caught == total

# ──────────────────────────────────────────────────────────────────────
# M2. 미분의 정체: 전체 변화량에서 a^n을 빼고 b로 나눈 몫의 b^1 계수 추출
# ──────────────────────────────────────────────────────────────────────
def m2():
    head("M2 차분 몫과 b^1 계수 추출")
    for n in (2, 3, 5, 8):
        a = Fr(3, 2)
        row = pascal_row(n)
        for b in (Fr(1), Fr(1, 10), Fr(1, 1000)):
            q = ((a + b) ** n - a ** n) / b
            head_term = row[1] * a ** (n - 1)                                  # C(n,1) a^(n-1) = n a^(n-1)
            tail = sum(row[r] * a ** (n - r) * b ** (r - 1) for r in range(2, n + 1))
            assert q == head_term + tail and head_term == n * a ** (n - 1)
        print(f"  n={n}: ((a+b)^n−a^n)/b = n·a^(n−1) + (b로 소멸하는 꼬리)  a=3/2, b=1,1/10,1/1000 에서 정확히 성립")
    # 꼬리는 b가 줄면 0으로 간다: 정확한 단조 감소 확인
    n, a = 6, Fr(5)
    row = pascal_row(n)
    tails = [sum(row[r] * a ** (n - r) * Fr(1, 10 ** k) ** (r - 1) for r in range(2, n + 1)) for k in range(0, 6)]
    assert all(tails[i] > tails[i + 1] for i in range(5))
    print("  꼬리 크기 b=1,.1,...,1e-5:", [float(t) for t in tails])
    # 적용: 누적 전송량 c(t)=t^2 (전송률이 선형으로 늘어남). 창 b로 잰 전송률 = 2t + b ∈ [2t, 2t+b]
    t, b = Fr(100), Fr(1, 4)
    measured = (( t + b) ** 2 - t ** 2) / b
    assert measured == 2 * t + b and 2 * t <= measured <= 2 * t + b
    print(f"  적용: 누적량 t^2에서 창 b={b}로 잰 전송률 {measured} ∈ [2t, 2t+b] = [{2*t}, {2*t+b}] (허용 구간 = b)")

# ──────────────────────────────────────────────────────────────────────
# M3. 나머지 사다리(정수 점화식)와 적분 점화식(유리수 점화식)의 구간
# ──────────────────────────────────────────────────────────────────────
def e_dec():
    return Decimal(1).exp()

def m3():
    head("M3 나머지 사다리 a_n 과 적분 점화식 I_n")
    E = e_dec()
    a = 1
    for n in range(1, 13):
        a = n * a + 1
        x = Decimal(factorial(n)) * E
        assert Decimal(a) < x < Decimal(a) + Decimal(1) / n, n
    print("  a_n = n·a_(n−1)+1: n=1..12에서 a_n < n!·e < a_n+1/n (60자리 십진수로 확인)")
    # 적분 점화식: c0=τ, c_n = c_(n−1)·x/(n+1), I_n = I_(n−1)+c_n,  x=λτ,  0<x<n+3 이면 In+L < (e^x−1)/λ < In+U
    lam, tau = Fr(1, 2), Fr(4)
    x = lam * tau
    true = (Decimal(2).exp() - 1) / Decimal(lam.numerator) * Decimal(lam.denominator)  # (e^2−1)/(1/2)
    I, c = tau, tau
    print(f"  λ=1/2, τ=4 (x=2): 참값 (e^x−1)/λ = {float(true):.9f}")
    print("   n   I_n(정확 분수)                 하한         상한         폭")
    for n in range(0, 9):
        if n > 0:
            c = c * x / (n + 1); I = I + c
        cn1 = c * x / (n + 2)
        L = cn1
        U = cn1 * (n + 3) / (n + 3 - x)
        lo, hi = Decimal(I.numerator) / Decimal(I.denominator) + Decimal(L.numerator) / Decimal(L.denominator), \
                 Decimal(I.numerator) / Decimal(I.denominator) + Decimal(U.numerator) / Decimal(U.denominator)
        assert lo < true < hi, n
        print(f"  {n:2d}  {str(I):28s} {float(lo):10.6f} {float(hi):10.6f} {float(hi-lo):.2e}")
    print("  적용: 지수 성장(슬로 스타트형 e^(t/2))의 4구간 누적량을 유리수 점화식 + 구간으로 가둔다")

# ──────────────────────────────────────────────────────────────────────
# M4. 조립제법: 유리수 다항식을 (x−c)로 나눈 나머지.  c=e 이면 나머지가 0이 되지 않는다. GF(2)에서는 0이 된다(CRC).
# ──────────────────────────────────────────────────────────────────────
def horner(coefs, c):
    """계수는 최고차부터. 조립제법: 몫 계수와 나머지 p(c)."""
```

```
    q = []
    acc = 0
    for k in coefs:
        acc = acc * c + k
        q.append(acc)
    return q[:-1], q[-1]

class Iv:
    """정확한 유리수 구간 산술 (증명용)."""
    def __init__(s, lo, hi=None): s.lo, s.hi = lo, (lo if hi is None else hi)
    def __add__(s, o): o = o if isinstance(o, Iv) else Iv(o); return Iv(s.lo + o.lo, s.hi + o.hi)
    def __mul__(s, o):
        o = o if isinstance(o, Iv) else Iv(o)
        p = [s.lo * o.lo, s.lo * o.hi, s.hi * o.lo, s.hi * o.hi]; return Iv(min(p), max(p))
    def __contains__(s, v): return s.lo <= v <= s.hi

def horner_iv(coefs, civ):
    acc = Iv(0)
    for k in coefs: acc = acc * civ + Iv(k)
    return acc

def crc_rem(data_bits, poly, width):
    """GF(2) 위의 다항식 나눗셈 나머지 (비트열을 최고차부터 넣는다)."""
    r = 0
    for bit in data_bits:
        top = (r >> (width - 1)) & 1
        r = ((r << 1) & ((1 << width) - 1)) | bit
        if top: r ^= (poly & ((1 << width) - 1))
    return r

def m4():
    head("M4 조립제법과 나머지")
    n = 10
    S = sum(Fr(1, factorial(k)) for k in range(n + 1))
    e_iv = Iv(S, S + Fr(1, factorial(n) * n))       # e ∈ (S_n, S_n+1/(n!·n))
    polys = {"x−3": [1, -3], "x²−7": [1, 0, -7], "x³−2x²+x−5": [1, -2, 1, -5], "x²−x−1": [1, -1, -1]}
    for name, co in polys.items():
        r = horner_iv(co, e_iv)
        nonzero = not (Fr(0) in r)
        print(f"  p={name:12s} 나머지 p(e) ∈ ({float(r.lo):.6f}, {float(r.hi):.6f})  0 포함? {'예' if not nonzero else '아니오'} → 나머지 ≠ 0 증명
{nonzero}")
        assert nonzero
    q, rem = horner([1, -3, 2], Fr(2))             # x²−3x+2 = (x−1)(x−2), c=2는 유리수 근
    assert rem == 0 and q == [1, -1]
    print("  유리수 근 c=2: x²−3x+2 ÷ (x−2) 나머지 0, 몫 x−1 (정확한 등식)")
    # GF(2): 프레임 = 데이터·x^16 + 나머지.  온전하면 나머지 0
    data = [int(b) for b in format(int.from_bytes(b"OSI7 frame", "big"), "080b")]
    poly, w = 0x11021, 16
    rem = crc_rem(data + [0] * w, poly, w)
    frame = data + [int(b) for b in format(rem, "016b")]
    assert crc_rem(frame, poly, w) == 0
    bad = frame[:]; bad[7] ^= 1
    assert crc_rem(bad, poly, w) != 0
    print(f"  GF(2) CRC-16: 온전한 프레임 나머지 0, 비트 1개 뒤집으면 나머지 {crc_rem(bad, poly, w):#06x} ≠ 0")
    # 실제 이더넷 FCS(CRC-32) 잔여값: 데이터 + 리틀엔디언 CRC 를 다시 CRC하면 상수
    d = b"L2 payload"
    assert zlib.crc32(d + struct.pack("<I", zlib.crc32(d))) == 0x2144DF1C
    print("  이더넷 FCS(CRC-32): crc32(data‖crc) = 0x2144DF1C 상수 (유리수·실수에서는 0이 안 되고, GF(2)에서는 닫힌다)")

# ──────────────────────────────────────────────────────────────────────
# M5. 오일러 분해 패턴 분류기와 닫힘 검사(연분수), i^n mod 4
# ──────────────────────────────────────────────────────────────────────
def classify_series(u, tol=1e-9):
    n = len(u)
    if all(abs(x - u[0]) <= tol for x in u): return "기준값(상수열, 0·1 닫힘)", {}
    # 1차: 일정한 비 → e^(at)
    if all(abs(x) > tol for x in u[:-1]):
        rs = [u[i + 1] / u[i] for i in range(n - 1)]
        if max(rs) - min(rs) <= tol:
            r = rs[0]; return ("성장(e^(at), a>0)" if r > 1 else "감쇠(e^(at), a<0)"), {"r": r, "a": math.log(abs(r))}
    # 다항식 배수: k차 차분이 0 → t^k
    d = list(u)
    for k in range(1, n):
        d = [d[i + 1] - d[i] for i in range(len(d) - 1)]
        if d and all(abs(x) <= tol for x in d): return f"다항식 배수(t^{k-1})", {"deg": k - 1}
        if len(d) <= 1: break
    # 2차: u_{k+2} = 2cosθ u_{k+1} − u_k → 회전 e^(ibt)
    cs = []
    for i in range(n - 2):
        if abs(u[i + 1]) > tol: cs.append((u[i] + u[i + 2]) / (2 * u[i + 1]))
    if cs and max(cs) - min(cs) <= 1e-7 and abs(cs[0]) <= 1 + 1e-9:
        th = math.acos(max(-1, min(1, cs[0])))
        return "회전(i)·주기(π)", {"theta": th, "turn": th / (2 * math.pi)}
    return "분류 밖(정의: 패턴표 밖)", {}

def closure(turn, qmax=64, tol=1e-9):
    """연분수로 turn = θ/2π 에 가장 가까운 p/q 를 구해 닫힘(q걸음에 한 바퀴)을 판정."""
    f = Fr(turn).limit_denominator(qmax)
    return (f.numerator, f.denominator) if abs(float(f) - turn) <= tol else None

def m5():
```

```
    head("M5 오일러 패턴 분류와 닫힘")
    cases = {
        "TCP 재전송 간격 1,2,4,8,16": [1, 2, 4, 8, 16],
        "하향(TTL식 반감) 64,32,16,8": [64, 32, 16, 8],
        "ARP 5상태 순환(코사인 표본)": [math.cos(2 * math.pi * k / 5) for k in range(12)],
        "keepalive 주기 7샘플/바퀴": [math.cos(2 * math.pi * k / 7) for k in range(14)],
        "누적 카운터 선형 0,5,10,15,20": [0, 5, 10, 15, 20],
        "누적 카운터 2차 0,1,4,9,16": [0, 1, 4, 9, 16],
        "링크 UP 상수열 1,1,1,1": [1, 1, 1, 1],
        "무작위 3,1,4,1,5,9": [3, 1, 4, 1, 5, 9],
    }
    for k, u in cases.items():
        lab, info = classify_series(u)
        extra = ""
        if "turn" in info:
            cl = closure(info["turn"]); extra = f"  한 바퀴 = {cl[1]}걸음 (θ/2π={cl[0]}/{cl[1]})" if cl else "  닫히지 않음"
        print(f"  {k:30s} → {lab}{extra}")
    assert classify_series([1, 2, 4, 8, 16])[0].startswith("성장")
    assert classify_series([math.cos(2 * math.pi * k / 7) for k in range(14)])[1]["turn"] > 0
    assert closure(classify_series([math.cos(2 * math.pi * k / 7) for k in range(14)])[1]["turn"]) == (1, 7)
    assert closure(classify_series([math.cos(2 * math.pi * k / 5) for k in range(12)])[1]["turn"]) == (1, 5)
    # i^n mod 4 규칙: 정수 회전은 4걸음마다 닫힌다
    z = 1 + 0j; seq = []
    for n in range(8):
        seq.append(complex(round(z.real), round(z.imag))); z *= 1j
    assert seq[:4] == [1, 1j, -1, -1j] and seq[4:] == seq[:4]
    print("  i^n mod 4: 1, i, −1, −i 반복 (4걸음 닫힘). TCP 상태 4개를 한 바퀴 순환시키면 같은 규칙")
    # 무리수 각도는 닫히지 않는다: θ/2π 가 유리수가 아니면 limit_denominator 안에서 닫힘이 없다
    irr = math.sqrt(2) - 1
    assert closure(irr, qmax=64, tol=1e-9) is None
    print("  √2−1 (무리수) 한 바퀴 비율: 분모 64 이하로는 닫히지 않음 → 패턴표 밖(정의)으로 보고")

# ──────────────────────────────────────────────────────────────────────
# M6. 그래프 상수 L=(V,T): 라플라시안, 2계 = L², 홉 수 = 인접행렬 거듭제곱, 수식↔그림 왕복
# ──────────────────────────────────────────────────────────────────────
def laplacian(nodes, edges):
    ix = {v: i for i, v in enumerate(nodes)}
    n = len(nodes)
    A = [[Fr(0)] * n for _ in range(n)]
    for (u, v, t) in edges:
        A[ix[u]][ix[v]] += Fr(t); A[ix[v]][ix[u]] += Fr(t)
    L = [[-A[i][j] for j in range(n)] for i in range(n)]
    for i in range(n): L[i][i] = sum(A[i])
    return A, L

def matmul(X, Y):
    n = len(X)
    return [[sum(X[i][k] * Y[k][j] for k in range(n)) for j in range(n)] for i in range(n)]

def draw(nodes, edges):
    return "  ".join(f"{u} -({t})- {v}" for u, v, t in edges)
def parse(text):
    es = []
    for part in text.split("  "):
        u, mid, v = part.split(" ")[0], part.split(" ")[1], part.split(" ")[2]
        es.append((u, v, int(mid.strip("-()"))))
    return es

def m6():
    head("M6 그래프 상수 L")
    for title, nodes, edges in (("랩 (client–server)", ["client", "server"], [("client", "server", 1)]),
                                ("라우터 경유 (client–router–server)", ["client", "router", "server"],
                                 [("client", "router", 1), ("router", "server", 1)])):
        A, L = laplacian(nodes, edges)
        n = len(nodes)
        assert all(sum(row) == 0 for row in L), "행 합 = 0"
        assert all(L[i][j] == L[j][i] for i in range(n) for j in range(n))
        L2 = matmul(L, L)
        assert all(sum(row) == 0 for row in L2)
        # 홉 수: (A^k)[c][s] > 0 인 최소 k
        P = A; k = 1
        while P[0][n - 1] == 0 and k < n: P = matmul(P, A); k += 1
        print(f"  {title}: L 행 합 0 · 대칭 · L² 행 합 0 확인, client→server 홉 수 = {k} (traceroute 홉과 같아야 함)")
        print("   L =", [[str(x) for x in row] for row in L])
        print("   L² =", [[str(x) for x in row] for row in L2])
        # 수식 ↔ 그림 왕복
        pic = draw(nodes, edges)
        A2, L2b = laplacian(nodes, parse(pic))
        assert L2b == L
        print(f"   그림 \"{pic}\" → 다시 L: 정확히 같음")
    assert k == 2

# ──────────────────────────────────────────────────────────────────────
# M7. 국소화: 묶의 합 = 한도 (망원 합).  curl -w 의 누적 시각을 구간 묶으로 나눈다.
# ──────────────────────────────────────────────────────────────────────
def m7():
    head("M7 지연의 국소화 (망원 합, 실제 curl 측정)")
    import socket, threading, http.server, functools, tempfile
    d = tempfile.mkdtemp(); open(os.path.join(d, "index.html"), "w").write("ok\n" * 2000)
    handler = functools.partial(http.server.SimpleHTTPRequestHandler, directory=d)
    class Q(handler.func):
```

```
        def log_message(self, *a): pass
    handler = functools.partial(Q, directory=d)
    srv = http.server.ThreadingHTTPServer(("127.0.0.1", 0), handler)
    port = srv.server_address[1]
    threading.Thread(target=srv.serve_forever, daemon=True).start()
    fmt = "%{time_namelookup} %{time_connect} %{time_appconnect} %{time_pretransfer} %{time_starttransfer} %{time_total}"
    ok = 0
    shares_acc = None
    for _ in range(20):
        out = subprocess.run(["curl", "-s", "-o", "/dev/null", "-w", fmt, f"http://127.0.0.1:{port}/index.html"],
                             capture_output=True, text=True).stdout.split()
        us = [round(float(x) * 1_000_000) for x in out]        # 정수 μs로 내림(반올림 엄밀성)
        dns, con, app, pre, st, tot = us
        tls = max(app - con, 0)
        parts = {"DNS(L7)": dns, "TCP 연결(L4)": con - dns, "TLS(L6)": tls,
                 "요청 준비": pre - max(con, app), "서버 처리(L7)": st - pre, "전송(L7)": tot - st}
        assert sum(parts.values()) == tot                       # 몫의 합 = 한도, 정확한 정수 등식
        ok += 1; shares_acc = parts
    print(f"  curl 20회 모두 몫의 합 == time_total (정수 μs 정확 등식) {ok}/20")
    print("  마지막 측정 몫(μs):", shares_acc)
    srv.shutdown()

# ──────────────────────────────────────────────────────────────────────────
# M8. 정수 추적과 대표 크기 (명제 20.3): 포트 분류 함수의 술어 칸마다 대표 입력
# ──────────────────────────────────────────────────────────────────────────
def port_class(p):
    if p <= 1023: return "시스템"
    if p <= 49151: return "등록"
    return "동적"

def rtp_even(p):                        # 주기 술어(나머지)가 있는 예: 16384..32767 짝수 포트
    return 16384 <= p <= 32767 and p % 2 == 0

def classes_of(pred_fn, domain):
    d = {}
    for p in domain: d.setdefault(pred_fn(p), []).append(p)
    return d

def m8():
    head("M8 정수 추적과 대표 크기")
    dom = range(0, 65536)
    for name, fn, thr, mods in (("port_class", port_class, (1023, 49151), (1,)), ("rtp_even", rtp_even, (16383, 32767), (2,))):
        cl = classes_of(fn, dom)
        M, L = max(thr) + 1, 1
        for m in mods: L = L * m // gcd(L, m)
        reps = set()
        for c in thr: reps |= {c - 1, c, c + 1}
        reps |= {0, 1, 65535}
        for c in thr:                   # 문턱 양쪽에서 각 나머지류를 모두 지나게
            for r in range(L): reps.add(c + 1 + r); reps.add(c - r)
        reps = {x for x in reps if 0 <= x < 65536}
        hit = {fn(x) for x in reps}
        assert hit == set(cl), (name, hit, set(cl))
        print(f"  {name}: 전수 열거 칸 {len(cl)}개, 대표 {len(reps)}개로 모든 칸을 덮음 (M={M}, L={L}), 전수 {len(dom)}개 대 대표 {len(reps)}개")
    print("  대표 입력만 통과하면 65536개 포트 전체 칸이 통과 — 술어가 아핀(상수 비교)·주기(나머지)뿐일 때")

# ──────────────────────────────────────────────────────────────────────────
# M9. 유한체 GF(p) 실행과 슈바르츠–지펠 오판 상한
# ──────────────────────────────────────────────────────────────────────────
def m9():
    head("M9 GF(p) 무작위 실행 (p = 2^61−1)")
    rnd = random.Random(7)
    trials = 20
    for n in range(2, 41, 6):
        row = pascal_row(n)
        for _ in range(trials):
            a, b = rnd.randrange(P61), rnd.randrange(P61)
            lhs = pow(a + b, n, P61)
            rhs = sum(row[r] % P61 * pow(a, n - r, P61) * pow(b, r, P61) for r in range(n + 1)) % P61
            assert lhs == rhs
    print("  이항 계수표가 n=2,8,...,38에서 GF(p) 무작위 a,b 20회씩 통과")
    caught = total = 0
    for n in (6, 12, 24):
        row = pascal_row(n)
        for r in range(1, n):
            bad = row[:]; bad[r] += 1
            total += 1
            for _ in range(2):          # 독립 시행 두 번
                a, b = rnd.randrange(P61), rnd.randrange(P61)
                rhs = sum(bad[k] % P61 * pow(a, n - k, P61) * pow(b, k, P61) for k in range(n + 1)) % P61
                if rhs != pow(a + b, n, P61): caught += 1; break
    print(f"  틀린 계수표(변이체) {total}개 중 {caught}개가 반례로 잡힘")
    assert caught == total
    n = 40
    print(f"  오판 확률 상한: 비교 1회 ≤ deg/p = {n}/(2^61−1) = {n / P61:.2e}, 독립 2회 ≤ {(n / P61) ** 2:.2e}")

# ──────────────────────────────────────────────────────────────────────────
# M10. 변이 분석: 설정(상수·매개변수) 표를 일부러 틀리게 해 검증기가 잡는가 (K_ 는 잡히고 P_ 는 잡을 기준이 없다)
# ──────────────────────────────────────────────────────────────────────────
def m10():
    head("M10 변이 분석 (설정 표)")
```

```
    K = {n: (v, d) for n, v, d in ITEMS if d is not None}
    P = {n: v for n, v, d in ITEMS if d is None}
    tot = caught = 0
    for n, (v, d) in K.items():
        for mut in (v + 1, v - 1, v * 2, v // 2):
            tot += 1; caught += classify(n, mut, d)[0] == "반례"
    print(f"  K_ 변이체 {tot}개 중 {caught}개 반례 → 변이 점수 {caught}/{tot}")
    assert caught == tot
    print(f"  P_ {len(P)}개 항목은 도출 규칙이 없어 판정 기준이 없다 → 값 자체는 호출 때 정하고, 형태(정수인가, 범위인가)만 검사")
    def v_param(name, val):
        if name.startswith("P_PORT") or name.startswith("P_L4_PORT"): return isinstance(val, int) and 0 <= val < 65536
        if name.startswith("P_L3_TTL"): return isinstance(val, int) and 1 <= val <= 255
        return True
    bad = [(n, v) for n, v in (("P_PORT     실습 HTTP 포트", 70000), ("P_L3_TTL    기본 TTL", 0)) if not v_param(n, v)]
    print("  P_ 형태 검사 반례:", bad)

# ──────────────────────────────────────────────────────────────────────
# M11. 확률 명세: 비율 편차 구간, 손실 검출, 놓칠 확률
# ──────────────────────────────────────────────────────────────────────
def hoeffding(N, delta): return math.sqrt(math.log(2 / delta) / (2 * N))

def m11():
    head("M11 비율 편차 구간 (손실률 판정)")
    rnd = random.Random(11)
    N, delta, p = 200, 0.05, 0.10
    eps = hoeffding(N, delta)
    cover = 0; trials = 2000; detect = 0
    for _ in range(trials):
        loss = sum(rnd.random() < p for _ in range(N))
        ph = loss / N
        cover += (ph - eps <= p <= ph + eps)
        detect += (ph - eps > 0.01)              # 정상 기준 1% 초과 손실을 검출
    print(f"  N={N}, δ={delta}: ε = {eps:.4f}  → 참 손실률을 덮은 시행 {cover}/{trials} = {cover/trials:.3f} (보증 ≥ {1-delta})")
    print(f"  netem loss 10% 검출(하한 > 1%) {detect}/{trials}  ← N이 작으면 구간이 넓어 놓칠 수 있다")
    N2 = 1000; eps2 = hoeffding(N2, delta); det2 = 0
    for _ in range(trials):
        ph = sum(rnd.random() < p for _ in range(N2)) / N2
        det2 += (ph - eps2 > 0.01)
    print(f"  N={N2}: ε = {eps2:.4f}, 검출 {det2}/{trials}  (표본이 늘면 같은 보증으로 검출력이 오른다)")
    assert det2 / trials > detect / trials
    assert cover / trials >= 1 - delta
    for Nn in (10, 20, 50, 100):
        print(f"  손실률 10%에서 ping -c {Nn:3d} 이 손실을 하나도 못 볼 확률 (1−p)^N = {(1-p)**Nn:.4f}")
    need = math.ceil(math.log(0.01) / math.log(1 - p))
    print(f"  손실 놓침 확률을 1% 아래로 하려면 ping 횟수 N ≥ {need}")

# ──────────────────────────────────────────────────────────────────────
# M12. 육하원칙 보고 (지위: 증명·반례·정의)
# ──────────────────────────────────────────────────────────────────────
def report(verdict, who, what, when, where, why, how, status):
    rows = [("판정", verdict, status["판정"]), ("누가", who, status["누가"]), ("무엇을", what, status["무엇을"]),
            ("언제", when, status["언제"]), ("어디서", where, status["어디서"]), ("왜", why, status["왜"]), ("어떻게", how, status["어떻게"])]
    return rows

def m12():
    head("M12 육하원칙 보고 (M1b 변이체 하나를 보고)")
    name, val, d = ITEMS[0]
    mut = val - 1
    c, why = classify(name, mut, d)
    assert c == "반례"
    rows = report("반례", f"K_ 검증기 classify()", f"{name.split()[0]} 적힌 값 {mut} ≠ 계산값 {derive_count(16)}",
                  time.strftime("%Y-%m-%d %H:%M:%S"), "layers.env / lab.env 의 해당 상수 한 줄",
                  "적힌 값이 sum C(16,r) = 2^16 과 1 어긋남", f"값을 {derive_count(16)}로 되돌려 다시 판정 → 통과",
                  {"판정": "반례", "누가": "증명", "무엇을": "정의", "언제": "정의", "어디서": "정의", "왜": "증명", "어떻게": "증명"})
    assert classify(name, derive_count(16), d)[0] == "K_"
    for q, a, s in rows: print(f"  {q:4s}: {a}   [{s}]")

# ──────────────────────────────────────────────────────────────────────
# M13. 판정 지도: 명세의 종류 → 쓸 장치 (본문 21.5, 32절 질문 트리의 축약)
# ──────────────────────────────────────────────────────────────────────
def route(spec):
    if not spec.get("computable", True): return "정의 25.2: 자원을 늘려 가는 후보 목록과 육하원칙 처방"
    if spec.get("external_value"): return "블룩체인형: 계산 층 사다리, 집계 층 그래프 상수, 진실 층 사이클"
    if spec.get("no_label"): return "세계 증인(반례) 또는 δ-증명"
    kind = spec["kind"]
    return {"algebraic": "정확 등식 (GF 실행·정규형 일치)",
            "probability": "비율 편차 δ-닫힘",
            "order": "결정 열거(대수 부분) + 실행 검사의 부등식",
            "limit": "증폭률 거듭제곱 상한 + 사다리 허용치",
            "transcendental": "사다리 꼬리 + π 연분수",
            "rounding": "반올림 부등식(정수로 내려 비교)"}[kind]

def m13():
    head("M13 판정 지도")
    obs = [("MTU 설정 일치(1500=1500)", dict(kind="algebraic")), ("헤더 필드·MAC 일치", dict(kind="algebraic")),
           ("패킷 손실률", dict(kind="probability")), ("TCP 상태 전이 순서", dict(kind="order")),
           ("재전송 백오프 수렴", dict(kind="limit")), ("지연 시간(ms, 참값은 연속)", dict(kind="transcendental")),
           ("curl 시간 초→μs 변환", dict(kind="rounding")),
           ("외부 DNS 응답이 실제와 같은가", dict(external_value=True)),
           ("정답 표지 없는 이상 탐지", dict(no_label=True)),
           ("정지 문제형 질문", dict(computable=False))]
```

```
    for n, s in obs: print(f"  {n:30s} → {route(s)}")

# ──────────────────────────────────────────────────────────────────────
# M14. 세션 파이프라인: 점화식 씨앗 하나로 과거·현재·미래 값 → 쿠키(L5), 세션 ID(L5), AES(L6)
# ──────────────────────────────────────────────────────────────────────
class Oracle:
    def __init__(self, secret, ctx): self.seed = hmac.new(secret, b"seed|" + ctx, hashlib.sha256).digest()
    def term(self, k): return hmac.new(self.seed, b"term|" + struct.pack(">q", k), hashlib.sha256).digest()[:16]

def m14():
    head("M14 세션 보안 파이프라인 (L5·L6)")
    secret = b"x" * 32
    O = Oracle(secret, b"lab")
    k = 100
    past, present, future = O.term(k - 1), O.term(k), O.term(k + 1)
    assert O.term(k - 1) == past                                  # 씨앗 하나로 같은 순서 값 재생성(표 저장 불필요)
    body = struct.pack(">q", k) + past
    sig = hmac.new(secret, b"cookie|" + body, hashlib.sha256).digest()[:16]
    cookie = (body + sig).hex()
    def verify(tok):
        raw = bytes.fromhex(tok); b, s = raw[:-16], raw[-16:]
        if not hmac.compare_digest(s, hmac.new(secret, b"cookie|" + b, hashlib.sha256).digest()[:16]): return None
        o = struct.unpack(">q", b[:8])[0]
        return o if hmac.compare_digest(b[8:], O.term(o - 1)) else None
    assert verify(cookie) == k
    forged = bytearray(bytes.fromhex(cookie)); forged[10] ^= 1
    assert verify(forged.hex()) is None
    print("  L5 쿠키(과거값): 정상 통과, 1비트 위조는 거부")
    sid = hmac.new(present, b"session|req-1", hashlib.sha256).hexdigest()
    sid2 = hmac.new(O.term(k + 5), b"session|req-1", hashlib.sha256).hexdigest()
    assert sid != sid2
    print(f"  L5 세션 ID(현재값) {sid[:16]}… , 다른 순서의 값이면 다른 ID")
    try:
        from cryptography.hazmat.primitives.ciphers.aead import AESGCM
        from cryptography.hazmat.primitives.kdf.hkdf import HKDF
        from cryptography.hazmat.primitives import hashes
        key = HKDF(algorithm=hashes.SHA256(), length=32, salt=sid.encode(), info=b"aes-response").derive(future)
        nonce = os.urandom(12); ct = AESGCM(key).encrypt(nonce, b"response-ok", sid.encode())
        assert AESGCM(key).decrypt(nonce, ct, sid.encode()) == b"response-ok"
        wrong = HKDF(algorithm=hashes.SHA256(), length=32, salt=sid.encode(), info=b"aes-response").derive(O.term(k + 2))
        try: AESGCM(wrong).decrypt(nonce, ct, sid.encode()); raise SystemExit("복호가 되면 안 됨")
        except Exception as ex:
            if isinstance(ex, SystemExit): raise
        print("  L6 AES-GCM(미래값): 맞는 미래값으로만 복호, 다른 순서의 값은 복호 실패")
    except ImportError:
        print("  (cryptography 없음: AES 단계는 건너뜀 — 정의(범위 밖))")
    # 일방향 확인: 토큰에서 씨앗을 되짚을 수 없다(HMAC은 다대일)
    print("  일방향: 쿠키·세션 ID에서 씨앗이나 다른 순서의 값을 복원하는 경로가 없다 (HMAC은 다대일 수축)")


def m15():
    head("M15 bash가 만든 K_ 와 파이썬 도출값의 일치")
    here = os.path.dirname(os.path.abspath(__file__))
    out = subprocess.run(["bash", "-c", f"source {here}/labtest.env; source {here}/layers.env; source {here}/ascii.env; "
                          "echo $K_N_PORT $K_N_IPV4 $K_N_MAC $K_N_TTL $K_LINKS_LAB $K_ASCII_SIZE"],
                         capture_output=True, text=True).stdout.split()
    want = [derive_count(16), derive_count(32), derive_count(48), derive_count(8), derive_pairs(2), derive_count(7)]
    assert [int(x) for x in out] == want, (out, want)
    print("  bash:  ", out)
    print("  python:", [str(x) for x in want], " → 정확히 일치 (같은 점화식을 두 언어로 따로 구현)")
    # readonly: 변경 시도는 실패
    r = subprocess.run(["bash", "-c", f"source {here}/labtest.env; (K_N_PORT=1); echo $K_N_PORT"], capture_output=True, text=True)
    assert "readonly" in r.stderr and r.stdout.strip() == "65536"
    print("  K_N_PORT=1 시도 → readonly variable 오류, 값은 65536 유지")
    # P_ 는 덮어쓰기 가능
    r = subprocess.run(["bash", "-c", f"P_PORT=9090; source {here}/labtest.env; echo $P_PORT"], capture_output=True, text=True)
    assert r.stdout.strip() == "9090"
    print("  P_PORT=9090 (호출 때 확정) → source 후에도 9090 유지: 매개변수는 호출이 정한다")

ALL = ["m1", "m1b", "m2", "m3", "m4", "m5", "m6", "m7", "m8", "m9", "m10", "m11", "m12", "m13", "m14", "m15"]
if __name__ == "__main__":
    want = sys.argv[1:] or ["all"]
    for f in (ALL if want == ["all"] else want):
        globals()[f]()
    print("\n모든 단언 통과")
```

이 절의 대응도 정의(설계 결정)입니다. 알고리즘이 돌고 단언이 통과한다는 것은 구현이 의도한 식과 같다는 뜻이며, P≠NP로 가는 연결은 여전히 H-11의 대응 가정으로 분리되어 있습니다.

H-15. 남은 일

- ☐ "만난다 := 정확한 등식으로 일치함"을 일방향 벡터 정의의 첫 문장에 명시
- ☐ '일방향 벡터'와 조립제법 논증을 본문으로 올리기(본문에는 두 단어가 0회)
- ☐ '패턴' 용어 3종 구분표 작성(H-3)
- ☐ MQ 다리의 간극(체, 최악/평균)을 한 문단으로 명시(H-11)
- ☐ 패턴 검증기를 `tcpdump`·`ss` 출력과 연결(재전송 간격, ARP 상태 순환). 이번에는 랩 네임스페이스가 없어 단위 검증까지만 확인함
- ☐ L2~L4 헤더 크기·오프셋(`P_ETH_*`, `P_IP_*`, `P_TCP_*`)을 `layers.env`에 통합
- ☐ 유지보수_261103_032620 (1).pdf는 텍스트가 없어 읽지 못함

부록 I. 개념별 시니어 학습 가이드 — 수학 개념 15종 × 실제 상황

각 개념마다 시니어가 실제로 마주치는 상황, 보는 곳, 적용 수식, 판독 기준을 표로 묶었다. 수치는 `seniorlab.py`(부록 L)가 실행해 검증한 값이다. 마지막 칸은 수식이 현상을 얼마나 직접 기술하는지를 표시한다: **정확** = 수식이 현상 자체를 기술, **근사** = 조건부로 성립, **유추** = 구조가 닮은 운영 규칙.

I-1. 점화식

정의 $a_n = f(a_{n-1})$: 앞 값에서 다음 값이 정해진다 (파스칼, 나머지 사다리, 지수 이동 평균)

상황	보는 곳	수식·적용	판독	상태
연결이 안 끊기고 15분 매달림	`ss -tno` 의 timer:(on,⋯,N), `sysctl net.ipv4.tcp_retries2`	$RTO_n = \min(2 \cdot RTO_{n-1}, 120초)$. 합 = 924.6초 (S1)	타이머 값이 0.2 → 0.4 → 0.8 ⋯로 배가되면 재전송 중. 응용 타임아웃이 이 합보다 짧아야 한다	정확
SYN 후 연결 실패까지 127초	`sysctl net.ipv4.tcp_syn_retries`	1+2+4+8+16+32+64 = 127초 (S1)	connect()가 약 2분 뒤 ETIMEDOUT이면 SYN에 응답이 전혀 없는 것(DROP). RST면 거부(REJECT·포트 닫힘)	정확
RTT 한 번 튀었는데 RTO가 한동안 높다	`ss -ti` 의 rto:, rtt: 평균/편차	$SRTT = \frac{7}{8} \cdot SRTT + \frac{1}{8} \cdot R$, $RTTVAR = \frac{3}{4} \cdot RTTVAR + \frac{1}{4} \cdot \lvert SRTT - R \rvert$ (S2)	편차(var)가 크면 RTO도 크다. 평균만 보지 말고 편차를 본다	정확
요청이 쌓이며 대기 시간이 선형 증가	`ss -ltn` Recv-Q, `tc -s qdisc`	$Q_n = \max(0, Q_{n-1} + 도착 - 처리)$ (S3)	도착률 > 처리율이면 Q가 t^1로 늘어난다. 증설이나 유입 제한 외에는 해소되지 않는다	정확
재시도가 재시도를 부른다	클라이언트 재시도 정책, 요청률 그래프	$r_n = r_{n-1}(1 + 재시도율 \cdot 실패율)$ 꼴이면 e^{at} 증가	실패율이 오르면 요청률이 비율로 증가. 지수 백오프+지터와 서킷 브레이커로 끊는다	근사

연습: 첫 RTO=0.3초, 상한 60초로 바꾸면 tcp_retries2=15의 총 시간은? 손으로 점화식을 돌려 seniorlab.py의 give_up_time으로 확인하라.

I-2. 이항정리·이항계수

정의 $(a+b)^n = \Sigma\, C(n,k) a^{n-k} b^k$. C(n,k)는 n개 중 k개를 고르는 방법의 수 (파스칼 삼각형)

상황	보는 곳	수식·적용	판독	상태
요청 하나가 느릴 확률은 작은데 화면은 자주 느림	팬아웃(한 요청이 부르는 하위 요청 수)	$1-(1-q)^n \approx n \cdot q$, 오차 ≤ $C(n,2)q^2$ (S6, S19)	n=100, q=0.1%면 화면 지연 확률 9.5%. 꼬리 지연은 팬아웃에 곱해진다	정확
직렬 구성요소가 늘수록 가용성이 떨어짐	구성도(LB→웹→앱→DB→스토리지)	$(1-q)^n \approx 1-nq$. 10단 × 99.9% → 99.0% (S6)	구성요소 수가 많으면 각각의 가용성보다 개수가 지배한다	정확
노드 5대에서 몇 대까지 죽어도 되나	클러스터 쿼럼(etcd, ZooKeeper, Raft)	$P(과반 생존) = \Sigma_{k \geq 3} C(5,k) p^k (1-p)^{5-k}$ (S6)	p=99%에서 0.99999. 동시에 2대 죽는 경우 C(5,2)=10가지를 각각 대비	정확
여러 계층이 동시에 고장 난 듯함	증상 + 계층별 검증기	후보 조합 C(7,1)+C(7,2) = 28가지 (S6)	검증기 결과가 계층 하나만 가리키지 않으면 쌍 후보로 넓혀 본다	정확

트래픽이 서버 4대에 안 고르게 갈림	LB 통계, `ip route get`	한 서버가 받는 흐름 수 ~ 이항분포 B(N, 1/k) (S10)	흐름 100개면 상대 편차 17%, 10000개면 1.7%. 적은 흐름에서는 쏠림이 정상 범위일 수 있다	정확

연습: 서버 3대, 각 가용성 99.5%에서 "최소 2대 생존" 확률을 이항합으로 계산하고, 1차 근사와 비교하라.

I-3. 미분 (b^1 계수 추출, 변화율)

정의 $((a+b)^n - a^n)/b = n \cdot a^{n-1}$ + (b의 고차). 변화율은 차분 몫에서 b^1 계수

상황	보는 곳	수식·적용	판독	상태
이용률 90%를 넘자 갑자기 느려짐	CPU 이용률, 큐 길이, p99 지연	$W = 1/(\mu-\lambda)$, $dW/d\lambda = 1/(\mu-\lambda)^2$ (S4)	50%→99%에서 지연 50배, 민감도 2500배. 이용률 한계를 ~70–80%로 두는 근거	근사
카운터에서 초당 값을 구해야 함	`nstat`, `ip -s link`, `/proc/net/dev`	rate = $(C_t - C_{t-\Delta})/\Delta$ (차분 몫)	누적 값은 절대값이 아니라 기울기를 본다. 카운터 되감김(wrap)과 재부팅 구간은 제외	정확
작은 설정 변경이 큰 효과	설정 변경 전후 지표	1차 효과 = $n \cdot a^{n-1} \cdot b$. b가 작으면 1차가 지배, 크면 고차 꼬리가 중요	변경폭을 키우기 전에 작은 폭으로 기울기를 먼저 잰다	근사

연습: μ=200일 때 λ=150, 180, 195에서 $dW/d\lambda$를 구하고 "여유 25%"가 안전한지 판단하라.

I-4. 적분과 적분부등식

정의 ∫ 누적량. 구간 점화식 I_n의 하한·상한으로 값을 가두는 부등식 $[I_n+L, I_n+U]$

상황	보는 곳	수식·적용	판독	상태
월 가용성 99.9%를 지켰나	시간별 오류율 집계	허용 오류 43.2분. 하한 ≤ ∫e ≤ 상한로 가둔 뒤 상한≤예산이면 합격, 하한>예산이면 불합격, 걸치면 불확정 (S12)	불확정이면 해상도를 높인다. 판정 3값(증명·반례·정의)과 같은 구조	정확
월 트래픽 요금·용량	`vnstat`, `sar -n DEV`	누적 바이트 = ∫ rate dt (구간 합으로 근사, 상·하합 사이에 가둠)	샘플 간격이 길수록 구간이 넓다. 95퍼센타일 과금은 적분이 아니라 정렬 문제	근사
디스크가 언제 차나	`df -h` 추이	사용량 U(t), 증가율 u′ 일정이면 t* = (용량−U)/u′	증가율이 일정(t^1)인지 가속(e^{at})인지를 I-6으로 먼저 분류	근사

연습: 오류 예산 43.2분 중 첫 2주에 30분 소진했다. 남은 2주의 허용 평균 오류율은?

I-5. 변수분리

정의 $dy/dx = f(x)g(y)$를 $\int dy/g = \int f\,dx$로 갈라 따로 푼다. 곱 꼴은 로그에서 합

상황	보는 곳	수식·적용	판독	상태
1Gbps 회선인데 WAN에서 10Mbps	`ss -ti` 의 rwnd, wscale, `sysctl net.ipv4.tcp_rmem`	처리량 = min(링크, 창/RTT, Mathis) 인자별 독립 계산 (S7)	창 64KB / 50ms = 10.5Mbps. 창 확대(wscale)와 버퍼 한도를 본다	정확
손실 0.01%로도 처리량이 낮음	`ping -c 1000`, `nstat` 재전송	$T \approx (MSS/RTT)\cdot(C/\sqrt{p})$. RTT와 손실이 곱으로 들어옴 (S7)	RTT ×2, 손실 ×4 → T ×¼. 어느 인자가 바뀌었는지 따로 본다	근사

"한 번에 하나만 바꾼다"	변경 관리 절차	독립 인자를 한 번에 하나씩 → 효과가 분리됨 (변수 분리의 운영 규칙)	두 개를 동시에 바꾸면 곱 꼴 효과를 나눌 수 없다	**유추**
계층별 독립 점검	lab.env의 P_ 매개변수와 v_ 검증기	계층 변수(주소·포트·경로)를 서로 독립으로 두고 계층마다 따로 검증	독립이 깨지는 곳(NAT, 터널)에서는 한 변수가 여러 계층에 걸친다	**유추**

연습: RTT 20ms, 손실 0.1%, 창 256KB, 링크 100Mbps의 병목 인자를 각각 계산해 정하라.

I-6. 오일러 분해·패턴 분류

정의 $t^k e^{\lambda t}(\cos bt + i \sin bt)$: 성장·감쇠($e^{\lambda t}$), 회전·주기(cos, sin), 다항식 배수(t^k), 기준값(0·1)

상황	보는 곳	수식·적용	판독	상태
메모리가 서서히 오름	`ps -eo pid,rss,cmd --sort=-rss` 시계열	일정 속도 증가 → t^1 (누수) (S11)	기울기로 OOM 시각을 예측하고 재시작 주기를 정한다	**정확**
요청이 비율로 폭증	요청률·큐 길이 그래프	일정한 비율 증가 → e^{at} (증폭) (S11)	로그 축에서 직선이면 지수. 되먹임 고리(재시도, 큐)를 찾는다	**정확**
장애 후 에러율이 줄어듦	에러율 그래프	일정 비율 감소 → e^{-at} (회복)	반감기 = ln2/a. 반감기가 길면 회복 경로(캐시 워밍, 큐 소진)를 확인	**정확**
정확히 같은 주기로 튐	cron 표, GC 로그, `ip monitor link`	cos 성분 → 주기 T (S11). 연분수로 주기 닫힘 확인	주기 = 원인의 주기(cron, GC, flap, keepalive). 주기부터 알면 원인 후보가 줄어든다	**정확**
기준값에서 벗어남	정상 상태 값(0 또는 1에 대응)	기준값(상수열). 이탈 = 이상	정상 기준을 먼저 정해 두어야 이상을 판정한다	**정확**

연습: 자신의 서버 지표 5개를 5가지 패턴 중 하나로 분류하고, 분류 밖이면 "모름"으로 남겨라.

I-7. 조립제법·나머지·유한체

정의 f(x)를 (x−c)로 나눈 나머지 f(c). GF(2) 나머지가 CRC, mod k가 순환

상황	보는 곳	수식·적용	판독	상태
프레임 손상	`ethtool -S eth0 | grep -i crc`	FCS = GF(2)에서 나눈 나머지. 온전하면 나머지 0 (M4)	rx_crc_errors 증가 = 케이블·SFP·듀플렉스. 놓칠 확률 2^{-32} (S17)	**정확**
4대로 순환 분배	LB 라운드 로빈	n mod 4 (S17)	순환 분배에서 요청 순서와 서버 상태가 주기적으로 겹치면 특정 서버에 느린 요청이 몰린다	**근사**
시퀀스 번호가 한 바퀴	고속 링크, PAWS(타임스탬프) 설정	2^{32} 바이트 되감김: 10Gbps에서 3.4초 (S17)	타임스탬프가 꺼져 있으면 지연 패킷이 새 데이터로 오인될 수 있다	**정확**

연습: 100Gbps에서 PAWS 없이 시퀀스 공간이 몇 ms마다 되감기는지 계산하라.

I-8. 망원 합·국소화

정의 $\Sigma(a_{k+1}-a_k) = a_n - a_0$. 누적 시각을 구간 몫으로 분해

상황	보는 곳	수식·적용	판독	상태

요청이 느린데 어느 단계가 느린가	`curl -w "%{time_namelookup} %{time_connect} %{time_appconnect} %{time_starttransfer} %{time_total}\n"`	각 구간 = 누적값의 차. 합 = time_total (S13)	가장 큰 몫이 서버면 앱, TLS면 인증서·핸드셰이크, DNS면 해석기를 본다	정확
경로 어디서 지연이 늘었나	`mtr -rwzbc 100`	홉 i의 증가분 = $RTT_i - RTT_{i-1}$	증가가 한 홉에서 갑자기 생기고 뒤 홉에서 유지되면 그 홉 구간이 원인. 중간 홉만 높고 뒤는 정상이면 ICMP 응답 지연일 뿐	정확
로그 타임스탬프로 단계 분해	`journalctl` 시각 차	이벤트 시각 차분. 합 = 전체 시간	시계가 어긋난 호스트 간 차분은 무의미(시간 동기 먼저)	근사

연습: curl -w 값 6개를 직접 재서 단계별 몫과 비율을 계산하고 합이 time_total과 같은지 확인하라.

I-9. 이분 탐색·역탐색

정의 단조 판정에서 경계를 $\log_2 n$번에 찾는다 (r_ 역탐색)

상황	보는 곳	수식·적용	판독	상태
어느 계층이 처음 고장 났나	누적 검증기 v(k)=L1..Lk 끝-끝 확인	고장 계층 8가지 모두 최대 3회 호출 (S15)	단조성(아래가 정상이면 위를 검사)일 때만 유효. 비대칭·간헐은 선형으로	정확
배포 후 에러가 늘었다: 어느 변경?	`git bisect`, 배포 이력	변경 N개 → $\lceil \log_2 N \rceil$번 배포·롤백	변경이 독립이 아니면(상호작용) 이분이 틀린다. 변경을 쪼개 독립으로 만든다	정확
언제부터 문제였나	시계열의 정상→이상 경계	시간 구간 이분. 기준 구간과 비교	경계 시각 주변의 배포·설정·트래픽 변화를 찾는다	정확

연습: 설정 파일 64줄 중 문제 줄을 이분으로 찾는 절차를 쓰고 최대 몇 번 시험하는지 구하라.

I-10. GF(p) 다항식 지문·Schwartz–Zippel

정의 다항식 f를 무작위 점 x에서 평가하면 서로 다른 f, g가 같은 값일 확률 ≤ 차수/p

상황	보는 곳	수식·적용	판독	상태
수백 대 서버 중 설정이 다른 한 대	`sysctl -a | sort | sha256sum` 와 비교	줄 해시를 계수로 하는 다항식을 mod p에서 평가한 지문 (S14)	지문이 다르면 차이가 있다. 같으면 차수/p ≈ 10^{-17} 확률로만 틀림	정확
캐나리 배포	일부 서버만 먼저 배포	무작위 표본으로 전체를 추정 (Hoeffding과 함께)	표본이 대표성을 가지려면 무작위로 뽑아야 한다	근사

연습: 설정 줄 순서가 달라도 같은 설정으로 취급하려면 지문을 어떻게 바꿔야 하나(힌트: 줄 해시의 합).

I-11. 표본 크기·Hoeffding

정의 $P(|\hat{p}-p| \geq \varepsilon) \leq 2e^{-2n\varepsilon^2}$

상황	보는 곳	수식·적용	판독	상태
ping 100번 무손실이면 손실이 없다고 해도 되나	`ping -c 459`	손실 0건에서 p<1%를 99% 확신: $n \geq \ln \delta / \ln(1-p) = 459$ (S16)	횟수가 모자라면 "없음"이 아니라 "아직 모름"이다	정확

손실률 ±1%p 추정	부하 시험 시간 결정	$n \geq \ln(2/\delta)/(2\epsilon^2)$ = 26,492 (S16)	정밀도를 10배 높이려면 표본은 100배	정확

연습: p<0.1%, 95% 확신이면 무손실 몇 번이 필요한가.

I-12. 곱 원리·인수분해

정의 독립 선택의 수는 곱, 작은 확률의 합은 근사

상황	보는 곳	수식·적용	판독	상태
DNS가 느리다	`cat /etc/resolv.conf`	대기 = timeout × attempts × 서버 수 = 30초 (S18)	search 도메인과 ndots로 질의 수가 곱해진다 (4×2=8건)	정확
"Cannot assign requested address"	`ss -s`, `sysctl net.ipv4.ip_local_port_range`	28,232포트 / TIME_WAIT 60초 ≈ 471 연결/초 (S9)	같은 목적지로 초당 500건이 넘으면 고갈. 연결 재사용	정확
conntrack 가득 참	`conntrack -S`, `dmesg | grep conntrack`	항목 수 = r·T (Little) (S5)	2,200/s × 120초 = 264,000 > 262,144	정확

연습: 외부 호출이 하루 4천만 건, 한 목적지로 평균 500/s 연결이라면 포트 범위를 어떻게 잡나.

I-13. 연분수·주기 닫힘

정의 θ/2π를 p/q로 근사해 q걸음에 한 바퀴 닫히는지 판정

상황	보는 곳	수식·적용	판독	상태
튀는 간격이 일정하지 않아 보임	이벤트 간격 목록	간격 비율을 연분수로 p/q 근사 → 공통 주기	두 주기의 비가 유리수면 합성 패턴도 주기. 무리수에 가까우면 두 주기를 분리	근사
keepalive·ARP·DHCP 갱신 주기	`sysctl net.ipv4.tcp_keepalive_time`, ARP 상태 순환	상태 순환(i^n mod 4, 5상태)	주기가 만나는 시각에 부하가 겹친다	유추

연습: 주기 7분과 10분의 작업이 동시에 도는 간격은?

I-14. 그래프 상수 L=(V,T)

정의 라플라시안 L, 홉 수는 거듭제곱으로

상황	보는 곳	수식·적용	판독	상태
장애 영향 범위	토폴로지(LLDP, 클라우드 구성도)	연결 요소 수 = L의 영고유값 수	노드를 지웠을 때 연결 요소가 늘면 단일 장애점	정확
홉 수와 지연 하한	`traceroute -n`	홉 수 = 인접행렬 거듭제곱에서 처음 0이 아닌 거듭제곱	홉 수 × 홉당 지연이 하한. 홉이 늘어나는 우회가 생겼는지 본다	정확

연습: 서버 2대+스위치 2대의 이중화 구성에서 단일 장애점을 그래프로 찾으라.

I-15. 변이 분석

정의 코드·설정에 일부러 결함을 넣어 검사가 잡는지 센다

상황	보는 곳	수식·적용	판독	상태
내 진단 절차가 믿을 만한가	랩에서 결함 주입 (INPUT DROP, MTU, 지연, 손실)	탐지율 = 잡힌 결함 / 주입한 결함	못 잡은 결함이 검증기의 빈틈이다. 빈틈을 먼저 메운다	**정확**
카오스 실험	운영 환경 결함 주입	같은 계산을 운영에 적용	반드시 폭발 반경을 제한하고 롤백을 먼저 준비	**근사**

연습: 랩에 결함 10가지를 주입하고 v_ 검증기가 몇 개를 잡는지 표로 기록하라.

부록 J. 시니어 영역 — 커널·TCP 내부·추적·한도·LB·쿠버네티스·클라우드·TLS·DNS

각 절은 **명령 → 판독 규칙 → 수학 연결** 순서다. 출력이 있는 것은 모두 형식 설명용 예시이고, 판독 규칙이 핵심이다. 이 문서의 작성 환경에는 `ip`, `ss`, `tcpdump`, `perf`, `bpftrace`가 없어 실행 출력은 싣지 않았다(수치 계산은 부록 L에서 실행).

J-1. TCP 연결 내부 읽기 (ss -ti)

시니어는 "연결됨/안 됨"이 아니라 한 연결의 윈도우·타이머·재전송을 읽는다. 아래 출력은 형식 설명용 **예시**이며 값은 환경마다 다르다.

```
ss -tin dst 203.0.113.10
# 예시 출력 (한 연결, 줄바꿈 편집)
ESTAB 0 0 10.0.0.5:51834 203.0.113.10:443
 cubic wscale:7,7 rto:204 rtt:3.2/1.6 ato:40 mss:1448 pmtu:1500
 cwnd:10 ssthresh:20 bytes_sent:81234 bytes_acked:81234
 retrans:0/2 lost:0 unacked:0 send 36.2Mbps pacing_rate 72.4Mbps
 delivery_rate 30.1Mbps app_limited rcv_space:14600 minrtt:2.9
```

보이는 것	뜻 · 해야 할 일
`rto:204`	재전송 타이머 204ms. 최소 200ms 근처가 정상. 수 초면 손실이 반복된 것
`rtt:3.2/1.6`	평균 RTT / 편차(ms). 편차가 평균에 비해 크면 경로가 불안정
`cwnd:10 ssthresh:20`	혼잡 창 10세그먼트. cwnd가 작게 유지되면 손실로 줄었거나 아직 슬로 스타트
`retrans:0/2`	지금 재전송 중 0 / 지금까지 누적 2. 누적이 계속 오르면 경로 손실
`unacked / lost`	응답 못 받은 세그먼트 / 손실로 판단한 세그먼트
`app_limited`	네트워크가 아니라 애플리케이션이 보낼 게 없어서 느린 것. 병목은 앱
`rwnd_limited / sndbuf_limited`	(해당 항목이 보이면) 수신 창이나 송신 버퍼가 병목. 창 확대·버퍼 조정 대상
`pmtu:1500 mss:1448`	경로 MTU와 세그먼트 크기(타임스탬프 옵션 12바이트 제외). 작아졌다면 경로 MTU 문제

병목 판별 순서: ① app_limited면 앱 ② rwnd_limited면 수신측 ③ cwnd_limited(cwnd가 가득 참)면 혼잡 제어·손실 ④ sndbuf_limited면 송신 버퍼. 수학 연결: I-1(RTO·SRTT 점화식), I-5(창/RTT, Mathis).

J-2. 커널 수신 경로와 드롭 위치 찾기

NIC 링 버퍼 → 하드 인터럽트 → softirq(NET_RX) → backlog → 소켓 버퍼 → accept 큐. 패킷이 어디서 버려지는지는 단계마다 보는 카운터가 다르다.

```
ethtool -S eth0 | egrep "rx_missed|rx_fifo|rx_dropped|rx_no_buffer"   # 1) NIC 링 버퍼 넘침
ethtool -g eth0                          # 링 버퍼 크기(현재/최대)
cat /proc/net/softnet_stat               # 2) CPU별 softirq 처리: 3번째 열 time_squeeze, 2번째 열 dropped
mpstat -P ALL 1 3                        # %soft 가 한 코어에 몰리는지
cat /proc/interrupts | egrep "eth0|mlx|ens"   # 인터럽트가 코어에 퍼지는지
nstat -az TcpExtListenOverflows TcpExtListenDrops   # 3) accept 큐 넘침
ss -ltn                                  # LISTEN 의 Recv-Q = 대기 중, Send-Q = 최대 backlog
nstat -az UdpRcvbufErrors UdpInErrors    # 4) UDP 수신 버퍼 넘침
cat /proc/net/sockstat                   # 소켓 개수·메모리
```

보이는 것	뜻 · 해야 할 일
`rx_missed / rx_fifo_errors 증가`	NIC가 링 버퍼가 차서 버림. ethtool -G로 링 확대, 인터럽트 분산(RSS), 큐 수(ethtool -l)

`softnet_stat 의 dropped 증가`	소프트웨어 backlog 넘침. net.core.netdev_max_backlog 증가 또는 CPU 분산(RPS/RFS)
`time_squeeze 증가`	softirq가 예산(netdev_budget) 안에 다 못 끝냄. 한 코어에 부하 집중
`ListenOverflows 증가`	애플리케이션이 accept()를 못 따라감. somaxconn과 앱 backlog, 워커 수 점검
`UdpRcvbufErrors 증가`	UDP 소켓 버퍼 부족. net.core.rmem_max와 앱의 SO_RCVBUF 조정
`ss -ltn 의 Recv-Q가 Send-Q에 근접`	대기열이 가득 참(오버플로 직전)

드롭 위치는 "수신 경로의 앞에서 뒤로" 순서로 카운터를 본다(I-9 이분 탐색). 카운터는 누적이므로 두 번 재서 차분(I-3)으로 초당 값을 구한다.

J-3. 연결 추적(conntrack)과 NAT

상태 기반 방화벽·NAT는 연결 추적표에 의존한다. 표가 차면 새 연결이 조용히 버려진다.

```
cat /proc/sys/net/netfilter/nf_conntrack_count /proc/sys/net/netfilter/nf_conntrack_max
sysctl net.netfilter.nf_conntrack_buckets
conntrack -S                                  # insert_failed, drop, early_drop 증가 여부
conntrack -L | awk '{print $4}' | sort | uniq -c | sort -rn | head   # 상태별 분포
dmesg -T | grep -i "nf_conntrack: table full"
```

보이는 것	뜻 · 해야 할 일
`count가 max에 근접`	표가 가득 참. 새 연결률 r × 유지 시간 T = 항목 수 (S5)
`insert_failed 증가`	동시 연결 경합(같은 5-튜플 충돌). DNS(UDP) 병렬 질의와 NAT에서 흔함
`drop / early_drop 증가`	표가 차서 버림. max 증가 또는 TIME_WAIT 항목 타임아웃 단축, 불필요한 추적 제외(NOTRACK)
`dmesg: table full, dropping packet`	실제 드롭 발생. 증상: 새 연결만 실패, 기존 연결은 정상

평균 체인 길이 = 항목 수 / 버킷 수. 버킷이 적으면 같은 계산량이 늘어난다. 수학 연결: I-12(r·T), I-7(해시 mod).

J-4. perf·bpftrace·strace — 무엇이 CPU를 쓰고 어디서 기다리나

애플리케이션이 원인인지 커널이 원인인지는 추적으로 가른다. 운영 환경에서는 오버헤드가 작은 도구부터 쓴다.

```
vmstat 1 5                                  # r(실행 대기) > 코어 수, b(블록), wa(IO 대기), cs(문맥 전환)
pidstat -u -w -p <PID> 1 5                  # 프로세스별 CPU, 자발/비자발 문맥 전환
perf top -g                                 # 지금 CPU를 쓰는 함수(실시간)
perf record -F 99 -g -p <PID> -- sleep 30 && perf report   # 30초 표본
strace -c -f -p <PID>                       # 시스템 호출 횟수·시간 요약 (오버헤드 큼, 짧게)
strace -f -tt -T -e trace=network -p <PID>   # 네트워크 호출의 시각·소요
bpftrace -e 'tracepoint:tcp:tcp_retransmit_skb { @[comm] = count(); }'   # 재전송을 일으키는 프로세스
bpftrace -e 'kprobe:tcp_v4_connect { @[comm] = count(); }'               # 누가 연결을 맺는가
# bcc 도구(설치된 경우): tcplife, tcpretrans, tcpconnect, runqlat, biolatency
```

보이는 것	뜻 · 해야 할 일
`vmstat r 가 코어 수보다 지속적으로 큼`	CPU 포화(대기열). 이용률보다 포화를 본다(I-3)
`wa 가 높고 b 가 큼`	디스크 IO 대기. iostat -xz로 장치 확인
`비자발 문맥 전환이 많음`	CPU 경쟁(코어 부족, cgroup 제한)

`strace -c` 에서 한 호출이 시간 대부분	그 호출(예: futex, epoll_wait, connect)이 병목 또는 대기
`perf report` 의 상위 함수	커널 함수가 상위면 네트워크 스택·잠금, 앱 함수면 앱 코드

순서: ① vmstat/pidstat로 자원 종류 ② perf로 함수 ③ strace/bpftrace로 호출·이벤트. 도구가 없으면 설치부터가 아니라 /proc 카운터로 시작한다. 이 환경에는 이 도구들이 없어 출력 예시는 생략하고 판독 규칙만 적었다.

J-5. 메모리·파일 디스크립터·cgroup 한도

"서버가 멈춘다"의 상당수는 한도다. 시스템 전체 한도, 프로세스 한도, 컨테이너 한도가 각각 따로 있다.

```
free -h; grep -E "MemAvailable|Dirty|Slab" /proc/meminfo
dmesg -T | egrep -i "out of memory|oom-kill"       # OOM 킬러 기록
cat /proc/<PID>/limits | grep -i "open files"       # 프로세스 FD 한도
ls /proc/<PID>/fd | wc -l                           # 현재 열린 FD 수
cat /proc/sys/fs/file-nr                            # 시스템 전체 FD(사용 0 최대)
cat /sys/fs/cgroup/<그룹>/memory.max /sys/fs/cgroup/<그룹>/memory.events   # oom_kill 횟수(cgroup v2)
cat /sys/fs/cgroup/<그룹>/cpu.stat | egrep "nr_throttled|throttled_usec"    # CPU 제한 대기
```

보이는 것	뜻 · 해야 할 일
`MemAvailable` 이 계속 감소	RSS 누수(I-6의 t^1) 또는 캐시 아닌 사용 증가. 기울기로 소진 시각 예측
`dmesg` 에 `oom-kill`	커널이 프로세스를 죽임. 누가·왜(메모리 한도, 시스템 부족) 로그에 있음
FD 수가 한도에 근접	"Too many open files". 누수(연결 미종료)인지 한도 설정인지 구분
`memory.events oom_kill > 0`	컨테이너가 자기 한도로 죽음(호스트는 여유 있어도)
`cpu.stat nr_throttled` 증가	컨테이너 CPU 한도에 걸려 대기. 평균 이용률이 낮아도 p99가 오른다

한도는 위에서 아래로(시스템 → cgroup → 프로세스) 가장 낮은 값이 적용된다. 수학 연결: I-6(누수 t^1, 소진 시각), I-12(FD = 동시 연결 수 = $\lambda \cdot W$).

J-6. 로드밸런서·ECMP·유휴 타임아웃

장애의 상당수는 "중간 장비의 유휴 타임아웃이 양 끝의 keepalive보다 짧을 때" 생긴다.

```
# 클라이언트·서버가 모르는 중간 장비의 유휴 타임아웃 확인
ss -tno state established '( dport = :443 )'       # keepalive 타이머 상태 (timer:(keepalive,…))
sysctl net.ipv4.tcp_keepalive_time net.ipv4.tcp_keepalive_intvl net.ipv4.tcp_keepalive_probes
# HAProxy/Nginx 상위 서버 연결 재사용 확인
curl -sv -o /dev/null http://$P_S_IP:$P_PORT/ 2>&1 | egrep -i "connection|keep-alive"
ip route get $P_S_IP                                 # 어느 경로로 나가는지(ECMP면 해시에 따라 다름)
ip route show | grep -i nexthop
```

보이는 것	뜻 · 해야 할 일
앱 keepalive > LB 유휴 타임아웃	유휴 중 LB가 연결을 지움 → 다음 요청이 RST 또는 502. 앱/서버 keepalive를 LB 값보다 짧게
간헐 502 / connection reset	위 불일치, 업스트림 재시작, 헬스체크 간격과 배포 순서
ECMP 쏠림	흐름 수가 적거나 코끼리 흐름. 해시는 5-튜플 기준이라 한 흐름은 한 경로(S10)
비대칭 라우팅	갈 때와 올 때 경로가 달라 상태 기반 방화벽이 응답을 버림(일방향 장애)

참고: AWS ALB 기본 유휴 타임아웃 60초, NLB TCP 유휴 350초(공식 문서 기준, 변경 가능). 수학 연결: I-2(이항 쏠림), I-7(해시 mod).

J-7. 쿠버네티스 네트워크

파드 안에서 보는 네트워크와 노드에서 보는 네트워크가 다르다. 계층 순서는 같다: 파드 veth → 노드 → 서비스 변환(iptables/ipvs) → 대상 파드.

```
kubectl get pods -o wide                          # 파드 IP, 노드
kubectl exec -it <파드> -- cat /etc/resolv.conf   # nameserver, search, options ndots:5
kubectl get svc,ep <서비스>                       # 엔드포인트가 비었는지
kubectl run tmp --rm -it --image=nicolaka/netshoot -- bash   # 디버그 파드
iptables -t nat -L KUBE-SERVICES -n | head        # kube-proxy(iptables 모드) 규칙
nsenter -t <컨테이너PID> -n ss -tn                # 컨테이너 네임스페이스에서 소켓 확인
kubectl get networkpolicy -A
```

보이는 것	뜻 · 해야 할 일
`엔드포인트가 비어 있음`	서비스 선택자(라벨)와 파드 라벨 불일치 또는 준비 안 됨
`DNS 가끔 5초 지연`	UDP DNS의 conntrack 경합(insert_failed)과 A/AAAA 동시 질의. NodeLocal DNSCache나 ndots 조정
`ndots:5`	점이 5개 미만인 이름은 search 목록을 먼저 시도. 외부 이름은 끝에 점을 찍거나 ndots를 낮춤 (S18)
`오버레이에서 큰 응답만 멈춤`	VXLAN 오버헤드 50바이트로 MTU 1450. 파드 MTU 점검 (S8)
`p99만 높음`	cpu.stat nr_throttled(CPU 제한). 평균 이용률이 낮아도 발생(J-5)

파드 안과 노드에서 같은 검사를 두 번 한다(I-9). 서비스 변환은 conntrack을 쓰므로 J-3의 카운터도 같이 본다.

J-8. 클라우드 VPC (보안 그룹·NACL·NAT 게이트웨이)

클라우드 장애의 다수는 서버가 아니라 구성(라우팅 표, 보안 그룹, NACL, NAT)이다. 보안 그룹은 상태 기반, NACL은 상태 비저장이다.

```
aws ec2 describe-security-groups --group-ids <sg-id>
aws ec2 describe-network-acls --filters Name=vpc-id,Values=<vpc-id>
aws ec2 describe-route-tables --filters Name=vpc-id,Values=<vpc-id>
aws ec2 describe-instance-status --instance-ids <i-id>
# VPC Reachability Analyzer 로 출발지-목적지 경로 분석(콘솔/CLI)
```

보이는 것	뜻 · 해야 할 일
`NACL에 들어오는 규칙만 허용`	NACL은 상태 비저장이라 되돌아오는 임시 포트(1024–65535) 규칙이 없으면 응답이 막힘 → 일방향 장애처럼 보임
`보안 그룹은 허용만, 상태 기반`	들어오는 허용이 있으면 응답은 자동 허용. 나가는 규칙도 따로 있음
`NAT 게이트웨이 포트 고갈`	한 목적지(IP:포트)당 동시 연결 한도(공식 문서상 약 55,000)에 도달하면 신규 연결 실패. ErrorPortAllocation 지표 확인
`MTU`	VPC 내부는 점보 프레임(9001) 가능, 인터넷·VPN 경로는 1500(터널은 더 작음). 경로 중 작은 값이 적용

일방향 장애는 "응답 경로의 규칙이 없다" 또는 "응답이 다른 경로로 간다"로 거의 수렴한다. 수학 연결: I-9(경로를 구간으로 이분), I-14(단일 장애점).

J-9. TLS·PKI·시간 동기

인증서 장애의 상당수는 만료·체인 누락·시계 오차다. 클라이언트마다 신뢰 저장소가 달라 "일부만 실패"가 흔하다.

```
openssl s_client -connect $P_TLS_CN:443 -servername $P_TLS_CN -showcerts </dev/null 2>/dev/null | egrep "Verify
return code|depth|subject|issuer"
openssl s_client -connect $P_TLS_CN:443 -servername $P_TLS_CN </dev/null 2>/dev/null | openssl x509 -noout -date
s -issuer -subject
openssl x509 -in $P_CRT -noout -checkend 604800 && echo "7일 이상 남음"
chronyc tracking | egrep "System time|Leap status"      # 시계 오차
timedatectl | egrep "synchronized|NTP"
```

보이는 것	뜻 · 해야 할 일
`Verify return code: 0 (ok)`	이 호스트의 신뢰 저장소 기준 검증 성공
`unable to get local issuer certificate`	중간 CA 누락 또는 신뢰 저장소에 루트 없음. 서버가 체인을 모두 보내야 함
`certificate has expired`	만료. 또는 클라이언트 시계가 앞서 있음(시계 확인)
`System time 이 수 초 이상 어긋남`	인증서 유효 시작 전/후로 오판, Kerberos(허용 5분) 실패
`일부 클라이언트만 실패`	오래된 루트 저장소, SNI 미지원, TLS 버전·암호 제한

만료는 임계 비교(참/거짓)이므로 I-9의 이분이나 점화식이 아니라 날짜 감시(-checkend)로 다룬다. 시계는 I-8의 차분이 맞으려면 먼저 동기되어야 한다.

J-10. DNS 심화

DNS는 캐시와 위임이 겹쳐서 "어떤 서버가 어떤 답을 줬나"를 구분해야 한다.

```
dig +trace example.com                          # 루트부터 위임을 따라 어디서 끊기는지
dig @$P_S_IP example.com +tcp +time=2 +tries=1  # UDP 단편화·잘림 문제면 TCP로는 성공
dig example.com SOA +short                      # 부정 캐시(NXDOMAIN) 유지시간은 SOA 마지막 값
resolvectl statistics                           # systemd-resolved 캐시 적중·실패
getent hosts example.com                        # 앱이 쓰는 해석 경로(nsswitch)로 확인
```

보이는 것	뜻 · 해야 할 일
`dig는 되는데 앱은 실패`	앱은 nsswitch/캐시/IPv6 우선순위를 따름. getent hosts로 앱 경로를 재현
`UDP 실패, TCP 성공`	큰 응답(DNSSEC 등)이 단편화·MTU에서 막힘. EDNS 크기와 방화벽 확인
`수정했는데 반영 안 됨`	TTL과 부정 캐시. 이미 NXDOMAIN을 받은 해석기는 SOA 값만큼 유지
`질의 수가 많음`	ndots·search 목록(S18). 끝에 점을 찍은 이름은 질의가 줄어든다

지연은 I-12의 곱(timeout×attempts×서버 수), 원인 단계는 I-8의 망원 분해(+trace)로 찾는다.

J-11. 성능 방법론: USE · RED · Little

지표가 수백 개라도 질문은 세 가지다: 얼마나 쓰나(Utilization), 줄이 생기나(Saturation), 오류가 있나(Errors).

```
# 자원마다 U/S/E 한 줄씩
CPU    U: mpstat -P ALL 1   S: vmstat 1 의 r, 비자발 문맥 전환   E: dmesg 의 MCE
메모리 U: free -h           S: vmstat 의 si/so, 페이지 스캔       E: dmesg oom-kill
디스크 U: iostat -xz 의 %util S: aqu-sz(평균 대기열)              E: smartctl -a, dmesg I/O error
NIC    U: sar -n DEV 의 rx/tx S: tc -s qdisc backlog, 드롭          E: ethtool -S 의 *_errors
```

보이는 것	뜻 · 해야 할 일
`이용률이 낮은데 느림`	포화나 제한(cgroup, 큐)일 수 있음. 이용률만으로 판단 금지
`큐(Saturation)가 길다`	M/M/1에서 이용률 90%대 진입(S4). 증설 또는 유입 제한
`서비스 관점 RED`	Rate(요청률), Errors(오류율), Duration(지연 분포). 평균이 아니라 분위수

Little: L = λ·W. 동시 처리 수와 도착률, 체류 시간 중 둘을 알면 나머지를 안다(S5). 큐 점화식(I-1)과 민감도(I-3)가 이 방법론의 수학적 뼈대다.

J-12. 장애 대응 절차 (판정·기록)

시니어의 차이는 도구보다 절차다. 가설을 세우고, 한 번에 하나만 바꾸고, 되돌릴 방법을 먼저 정한다.

```
# 1) 범위: 누가·어디서·언제부터·무엇이 (영향 범위, 시작 시각)
# 2) 경계: 계층 이분 탐색 (부록 I-9, oracle.env 의 v_/r_)
# 3) 가설: 후보 원인 목록(이항 조합) → 각각을 반증할 명령 하나
# 4) 변경: 한 번에 하나, 변경 전 상태 저장, 롤백 방법 먼저 기록
# 5) 확인: 증상 재측정(표본 크기 확인, I-11), 부작용 확인
# 6) 기록: 육하원칙 + 판정(증명/반례/정의)
```

보이는 것	뜻 · 해야 할 일
판정 3값	증명(재현 가능한 증거) / 반례(가설을 깨는 증거) / 정의(그렇게 정한 것, 증거 아님)로 나눠 적는다
육하원칙	누가·무엇을·언제·어디서·왜·어떻게. 원인 칸에는 증거 명령과 출력을 붙인다
재발 방지	검증기를 추가(해당 장애를 잡는 v_ 한 개), 지문(I-10)이나 경보로 연결

운영 절차는 수학이 아니라 규율이다. 수학 도구는 5단계의 표본 크기(I-11)와 2단계의 이분 탐색(I-9)에서 쓰인다.

부록 K. 장애 사례 28건 — 증상 → 경계 → 증거 → 원인 → 조치

사례는 이름·수치를 일반화한 전형적인 패턴이다. 명령 출력은 싣지 않고 "무엇을 보면 무엇이 확인되는가"를 적었다. 마지막 칸은 쓰인 수학 개념(부록 I)과 영역(부록 J)의 번호다.

#	증상	증거(보는 순서)	원인	조치·재발 방지	개념
1	**특정 서버만 접속 안 됨, 같은 대역 ping은 됨** 포트 타임아웃	1. L3 ping 정상 → L4로 이분 2. `iptables -L INPUT -n -v` 의 DROP 카운터가 접속 시도마다 증가 3. server에서 <code>tcpdump -ni eth0 port 8080</code> 에 SYN은 보이는데 SYN-ACK가 없음	INPUT 체인 DROP 규칙(또는 서비스 미기동). 서비스 미기동이면 RST가 오지 DROP이 아니다	규칙 삭제 또는 허용 추가. 변경은 한 줄씩, 전후 카운터 비교	I-9, I-1(127초 판별)
2	**한쪽 방향만 통신됨** 요청은 가는데 응답이 안 옴	1. 양 끝에서 동시에 캡처. 응답이 어디서 사라지나 2. `ip route get <출발지>` 로 응답 경로 확인, `sysctl net.ipv4.conf.all.rp_filter` 3. 클라우드면 NACL(상태 비저장)·비대칭 라우팅 점검	rp_filter 엄격 모드로 비대칭 응답 폐기, NACL에 임시 포트 규칙 없음, OUTPUT DROP	응답 경로 라우팅 수정, rp_filter=2(느슨) 또는 정책 라우팅, NACL에 임시 포트 허용	I-9, J-8
3	**간헐적 502/Connection reset** 배포와 무관, 유휴 후 첫 요청에서 실패	1. 실패 요청의 직전 유휴 시간 측정 2. `ss -tno` 의 keepalive 타이머와 LB 유휴 타임아웃 비교 3. `curl -w` 로 재현(유휴 N초 후 요청)	중간 장비 유휴 타임아웃 < 앱 keepalive	앱/서버 keepalive를 LB 값보다 짧게, 업스트림 연결 재사용 설정	I-1, J-6
4	**새 연결 때만 5초 지연** 이후 요청은 빠름	1. `curl -w "%{time_namelookup}"` 가 5초 근처 2. `dig +time=2 +tries=1` 로 서버별 응답 확인 3. `nstat -az | grep -i insert_failed` , `conntrack -S`	DNS 서버 하나가 응답 없음(timeout 5초 후 다음 서버), 또는 A/AAAA 병렬 질의의 conntrack 경합	죽은 서버 제거, resolv.conf에 single-request-reopen·로컬 캐시, 쿠버네티스는 NodeLocal DNSCache	I-8, I-12(30초 최악), J-10
5	**"Cannot assign requested address"** 새 연결 생성 실패, 기존은 정상	1. `ss -s` 의 timewait 수천~수만 2. `sysctl net.ipv4.ip_local_port_range` 3. 목적지별 연결 수 <code>ss -tan | awk '{print $5}' | sort | uniq -c | sort -rn | head</code>	같은 (출발IP, 목적IP, 목적포트)로 초당 ~471건 초과	keep-alive/커넥션 풀로 재사용, 범위 확대, 출발 IP 추가. tw_reuse는 출발 연결에만 영향	I-12(S9)
6	**새 연결만 실패, 기존 연결은 정상** 로그에 table full	1. `dmesg -T | grep conntrack` 2. `cat /proc/sys/net/netfilter/nf_conntrack_count` 3. `conntrack -S`	conntrack 표 가득 참(항목 수 = 새 연결률 × 유지 시간)	max 증가와 버킷 조정, TIME_WAIT 타임아웃 단축, 추적 불필요 트래픽은 NOTRACK	I-12(S5), J-3

7	**SSH 접속은 되는데 큰 출력에서 멈춤** ls 같은 짧은 명령은 됨	1. `ping -M do -s 1472 <대상>` 실패, 1400은 성공 2. `tracepath -n <대상>` 로 MTU가 줄어드는 지점 3. ICMP "fragmentation needed" 차단 여부	MTU 블랙홀(터널 오버헤드 + ICMP 차단)	경로 ICMP 허용, 인터페이스 MTU/MSS 클램핑 (iptables TCPMSS)	S8, I-8
8	**일부 클라이언트만 HTTPS 실패** 브라우저는 되고 curl·구형 클라이언트는 실패	1. `openssl s_client -showcerts` 로 서버가 보내는 체인 2. Verify return code가 20/21 3. `openssl x509 -noout -dates`	서버가 중간 CA를 보내지 않음(체인 누락) 또는 만료	서버에 전체 체인(leaf+intermediate) 설치, 만료 감시 -checkend	J-9
9	**인증·TLS가 갑자기 실패** 인증서는 유효한데 "not yet valid"	1. `chronyc tracking` System time 오차 2. `timedatectl` synchronized 3. 다른 호스트와 날짜 비교	시계 오차(NTP 중단, VM 일시정지 후 복귀)	시간 동기 복구, 동기 상태 경보	J-9, I-8
10	**메모리가 서서히 증가해 며칠마다 OOM** 재시작하면 회복	1. `ps -eo pid,rss,cmd --sort=-rss | head` 를 주기 수집 2. 시계열을 I-6으로 분류(t^1), 소진 시각 = (한도−현재)/기울기 3. `dmesg | grep -i oom`	일정 속도의 누수(t^1). 지수 증가면 증폭 고리(캐시·재시도)	누수 수정, 임시로 재시작 주기를 소진 시각의 절반으로	I-6(S11), J-5
11	**정확히 7분마다 지연 스파이크** 원인 불명	1. 지연 시계열의 주기 추정(7분) 2. cron 표, GC 로그, 스케줄러에서 7분·그 약수 후보 3. `ip monitor link` 로 링크 변화	주기 작업(백업·로그 회전·GC)과 요청 경로의 자원 경합	작업을 이동·분산, 자원 격리	I-6, I-13
12	**장애 중 요청이 계속 늘어 복구 안 됨** 서버를 늘려도 더 나빠짐	1. 요청률 그래프를 로그 축으로: 직선이면 지수 2. 클라이언트 재시도 정책과 타임아웃 3. 서버 큐 길이	재시도 폭풍(되먹임). 실패 → 재시도 → 더 많은 부하	지수 백오프+지터, 서킷 브레이커, 부하 차단(shedding) 후 점진 복구	I-1, I-6
13	**평소와 큰 차이 없는 트래픽인데 지연 급증** CPU는 80~90%	1. 이용률과 큐 길이 동시 확인 2. $dW/d\lambda = 1/(\mu-\lambda)^2$로 현재 위치 계산 3. p99와 p50 차이	큐 이론의 무릎 구간(이용률 90%대)	증설 또는 유입 제한. 이용률 목표를 70~80%로	I-3(S4)
14	**1Gbps 회선인데 원격지 전송이 10Mbps** 근거리는 빠름	1. `ss -ti` 의 rwnd_limited, wscale 2. `sysctl net.ipv4.tcp_rmem net.ipv4.tcp_window_scaling` 3. RTT 측정	창 크기가 RTT×대역폭(BDP 6.25MB)보다 작음	window scaling 켜기, tcp_rmem/wmem 최대값 확대(BDP 이상)	I-5(S7)
15	**손실 0.01%인데 처리량이 낮음** 재전송 약간	1. `nstat -az TcpRetransSegs` 차분 2. `mtr -rwzbc 100` 로 손실 구간 3. Mathis 한계 계산	손실률이 $\sqrt{p}$로 처리량을 제한	손실 구간 제거(케이블·혼잡), BBR 같은 손실에 덜 민감한 혼잡 제어 검토	I-5

16	**접속이 간헐적으로 거부·지연** accept 큐 문제	1. `nstat -az TcpExtListenOverflows TcpExtListenDrops` 2. `ss -ltn` Recv-Q가 Send-Q에 근접 3. 워커 수와 backlog	앱이 accept를 못 따라감 또는 backlog 작음	somaxconn·앱 backlog 증가, 워커 증설, 처리 병목 제거	J-2
17	**특정 NIC에서 패킷 손실** 재전송이 늘고 CRC 에러	1. `ethtool -S eth0 | egrep "crc|errors|missed"` 차분 2. `ethtool eth0` 속도/듀플렉스 3. `ethtool -m eth0` 광 수신 세기	케이블·SFP 불량 또는 듀플렉스 불일치(CRC), 링 버퍼 넘침(missed)	케이블/모듈 교체, 링 버퍼 확대(ethtool -G), 자동 협상 통일	I-7, J-2
18	**한 코어만 100%, 나머지는 한가함** 네트워크 처리량 한계	1. `mpstat -P ALL 1` 의 %soft 편중 2. `cat /proc/interrupts` 한 코어에 몰림 3. `ethtool -l eth0` 큐 수	인터럽트·softirq가 한 큐/코어에 집중(단일 흐름 또는 RSS 미설정)	큐 수 확대, irqbalance, RPS/RFS. 단일 흐름이면 한계가 있음	J-2, I-2
19	**"Too many open files"** 새 연결·파일 열기 실패	1. `cat /proc/<PID>/limits` 의 open files 2. `ls /proc/<PID>/fd | wc -l` 3. `ss -s`	FD 한도 부족 또는 연결 미종료 누수. FD = 동시 연결 수 (L=λW)	누수면 수정, 한도면 LimitNOFILE·ulimit 조정 (재시작 필요 여부 확인)	I-12, J-5
20	**쿠버네티스에서 외부 호출만 느림** 내부 서비스는 빠름	1. `kubectl exec <파드> -- cat /etc/resolv.conf` 의 ndots:5 2. `kubectl exec <파드> -- time getent hosts example.com` 3. DNS 질의 8건 계산	ndots 때문에 외부 이름이 search 목록을 먼저 질의	이름 끝에 점, ndots 조정, NodeLocal DNSCache	I-12(S18), J-7
21	**쿠버네티스 p99만 오르고 평균은 정상** 이용률 낮음	1. `cat cpu.stat` 의 nr_throttled, throttled_usec 2. 컨테이너 limits.cpu 대비 순간 사용	CPU 한도 제한으로 주기적 대기(throttling)	limits 조정(또는 제거), 요청량 정확화, 스레드 수 정리	J-5
22	**클라우드에서 외부 호출 일부 실패** 트래픽이 많을 때만	1. NAT 게이트웨이의 포트 할당 오류 지표 2. 같은 목적지로의 동시 연결 수 3. 목적지별 분포	한 목적지당 동시 연결 한도 도달(NAT 포트 고갈)	연결 재사용, NAT 추가·서브넷 분산, 목적지 분산	I-12(S9), J-8
23	**서버 한 대만 가끔 이상** 같은 이미지로 배포했는데	1. `sysctl -a | sort | sha256sum` 지문을 서버 전체에서 비교 2. 다른 서버만 diff 3. GF(p) 지문으로 대량 비교	설정 드리프트(수동 변경)	구성 관리로 되돌리고 지문 경보	I-10(S14)
24	**배포 후 에러율 상승** 변경 20개가 한 번에 나감	1. 변경 목록을 반으로 나눠 절반만 롤백 2. `git bisect` 와 같은 절차 3. 시간 경계도 이분	변경 중 하나(또는 상호작용)	⌈log2 20⌉=5번 안에 특정. 상호작용이면 쌍으로	I-9(S15)
25	**트래픽이 서버 4대에 쏠림** 한 서버가 과부하	1. 흐름 수·바이트 분포(상위 흐름) 2. LB 알고리즘(해시/최소 연결) 3. `ip route show | grep nexthop`	흐름이 적으면 이항 쏠림, 코끼리 흐름은 해시로 분산 불가	최소 연결 방식, 흐름 쪼개기(여러 연결), 가중치	I-2(S10)

26	**TIME_WAIT가 수만 개라 걱정** 실제 문제는 없음	1. `ss -s` 2. 새 연결 실패(포트 고갈) 여부 3. TIME_WAIT는 주로 먼저 끊은 쪽에 남음	대부분 정상. 고갈(S9)이 있을 때만 문제	수치만 보고 튜닝하지 말고 실패가 있는지부터 본다	I-12
27	**UDP 수신이 간헐적으로 유실** 로그·메트릭 수집기	1. `nstat -az UdpRcvbufErrors` 증가 2. `ss -uanp` 의 Recv-Q 3. `sysctl net.core.rmem_max`	UDP 소켓 수신 버퍼 부족(재전송이 없어 그대로 유실)	rmem_max·SO_RCVBUF 확대, 수신 스레드 증가	J-2
28	**화면 하나가 요청 100개를 부르는데 가끔 느림** 개별 요청은 99.9%가 빠름	1. 화면 지연 분포(p99)와 하위 요청 p99.9 2. 하위 요청 수 n	꼬리 지연 증폭: $1-(1-q)^n$ (n=100이면 9.5%)	요청 수 줄이기, 헤징(중복 요청), 타임아웃과 병렬화	I-2(S19)

부록 L. 학습 로드맵과 검증 코드 (seniorlab.py)

단계	목표	할 일	완료 기준(측정 가능)	쓰는 개념
0단계	랩 구축과 기준선	lab.env로 네임스페이스 랩 구성, v_ 검증기 7개 실행	v_l1~v_l7이 모두 1. 정상 상태의 지표 값을 기록	I-6(기준값)
1단계	주니어: 계층별 진단	1~7장 명령 전부를 랩에서 실행하고 출력을 읽음	의도적으로 결함 5가지(INPUT DROP, MTU, 지연, 손실, DNS)를 넣고 계층을 맞힘	I-9
2단계	미들: 출력 해석	ss -ti, nstat, ethtool -S, conntrack -S 출력을 읽고 원인을 말함	부록 J의 판독 규칙을 보지 않고 70% 이상 설명	I-1, I-3
3단계	미들+: 수치로 말하기	seniorlab.py의 계산을 자기 환경 값으로 바꿔 돌림	RTO 합, 포트 고갈률, conntrack r·T, 창/RTT를 자기 서버에서 계산	I-1~I-5
4단계	시니어: 한도와 포화	USE 방법으로 자원 4종의 U/S/E를 한 장에 정리	이용률이 낮은데 느린 사례를 포화·제한으로 설명	I-3, J-5
5단계	시니어: 추적	perf, strace, bpftrace(가능하면)로 원인 함수·호출 찾기	같은 장애를 추적 없이 / 추적으로 각각 특정하고 비교	I-9
6단계	시니어: 대규모 구조	LB·쿠버네티스·VPC 구성에서 일방향·비대칭·한도 장애 재현	부록 K 28건 중 15건을 랩이나 테스트 환경에서 재현	I-2, I-14
7단계	시니어: 절차와 검증	육하원칙 보고, 변경 이분, 지문 비교, 변이 분석으로 도구 점검	검증기 탐지율을 표로 제시하고 빈틈을 하나씩 메움	I-10, I-15

시니어 판단 체크리스트 (장애 때 스스로 묻기)

1. 증상의 시작 시각과 범위를 숫자로 말할 수 있는가? (I-8, I-9)
2. "되는 것"과 "안 되는 것"의 경계를 이분으로 좁혔는가? (I-9)
3. 이용률이 아니라 포화(큐)와 오류를 봤는가? (J-11)
4. 카운터는 누적값이 아니라 차분(초당)으로 봤는가? (I-3)
5. 시계열을 성장·감쇠·주기·누수로 분류했는가? (I-6)
6. 표본이 충분한가? 무손실 100번은 "없음"의 증거가 아니다 (I-11)
7. 변경은 한 번에 하나, 롤백이 준비돼 있는가? (I-5, J-12)
8. 원인 칸에 증거(명령과 출력)가 붙어 있는가? 증명·반례·정의를 구분했는가?
9. 같은 장애를 다음에 잡을 검증기나 경보를 추가했는가? (I-15)

seniorlab.py 실행 결과 (모든 단언 통과)

```
=== S1 점화식: 재전송 백오프 RTO_n = min(2·RTO_(n-1), 120초) ===
  SYN 재시도 tcp_syn_retries=6, 첫 대기 1초: 1+2+4+8+16+32+64 = 127초 후 connect 실패(ETIMEDOUT)
  전송 중 tcp_retries2=15, 첫 RTO 0.2초(리눅스 하한), 상한 120초: 합 = 924.6초 ≈ 15.4분 뒤 연결 포기
  → 케이블을 뽑은 뒤 연결이 15분 가까이 살아 있는 것처럼 보이는 이유. 앱 쪽 타임아웃이 먼저 필요하다.
  RTO 열: 0.2, 0.4, 0.8, 1.6, 3.2, 6.4, 12.8, 25.6, 51.2, 102.4, 120, 120, 120, 120, 120, 120

=== S2 점화식: RTT 지수 이동 평균 SRTT_n = (7/8)·SRTT_(n-1) + (1/8)·R_n ===
  측정 RTT(ms) →  SRTT     RTTVAR   RTO=max(200, SRTT+4·RTTVAR)
      52          50.25    19.25   200.00
      49          50.09    14.75   200.00
      51          50.21    11.29   200.00
      50          50.18     8.52   200.00
     400          93.91    93.84   469.28
      50          88.42    81.36   413.86
      51          83.74    70.37   365.24
      50          79.52    61.22   324.39
      49          75.71    53.54   289.88
  → 한 번의 400ms 스파이크가 SRTT를 크게 올리지 않지만 RTTVAR를 키워 RTO가 여러 표본 동안 높게 남는다.

=== S3 점화식: 큐 적체 Q_n = max(0, Q_(n-1) + 도착 − 처리) ===
  도착 95/s, 처리 100/s, 60초 뒤 대기열 0개, 마지막 요청의 대기 0.0초
  도착 100/s, 처리 100/s, 60초 뒤 대기열 0개, 마지막 요청의 대기 0.0초
  도착 105/s, 처리 100/s, 60초 뒤 대기열 300개, 마지막 요청의 대기 3.0초
  → 처리율이 도착률에 못 미치면 대기열은 시간에 비례(t¹)해 늘어난다. 정상 구간에서는 0에 붙는다.
```

```
=== S4 미분: 지연의 민감도 dW/dλ = 1/(μ−λ)²  (M/M/1, μ=100/s) ===
  부하λ  이용률  지연W(ms)  민감도dW/dλ(ms per 1/s)
    50    50%      20.0         0.4
    80    80%      50.0         2.5
    90    90%     100.0        10.0
    95    95%     200.0        40.0
    99    99%    1000.0      1000.0
  → 이용률 50%→99%에서 지연은 50배, 민감도는 2500배. 90%대에서 '조금 늘었는데 갑자기 느려진다'의 정체.

=== S5 항등식: Little의 법칙 L = λ·W (동시 처리 수 = 도착률 × 체류 시간) ===
  λ=200 req/s, W=0.5초 → 동시에 처리 중인 요청 L=100개. 워커/스레드/FD가 64개면 포화.
  conntrack: 새 연결률 r, 항목 유지 시간 T → 항목 수 = r·T
   r=1000/s, TIME_WAIT 항목 120초 유지 → 120000개 (여유, nf_conntrack_max=262144)
   r=2000/s, TIME_WAIT 항목 120초 유지 → 240000개 (여유, nf_conntrack_max=262144)
   r=2200/s, TIME_WAIT 항목 120초 유지 → 264000개 (넘침, nf_conntrack_max=262144)

=== S6 이항정리: (1−q)^n ≈ 1 − n·q (b¹ 계수 추출) 과 조합 개수 C(n,k) ===
  직렬 n개 구성요소(각 가용성 99.9%) 전체 가용성: 정확 vs 1차 근사
   n= 3: 정확 0.997003  1차 0.997000  오차 -3.00e-06  (2차 꼬리 C(n,2)q² = 3.00e-06가 상한)
   n=10: 정확 0.990045  1차 0.990000  오차 -4.49e-05  (2차 꼬리 C(n,2)q² = 4.50e-05가 상한)
   n=30: 정확 0.970431  1차 0.970000  오차 -4.31e-04  (2차 꼬리 C(n,2)q² = 4.35e-04가 상한)
  노드 5대 각 99%: 과반(3대 이상) 생존 확률 = Σ C(5,k)p^k(1−p)^(5−k) = 0.99999015
   정확히 2대 죽는 경우의 수 C(5,2) = 10가지, 확률 0.000970
  7계층에서 동시에 고장 난 계층이 최대 2개일 때의 후보 조합 수 = C(7,1)+C(7,2) = 28가지 (검증기 1회로 한 계층씩 배제)

=== S7 변수분리: TCP 처리량 = f(RTT)·g(손실) (Mathis), 한계는 독립 인자의 최솟값 ===
  MSS 1460B, RTT 50ms, 손실 0.01%: Mathis 한계 ≈ 28.6 Mbps
  수신창 64KB 고정이면 창/RTT 한계 = 10.5 Mbps  (창 확대 없으면 1Gbps 링크라도 이 값)
  1Gbps × 50ms 의 BDP(창이 있어야 하는 크기) = 6.25 MB
  RTT 2배·손실 4배 → 처리량 ×1/2 ×1/2 = ×1/4 (실측 계산과 일치): 인자별로 따로 계산해 곱한다
  처리량 상한 = min(링크, 창/RTT, Mathis) : 가장 작은 인자가 병목

=== S8 정수 산술: MTU·MSS·ping -s 크기 ===
  MTU 1500 → TCP MSS 1460, 단편화 없이 보낼 최대 ping -s 1472
  VXLAN 터널 오버헤드 50B → 안쪽 MTU 1450, 안쪽 MSS 1410
  GRE 터널 오버헤드 24B → 안쪽 MTU 1476, 안쪽 MSS 1436
  IPsec ESP(대략) 터널 오버헤드 73B → 안쪽 MTU 1427, 안쪽 MSS 1387

=== S9 계산: 임시 포트 고갈 속도 ===
  net.ipv4.ip_local_port_range 32768 60999 = 28232개, TIME_WAIT 60초 → 같은 (출발IP, 목적IP, 목적포트)로 지속 가능한 새 연결률 ≈ 471/s
  net.ipv4.ip_local_port_range 32768 60999 = 28232개, TIME_WAIT 30초 → 같은 (출발IP, 목적IP, 목적포트)로 지속 가능한 새 연결률 ≈ 941/s
  → 목적지 한 곳으로 초당 500건 넘게 새 연결을 맺으면 'Cannot assign requested address'. 해법은 연결 재사용(keep-alive)이다.

=== S10 이항 분포: 해시 분산(ECMP·LB)에서 흐름 수가 적으면 쏠린다 ===
  흐름 100개, 경로 4개: 한 경로의 평균 25개, 표준편차 4.33
  한 경로가 35개 이상 받을 확률(정확 이항) = 0.0164
  모의 20000회 빈도 0.0175 (정확값과 일치, 허용 0.004)
  흐름 10000개면 상대 편차 1.73% (100개일 때 17.3%) → 흐름이 많을수록 고르다
  단, 흐름 1개가 전체의 40%를 차지(코끼리 흐름)하면 해시로는 못 나눈다

=== S11 오일러 분해: 시계열을 성장·감쇠·회전·다항식·기준값으로 분류 ===
  RSS 메모리(MB), 누수                    → 다항식 배수 t¹: 일정 속도로 늘어남(누수)
  재시도 폭주 요청 수/s                     → 성장 e^(at): 비율로 늘어남
  장애 후 에러율 감쇠                       → 감쇠 e^(−at): 비율로 줄어듦
  링크 flapping(up=1/down=0)에 가까운 지연   → 회전·주기(i, π): 반복되는 흔들림(flapping)
  정상 CPU(%)                           → 기준값(상수)
  이상적(잡음 없는) 열의 정확 분류는 부록 H-14의 classify_series가 담당 (같은 패턴표).
  판독: 메모리 t¹ → 누수(재시작 시점 예측), e^(at) → 증폭(재시도·큐), cos → 주기 원인(cron·GC·flap)

=== S12 적분부등식: 누적 오류 시간 ∫e(t)dt 를 하한·상한 구간으로 가두어 판정 ===
  30일 가용성 99.9% → 허용 오류 시간 43.2분
  시간별 오류 비율을 [최솟값, 최댓값]으로만 아는 720시간: 하한 15.0분 ≤ ∫e ≤ 상한 105.5분
  판정: 불확정 (상한 ≤ 예산이면 합격, 하한 > 예산이면 불합격, 걸치면 측정 해상도를 높인다)
  해상도가 좋은 경우 상한 21.6분 ≤ 43.2분 → 합격(구간이 좁아져야 결론이 난다)

=== S13 망원 합: curl -w 누적 시각을 단계별 몫으로 분해 (합 = time_total) ===
    DNS                        3.0 ms    0.7%
    TCP 연결                   18.0 ms    4.0%
    TLS                       68.0 ms   14.9%
    요청 준비                    0.0 ms    0.0%
    서버 처리(첫 바이트까지)        323.0 ms   71.0%
    본문 전송                   43.0 ms    9.5%
    합 455.0 ms = time_total (정확한 분수 합으로 일치 확인)
  → 최대 몫: 서버 처리(첫 바이트까지). 네트워크가 아니라 애플리케이션 쪽을 본다.

=== S14 GF(p) 다항식 지문: 서버 수백 대의 설정이 같은지 한 숫자로 비교 (Schwartz–Zippel) ===
  같은 설정 지문 일치: True, 한 줄이 다른 설정 지문 불일치: True
  서로 다른 두 설정이 우연히 같은 지문일 확률 ≤ 차수/p = 60/(2^61−1) ≈ 2.60e-17
  → 비교는 x(무작위 점)와 지문 한 개씩만 주고받으면 되고, 서로 다른 줄 위치가 틀려도 잡힌다(줄 순서도 반영).

=== S15 이분 탐색: 첫 고장 계층 찾기 (누적 검증기 v(k)=L1..Lk 모두 정상이면 1) ===
  고장 계층 1..7(8=고장 없음) 8가지 모두 정확히 찾음, 검증 호출 최대 3회 (선형 L1→L7은 최대 7회, ⌈log2 8⌉=3)
  전제: 검증기가 '그 계층까지 끝-끝으로' 확인할 것(단조성). 비대칭 경로·간헐 장애는 단조성이 깨지니 선형으로 돌아간다.
```

```
=== S16 측정 횟수: 손실률 p<1%를 99% 확신으로 말하려면 몇 번 보내야 하나 ===
  손실 0건 관측 시 정확한 한계: (1−p)^n ≤ δ → n ≥ ln δ/ln(1−p) = 459회
  손실률을 ±1%p로 추정(Hoeffding 양쪽): n ≥ ln(2/δ)/(2ε²) = 26492회
  → ping 100번 무손실로는 '1% 미만'을 99% 확신으로 단정할 수 없다. 459번 무손실이어야 한다.

=== S17 나머지 연산: 라운드 로빈 mod k, 시퀀스 번호 되감김 2^32 ===
  백엔드 4대 라운드 로빈 n mod 4: [0, 1, 2, 3, 0, 1, 2, 3, 0, 1]
  i^n 도 mod 4 로 순환(1, i, −1, −i): 주기 4 현상은 같은 구조
  TCP 시퀀스(32비트)가 1Gbps에서 한 바퀴 도는 시간 34.36초
  TCP 시퀀스(32비트)가 10Gbps에서 한 바퀴 도는 시간 3.44초
  TCP 시퀀스(32비트)가 100Gbps에서 한 바퀴 도는 시간 0.34초
  → 고속 구간에서 오래된 세그먼트와 새 세그먼트를 구분하려면 타임스탬프(PAWS)가 필요하다.
  CRC-32가 무작위 손상을 놓칠 확률 2^-32 = 2.33e-10

=== S18 곱셈 원리: DNS 최악 대기 시간과 쿠버네티스 ndots ===
  resolv.conf timeout:5 attempts:2, 서버 3개 → 응답이 전혀 없을 때 한 질의 30초
  ndots:5, 검색 도메인 3개, 이름의 점이 5개 미만: 후보 이름 4개 × (A, AAAA) 2종 = 질의 8건
  → 외부 도메인 하나를 열 때 질의 8건. 끝에 점을 찍은 이름(example.com.)이면 2건.

=== S19 인수분해: 지연 꼬리 — 연결 N회 중 하나라도 느릴 확률 1−(1−q)^n ≈ n·q ===
  요청 하나가 느릴 확률 0.1%, 한 화면이 요청 1개를 부르면 화면이 느릴 확률 0.10% (1차 근사 0.10%)
  요청 하나가 느릴 확률 0.1%, 한 화면이 요청 10개를 부르면 화면이 느릴 확률 1.00% (1차 근사 1.00%)
  요청 하나가 느릴 확률 0.1%, 한 화면이 요청 100개를 부르면 화면이 느릴 확률 9.52% (1차 근사 10.00%)

모든 단언 통과
```

seniorlab.py 전체 코드

```
#!/usr/bin/env python3
"""seniorlab.py — 시니어가 마주하는 상황에 수학 개념(점화식·이항정리·미분·적분부등식·변수분리·오일러 분해·망원 합·GF(p)·이분 탐색)을
적용한 계산. 모든 단언(assert)이 통과해야 끝난다. 사용: python3 seniorlab.py
"""
import math, random, zlib
from fractions import Fraction as Fr
from math import comb

def head(t): print(f"\n=== {t} ===")

# S1 점화식: 재전송 대기시간 RTO_n = min(2·RTO_{n-1}, 120)
def give_up_time(rto0, retries, cap=120.0):
    t, rto = 0.0, rto0
    for _ in range(retries + 1):
        t += rto; rto = min(rto * 2, cap)
    return t
def s1():
    head("S1 점화식: 재전송 백오프 RTO_n = min(2·RTO_(n-1), 120초)")
    syn = give_up_time(1.0, 6, cap=1e9)
    assert syn == 127.0
    print(f"  SYN 재시도 tcp_syn_retries=6, 첫 대기 1초: 1+2+4+8+16+32+64 = {syn:.0f}초 후 connect 실패(ETIMEDOUT)")
    est = give_up_time(0.2, 15)
    assert abs(est - 924.6) < 1e-9
    print(f"  전송 중 tcp_retries2=15, 첫 RTO 0.2초(리눅스 하한), 상한 120초: 합 = {est:.1f}초 ≈ 15.4분 뒤 연결 포기")
    print("  → 케이블을 뽑은 뒤 연결이 15분 가까이 살아 있는 것처럼 보이는 이유. 앱 쪽 타임아웃이 먼저 필요하다.")
    ladder = [min(0.2 * 2 ** k, 120) for k in range(16)]
    print("  RTO 열:", ", ".join(f"{x:g}" for x in ladder))

# S2 점화식: RTT 추정 (RFC 6298)
def s2():
    head("S2 점화식: RTT 지수 이동 평균 SRTT_n = (7/8)·SRTT_(n-1) + (1/8)·R_n")
    rtts = [50, 52, 49, 51, 50, 400, 50, 51, 50, 49]
    srtt = rtts[0]; var = rtts[0] / 2
    print("  측정 RTT(ms) →  SRTT     RTTVAR   RTO=max(200, SRTT+4·RTTVAR)")
    for r in rtts[1:]:
        var = 0.75 * var + 0.25 * abs(srtt - r)
        srtt = 0.875 * srtt + 0.125 * r
        rto = max(200, srtt + 4 * var)
        print(f"   {r:>5}         {srtt:7.2f}  {var:7.2f}  {rto:7.2f}")
    assert 50 < srtt < 120
    print("  → 한 번의 400ms 스파이크가 SRTT를 크게 올리지 않지만 RTTVAR를 키워 RTO가 여러 표본 동안 높게 남는다.")

# S3 점화식: 큐 적체 (Lindley) Q_n = max(0, Q_(n-1) + a - s)
def s3():
    head("S3 점화식: 큐 적체 Q_n = max(0, Q_(n-1) + 도착 − 처리)")
    for lam, mu in ((95, 100), (100, 100), (105, 100)):
        q = 0
        for _ in range(60): q = max(0, q + lam - mu)
        print(f"  도착 {lam}/s, 처리 {mu}/s, 60초 뒤 대기열 {q}개, 마지막 요청의 대기 {q/mu:.1f}초")
    q = 0
    for _ in range(60): q = max(0, q + 105 - 100)
    assert q == 300
    print("  → 처리율이 도착률에 못 미치면 대기열은 시간에 비례(t¹)해 늘어난다. 정상 구간에서는 0에 붙는다.")

# S4 미분: M/M/1 지연 W(λ)=1/(μ-λ), dW/dλ = 1/(μ-λ)^2
def s4():
    head("S4 미분: 지연의 민감도 dW/dλ = 1/(μ−λ)²  (M/M/1, μ=100/s)")
    mu = 100; prev = None
    print("  부하λ  이용률  지연W(ms)  민감도dW/dλ(ms per 1/s)")
    for lam in (50, 80, 90, 95, 99):
        W = 1 / (mu - lam); d = 1 / (mu - lam) ** 2
        # 수치 미분과 대조
        h = 1e-6; nd = (1 / (mu - lam - h) - 1 / (mu - lam)) / (-h) * -1
        nd = (1 / (mu - (lam + h)) - 1 / (mu - lam)) / h
        assert abs(nd - d) / d < 1e-4
        print(f"  {lam:>4}  {lam/mu:5.0%}  {W*1000:8.1f}  {d*1000:10.1f}")
    assert (1 / (mu - 99) ** 2) / (1 / (mu - 50) ** 2) == 2500
    print("  → 이용률 50%→99%에서 지연은 50배, 민감도는 2500배. 90%대에서 '조금 늘었는데 갑자기 느려진다'의 정체.")

# S5 Little's law: L = λ·W
def s5():
    head("S5 항등식: Little의 법칙 L = λ·W (동시 처리 수 = 도착률 × 체류 시간)")
    lam, W = 200, 0.5
    L = lam * W
    assert L == 100
    print(f"  λ=200 req/s, W=0.5초 → 동시에 처리 중인 요청 L={L:.0f}개. 워커/스레드/FD가 64개면 포화.")
    print("  conntrack: 새 연결률 r, 항목 유지 시간 T → 항목 수 = r·T")
    for r in (1000, 2000, 2200):
        n = r * 120; mx = 262144
        print(f"   r={r}/s, TIME_WAIT 항목 120초 유지 → {n}개 ({'넘침' if n>mx else '여유'}, nf_conntrack_max={mx})")
    assert 2200 * 120 > 262144 >= 2000 * 120

# S6 이항정리·이항계수
def s6():
    head("S6 이항정리: (1−q)^n ≈ 1 − n·q (b¹ 계수 추출) 과 조합 개수 C(n,k)")
    q = 0.001
    print("  직렬 n개 구성요소(각 가용성 99.9%) 전체 가용성: 정확 vs 1차 근사")
```

```
    for n in (3, 10, 30):
        ex = (1 - q) ** n; ap = 1 - n * q
        assert abs(ex - ap) < comb(n, 2) * q * q
        print(f"   n={n:>2}: 정확 {ex:.6f}  1차 {ap:.6f}  오차 {ap-ex:+.2e}  (2차 꼬리 C(n,2)q² = {comb(n,2)*q*q:.2e}가 상한)")
    p = 0.99
    n = 5
    pr = lambda k: comb(n, k) * p ** k * (1 - p) ** (n - k)
    quorum = sum(pr(k) for k in range(3, 6))
    assert abs(sum(pr(k) for k in range(n + 1)) - 1) < 1e-12
    print(f"  노드 5대 각 99%: 과반(3대 이상) 생존 확률 = Σ C(5,k)p^k(1−p)^(5−k) = {quorum:.8f}")
    print(f"   정확히 2대 죽는 경우의 수 C(5,2) = {comb(5,2)}가지, 확률 {pr(3):.6f}")
    c1, c2 = comb(7, 1), comb(7, 2)
    assert (c1, c2) == (7, 21)
    print(f"  7계층에서 동시에 고장 난 계층이 최대 2개일 때의 후보 조합 수 = C(7,1)+C(7,2) = {c1+c2}가지 (검증기 1회로 한 계층씩 배제)")

# S7 변수분리: Mathis 곱 꼴, 창/RTT 한계
def s7():
    head("S7 변수분리: TCP 처리량 = f(RTT)·g(손실) (Mathis), 한계는 독립 인자의 최솟값")
    MSS, rtt, p, C = 1460, 0.050, 1e-4, math.sqrt(1.5)
    T = (MSS * 8 / rtt) * (C / math.sqrt(p))
    print(f"  MSS 1460B, RTT 50ms, 손실 0.01%: Mathis 한계 ≈ {T/1e6:.1f} Mbps")
    win = 65535 * 8 / rtt
    print(f"  수신창 64KB 고정이면 창/RTT 한계 = {win/1e6:.1f} Mbps  (창 확대 없으면 1Gbps 링크라도 이 값)")
    bdp = 1e9 * rtt / 8
    print(f"  1Gbps × 50ms 의 BDP(창이 있어야 하는 크기) = {bdp/1e6:.2f} MB")
    assert abs(win/1e6 - 10.49) < 0.01 and abs(bdp - 6.25e6) < 1
    # 로그를 취하면 곱 → 합: d ln T = −d ln RTT − ½ d ln p
    T2 = (MSS * 8 / (rtt * 2)) * (C / math.sqrt(p * 4))
    assert abs(T2 / T - 1 / 4) < 1e-12
    print("  RTT 2배·손실 4배 → 처리량 ×1/2 ×1/2 = ×1/4 (실측 계산과 일치): 인자별로 따로 계산해 곱한다")
    print("  처리량 상한 = min(링크, 창/RTT, Mathis) : 가장 작은 인자가 병목")

# S8 MTU 산술
def s8():
    head("S8 정수 산술: MTU·MSS·ping -s 크기")
    mtu = 1500; ip = 20; tcp = 20; icmp = 8
    mss = mtu - ip - tcp; ping_payload = mtu - ip - icmp
    assert (mss, ping_payload) == (1460, 1472)
    print(f"  MTU {mtu} → TCP MSS {mss}, 단편화 없이 보낼 최대 ping -s {ping_payload}")
    for name, ov in (("VXLAN", 50), ("GRE", 24), ("IPsec ESP(대략)", 73)):
        print(f"  {name} 터널 오버헤드 {ov}B → 안쪽 MTU {mtu-ov}, 안쪽 MSS {mtu-ov-40}")
    assert 1500 - 50 == 1450

# S9 포트 고갈
def s9():
    head("S9 계산: 임시 포트 고갈 속도")
    lo, hi = 32768, 60999
    n = hi - lo + 1
    assert n == 28232
    for tw in (60, 30):
        print(f"  net.ipv4.ip_local_port_range {lo} {hi} = {n}개, TIME_WAIT {tw}초 → 같은 (출발IP, 목적IP, 목적포트)로 지속 가능한 새 연결률 ≈ {n/tw:.
0f}/s")
    print("  → 목적지 한 곳으로 초당 500건 넘게 새 연결을 맺으면 'Cannot assign requested address'. 해법은 연결 재사용(keep-alive)이다.")

# S10 해시 분산 (ECMP/LB)
def s10():
    head("S10 이항 분포: 해시 분산(ECMP·LB)에서 흐름 수가 적으면 쏠린다")
    N, k = 100, 4; p = 1 / k
    ex = sum(comb(N, j) * p ** j * (1 - p) ** (N - j) for j in range(35, N + 1))
    sd = math.sqrt(N * p * (1 - p))
    print(f"  흐름 {N}개, 경로 {k}개: 한 경로의 평균 {N*p:.0f}개, 표준편차 {sd:.2f}")
    print(f"  한 경로가 35개 이상 받을 확률(정확 이항) = {ex:.4f}")
    rnd = random.Random(7); hit = 0; T = 20000
    for _ in range(T):
        if sum(1 for _ in range(N) if rnd.randrange(k) == 0) >= 35: hit += 1
    assert abs(hit / T - ex) < 0.004
    print(f"  모의 {T}회 빈도 {hit/T:.4f} (정확값과 일치, 허용 0.004)")
    N2 = 10000
    sd2 = math.sqrt(N2 * p * (1 - p)); print(f"  흐름 {N2}개면 상대 편차 {sd2/(N2*p):.2%} (100개일 때 {sd/(N*p):.1%}) → 흐름이 많을수록 고르다")
    print("  단, 흐름 1개가 전체의 40%를 차지(코끼리 흐름)하면 해시로는 못 나눈다")

# S11 오일러 분해: 시계열 패턴 분류 (잡음 포함)
def linfit(x, y):
    n = len(x); mx = sum(x) / n; my = sum(y) / n
    sxx = sum((a - mx) ** 2 for a in x); sxy = sum((a - mx) * (b - my) for a, b in zip(x, y))
    b = sxy / sxx if sxx else 0; a = my - b * mx
    ss = sum((v - my) ** 2 for v in y); se = sum((v - (a + b * u)) ** 2 for u, v in zip(x, y))
    return a, b, (1 - se / ss) if ss else 1.0
def classify_noisy(y):
    x = list(range(len(y)))
    a, b, r2lin = linfit(x, y)
    if min(y) > 0:
        la, lb, r2exp = linfit(x, [math.log(v) for v in y])
    else:
        lb, r2exp = 0, 0
    det = [v - (a + b * u) for u, v in zip(x, y)]
    sc = sum(1 for i in range(len(det) - 1) if det[i] * det[i + 1] < 0)
    rng = max(y) - min(y)
    mean = sum(y) / len(y); sd = math.sqrt(sum((v - mean) ** 2 for v in y) / len(y))
    if sd < 0.03 * (abs(mean) + 1e-9): return "기준값(상수)"
    if sc >= len(y) * 0.2 and r2lin < 0.5 and sd > 0.1 * abs(mean): return "회전·주기(i, π): 반복되는 흔들림(flapping)"
```

```
    if r2exp > 0.98 and lb > 0.01 and r2exp > r2lin + 0.01: return "성장 e^(at): 비율로 늘어남"
    if r2exp > 0.98 and lb < -0.01 and r2exp > r2lin + 0.01: return "감쇠 e^(−at): 비율로 줄어듦"
    if r2lin > 0.95 and b > 0: return "다항식 배수 t¹: 일정 속도로 늘어남(누수)"
    if r2lin > 0.95 and b < 0: return "다항식 배수 t¹: 일정 속도로 줄어듦"
    return "패턴표 밖(정의: 모름)"
def s11():
    head("S11 오일러 분해: 시계열을 성장·감쇠·회전·다항식·기준값으로 분류")
    rnd = random.Random(3); n = 40
    series = {
     "RSS 메모리(MB), 누수": [500 + 3.2 * t + rnd.gauss(0, 1.5) for t in range(n)],
     "재시도 폭주 요청 수/s": [20 * math.exp(0.11 * t) * (1 + rnd.gauss(0, 0.01)) for t in range(n)],
     "장애 후 에러율 감쇠": [900 * math.exp(-0.12 * t) * (1 + rnd.gauss(0, 0.01)) + 5 for t in range(n)],
     "링크 flapping(up=1/down=0)에 가까운 지연": [100 + 40 * math.cos(2 * math.pi * t / 7) + rnd.gauss(0, 3) for t in range(n)],
     "정상 CPU(%)": [32 + rnd.gauss(0, 0.4) for t in range(n)],
    }
    want = ["다항식 배수 t¹: 일정 속도로 늘어남(누수)", "성장 e^(at): 비율로 늘어남", None, "회전·주기(i, π): 반복되는 흔들림(flapping)", "기준값(상수)"]
    for (name, y), w in zip(series.items(), want):
        c = classify_noisy(y)
        print(f"  {name:<34} → {c}")
        if w: assert c == w, (name, c)
    # 이상적(잡음 없는) 열은 mathlab의 정확 분류기로 분류
    print("  이상적(잡음 없는) 열의 정확 분류는 부록 H-14의 classify_series가 담당 (같은 패턴표).")
    print("  판독: 메모리 t¹ → 누수(재시작 시점 예측), e^(at) → 증폭(재시도·큐), cos → 주기 원인(cron·GC·flap)")

# S12 적분부등식: 오류 예산을 구간으로 판정
def s12():
    head("S12 적분부등식: 누적 오류 시간 ∫e(t)dt 를 하한·상한 구간으로 가두어 판정")
    budget_min = 30 * 24 * 60 * 0.001
    assert abs(budget_min - 43.2) < 1e-9
    print(f"  30일 가용성 99.9% → 허용 오류 시간 {budget_min:.1f}분")
    hours = [(0.0, 0.0005)] * 716 + [(0.05, 0.30)] * 3 + [(0.10, 0.50)] * 1
    lo = sum(a * 60 for a, b in hours); hi = sum(b * 60 for a, b in hours)
    print(f"  시간별 오류 비율을 [최솟값, 최댓값]으로만 아는 720시간: 하한 {lo:.1f}분 ≤ ∫e ≤ 상한 {hi:.1f}분")
    verdict = "합격" if hi <= budget_min else ("불합격" if lo > budget_min else "불확정")
    print(f"  판정: {verdict} (상한 ≤ 예산이면 합격, 하한 > 예산이면 불합격, 걸치면 측정 해상도를 높인다)")
    assert verdict == "불확정"
    hours2 = [(0.0, 0.0005)] * 720
    lo2 = sum(a * 60 for a, b in hours2); hi2 = sum(b * 60 for a, b in hours2)
    assert hi2 <= budget_min
    print(f"  해상도가 좋은 경우 상한 {hi2:.1f}분 ≤ {budget_min:.1f}분 → 합격(구간이 좁아져야 결론이 난다)")

# S13 망원 합: curl -w 단계 분해
def s13():
    head("S13 망원 합: curl -w 누적 시각을 단계별 몫으로 분해 (합 = time_total)")
    t = dict(namelookup=Fr(3, 1000), connect=Fr(21, 1000), appconnect=Fr(89, 1000), pretransfer=Fr(89, 1000), starttransfer=Fr(412, 1000), t
otal=Fr(455, 1000))
    keys = list(t)
    parts = {"DNS": t["namelookup"], "TCP 연결": t["connect"] - t["namelookup"], "TLS": t["appconnect"] - t["connect"],
             "요청 준비": t["pretransfer"] - t["appconnect"], "서버 처리(첫 바이트까지)": t["starttransfer"] - t["pretransfer"], "본문 전송": t["tot
al"] - t["starttransfer"]}
    assert sum(parts.values()) == t["total"]
    for k, v in parts.items(): print(f"   {k:<22} {float(v)*1000:7.1f} ms  {float(v/t['total']):6.1%}")
    print(f"   합 {float(sum(parts.values()))*1000:.1f} ms = time_total (정확한 분수 합으로 일치 확인)")
    worst = max(parts, key=parts.get)
    assert worst == "서버 처리(첫 바이트까지)"
    print(f"  → 최대 몫: {worst}. 네트워크가 아니라 애플리케이션 쪽을 본다.")

# S14 GF(p): 설정 지문 (Schwartz–Zippel)
P61 = (1 << 61) - 1
def fingerprint(lines, x):
    acc = 0
    for ln in lines:
        h = zlib.crc32(ln.encode()) | (len(ln) << 32)
        acc = (acc * x + h) % P61
    return acc
def s14():
    head("S14 GF(p) 다항식 지문: 서버 수백 대의 설정이 같은지 한 숫자로 비교 (Schwartz–Zippel)")
    base = ["net.ipv4.ip_forward = 1", "net.core.somaxconn = 4096", "net.ipv4.tcp_tw_reuse = 1"] * 20
    drift = list(base); drift[17] = "net.core.somaxconn = 1024"
    x = random.Random(11).randrange(2, P61)
    f1, f2, f3 = fingerprint(base, x), fingerprint(list(base), x), fingerprint(drift, x)
    assert f1 == f2 and f1 != f3
    print(f"  같은 설정 지문 일치: {f1 == f2}, 한 줄이 다른 설정 지문 불일치: {f1 != f3}")
    d = len(base)
    print(f"  서로 다른 두 설정이 우연히 같은 지문일 확률 ≤ 차수/p = {d}/(2^61−1) ≈ {d/P61:.2e}")
    print("  → 비교는 x(무작위 점)와 지문 한 개씩만 주고받으면 되고, 서로 다른 줄 위치가 틀려도 잡힌다(줄 순서도 반영).")

# S15 이분 탐색: 계층 경계
def s15():
    head("S15 이분 탐색: 첫 고장 계층 찾기 (누적 검증기 v(k)=L1..Lk 모두 정상이면 1)")
    worst = 0
    for f in range(1, 9):
        calls = 0
        def v(k):
            global_calls[0] += 1
            return 1 if k < f else 0
        global_calls = [0]
        lo, hi = 0, 7
        while lo < hi:
            mid = (lo + hi) // 2 + 1 if False else (lo + hi + 1) // 2
            if v(mid): lo = mid
```

```
            else: hi = mid - 1
        first_bad = lo + 1
        assert first_bad == min(f, 8)
        worst = max(worst, global_calls[0])
    assert worst <= 3
    print(f"  고장 계층 1..7(8=고장 없음) 8가지 모두 정확히 찾음, 검증 호출 최대 {worst}회 (선형 L1→L7은 최대 7회, ⌈log2 8⌉=3)")
    print("  전제: 검증기가 '그 계층까지 끝-끝으로' 확인할 것(단조성). 비대칭 경로·간헐 장애는 단조성이 깨지니 선형으로 돌아간다.")

# S16 필요한 측정 횟수
def s16():
    head("S16 측정 횟수: 손실률 p<1%를 99% 확신으로 말하려면 몇 번 보내야 하나")
    delta, eps = 0.01, 0.01
    n0 = math.ceil(math.log(delta) / math.log(1 - eps))
    nh = math.ceil(math.log(2 / delta) / (2 * eps * eps))
    assert (n0, nh) == (459, 26492)
    print(f"  손실 0건 관측 시 정확한 한계: (1−p)^n ≤ δ → n ≥ ln δ/ln(1−p) = {n0}회")
    print(f"  손실률을 ±1%p로 추정(Hoeffding 양쪽): n ≥ ln(2/δ)/(2ε²) = {nh}회")
    print("  → ping 100번 무손실로는 '1% 미만'을 99% 확신으로 단정할 수 없다. 459번 무손실이어야 한다.")

# S17 나머지·주기
def s17():
    head("S17 나머지 연산: 라운드 로빈 mod k, 시퀀스 번호 되감김 2^32")
    seq = [n % 4 for n in range(10)]
    print("  백엔드 4대 라운드 로빈 n mod 4:", seq)
    I = [1, 1j, -1, -1j]
    assert all(abs((1j) ** n - I[n % 4]) < 1e-12 for n in range(20))
    print("  i^n 도 mod 4 로 순환(1, i, −1, −i): 주기 4 현상은 같은 구조")
    for gbps in (1, 10, 100):
        t = 2 ** 32 * 8 / (gbps * 1e9)
        print(f"  TCP 시퀀스(32비트)가 {gbps}Gbps에서 한 바퀴 도는 시간 {t:.2f}초")
    assert abs(2 ** 32 * 8 / 10e9 - 3.4359738368) < 1e-9
    print("  → 고속 구간에서 오래된 세그먼트와 새 세그먼트를 구분하려면 타임스탬프(PAWS)가 필요하다.")
    print(f"  CRC-32가 무작위 손상을 놓칠 확률 2^-32 = {2**-32:.2e}")

# S18 DNS 최악 시간
def s18():
    head("S18 곱셈 원리: DNS 최악 대기 시간과 쿠버네티스 ndots")
    timeout, attempts, servers = 5, 2, 3
    print(f"  resolv.conf timeout:{timeout} attempts:{attempts}, 서버 {servers}개 → 응답이 전혀 없을 때 한 질의 {timeout*attempts*servers}초")
    assert timeout * attempts * servers == 30
    search = 3; names = search + 1; types = 2
    print(f"  ndots:5, 검색 도메인 {search}개, 이름의 점이 5개 미만: 후보 이름 {names}개 × (A, AAAA) {types}종 = 질의 {names*types}건")
    assert names * types == 8
    print("  → 외부 도메인 하나를 열 때 질의 8건. 끝에 점을 찍은 이름(example.com.)이면 2건.")

def s19():
    head("S19 인수분해: 지연 꼬리 — 연결 N회 중 하나라도 느릴 확률 1−(1−q)^n ≈ n·q")
    q = 0.001
    for n in (1, 10, 100):
        ex = 1 - (1 - q) ** n
        print(f"  요청 하나가 느릴 확률 {q:.1%}, 한 화면이 요청 {n}개를 부르면 화면이 느릴 확률 {ex:.2%} (1차 근사 {n*q:.2%})")
    assert abs((1 - (1 - q) ** 100) - 0.0952) < 1e-3

ALL = [s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11, s12, s13, s14, s15, s16, s17, s18, s19]
if __name__ == "__main__":
    for f in ALL: f()
    print("\n모든 단언 통과")
```

부록 M. 진단은 수식으로, 해결은 기존 방식으로 — 호환성 가이드

부록 I~K의 진단 결과(수식·판정)를 받아 **널리 쓰이는 표준 해결 방법**으로 잇는다. 기존 방식을 쓰는 이유는 배포판·버전·환경이 달라도 통하고, 팀과 문서가 이미 아는 방식이기 때문이다. 값은 예시이며, 실제 값은 진단 수식으로 자기 환경에서 계산해 넣는다.

M-1. 진단 결과 → 표준 해결 매핑

진단 결과(수식·판정)	해결 범주	기존 방식(명령·설정)	호환성 주의	확인(같은 수식으로 재측정)
RTO 합이 앱 타임아웃보다 길다 (I-1)	타임아웃 정렬	앱 연결·읽기 타임아웃을 먼저 설정. 커널 값을 낮추려면 `sysctl -w net.ipv4.tcp_retries2=8` (값 변경은 전체 TCP에 영향)	컨테이너는 네임스페이스 sysctl만 변경 가능. 앱 타임아웃은 언어·라이브러리마다 이름이 다름	장애 재현 시 실패까지 걸리는 시간 재측정
도착률 > 처리율, 큐가 t^1로 증가 (I-1)	용량 증설·유입 제한	워커·인스턴스 증설, 큐 상한과 거부(503), 속도 제한(nginx `limit_req` 등)	무한 큐는 지연만 키운다. 상한을 두고 거부하는 쪽이 복구가 빠르다	Q_n 증가가 멈추는지, 대기 시간 재계산
재시도 폭주 (e^{at}) (I-6)	재시도 정책 교정	지수 백오프+지터, 재시도 횟수 상한, 서킷 브레이커, 부하 차단 후 점진 복구	클라이언트 라이브러리 설정이 서로 달라 한 곳만 고치면 효과가 적다	요청률을 로그 축으로 보아 기울기가 꺾였는지
이용률 90% 대, 민감도 급증 (I-3)	여유 확보	증설, 이용률 목표 70~80%, 오토스케일 임계를 그 아래로	오토스케일은 기동 지연이 있으니 임계를 더 낮게	dW/dλ 재계산, p99 확인
연결 포트 고갈 (I-12)	연결 재사용	커넥션 풀·keep-alive. 보조: `sysctl -w net.ipv4.ip_local_port_range="20000 60999"`, 출발 IP 추가	`tcp_tw_recycle` 는 커널 4.12에서 제거되어 쓰면 안 된다. `tcp_tw_reuse` 는 출발 연결에만 영향	실패율과 timewait 수, 471건/초 한계 재계산
conntrack 항목 수 r·T가 max 근접 (I-12)	표 확대·유지 시간 축소	`sysctl -w net.netfilter.nf_conntrack_max=524288`, 버킷은 `echo 131072 > /sys/module/nf_conntrack/parameters/hashsize`, 불필요 트래픽은 NOTRACK(raw 테이블)	항목은 메모리를 쓴다(수백 바이트씩). 증설 전 메모리 확인. nf_conntrack 모듈이 로드돼 있어야 경로 존재	count/max 비율, insert_failed·drop 증가 멈춤
MTU 블랙홀 (S8, 사례 7)	MSS 클램핑·MTU 정렬	iptables: `iptables -t mangle -A FORWARD -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --clamp-mss-to-pmtu` / nft: `tcp option maxseg size set rt mtu`, 인터페이스 MTU 수정, ICMP type 3 code 4 허용	영구 MTU 설정은 배포판 네트워크 관리 방식(NM·netplan·networkd·ifupdown)마다 다름(M-2)	`ping -M do -s 1472` 재시도
창/RTT 한계가 병목 (I-5)	창 확대	`sysctl -w net.ipv4.tcp_window_scaling=1`, `net.ipv4.tcp_rmem="4096 131072 16777216"`, `net.ipv4.tcp_wmem="4096 65536 16777216"` (최대값 ≥ BDP)	최대값을 크게 잡으면 연결 수가 많을 때 메모리 압박. net.core.rmem_max/wmem_max도 함께	ss -ti 의 rwnd_limited 사라짐, 처리량 재측정

손실이 $\sqrt{p}$로 처리량 제한 (I-5)	손실 구간 제거	케이블·SFP 교체, 혼잡 해소, 필요 시 혼잡 제어 변경 `modprobe tcp_bbr; sysctl -w net.ipv4.tcp_congestion_control=bbr` (+ `net.core.default_qdisc=fq`)	BBR은 커널 4.9 이상. 공유 회선에서는 다른 흐름과의 공정성 검토	mtr 손실 구간, 재전송 차분
accept 큐 넘침 (J-2)	backlog·워커	`sysctl -w net.core.somaxconn=4096` 와 앱의 listen backlog(nginx `listen ... backlog=4096`)를 같이	둘 중 작은 값이 적용. 앱 재시작이 필요한 경우가 많음	ListenOverflows 증가 멈춤
NIC 링 버퍼 넘침 (J-2)	링·큐 확대	`ethtool -G eth0 rx 4096`, `ethtool -L eth0 combined 8`, irqbalance	ethtool 변경은 재부팅·링크 재설정 시 사라진다. 영구화는 NM·networkd·udev 규칙(M-2). 드라이버·가상 NIC는 일부 미지원	rx_missed 차분이 0
한 코어에 softirq 집중 (J-2)	처리 분산	RSS 큐 수 확대, irqbalance, RPS/RFS(`/sys/class/net/eth0/queues/rx-0/rps_cpus`)	단일 흐름은 분산 불가. 가상화 환경은 큐 수 제한	mpstat %soft 분포 평탄화
FD 한도 근접 (J-5)	한도 상향 또는 누수 수정	systemd 서비스: `systemctl edit <서비스>` 에 `LimitNOFILE=65536`, 비 systemd 로그인 셸: `/etc/security/limits.conf`	limits.conf는 PAM 로그인에만 적용, systemd 서비스는 무시한다(unit의 LimitNOFILE이 필요)	/proc//limits 와 fd 수 비교
RSS가 t^1로 증가 (I-6)	누수 수정, 임시 재시작	근본: 코드·라이브러리 수정. 임시: 소진 시각의 절반 주기로 재시작 (systemd `RuntimeMaxSec=` 또는 롤링 재시작), 메모리 한도 (cgroup `MemoryMax=`)	한도는 OOM 킬을 부를 수 있으므로 재시작 정책과 함께	기울기 감소·소진 시각 재계산
CPU throttling (J-5)	한도 조정	k8s limits.cpu 상향 또는 제거, requests 정확화, 스레드 수 정리	cgroup v1은 cpu.stat, v2는 cpu.stat 위치와 이름이 다름(경로 확인)	nr_throttled 증가 멈춤
keepalive가 LB 유휴 타임아웃보다 김 (사례 3)	타임아웃 정렬	서버·앱 keepalive를 LB 유휴 값보다 짧게, 업스트림 재사용 설정	LB 종류(ALB 60초, NLB 350초 기본)와 앱 서버(nginx·Apache·앱 프레임워크)마다 설정 이름이 다름	유휴 후 첫 요청 재현
NACL·SG 응답 경로 누락 (J-8)	규칙 보완	NACL에 임시 포트(1024–65535) 되돌아오는 규칙 추가, IaC(Terraform 등)로 반영	콘솔에서 고치면 IaC 상태와 어긋난다. 코드에도 반영	Reachability Analyzer 재실행
rp_filter가 비대칭 응답을 버림 (사례 2)	정책 완화·경로 정리	`sysctl -w net.ipv4.conf.all.rp_filter=2` 와 인터페이스별 값(적용값은 all과 인터페이스 값 중 큰 쪽)	배포판 기본값이 다름(Debian 계열은 보통 0/1, RHEL 계열은 1). 근본은 비대칭 라우팅 해소	IPReversePathFilter 카운터 증가 멈춤
인증서 체인 누락·만료 (J-9)	체인 설치·갱신	서버에 leaf+intermediate 설치 (fullchain), 갱신 자동화(ACME 클라이언트), 만료 감시 `openssl x509 -checkend`	신뢰 저장소 갱신: Debian `update-ca-certificates` / RHEL `update-ca-trust`	Verify return code: 0
시계 오차 (J-9)	시간 동기	chrony(기본) 또는 systemd-timesyncd로 동기, 가상화는 호스트 시간 동기와 중복 확인	chronyd·ntpd·timesyncd 중 하나만 쓴다(동시 실행 금지). VM 일시정지 후 급격한 점프 주의	`chronyc tracking` 오차

DNS 최악 30초 구조 (I-12)	해석 설정 정리	죽은 네임서버 제거, `options timeout:1 attempts:2`, 로컬 캐시(systemd-resolved·dnsmasq·NodeLocal DNSCache), 이름 끝 점	/etc/resolv.conf는 systemd-resolved·NM이 덮어쓰므로 해당 관리 도구에서 수정	질의 시간 재측정
ndots 때문에 질의 8건 (S18)	ndots 조정	파드 `dnsConfig.options: ndots=2`, 외부 이름은 끝에 점	내부 서비스 단축 이름(svc 이름만) 해석이 영향을 받을 수 있음	DNS 질의 수 재계산
설정 지문 불일치 (I-10)	구성 관리로 수렴	Ansible·Puppet 등으로 원하는 상태에 맞춤, 수동 변경 금지, 지문 경보	수동으로 고치면 다음 실행에서 되돌려질 수 있다	지문 일치
변경 이분에서 범위가 좁혀짐 (I-9)	롤백·분할 배포	문제 변경 롤백, 변경을 작게 쪼개 배포, 카나리	변경 간 상호작용이 있으면 쌍으로 롤백	에러율 복귀
흐름 쏠림(이항·코끼리) (I-2)	분산 방식 변경	최소 연결 방식, 가중치, 흐름 쪼개기, 해시 입력에 포트 포함	장비마다 지원 알고리즘이 다름	서버별 부하 분포
꼬리 지연 증폭 (I-2)	팬아웃 줄이기	요청 병합·캐시, 타임아웃과 헤징(중복 요청), 느린 서버 제외	헤징은 부하를 늘리므로 제한	화면 p99
UDP 수신 버퍼 오류 (J-2)	버퍼 확대	`sysctl -w net.core.rmem_max=26214400` + 앱의 SO_RCVBUF	앱이 요청하지 않으면 기본값 사용. 재시작 필요	UdpRcvbufErrors 증가 멈춤

M-2. 환경별 호환성 표 — 같은 목적, 다른 방법

목적	확인 / 임시 적용	영구 적용 (배포판별)	호환성 주의
방화벽 규칙	`iptables -V` 로 (legacy) / (nf_tables) 확인. 직접 nft 면 `nft list ruleset`	영구화: Debian `iptables-save > /etc/iptables/rules.v4` (+ netfilter-persistent), RHEL firewalld `--permanent` + `--reload`, Ubuntu ufw	legacy와 nft 규칙이 따로 존재할 수 있다. 한쪽만 보면 놓침. firewalld가 있으면 iptables 직접 수정은 덮어쓰임
커널 파라미터	`sysctl -w k=v` (즉시, 재부팅 시 소멸)	`/etc/sysctl.d/99-lab.conf` 에 쓰고 `sysctl --system`	컨테이너는 네임스페이스 sysctl만 허용(k8s는 safe/unsafe sysctl 구분). 값이 적용 안 되면 상위 기본값·모듈 로드 여부 확인
프로세스 한도	systemd: `systemctl edit <서비스>` → `[Service] LimitNOFILE=`, 재시작	non-systemd 로그인: `/etc/security/limits.d/*.conf`	limits.conf는 systemd 서비스에 적용되지 않는다
인터페이스 설정(MTU 등)	임시: `ip link set eth0 mtu 1450`	NetworkManager `nmcli con mod ... 802-3-ethernet.mtu 1450` / Ubuntu netplan `mtu: 1450` + `netplan apply` / systemd-networkd `[Link] MTUBytes=` / ifupdown `mtu 1450`	배포판 관리 도구 하나를 정해 거기서만 수정. ip 명령 직접 변경은 재부팅·서비스 재시작에 사라진다
NIC 튜닝 (ethtool)	`ethtool -G/-L/-K/-s` (즉시)	NM 연결 속성, systemd-networkd `[Link]` 항목, udev 규칙으로 부팅 시 적용	가상 NIC·드라이버에 따라 일부 항목 미지원(Operation not supported)
DNS 설정	`resolvectl status` 로 관리 주체 확인	systemd-resolved: `/etc/systemd/resolved.conf`, NM: `nmcli con mod ... ipv4.dns`, 직접: `/etc/resolv.conf`	resolv.conf가 심볼릭 링크이면 직접 편집은 덮어쓰임
시간 동기	`timedatectl` 로 동기 여부, `chronyc tracking`	chrony 설정 `/etc/chrony.conf` (Debian은 `/etc/chrony/chrony.conf`)	동기 데몬은 하나만. VM 호스트 시간 동기 기능과 중복 점검
인증서 신뢰	Debian: `cp ca.crt /usr/local/share/ca-certificates/ && update-ca-certificates`	RHEL: `cp ca.crt /etc/pki/ca-trust/source/anchors/ && update-ca-trust`	자바·파이썬(certifi)·브라우저는 자체 저장소를 쓸 수 있다
cgroup 경로	v2: `/sys/fs/cgroup/<경로>/memory.max`, `cpu.stat`	v1: `/sys/fs/cgroup/memory/<경로>/memory.limit_in_bytes`, `cpu/<경로>/cpu.stat`	`stat -fc %T /sys/fs/cgroup` 가 cgroup2fs면 v2
패키지	Debian 계열 `apt`	RHEL 계열 `dnf` (구형 yum)	패키지 이름이 다르다(예: iproute2 ↔ iproute, dnsutils ↔ bind-utils)
서비스 재적용	`systemctl reload <서비스>` (끊김 없음)	`systemctl restart <서비스>` (끊김)	reload가 지원되는지 확인. 설정 검사(`nginx -t`, `sshd -t`, `haproxy -c`)를 먼저

M-3. 변경 절차 (환경과 무관한 순서)

```
# 호환성 높은 변경 절차 (모든 배포판·환경에서 같은 순서)
1) 증거 저장   전후 비교용으로 카운터·설정을 파일로 남김
                 sysctl -a > /tmp/before.sysctl; iptables-save > /tmp/before.rules   # nft면 nft list ruleset
2) 롤백 준비   되돌리는 명령을 먼저 적고, 원격이면 자동 복귀 타이머를 건다
                 echo 'iptables-restore < /tmp/before.rules' | at now + 5 minutes    # 확인되면 atrm 으로 취소
```

```
3) 임시 적용    재부팅하면 사라지는 방식으로 먼저(sysctl -w, ip link set, ethtool, iptables -I)
4) 같은 수식으로 재측정    진단에 쓴 같은 명령·같은 계산으로 개선을 숫자로 확인(표본 크기 확인, I-11)
5) 영구 적용    배포판의 관리 도구 한 곳에만(M-2 표), 설정 검사 후 reload
6) 기록·감시    원인 증거 + 변경 내용 + 검증기(v_) 또는 경보 추가, 구성 관리 코드에 반영
```

M-4. 하지 말아야 할 해결

하지 말 것	이유와 대안
`tcp_tw_recycle`	커널 4.12에서 제거되었고 NAT 뒤 클라이언트를 끊는 문제가 있었다. 쓰지 않는다
`iptables -F` 를 기본 정책 DROP 상태로 SSH에서 실행	내 세션이 끊긴다. 먼저 ACCEPT 규칙·타이머 롤백을 준비
rp_filter를 전체 0으로 끔	스푸핑 방어를 잃는다. 비대칭 경로를 고치거나 2(느슨)로
`echo 3 > /proc/sys/vm/drop_caches` 로 메모리 문제 해결	캐시를 비울 뿐 원인(누수)을 안 고친다. 직후 성능 저하
SELinux를 영구 끔	로그(`ausearch -m avc`)로 원인을 보고 정책·라벨(`restorecon`)을 고친다. 임시 점검은 `setenforce 0` 으로만
conntrack_max만 키움	메모리 소비 증가와 해시 버킷 부족. 버킷·메모리·타임아웃을 함께
somaxconn만 키움	앱의 listen backlog가 작으면 효과 없음
여러 값을 한꺼번에 변경	효과를 분리할 수 없다(I-5). 한 번에 하나
튜닝 값을 블로그에서 복사	값은 환경(RTT, 메모리, 흐름 수)에 따라 다르다. 진단 수식으로 필요값을 계산

M-5. 어디서 고치는가 — 환경별 수정 위치

환경	고치는 곳	주의
물리·가상 서버	호스트 OS(sysctl, 네트워크 관리 도구, systemd unit)	구성 관리(Ansible 등)에 반영해 재현 가능하게
컨테이너 (Docker)	이미지·실행 옵션(`--sysctl` , `--ulimit` , `--memory`)	컨테이너 안에서 바꾼 값은 재생성 시 사라짐. 호스트 전역 값은 호스트에서
쿠버네티스	Deployment·Pod 스펙(`resources` , `dnsConfig` , `securityContext.sysctls`), 노드 설정(DaemonSet·노드 이미지)	unsafe sysctl은 노드에서 허용해야 한다. kubectl로 직접 고치면 다음 배포에서 되돌아감
클라우드	IaC(Terraform·CloudFormation) 상태, 보안 그룹·NACL·라우팅 표, 로드밸런서 설정	콘솔에서만 고치면 IaC와 어긋난다. 코드에도 반영
관리형 서비스	서비스의 파라미터 그룹·옵션	OS 수준 조정이 불가능할 수 있다. 지원되는 항목만